
DATA SCIENCE

Série 1 — Volume I

Cahier d'exercices et d'applications

Exercices corrigés · Applications guidées · Projet
Compagnon du volume I : Fondations

À utiliser avec le livre : chaque chapitre renvoie ici
pour s'entraîner.

Table des matières

Mode d'emploi	1
Comment travailler avec ce cahier	1
Numérotation et difficulté	1
Les données et l'environnement	2
Test de départ	2
Questions	2
Corrigé	3
Interpréter votre score	3
Exercices du rappel express	3
Exercice 0.1 ★ — Une taxe (section R.1)	3
Exercice 0.2 ★ — Une somme (section R.4)	3
Exercice 0.3 ★ — Une pente (section R.3)	3
Exercice 0.4 ★ — Un logarithme (section R.5)	3
Exercice 0.5 ★ — Puissance et racine (section R.2)	3
Corrigés du rappel express	4
Chapitre 1 : Mathématiques pour la data science — exercices et applications	5
Préparation	5
Applications	5
Application 1.1 — Des colonnes redondantes : le rang d'une table	5
Application 1.2 — La matrice des similarités entre clients	6
Application 1.3 — La direction principale d'un nuage de clients	7
Application 1.4 — Résumer un tableau de ventes par la SVD	9
Application 1.5 — À quel niveau de ventes le bénéfice est-il maximal ?	10
Application 1.6 — Le gradient de l'erreur d'une droite	11
Application 1.7 — Une commande dans les trois prochaines minutes ?	12
Application 1.8 — La descente de gradient pour ajuster une droite	13
Application 1.9 — Le prix qui maximise les recettes	14
Application 1.10 — Répartir un budget publicitaire (multiplicateurs de Lagrange)	16
Application 1.11 — Le graphe des produits achetés ensemble	17
Exercices	18
Exercice 1.1 ★ — Similarité cosinus (section 1.1.1)	18
Exercice 1.2 ★ — Produit matriciel et système (section 1.1.2)	18
Exercice 1.3 ★★ — Valeurs propres (section 1.1.3)	19
Exercice 1.4 ★ — Dérivées (section 1.2.1)	19
Exercice 1.5 ★★ — Gradient (section 1.2.2)	19
Exercice 1.6 ★★ — Pas d'apprentissage (section 1.3.3)	19
Exercice 1.7 ★★ — Lagrange (section 1.3.5)	19
Exercice 1.8 ★★★ — Régression à la main (sections 1.3.3 et 1.3.4)	19

Exercice 1.9 ★★ — Analyse numérique (section 1.5)	19
Exercice 1.10 ★★ — Graphes (section 1.6)	19
Corrigés	19
Corrigé 1.1	20
Corrigé 1.2	20
Corrigé 1.3	20
Corrigé 1.4	20
Corrigé 1.5	21
Corrigé 1.6	21
Corrigé 1.7	21
Corrigé 1.8	22
Corrigé 1.9	22
Corrigé 1.10	22
Chapitre 2 : Probabilités — exercices et applications	23
Applications	23
Application 2.1 — Classer des avis clients avec la formule de Bayes	23
Application 2.2 — Voir pour croire : trois résultats par simulation	24
Application 2.3 — Dimensionner un échantillon : Tchebychev contre TCL	26
Application 2.4 — Chaîne de Markov : la valeur à long terme d'un client	26
Application 2.5 — Diversification et matrice de covariance	27
Application 2.6 — Les dépenses « à zéros » : une loi mixte	28
Exercices	29
Exercice 2.1 ★ — Union (section 2.1)	29
Exercice 2.2 ★★ — Bayes (section 2.1)	29
Exercice 2.3 ★ — « Au moins un » (section 2.1)	29
Exercice 2.4 ★ — Binomiale (section 2.2)	29
Exercice 2.5 ★★ — Poisson et exponentielle (section 2.2)	29
Exercice 2.6 ★★ — Normale (section 2.2)	29
Exercice 2.7 ★★ — Espérance et variance (section 2.3)	29
Exercice 2.8 ★★ — Covariance (section 2.3)	30
Exercice 2.9 ★★ — Théorème central limite (section 2.4)	30
Exercice 2.10 ★★ — Chaîne de Markov (section 2.6)	30
Corrigés	30
Corrigé 2.1	30
Corrigé 2.2	30
Corrigé 2.3	30
Corrigé 2.4	31
Corrigé 2.5	31
Corrigé 2.6	31
Corrigé 2.7	32
Corrigé 2.8	32
Corrigé 2.9	32
Corrigé 2.10	33
Chapitre 3 : Statistique — exercices et applications	34
Applications	34
Application 3.1 — Décrire un jeu de commandes	34
Application 3.2 — Biais, variance et maximum de vraisemblance par simulation	35
Application 3.3 — Intervalles de confiance : couverture et bootstrap	36
Application 3.4 — Comparer les trois canaux	38
Application 3.5 — Puissance, arrêt prématuré et tests multiples	39

Application 3.6 — Stratifier et pondérer	40
Application 3.7 — Tests sans hypothèse de loi : rangs et permutations	41
Exercices	41
Exercice 3.1 ★ — Résumer huit commandes (section 3.1 du livre)	41
Exercice 3.2 ★ — La variance sans biais (section 3.2 du livre)	41
Exercice 3.3 ★★ — Maximum de vraisemblance d'un taux d'arrivée (section 3.2 du livre)	42
Exercice 3.4 ★ — Intervalle de confiance d'une moyenne (section 3.3 du livre)	42
Exercice 3.5 ★★ — Intervalle de confiance d'une proportion (section 3.3 du livre)	42
Exercice 3.6 ★★ — Le transporteur tient-il sa promesse ? (section 3.4 du livre)	42
Exercice 3.7 ★★ — Un test A/B (section 3.4 et 3.5 du livre)	42
Exercice 3.8 ★★ — Un code promo a-t-il un effet ? (section 3.4 du livre)	42
Exercice 3.9 ★★ — Dimensionner une expérience (section 3.5 du livre)	42
Exercice 3.10 ★★ — Tests multiples (section 3.5 du livre)	42
Exercice 3.11 ★ — Marge d'erreur d'un sondage (section 3.6 du livre)	43
Exercice 3.12 ★★ — Mann-Whitney à la main (section 3.7 du livre)	43
Corrigés	43
Corrigé 3.1	43
Corrigé 3.2	43
Corrigé 3.3	44
Corrigé 3.4	44
Corrigé 3.5	44
Corrigé 3.6	44
Corrigé 3.7	45
Corrigé 3.8	45
Corrigé 3.9	46
Corrigé 3.10	46
Corrigé 3.11	46
Corrigé 3.12	47
Chapitre 4 : Programmation — exercices et applications	48
Préparation	48
Applications	48
Application 4.1 — Le ticket de caisse complet (section 4.1.10)	49
Application 4.2 — Le ticket de caisse en R (section 4.2)	50
Application 4.3 — Une file d'attente à l'atelier d'emballage (section 4.3.3)	50
Application 4.4 — Le seuil de livraison gratuite (section 4.3.5)	51
Application 4.5 — Le rapport hebdomadaire de la gérante (section 4.4.13)	51
Application 4.6 — Le tableau de bord de la gérante (section 4.5.7)	53
Application 4.7 — Le module de la boutique et ses tests (section 4.6)	54
Application 4.8 — Règles d'association : quels produits s'achètent ensemble ? (section 4.8.7)	58
Exercices	59
Exercice 4.1 ★ — dictionnaires et boucles (section 4.1)	59
Exercice 4.2 ★★ — un produit absent du catalogue (section 4.1.10)	60
Exercice 4.3 ★★ — un code promo (section 4.1.10)	60
Exercice 4.4 ★ — R (section 4.2)	60
Exercice 4.5 ★ — une pile (section 4.3.3)	60
Exercice 4.6 ★★ — une file à deux emballeuses (section 4.3.3)	60
Exercice 4.7 ★★ — une dichotomie à variante (section 4.3.5)	60
Exercice 4.8 ★ — tri stable (section 4.3.6)	60
Exercice 4.9 ★ — NumPy et broadcasting (section 4.4)	61

Exercice 4.10 ★★ — groupby et dates (section 4.4)	61
Exercice 4.11 ★★ — jointure et clients dormants (section 4.4.10)	61
Exercice 4.12 ★★ — un graphique honnête (section 4.5)	61
Exercice 4.13 ★★ — un test unitaire (section 4.6)	61
Exercice 4.14 ★★★ — coût d'un algorithme (section 4.8)	61
Corrigés	62
Corrigé 4.1	62
Corrigé 4.2	62
Corrigé 4.3	63
Corrigé 4.4	64
Corrigé 4.5	64
Corrigé 4.6	65
Corrigé 4.7	66
Corrigé 4.8	66
Corrigé 4.9	67
Corrigé 4.10	68
Corrigé 4.11	69
Corrigé 4.12	69
Corrigé 4.13	70
Corrigé 4.14	71
Chapitre 5 : Bases de données et SQL — exercices et applications	73
Préparation	73
Applications	73
Application 5.1 — Reconstruire la base de la boutique	74
Application 5.2 — Importer un CSV et lui donner un vrai schéma	75
Application 5.3 — Les dix meilleurs clients	76
Application 5.4 — Chiffre d'affaires par mois et par canal	77
Application 5.5 — Repérer les clients « dormants »	77
Application 5.6 — Segmenter la clientèle par quartiles de dépenses	78
Application 5.7 — Le temps dans les données : un calendrier complet, et le délai entre commandes	79
Application 5.8 — Le réseau de parrainage : chaînes et chiffre d'affaires généré	80
Application 5.9 — Mesurer le gain d'un index	80
Application 5.10 — De la base relationnelle aux documents JSON	81
Application 5.11 — Un cache en Python	82
Exercices	83
Exercice 5.1 ★ — Clés et contraintes (section 5.1 du livre)	83
Exercice 5.2 ★ — Filtrer (section 5.2 du livre)	83
Exercice 5.3 ★ — Agréger et joindre (section 5.2 du livre)	83
Exercice 5.4 ★★ — HAVING, jointure externe (section 5.2 du livre)	83
Exercice 5.5 ★★ — Anti-jointure (section 5.2 du livre)	83
Exercice 5.6 ★★ — NULL (section 5.2.7 du livre)	83
Exercice 5.7 ★★ — Dates, et regard statistique (sections 5.2.7 et 3.4 du livre)	84
Exercice 5.8 ★★★ — Fenêtre (section 5.3 du livre)	84
Exercice 5.9 ★★★ — Cte et fenêtres (section 5.3 du livre)	84
Exercice 5.10 ★★★ — Récursivité (section 5.3.6 du livre)	84
Exercice 5.11 ★★★ — Dépendances fonctionnelles (section 5.4 du livre)	84
Corrigés	84
Corrigé 5.1	84
Corrigé 5.2	85
Corrigé 5.3	85

Corrigé 5.4	86
Corrigé 5.5	86
Corrigé 5.6	87
Corrigé 5.7	88
Corrigé 5.8	88
Corrigé 5.9	89
Corrigé 5.10	90
Corrigé 5.11	90
Chapitre 6 : Outils de travail — exercices et applications	92
Applications	92
Application 6.1 — Une séance Git de A à Z	92
Application 6.2 — Branches, fusions et conflits	97
Application 6.3 — Un dépôt partagé, simulé sans Internet	99
Application 6.4 — Un notebook : le fabriquer, l'exécuter, l'exporter	100
Application 6.5 — L'état caché, le détecteur d'ordre et le nettoyeur de sorties	103
Application 6.6 — Explorer un fichier avec les outils du terminal	105
Application 6.7 — Un outil en ligne de commande, avec ses codes de sortie	107
Application 6.8 — Des scripts shell et un petit pipeline	108
Application 6.9 — Environnements virtuels : deux mondes sur une machine	110
Application 6.10 — Un rapport reproductible, de Markdown à PDF, avec make	112
Application 6.11 — R Markdown et LaTeX	115
Exercices	116
Exercice 6.1 ★ — Se repérer (section 6.3)	117
Exercice 6.2 ★ — Tuyaux (section 6.3)	117
Exercice 6.3 ★ — Premiers pas avec Git (section 6.1)	117
Exercice 6.4 ★★ — awk (section 6.3)	117
Exercice 6.5 ★★ — Conflit (section 6.1)	117
Exercice 6.6 ★★ — État caché (section 6.2)	117
Exercice 6.7 ★★ — Environnement (section 6.3)	117
Exercice 6.8 ★★ — Script shell (section 6.4)	117
Exercice 6.9 ★★ — Makefile (section 6.5)	118
Exercice 6.10 ★★ — Synthèse : un projet reproductible (section 6.5)	118
Corrigés	118
Corrigé 6.1	118
Corrigé 6.2	118
Corrigé 6.3	119
Corrigé 6.4	119
Corrigé 6.5	120
Corrigé 6.6	121
Corrigé 6.7	121
Corrigé 6.8	122
Corrigé 6.9	122
Corrigé 6.10	123
Projet du volume et auto-évaluation — exercices et applications	125
Projet du volume	125
P.1 Le cahier des charges	125
P.2 Étape 1 : charger et contrôler les données	126
P.3 Étape 2 : photographier l'année (question 1)	128
P.4 Étape 3 : les clients des réseaux sociaux dépensent-ils moins ? (question 2)	130
P.5 Étape 4 : les retards font-ils baisser la satisfaction ? (question 3)	132

P.6 Étape 5 : qui relancer en janvier ? (question 4)	134
P.7 Étape 6 : rédiger le rapport	135
P.8 Rendre le projet reproductible	137
P.9 Ce que cette étude ne dit pas, et la suite	138
Auto-évaluation	138
Mathématiques (chapitre 1)	138
Probabilités (chapitre 2)	138
Statistique (chapitre 3)	139
Programmation (chapitre 4)	139
SQL (chapitre 5)	139
Outils (chapitre 6)	139
Vérifier les réponses chiffrées	139
Corrigés des questions	140
Votre grille d'auto-évaluation	142

Mode d'emploi

« On apprend à nager dans l'eau, pas dans un livre. »

Ce **cahier** est le compagnon du volume I, *Fondations*. Le livre explique et démontre ; ici, **on s'entraîne**. Il contient, pour chaque chapitre du livre :

- des **applications** : de petites études guidées, sur des données réalistes, avec du code découpé en étapes ;
- des **exercices corrigés**, classés par difficulté ;
- et, à la fin du volume, le **projet** qui assemble tout, puis une **auto-évaluation**.

Le cahier est organisé comme le livre : le chapitre 3 du cahier accompagne le chapitre 3 du livre. Les sections du livre vous renvoient ici par une ligne de ce type :

Pour s'entraîner. Cahier, chapitre 3 : application 3.1, exercices 3.1 à 3.4.

Comment travailler avec ce cahier

1. **Lisez la section du livre**, puis ouvrez le chapitre correspondant du cahier.
2. **Cherchez d'abord**. Prenez un crayon et une feuille. Les exercices se font à la main, comme les exemples du livre. Ne regardez pas le corrigé avant d'avoir tenté sérieusement, même sans succès : l'effort de chercher est ce qui fait apprendre.
3. **Comparez** avec le corrigé. Si votre méthode diffère mais votre résultat est juste, c'est très bien ; s'il est faux, repérez où le raisonnement a dévié.
4. **Refaites** l'exercice quelques jours plus tard. Un exercice réussi une fois est une chance ; réussi deux fois, c'est une compétence.

Numérotation et difficulté

Repère	Signification
Exercice N.k	le k-ième exercice du chapitre N (énoncé dans « Exercices », corrigé dans « Corrigés »)
Application N.k	la k-ième application guidée du chapitre N
★	exercice d'application directe : une définition, un calcul
★★	exercice qui demande de combiner deux idées
★★★	exercice de réflexion : une démonstration, une modélisation, un piège

Chaque exercice indique la **section du livre** qu'il met en pratique, pour que vous puissiez y retourner.

Les données et l'environnement

Les applications utilisent les jeux de données du dossier `donnees/` : un tableau de commandes de la boutique (`commandes.csv`, 400 lignes) et une petite base SQL (`boutique.db`). Tout est fictif et généré avec une graine fixe : vous retrouverez exactement les mêmes nombres que dans le cahier.

Pour exécuter le code, installez l'environnement décrit dans l'avant-propos du livre (non exécuté ici, car il installe des paquets sur *votre* machine) :

```
python -m venv .venv
source .venv/bin/activate          # sous Windows : .venv\Scripts\activate
pip install numpy pandas scipy matplotlib seaborn
```

Chaque chapitre du cahier est **autonome** : il recharge lui-même ses données et refait ses imports, de sorte que vous pouvez commencer par n'importe lequel. Le code y est découpé en petites étapes, chacune suivie de sa sortie.

Vérifier une réponse avec Python. Pour les exercices de calcul, l'ordinateur est un excellent correcteur *après* avoir cherché à la main. Par exemple, pour vérifier une somme de fractions :

```
from fractions import Fraction

print(Fraction(3, 4) + Fraction(5, 6))
```

```
19/12
```

Le module `fractions` calcule en valeurs exactes : on retrouve $\frac{19}{12}$, le résultat de la première question du test ci-dessous.

Test de départ

Prenez dix minutes, un crayon, et répondez **sans calculatrice**. Les réponses sont juste après.

Questions

1. Calculez $\frac{3}{4} + \frac{5}{6}$.
2. Développez $(x + 2)^2$.
3. Résolvez $2x - 7 = 11$.
4. Quelle est la moyenne des nombres 4, 8, 15, 16, 23, 42 ?
5. On lance deux dés équilibrés. Quelle est la probabilité que la somme fasse 7 ?
6. Quelle est la dérivée de x^2 ?
7. Calculez $2^3 \times 2^4$.
8. Que vaut $\log_{10}(1000)$?
9. Un article coûte 120 €. On applique 25 % de remise. Quel est le nouveau prix ?
10. Dans une phrase : à quoi sert une « variable » en programmation ?

Corrigé

1. $\frac{3}{4} + \frac{5}{6} = \frac{9}{12} + \frac{10}{12} = \frac{19}{12}$.
2. $(x + 2)^2 = x^2 + 4x + 4$.
3. $2x = 18$, donc $x = 9$.
4. $(4 + 8 + 15 + 16 + 23 + 42)/6 = 108/6 = 18$.
5. Il y a $6 \times 6 = 36$ résultats possibles, dont 6 donnent 7 : (1,6), (2,5), (3,4), (4,3), (5,2), (6,1).
Probabilité : $6/36 = 1/6$.
6. $2x$.
7. $2^{3+4} = 2^7 = 128$.
8. 3, car $10^3 = 1000$.
9. $120 \times 0,75 = 90$ €.
10. Une variable est un nom qui désigne une valeur gardée en mémoire (par exemple prix = 45.0), que l'on peut relire et modifier.

Interpréter votre score

- **8 à 10 bonnes réponses** : vous êtes prêt(e). Commencez par le chapitre 1 du livre.
- **5 à 7** : lisez d'abord le « Rappel express » du livre, puis commencez.
- **Moins de 5** : lisez le « Rappel express » attentivement, en refaisant les calculs à la main. Ce n'est pas une affaire de talent, seulement de pratique.

Exercices du rappel express

Cinq questions pour vérifier que les bases du « Rappel express » sont en place. Si vous savez y répondre, vous avez tout ce qu'il faut pour commencer.

Exercice 0.1 ★ — Une taxe (section R.1)

Quel est le prix final d'un article à 80 € avec 19 % de TVA ?

Exercice 0.2 ★ — Une somme (section R.4)

Que vaut $\sum_{i=1}^4 i^2$?

Exercice 0.3 ★ — Une pente (section R.3)

Quelle est la pente de $y = -3x + 10$?

Exercice 0.4 ★ — Un logarithme (section R.5)

Simplifiez $\ln(e^2 \times e^3)$.

Exercice 0.5 ★ — Puissance et racine (section R.2)

Combien font $\sqrt{81} + 2^{-1}$?

Corrigés du rappel express

- **Corrigé 0.1.** $80 \times 1,19 = 95,2$ €.
- **Corrigé 0.2.** $1 + 4 + 9 + 16 = 30$.
- **Corrigé 0.3.** -3 : la droite descend de 3 quand x augmente de 1.
- **Corrigé 0.4.** $\ln(e^2 \times e^3) = \ln(e^5) = 5$.
- **Corrigé 0.5.** $\sqrt{81} = 9$ et $2^{-1} = \frac{1}{2}$, donc $9 + 0,5 = 9,5$.

Chapitre 1 : Mathématiques pour la data science — exercices et applications

Ce chapitre du cahier accompagne le chapitre 1 du livre. On y manipule vecteurs, matrices, dérivées, gradients et optimisation sur les petits exemples de la boutique. Aucune donnée externe : tout est défini dans le texte. Prérequis : le chapitre 1 du livre. Le code est écrit en Python avec NumPy et SciPy ; chaque application avance par petites étapes, avec le résultat sous chaque bloc.

Préparation

Une seule fois, au début de la séance :

```
import math
import numpy as np
```

Applications

Application 1.1 — Des colonnes redondantes : le rang d'une table

Section 1.1.2 du livre.

Contexte. Un fichier de produits contient le prix hors taxes (HT) et le prix toutes taxes comprises (TTC), égal à 1,19 fois le HT. Une des deux colonnes est-elle utile ?

Étape 1 : construire la table et mesurer son rang. Le rang est le nombre de colonnes réellement indépendantes.

```
ht = np.array([40.0, 25.0, 60.0, 15.0])
table = np.column_stack([ht, 1.19 * ht])      # deux colonnes : HT et TTC
print("colonnes :", table.shape[1], " rang :", np.linalg.matrix_rank(table))
```

```
colonnes : 2   rang : 1
```

Le rang vaut 1 : la colonne TTC ne dit rien de plus que la colonne HT.

Étape 2 : ajouter une vraie information. Ajoutons le poids de chaque produit (en kg), qui n'a aucun lien avec le prix.

```
poids = np.array([1.2, 0.4, 2.5, 0.3])
table3 = np.column_stack([ht, 1.19 * ht, poids])
print("colonnes :", table3.shape[1], "rang :", np.linalg.matrix_rank(table3))
```

```
colonnes : 3    rang : 2
```

Trois colonnes, mais le rang ne monte qu'à 2 : la redondance HT/TTC est toujours là.

Étape 3 : voir la conséquence. Une régression résout un système avec la matrice $\mathbf{X}^T \mathbf{X}$. Pour une table de rang insuffisant, elle est singulière :

```
G = table.T @ table
print("déterminant de X^T X :", round(np.linalg.det(G), 6))
print("valeurs propres      :", np.linalg.eigvalsh(G).round(4))
```

```
déterminant de X^T X : 0.0
valeurs propres      : [ -0.    14617.405]
```

Le déterminant est (numériquement) nul et une valeur propre est nulle : le système n'a pas de solution unique, on ne peut pas départager les rôles du HT et du TTC. C'est la **multicolinéarité**.

Pour aller plus loin. Retirez la colonne TTC et recalculez le déterminant de $\mathbf{X}^T \mathbf{X}$ pour la table (HT, poids). Que constatez-vous ?

Application 1.2 — La matrice des similarités entre clients

Section 1.1.2 du livre.

Contexte. Cinq clients sont décrits par leurs achats dans trois catégories (poteries, textiles, bijoux). On veut comparer **tous** les clients deux à deux d'un seul coup.

Étape 1 : la matrice des profils. Une ligne par client.

```
clients = {"Alix": [3, 1, 0], "Basile": [6, 2, 0], "Camille": [0, 1, 4],
           "Dominique": [1, 0, 5], "Eden": [2, 2, 2]}
noms = list(clients)
X = np.array(list(clients.values()), dtype=float)
print(X)
```

```
[[3. 1. 0.]
 [6. 2. 0.]
 [0. 1. 4.]
 [1. 0. 5.]
 [2. 2. 2.]]
```

Étape 2 : normaliser les lignes. Chaque ligne est divisée par sa norme, pour que sa longueur vaille 1.

```
U = X / np.linalg.norm(X, axis=1, keepdims=True)
print("normes des lignes de U :", np.linalg.norm(U, axis=1).round(3))
```

```
normes des lignes de U : [1. 1. 1. 1. 1.]
```

Étape 3 : toutes les similarités d'un coup. Le produit $\mathbf{U}\mathbf{U}^T$ rassemble les cosinus.

```
S = U @ U.T
print(" " * 10 + "...".join(f"{n:>10}" for n in noms))
for i, n in enumerate(noms):
    print(f"{n:<10}" + "...".join(f"{S[i, j]:10.2f}" for j in range(len(noms))))
```

	Alix	Basile	Camille	Dominique	Eden
Alix	1.00	1.00	0.08	0.19	0.73
Basile	1.00	1.00	0.08	0.19	0.73
Camille	0.08	0.08	1.00	0.95	0.70
Dominique	0.19	0.19	0.95	1.00	0.68
Eden	0.73	0.73	0.70	0.68	1.00

Lecture. La diagonale vaut 1 (chacun ressemble parfaitement à lui-même) et la matrice est symétrique. Alix et Basile valent 1,00 : même profil, simplement plus de volume. Camille et Dominique, tous deux amateurs de bijoux, sont très proches. Eden, qui achète un peu de tout, est moyennement proche de chacun (entre 0,68 et 0,73) sans avoir de « jumeau ».

Étape 4 : le plus proche voisin de chacun. Pour recommander des produits, on regarde ce qu'achète le client le plus semblable. On ignore la diagonale (la ressemblance à soi-même) avant de chercher le maximum.

```
S2 = S.copy()
np.fill_diagonal(S2, -1)
for i, n in enumerate(noms):
    j = S2[i].argmax()
    print(f"voisin le plus proche de {n:<9} : {noms[j]:<9} (similarité {S2[i, j]:.2f})")
```

```
voisin le plus proche de Alix      : Basile    (similarité 1.00)
voisin le plus proche de Basile    : Alix      (similarité 1.00)
voisin le plus proche de Camille   : Dominique (similarité 0.95)
voisin le plus proche de Dominique : Camille  (similarité 0.95)
voisin le plus proche de Eden      : Alix      (similarité 0.73)
```

Avec cinq mille clients, ce serait exactement le même code.

Pour aller plus loin. Remplacez la similarité cosinus par la distance euclidienne : les voisins les plus proches changent-ils ? Pourquoi (relisez la section 1.1.1 sur cosinus et distance) ?

Application 1.3 — La direction principale d'un nuage de clients

Section 1.1.3 du livre.

Contexte. Pour 200 clients, on mesure le nombre de visites mensuelles sur le site et la dépense mensuelle. Les deux semblent liées. Quel axe résume le mieux le nuage ?

Étape 1 : simuler les données (graine fixée pour retrouver les mêmes nombres).

```
rng = np.random.default_rng(42)
n = 200
visites = rng.normal(6, 2, n)
depense = 15 * visites + rng.normal(0, 12, n)
```

visites par mois
dépense en euros, liée aux visites

Étape 2 : standardiser (moyenne 0, écart-type 1), pour que les deux variables aient la même échelle.

```
def standardise(x):
    return (x - x.mean()) / x.std(ddof=1)

Z = np.column_stack([standardise(visites), standardise(depense)])
C = np.cov(Z.T)
print("Matrice de covariance :\n", C.round(3))
```

matrice de covariance 2 x 2

```
Matrice de covariance :
[[1.    0.903]
 [0.903 1.    ]]
```

C'est la matrice $\begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$ du livre, avec $r \approx 0,90$.

Étape 3 : valeurs propres et vecteurs propres.

```
valeurs, vecteurs = np.linalg.eigh(C)
print("Valeurs propres :", valeurs.round(3))
print("Part de la variance expliquée :", (valeurs / valeurs.sum()).round(3))
print("Direction principale :", np.abs(vecteurs[:, -1]).round(3))
```

```
Valeurs propres : [0.097 1.903]
Part de la variance expliquée : [0.049 0.951]
Direction principale : [0.707 0.707]
```

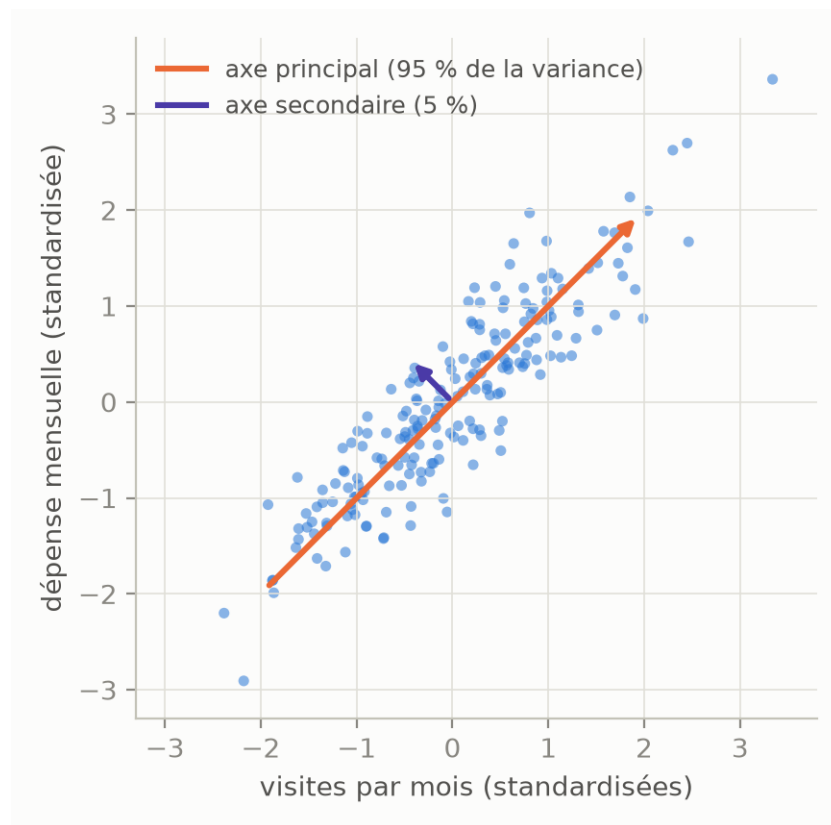


Figure 1: Nuage des 200 clients (variables standardisées) et son axe principal : le long de la diagonale, le nuage est étiré ; perpendiculairement, il est mince.

Lecture. La plus grande valeur propre (environ 1,90) correspond à la direction (0,71;0,71), la diagonale : l'axe « client actif et dépensier ». Elle capte environ 95 % de la variabilité, ce que prédit la formule $(1 + r)/2$ du livre. La seconde (0,10) correspond à la direction perpendiculaire : « dépense inhabituelle pour ce nombre de visites ». On peut donc résumer les deux variables par une seule en ne perdant que 5 % de l'information : c'est le principe de l'analyse en composantes principales (volume II).

Pour aller plus loin. Recommencez avec un bruit plus fort (par exemple `rng.normal(0, 40, n)` pour la dépense). Comment évoluent la corrélation et la part de variance de l'axe principal ? Comparez avec $(1 + r)/2$.

Application 1.4 — Résumer un tableau de ventes par la SVD

Section 1.1.4 du livre.

Contexte. Ventes hebdomadaires de 6 produits sur 8 semaines. Les données sont simulées selon une règle simple : $\text{ventes} = \text{popularité du produit} \times \text{effet de la semaine} + \text{bruit}$.

Étape 1 : construire le tableau.

```
rng = np.random.default_rng(7)
popularite = np.array([50, 30, 20, 12, 8, 5.0])           # un niveau par produit
saison = np.array([1.0, 1.1, 0.9, 1.2, 1.5, 1.4, 1.0, 0.8]) # un effet par semaine
M = (np.outer(popularite, saison) + rng.normal(0, 1.0, (6, 8))).round(0)
print(M.astype(int))
```

```
[[50 55 45 59 75 69 50 41]
 [30 32 27 36 45 41 30 25]
 [19 22 16 23 28 28 19 16]
 [12 13  8 14 18 17 10  9]
 [ 7  8  8  9 12 12  7  6]
 [ 5  6  3  6  9  5  6  4]]
```

Étape 2 : la décomposition en valeurs singulières.

```
U, s, Vt = np.linalg.svd(M)
part = 100 * s**2 / (s**2).sum()
print("valeurs singulières :", s.round(2))
print("part de l'énergie par couche (%) :", " ".join(f"{x:.2f}" for x in part))
```

```
valeurs singulières : [202.05  3.53  3.51  1.81  1.18  0.75]
part de l'énergie par couche (%) : 99.93 0.03 0.03 0.01 0.00 0.00
```

Étape 3 : approximation par une seule couche.

```
M1 = s[0] * np.outer(U[:, 0], Vt[0])
erreur = np.linalg.norm(M - M1) / np.linalg.norm(M)
print("erreur relative du rang 1 :", round(erreur, 4))
print("nombres à stocker : 48 (tableau) contre", 6 + 8 + 1, "(couche 1)")
```

```
erreur relative du rang 1 : 0.0271
nombres à stocker : 48 (tableau) contre 15 (couche 1)
```

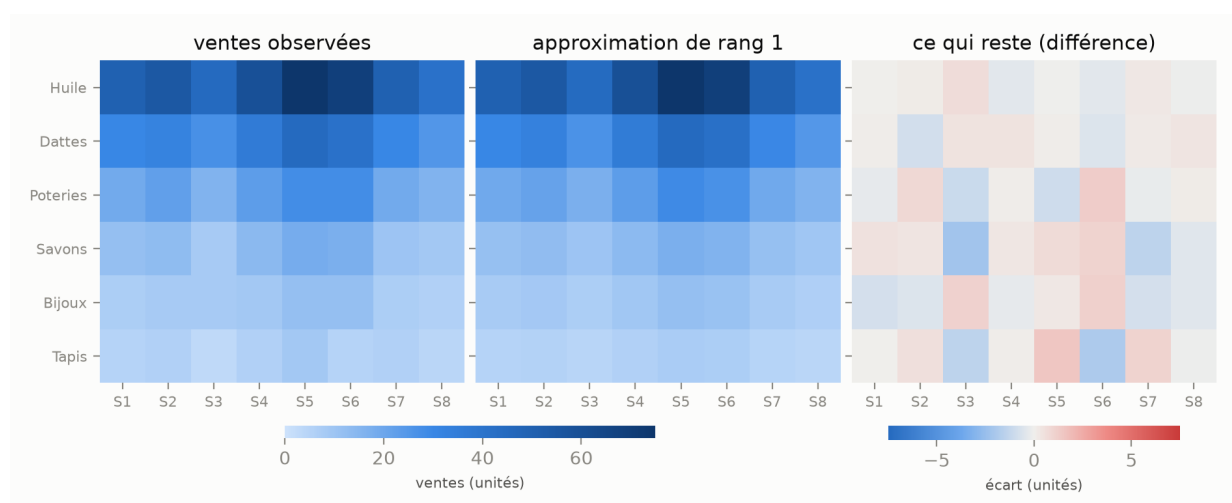


Figure 2: Ventes observées (à gauche), approximation par une seule couche (au centre) et ce qui reste (à droite). L'échelle de couleur est la même pour les deux premiers panneaux.

Lecture. La première valeur singulière (environ 202) est près de soixante fois plus grande que la deuxième (environ 3,5) : 99,9 % de l'« énergie » est dans une seule couche. Une approximation de rang 1, qui ne stocke que 15 nombres au lieu de 48, reproduit le tableau avec une erreur relative d'environ 2,7 %. La SVD a retrouvé la structure « popularité × saison » sans qu'on la lui indique.

Pour aller plus loin. Augmentez le bruit (écart-type 5 au lieu de 1) et regardez comment la part d'énergie de la première couche diminue. À partir de quel niveau de bruit la deuxième couche devient-elle utile ?

Application 1.5 — À quel niveau de ventes le bénéfice est-il maximal ?

Section 1.2.1 du livre.

Contexte. Le bénéfice hebdomadaire d'une poterie en fonction du nombre q de pièces vendues est $P(q) = -2q^2 + 80q - 300$.

Étape 1 : observer la fonction.

```
def benefice(q):
    return -2 * q**2 + 80 * q - 300

for q in [10, 15, 20, 25, 30]:
    print(f"q = {q:>2}    bénéfice = {benefice(q):>4} €")
```

```
q = 10    bénéfice = 300 €
q = 15    bénéfice = 450 €
q = 20    bénéfice = 500 €
q = 25    bénéfice = 450 €
q = 30    bénéfice = 300 €
```

Étape 2 : la dérivée et le bénéfice marginal. $P'(q) = -4q + 80$ s'annule en $q = 20$. Comparons la pente en $q = 10$ au gain réel de la pièce suivante.

```
print("gain réel de la 11e pièce : P(11) - P(10) =", benefice(11) - benefice(10), "€")
print("pente en q = 10           : P'(10) = -4*10 + 80 =", -4 * 10 + 80, "€")
```

```
gain réel de la 11e pièce : P(11) - P(10) = 38 €
pente en q = 10          : P'(10) = -4*10 + 80 = 40 €
```

Étape 3 : confirmer sans la dérivée, par une recherche numérique du maximum (on minimise $-P$).

```
from scipy.optimize import minimize_scalar

res = minimize_scalar(lambda q: -benefice(q), bounds=(0, 40), method="bounded")
print(f"optimum numérique : q = {res.x:.3f}, bénéfice = {-res.fun:.2f} €")
```

```
optimum numérique : q = 20.000, bénéfice = 500.00 €
```

Lecture. Le sommet est en $q = 20$ pour un bénéfice de 500 €. La pente en $q = 10$ (40 €) approche le gain réel de la pièce suivante (38 €) : la dérivée est exacte pour une variation infiniment petite, approchée pour une pièce.

Pour aller plus loin. Les frais fixes passent de 300 € à 500 €. Que devient le niveau optimal ? Et le bénéfice maximal ? (Expliquez pourquoi sans recalculer.)

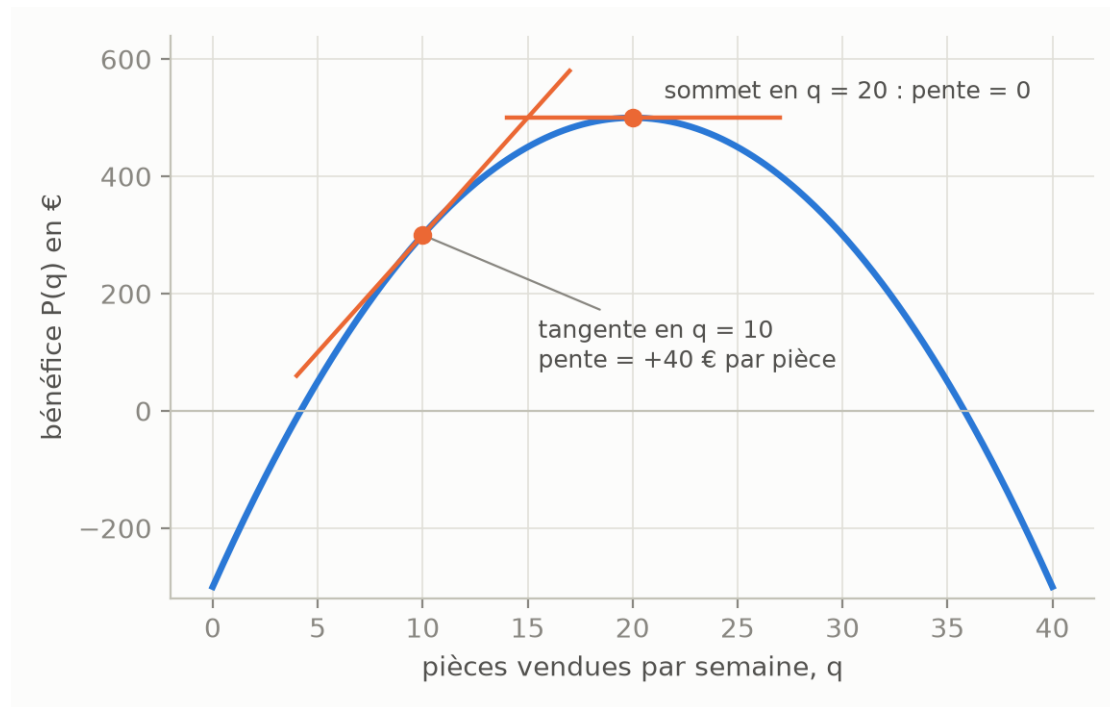


Figure 3: Bénéfice hebdomadaire en fonction du nombre de pièces vendues. En $q = 10$ la tangente monte avec une pente de 40 ; en $q = 20$ elle est horizontale : c'est le sommet.

Application 1.6 — Le gradient de l'erreur d'une droite

Section 1.2.2 du livre.

Contexte. On ajuste $y = ax + b$ à trois mesures $(1, 2)$, $(2, 3)$, $(3, 5)$; la perte est $L(a, b) = \sum_i (y_i - (ax_i + b))^2$.

Étape 1 : la perte et son gradient, écrits à partir des formules du livre.

```
x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 3.0, 5.0])

def perte(a, b):
    return np.sum((y - (a * x + b))**2)

def grad_perte(a, b):
    r = y - (a * x + b)
    return np.array([-2 * np.sum(x * r), -2 * np.sum(r)])

print("perte en (0, 0) :", perte(0, 0))
print("gradient en (0, 0) :", grad_perte(0, 0))
```

```
perte en (0, 0) : 38.0
gradient en (0, 0) : [-46. -20.]
```

On retrouve le calcul à la main du livre : $L = 38$ et un gradient $(-46, -20)$.

Étape 2 : vérifier le gradient par de petites différences. Si la formule est juste, la pente mesurée par un petit déplacement doit coïncider.

```
h = 1e-6
num_a = (perte(0 + h, 0) - perte(0 - h, 0)) / (2 * h)
num_b = (perte(0, 0 + h) - perte(0, 0 - h)) / (2 * h)
print("gradient numérique :", round(num_a, 4), round(num_b, 4))
```

```
gradient numérique : -46.0 -20.0
```

Étape 3 : le fond du bol. Au point $(1,5; 1/3)$ le gradient doit être nul.

```
print("perte en (1,5 ; 1/3) :", round(perte(1.5, 1 / 3), 4))
print("gradient en (1,5 ; 1/3) :", grad_perte(1.5, 1 / 3).round(10) + 0)
```

```
perte en (1,5 ; 1/3) : 0.1667
gradient en (1,5 ; 1/3) : [0. 0.]
```

Pour aller plus loin. Choisissez deux autres points (a, b) et vérifiez que $-\nabla L$ pointe bien dans la direction où la perte diminue, en comparant $\text{perte}(a, b)$ à perte après un petit pas de $-0,01 \nabla L$.

Application 1.7 — Une commande dans les trois prochaines minutes ?

Section 1.2.3 du livre.

Contexte. En heure de pointe, une commande arrive en moyenne toutes les 2 minutes. Le temps d'attente T (en minutes) suit une loi exponentielle de densité $f(t) = 0,5e^{-0,5t}$. Quelle est la probabilité que $T \leq 3$?

Étape 1 : intégrer avec des rectangles de Riemann.

```
def riemann(f, a, b, n):
    largeur = (b - a) / n
    xs = a + largeur * np.arange(1, n + 1)
    return np.sum(f(xs)) * largeur

densite = lambda t: 0.5 * np.exp(-0.5 * t)
for n in [3, 30, 300, 3000]:
    print(f"n = {n:>4} rectangles : P(T <= 3) ≈ {riemann(densite, 0, 3, n):.5f}")
```

```
n =      3 rectangles : P(T <= 3) ≈ 0.59877
n =     30 rectangles : P(T <= 3) ≈ 0.75761
n =    300 rectangles : P(T <= 3) ≈ 0.77493
n =   3000 rectangles : P(T <= 3) ≈ 0.77668
```

Étape 2 : comparer à l'intégration numérique et à la formule.

```
from scipy.integrate import quad

p_quad, _ = quad(densite, 0, 3)
print("intégration numérique (quad) :", round(p_quad, 5))
print("formule 1 - exp(-1,5)          :", round(1 - np.exp(-1.5), 5))

intégration numérique (quad) : 0.77687
formule 1 - exp(-1,5)          : 0.77687
```

Étape 3 : l'aire totale doit valoir 1 (c'est la condition pour qu'une densité soit une densité).

```
total, _ = quad(densite, 0, np.inf)
print("aire totale sous la courbe :", round(total, 5))
```

```
aire totale sous la courbe : 1.0
```

Lecture. Les rectangles convergent vers la valeur exacte $1 - e^{-1,5} \approx 0,777$: environ 77,7 % de chances qu'une commande arrive dans les 3 minutes.

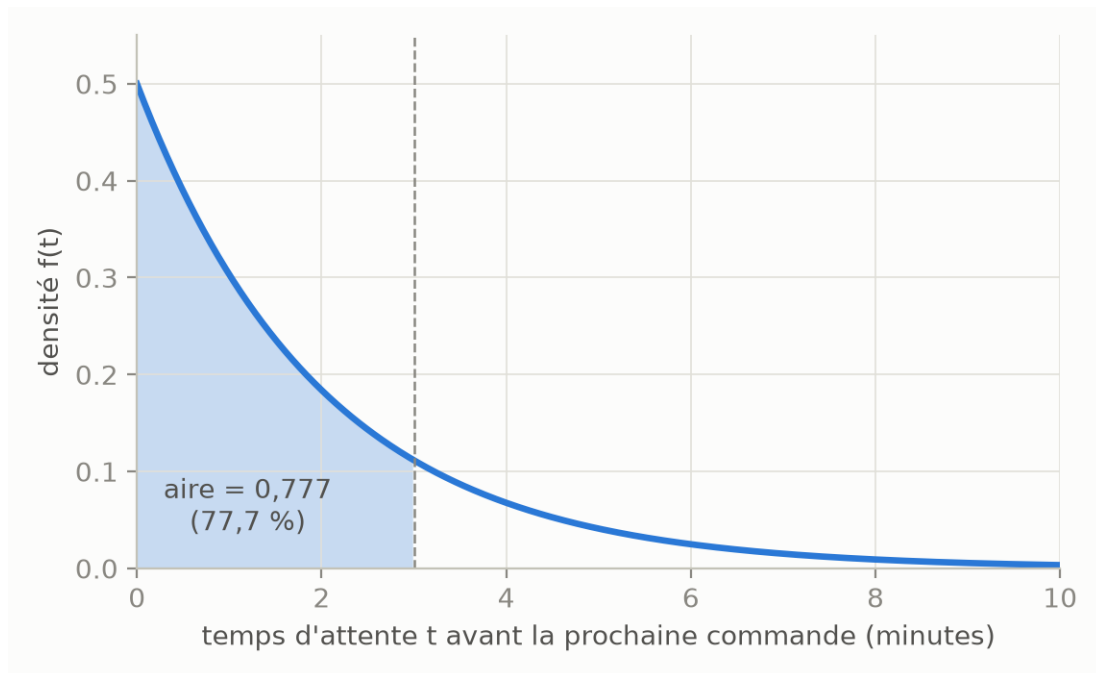


Figure 4: Densité du temps d'attente avant la prochaine commande. L'aire grisée entre 0 et 3 minutes vaut 0,777 : il y a environ 78 % de chances qu'une commande arrive dans les trois prochaines minutes.

Pour aller plus loin. Calculez, par la formule, la probabilité d'attendre **plus de 5 minutes**, puis vérifiez avec `quad(densite, 5, np.inf)`.

Application 1.8 — La descente de gradient pour ajuster une droite

Section 1.3.3 du livre.

Contexte. On reprend les trois points (1,2), (2,3), (3,5). On sait que la droite optimale est $y = 1,5x + 1/3$; voyons comment la descente de gradient la trouve, et comment le pas η et le centrage de x changent tout.

Étape 1 : l'algorithme, écrit une fois pour toutes.

```
def descente_de_gradient(grad, theta0, eta, n_iter=1000, tol=1e-8):
    """Renvoie le point final et le chemin parcouru."""
    theta = np.array(theta0, dtype=float)
    chemin = [theta.copy()]
    for _ in range(n_iter):
        g = grad(theta)
        if np.linalg.norm(g) < tol:           # presque à plat : on s'arrête
            break
        theta = theta - eta * g
        chemin.append(theta.copy())
    return theta, np.array(chemin)
```

Étape 2 : la perte des moindres carrés et son gradient.

```
x = np.array([1.0, 2.0, 3.0])
y = np.array([2.0, 3.0, 5.0])
X = np.column_stack([x, np.ones_like(x)])           # colonnes : x et 1 -> theta = (a, b)

grad_L = lambda theta, X, y: -2 * X.T @ (y - X @ theta)
perte_L = lambda theta, X, y: np.sum((y - X @ theta)**2)
```

```
H = 2 * X.T @ X
print("valeurs propres de la hessienne :", np.linalg.eigvalsh(H).round(2))
```

```
valeurs propres de la hessienne : [ 0.72 33.28]
```

Le pas doit rester inférieur à $2/\lambda_{\max} = 2/33,28 \approx 0,060$.

Étape 3 : un pas acceptable, puis un pas trop grand.

```
theta, chemin = descente_de_gradient(lambda t: grad_L(t, X, y), [0, 0], eta=0.05, n_iter=5000,
    ↪ tol=1e-6)
print("eta = 0,05 : itérations =", len(chemin) - 1, "   theta =", theta.round(4))

_, chemin = descente_de_gradient(lambda t: grad_L(t, X, y), [0, 0], eta=0.07, n_iter=15, tol=1e-6)
print("eta = 0,07 : pertes aux étapes 0, 5, 10, 15 :", [float(round(perte_L(chemin[k], X, y), 1)) for
    ↪ k in (0, 5, 10, 15)])
```

```
eta = 0,05 : itérations = 335   theta = [1.5   0.3333]
eta = 0,07 : pertes aux étapes 0, 5, 10, 15 : [38.0, 652.5, 11256.2, 194234.4]
```

Avec $\eta = 0,05$, la descente converge vers $(1,5; 0,3333)$ mais il lui faut **335 itérations**. Avec $\eta = 0,07$, la perte explose.

Étape 4 : centrer la variable x . On remplace $x = (1, 2, 3)$ par $x - \bar{x} = (-1, 0, 1)$, puis on revient à l'ordonnée d'origine par $b = b' - a\bar{x}$.

```
xc = x - x.mean()
Xc = np.column_stack([xc, np.ones_like(xc)])
print("valeurs propres (variable centrée) :", np.linalg.eigvalsh(2 * Xc.T @ Xc).round(3))

theta_c, chemin_c = descente_de_gradient(lambda t: grad_L(t, Xc, y), [0, 0], eta=0.15, n_iter=5000,
    ↪ tol=1e-6)
a, b_centre = theta_c
print("itérations =", len(chemin_c) - 1, "   retour : a =", round(a, 4), " b =", round(b_centre - a *
    ↪ x.mean(), 4))
```

```
valeurs propres (variable centrée) : [4. 6.]
itérations = 18   retour : a = 1.5   b = 0.3333
```

18 itérations au lieu de 335, pour le même résultat. C'est pourquoi on centre et on standardise presque toujours les variables.

Étape 5 : comparer avec la solution exacte des équations normales.

```
print("équations normales :", np.linalg.solve(X.T @ X, X.T @ y).round(4))
```

```
équations normales : [1.5   0.3333]
```

Pour aller plus loin. Avec la variable centrée, essayez $\eta = 0,3$ (limite théorique : $2/6 \approx 0,33$) puis $\eta = 0,4$. Combien d'itérations faut-il, et que se passe-t-il à $0,4$?

Application 1.9 — Le prix qui maximise les recettes

Section 1.3.4 du livre.

Contexte. On a testé huit prix pour un bol en céramique, chacun pendant une semaine.

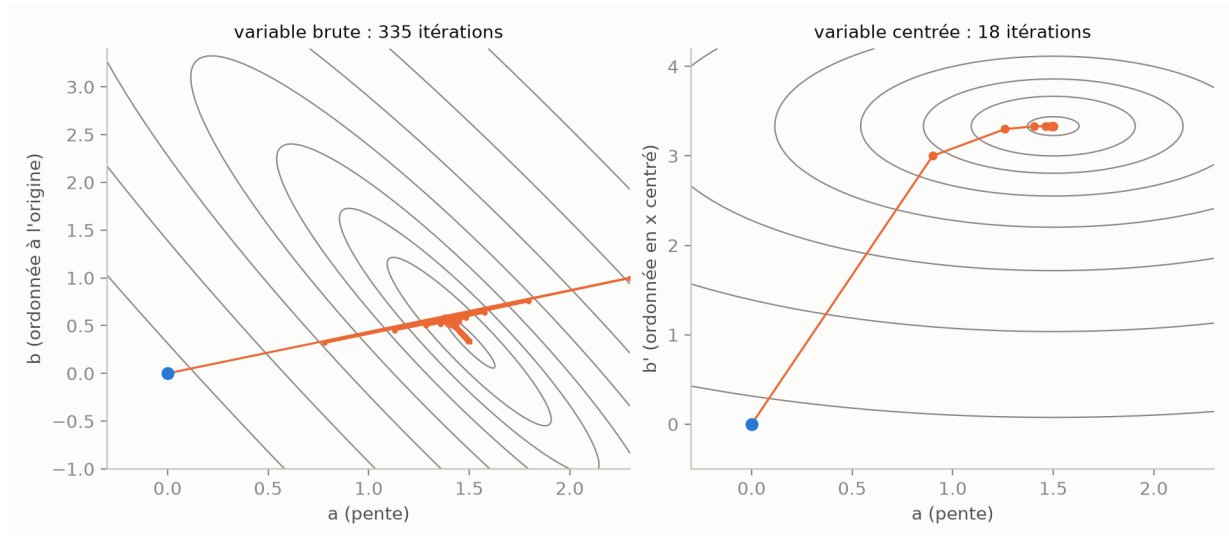


Figure 5: Chemin de la descente de gradient sur les courbes de niveau de la perte, avec la variable brute (à gauche) et la variable centrée (à droite). Le bol allongé force à zigzaguer ; le bol presque rond permet d'aller droit au but.

Prix p (€)	25	28	31	34	37	40	43	46
Ventes q (pièces)	93	82	82	69	67	56	56	47

Étape 1 : modéliser la demande par $q = \alpha + \beta p$, ajustée par descente de gradient sur le prix standardisé (les variables n'ont pas la même échelle).

```
prix = np.array([25, 28, 31, 34, 37, 40, 43, 46], dtype=float)
ventes = np.array([93, 82, 82, 69, 67, 56, 56, 47], dtype=float)

z = (prix - prix.mean()) / prix.std()
Z = np.column_stack([z, np.ones_like(z)])
theta = np.zeros(2)
for _ in range(2000):
    theta = theta + 0.05 * 2 * Z.T @ (ventes - Z @ theta)
    if np.linalg.norm(Z.T @ (ventes - Z @ theta)) < 1e-6:
        break

beta = theta[0] / prix.std()
alpha = theta[1] - theta[0] * prix.mean() / prix.std()
print(f"alpha = {alpha:.3f}    beta = {beta:.3f}")
print("contrôle avec np.polyfit :", np.polyfit(prix, ventes, 1).round(3)[:,-1])

alpha = 143.944    beta = -2.111
contrôle avec np.polyfit : [143.944 -2.111]
```

Chaque euro de hausse fait perdre environ 2,1 ventes par semaine.

Étape 2 : les recettes et leur maximum. $R(p) = p(\alpha + \beta p)$, maximale en $p^* = -\alpha/(2\beta)$ puisque $\beta < 0$.

```
p_opt = -alpha / (2 * beta)
R = lambda p: p * (alpha + beta * p)
print(f"prix optimal p* = {p_opt:.2f} €")
for p in (p_opt, 40, 46):
    print(f"recettes prévues à {p:5.2f} € : {R(p):7.1f} € par semaine")
```

prix optimal $p^* = 34.09$ €
 recettes prévues à 34.09 € : 2453.7 € par semaine
 recettes prévues à 40.00 € : 2380.0 € par semaine
 recettes prévues à 46.00 € : 2154.3 € par semaine

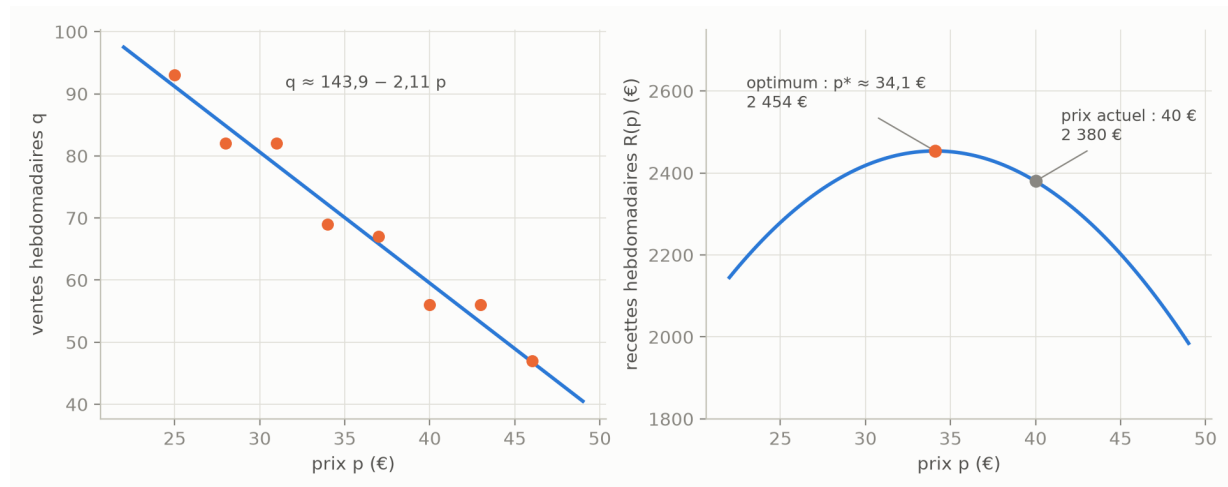


Figure 6: À gauche : les huit mesures et la droite de demande ajustée. À droite : les recettes prévues selon le prix, avec leur maximum vers 34 €.

Lecture. Le prix qui maximise les recettes est d'environ 34 €, pour 2 454 € par semaine. Au prix actuel de 40 €, elles seraient de 2 380 € : baisser le prix de 6 € rapporterait un peu plus de 70 € par semaine, soit environ 3 %.

Prudence. Huit points et un modèle très simple : aucune idée de l'incertitude de p^* , aucune prise en compte des coûts (on maximise des *recettes*), aucune garantie hors de la plage de prix testée. Ces questions relèvent du chapitre 3 et du volume II.

Pour aller plus loin. Chaque bol coûte 15 € à produire. Maximisez le **bénéfice** ($p - 15$) $q(p)$ au lieu des recettes : le prix optimal est-il plus haut ou plus bas ? Pourquoi ?

Application 1.10 — Répartir un budget publicitaire (multiplicateurs de Lagrange)

Section 1.3.5 du livre.

Contexte. On dispose de 1 000 € à répartir entre deux canaux de publicité, x euros sur le canal A et y euros sur le canal B, avec des recettes $R(x, y) = 80\sqrt{x} + 120\sqrt{y}$ sous la contrainte $x + y = 1000$.

Étape 1 : la solution par la formule de Lagrange, établie à la main dans le livre : $x = 1000/3,25$ et $y = 2,25x$.

```
R2 = lambda v: 80 * np.sqrt(v[0]) + 120 * np.sqrt(v[1])
x_th = 1000 / 3.25
print("formule : x = %.1f, y = %.1f, recettes = %.2f" % (x_th, 1000 - x_th, R2([x_th, 1000 - x_th])))
```

```
formule : x = 307.7, y = 692.3, recettes = 4560.70
```

Étape 2 : deux vérifications indépendantes. Un solveur numérique sous contrainte, puis une recherche exhaustive le long de la contrainte.

```
from scipy.optimize import minimize

res = minimize(lambda v: -R2(v), x0=[500, 500], method="SLSQP", bounds=[(1e-6, None), (1e-6, None)],
```

```

constraints=[{"type": "eq", "fun": lambda v: v[0] + v[1] - 1000}]
print("solveur SLSQP      : x = %.1f, y = %.1f, recettes = %.2f" % (res.x[0], res.x[1], -res.fun))

grille = np.linspace(1, 999, 9981)
valeurs = [R2([g, 1000 - g]) for g in grille]
i = int(np.argmax(valeurs))
print("recherche exhaustive : x = %.1f, y = %.1f, recettes = %.2f" % (grille[i], 1000 - grille[i],
↪ valeurs[i]))

```

```

solveur SLSQP      : x = 307.7, y = 692.3, recettes = 4560.70
recherche exhaustive : x = 307.7, y = 692.3, recettes = 4560.70

```

Étape 3 : le prix de l'ombre. Le multiplicateur $\lambda = 40/\sqrt{x}$ doit valoir le gain obtenu avec 1 € de budget en plus.

```

meilleur = lambda B: R2([B / 3.25, B - B / 3.25]) # répartition optimale pour un budget B
print("gain réel avec 1 € de plus :", round(meilleur(1001) - meilleur(1000), 4))
print("multiplicateur lambda      :", round(40 / np.sqrt(x_th), 4))

```

```

gain réel avec 1 € de plus : 2.2798
multiplicateur lambda      : 2.2804

```

Lecture. Chaque euro de budget supplémentaire rapporte environ 2,28 € de recettes ; il n'est donc rentable d'augmenter le budget que si la marge brute dépasse $1/2,28 \approx 44\%$.

Pour aller plus loin. Remplacez les coefficients (80, 120) par (80, 80). Quelle répartition obtenez-vous, et pourquoi est-elle plus simple à prévoir ?

Application 1.11 — Le graphe des produits achetés ensemble

Sections 1.6.1 à 1.6.4 du livre.

Contexte. Cinq produits ; deux produits sont reliés si au moins un client les a pris dans le même panier. On veut mesurer la ressemblance de paniers, compter des chemins, et trouver le plus court chemin de recommandation.

Étape 1 : ensembles et similarité de Jaccard.

```

panier_a = {"bol", "tapis", "lampe"}
panier_b = {"bol", "plateau"}
print("union :", sorted(panier_a | panier_b), " intersection :", sorted(panier_a & panier_b))
print("Jaccard(A, B) =", len(panier_a & panier_b) / len(panier_a | panier_b))

```

```

union : ['bol', 'lampe', 'plateau', 'tapis']  intersection : ['bol']
Jaccard(A, B) = 0.25

```

Étape 2 : le graphe en liste d'adjacence.

```

produits = ["bol", "tapis", "lampe", "plateau", "coussin"]
aretes = [("bol", "plateau"), ("bol", "tapis"), ("tapis", "lampe"),
          ("tapis", "coussin"), ("lampe", "coussin")]
voisins = {p: [] for p in produits}
for a, b in aretes:
    voisins[a].append(b)
    voisins[b].append(a)
print({p: len(v) for p, v in voisins.items()}) # le degré de chaque produit

```

```
{'bol': 2, 'tapis': 3, 'lampe': 2, 'plateau': 1, 'coussin': 2}
```

Étape 3 : la matrice d'adjacence et le comptage des chemins.

```

idx = {p: i for i, p in enumerate(produits)}
A = np.zeros((5, 5), dtype=int)
for a, b in aretes:
    A[idx[a], idx[b]] = A[idx[b], idx[a]] = 1

A2 = A @ A
print("chemins de longueur 2, bol -> lampe :", A2[idx["bol"], idx["lampe"]])
print("triangles :", np.trace(np.linalg.matrix_power(A, 3)) // 6)

```

```

chemins de longueur 2, bol -> lampe : 1
triangles : 1

```

Étape 4 : la recherche en largeur.

```

from collections import deque

def distances_depuis(depart, voisins):
    dist = {depart: 0}
    file = deque([depart])
    while file:
        courant = file.popleft()
        for v in voisins[courant]:
            if v not in dist:
                dist[v] = dist[courant] + 1
                file.append(v)
    return dist

print(distances_depuis("plateau", voisins))

{'plateau': 0, 'bol': 1, 'tapis': 2, 'lampe': 3, 'coussin': 3}

```

Lecture. Les deux paniers n'ont qu'un produit en commun sur quatre au total ($J = 0,25$). Le tapis (degré 3) est le produit le plus connecté. Il y a un seul chemin en deux pas entre le bol et la lampe (bol → tapis → lampe), et un seul triangle (tapis, lampe, coussin). Depuis le plateau, la chaîne de recommandation la plus courte vers une lampe passe par le bol puis le tapis (distance 3).

Pour aller plus loin. Ajoutez un sixième produit « bougie » relié seulement au coussin. Quelles distances depuis le plateau changent ? Combien de triangles y a-t-il maintenant ?

Exercices

Essayez chaque exercice sur papier ou dans un notebook *avant* de lire le corrigé. Difficulté croissante : ★ application directe, ★★ demande un raisonnement, ★★★ synthèse.

Exercice 1.1 ★ — Similarité cosinus (section 1.1.1)

Deux clientes sont décrites par leurs achats (bols, tapis, lampes) : $\mathbf{u} = (1, 2, 2)$ et $\mathbf{v} = (2, 0, 1)$. Calculez leur similarité cosinus. Sont-elles plutôt semblables ?

Exercice 1.2 ★ — Produit matriciel et système (section 1.1.2)

Soit $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$ et $\mathbf{b} = \begin{pmatrix} 5 \\ 10 \end{pmatrix}$. (a) Calculez $\mathbf{A}^2$. (b) Résolvez $\mathbf{Ax} = \mathbf{b}$ à la main (par substitution), puis vérifiez avec NumPy.

Exercice 1.3 ★★ — Valeurs propres (section 1.1.3)

Trouvez les valeurs propres et des vecteurs propres de $\mathbf{M} = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$. Vérifiez que la trace est la somme et le déterminant le produit des valeurs propres.

Exercice 1.4 ★ — Dérivées (section 1.2.1)

Pour $f(x) = x^3 - 6x^2 + 9x$, calculez f' , trouvez les points où la pente est nulle, et déterminez s'il s'agit de minima ou de maxima grâce à f'' .

Exercice 1.5 ★★ — Gradient (section 1.2.2)

Pour $f(x, y) = x^2 + 3y^2$, calculez le gradient en $(1, 1)$ puis effectuez **un pas** de descente de gradient avec $\eta = 0,1$. La valeur de f a-t-elle diminué ?

Exercice 1.6 ★★ — Pas d'apprentissage (section 1.3.3)

Soit $f(x) = 2(x-1)^2$. (a) Écrivez la récurrence de la descente de gradient. (b) Pour quelles valeurs de η converge-t-elle ? (c) Quelle valeur de η converge en un seul pas ?

Exercice 1.7 ★★ — Lagrange (section 1.3.5)

La gérante dispose de 10 mètres de ruban pour entourer un rectangle de tissu. Quel rectangle d'aire maximale peut-on former ? Utilisez un multiplicateur de Lagrange, c'est-à-dire maximisez xy sous la contrainte $x + y = 5$ (demi-périmètre), et interprétez λ .

Exercice 1.8 ★★★ — Régression à la main (sections 1.3.3 et 1.3.4)

Les ventes de bougies en fonction de la température sont : $x = (10, 15, 20, 25)$ et $y = (40, 35, 28, 22)$. (a) Ajustez la droite $y = ax + b$ avec les équations normales. (b) Retrouvez le résultat par descente de gradient sur la variable **centrée**. (c) Prédisez les ventes à 18 °C.

Exercice 1.9 ★★ — Analyse numérique (section 1.5)

Expliquez pourquoi `0.1 * 3 == 0.3` est faux en Python et écrivez le test correct. Puis donnez le nombre de chiffres perdus pour un conditionnement $\kappa = 10^6$.

Exercice 1.10 ★★ — Graphes (section 1.6)

Dans le graphe de co-achat de l'application 1.11, calculez avec $\mathbf{A}^2$ le nombre de chemins de longueur 2 entre « plateau » et « tapis », puis entre « bol » et « coussin ». Vérifiez en les énumérant à la main.

Corrigé 1.1

$\mathbf{u} \cdot \mathbf{v} = 1 \cdot 2 + 2 \cdot 0 + 2 \cdot 1 = 4$. $\|\mathbf{u}\| = \sqrt{1 + 4 + 4} = 3$, $\|\mathbf{v}\| = \sqrt{4 + 0 + 1} = \sqrt{5}$. Donc $\cos \theta = \frac{4}{3\sqrt{5}} \approx 0,596$.

```
u = np.array([1, 2, 2]); v = np.array([2, 0, 1])
cos = u @ v / (np.linalg.norm(u) * np.linalg.norm(v))
print("cosinus =", round(cos, 3), " angle =", round(np.degrees(np.arccos(cos)), 1), "degrés")
```

```
cosinus = 0.596   angle = 53.4 degrés
```

Un cosinus de 0,6 correspond à un angle d'environ 53° : **moyennement semblables** (1 = identiques, 0 = rien en commun). Elles aiment toutes deux les bols mais diffèrent sur les tapis.

Corrigé 1.2

(a) $\mathbf{A}^2 = \begin{pmatrix} 2 \cdot 2 + 1 \cdot 1 & 2 \cdot 1 + 1 \cdot 3 \\ 1 \cdot 2 + 3 \cdot 1 & 1 \cdot 1 + 3 \cdot 3 \end{pmatrix} = \begin{pmatrix} 5 & 5 \\ 5 & 10 \end{pmatrix}$. (b) Les équations sont $2x + y = 5$ et $x + 3y = 10$.

De la première : $y = 5 - 2x$. En remplaçant : $x + 15 - 6x = 10$, donc $-5x = -5$, $x = 1$ et $y = 3$.

```
A = np.array([[2, 1], [1, 3]]); b = np.array([5, 10])
print(A @ A)
print("solution :", np.linalg.solve(A, b))
```

```
[[ 5  5]
 [ 5 10]]
solution : [1. 3.]
```

Corrigé 1.3

Le polynôme caractéristique est $\det(\mathbf{M} - \lambda \mathbf{I}) = (4 - \lambda)(3 - \lambda) - 2 = \lambda^2 - 7\lambda + 10 = (\lambda - 5)(\lambda - 2)$.

Les valeurs propres sont 5 et 2. Pour $\lambda = 5$: $(\mathbf{M} - 5\mathbf{I})\mathbf{v} = \begin{pmatrix} -1 & 1 \\ 2 & -2 \end{pmatrix} \mathbf{v} = \mathbf{0}$ donne $\mathbf{v} = (1, 1)$.

Pour $\lambda = 2$: $\begin{pmatrix} 2 & 1 \\ 2 & 1 \end{pmatrix} \mathbf{v} = \mathbf{0}$ donne $\mathbf{v} = (1, -2)$. Trace : $4 + 3 = 7 = 5 + 2$ ✓. Déterminant : $4 \cdot 3 - 1 \cdot 2 = 10 = 5 \times 2$ ✓.

```
M = np.array([[4, 1], [2, 3]])
valeurs, vecteurs = np.linalg.eig(M)
print("valeurs propres :", np.sort(valeurs.real))
print("trace =", np.trace(M), " det =", round(np.linalg.det(M), 6))
print("M @ (1,1) =", M @ np.array([1, 1]), " = 5 * (1,1)")
```

```
valeurs propres : [2. 5.]
trace = 7   det = 10.0
M @ (1,1) = [5 5] = 5 * (1,1)
```

Corrigé 1.4

$f'(x) = 3x^2 - 12x + 9 = 3(x - 1)(x - 3)$, nulle en $x = 1$ et $x = 3$. $f''(x) = 6x - 12$. En $x = 1$: $f'' = -6 < 0$, c'est un **maximum local** ($f(1) = 4$). En $x = 3$: $f'' = 6 > 0$, c'est un **minimum local** ($f(3) = 0$).

```
f = lambda x: x**3 - 6*x**2 + 9*x
print("f(1) =", f(1), " f(3) =", f(3))
```

```
f(1) = 4  f(3) = 0
```

Corrigé 1.5

$\nabla f = (2x, 6y)$, donc $\nabla f(1,1) = (2,6)$. Le pas : $(1,1) - 0,1(2,6) = (0,8; 0,4)$. Avant : $f(1,1) = 1 + 3 = 4$. Après : $f(0,8; 0,4) = 0,64 + 3 \cdot 0,16 = 1,12$. La valeur a bien diminué (de 4 à 1,12).

```
grad = lambda p: np.array([2 * p[0], 6 * p[1]])
f2 = lambda p: p[0]**2 + 3 * p[1]**2
p = np.array([1.0, 1.0])
p_nouveau = p - 0.1 * grad(p)
print(p_nouveau, f2(p), round(f2(p_nouveau), 2))
```

```
[0.8 0.4] 4.0 1.12
```

Corrigé 1.6

- (a) $f'(x) = 4(x-1)$, donc $x_{k+1} = x_k - 4\eta(x_k - 1)$, soit $x_{k+1} - 1 = (1 - 4\eta)(x_k - 1)$. (b) L'écart à 1 est multiplié par $(1 - 4\eta)$ à chaque pas ; il tend vers 0 si $|1 - 4\eta| < 1$, c'est-à-dire $0 < \eta < \frac{1}{2}$. (On retrouve $\eta < 2/\lambda_{\max}$ avec $f'' = 4$.) (c) Le facteur est nul si $\eta = \frac{1}{4}$: on tombe sur le minimum en un seul pas.

```
for eta in (0.1, 0.25, 0.45, 0.5, 0.55):
    x = 5.0
    for _ in range(20):
        x = x - eta * 4 * (x - 1)
    print(f"eta = {eta:<5} -> x après 20 pas = {x:.6g}")
```

```
eta = 0.1  -> x après 20 pas = 1.00015
eta = 0.25 -> x après 20 pas = 1
eta = 0.45 -> x après 20 pas = 1.04612
eta = 0.5  -> x après 20 pas = 5
eta = 0.55 -> x après 20 pas = 154.35
```

Corrigé 1.7

On maximise $g(x,y) = xy$ sous $x + y = 5$. Lagrangien : $\mathcal{L} = xy - \lambda(x + y - 5)$. Les conditions $\partial_x \mathcal{L} = y - \lambda = 0$ et $\partial_y \mathcal{L} = x - \lambda = 0$ donnent $x = y = \lambda$. La contrainte donne $2\lambda = 5$, donc $x = y = 2,5$ m : le **carré** de 2,5 m de côté, d'aire 6,25 m². Le multiplicateur $\lambda = 2,5$ s'interprète comme le prix de l'ombre : un mètre de demi-périmètre en plus rapporte environ 2,5 m² d'aire en plus (l'aire optimale est $(c/2)^2$ et sa dérivée par rapport à c est $c/2 = 2,5$).

```
from scipy.optimize import minimize
res = minimize(lambda v: -v[0] * v[1], x0=[1, 1],
               constraints={"type": "eq", "fun": lambda v: v[0] + v[1] - 5})
print(res.x.round(3), "aire =", round(-res.fun, 3))
```

```
[2.5 2.5] aire = 6.25
```

Corrigé 1.8

- (a) On construit X avec une colonne x et une colonne de 1, puis on résout $X^T X \theta = X^T y$. (b) On centre x (moyenne 17,5), on descend le gradient, puis on revient à l'ordonnée d'origine par $b = b' - a\bar{x}$. (c) On évalue la droite en 18. Attention au pas : la hessienne vaut ici $2X_c^T X_c = \text{diag}(250, 8)$, donc il faut $\eta < 2/250 = 0,008$; nous prendrons $\eta = 0,003$.

```
x = np.array([10, 15, 20, 25.0]); y = np.array([40, 35, 28, 22.0])
X = np.column_stack([x, np.ones_like(x)])
a, b = np.linalg.solve(X.T @ X, X.T @ y)
print(f"(a) a = {a:.3f} b = {b:.3f}")

xc = x - x.mean()
Xc = np.column_stack([xc, np.ones_like(xc)])
theta = np.zeros(2)
for _ in range(500):
    theta = theta - 0.003 * (-2 * Xc.T @ (y - Xc @ theta))
a2, b2 = theta[0], theta[1] - theta[0] * x.mean()
print(f"(b) a = {a2:.3f} b = {b2:.3f}")
print("(c) ventes à 18 °C :", round(a * 18 + b, 1))
```

```
(a) a = -1.220 b = 52.600
(b) a = -1.220 b = 52.600
(c) ventes à 18 °C : 30.6
```

Lecture : chaque degré de plus fait perdre environ 1,2 ventes de bougies ; à 18 °C on prévoit environ 31 bougies.

Corrigé 1.9

Le nombre 0,1 n'a pas d'écriture finie en base 2 ; $0.1 * 3$ vaut 0.30000000000000004 , un flottant différent de celui de 0.3. Le bon test est `math.isclose(0.1 * 3, 0.3)`. Pour $\kappa = 10^6$ on perd environ $\log_{10} \kappa = 6$ chiffres significatifs sur les 16 disponibles : il en reste environ 10.

```
print(0.1 * 3, 0.1 * 3 == 0.3, math.isclose(0.1 * 3, 0.3))
```

```
0.30000000000000004 False True
```

Corrigé 1.10

Avec l'ordre des produits ["bol", "tapis", "lampe", "plateau", "coussin"] (indices 0 à 4), on lit $(A^2)_{\text{plateau}, \text{tapis}}$ et $(A^2)_{\text{bol}, \text{coussin}}$. À la main : plateau—bol—tapis est le seul chemin en deux pas du plateau au tapis (le plateau n'a qu'un voisin, le bol), donc 1. Bol—tapis—coussin est le seul chemin en deux pas du bol au coussin (les voisins du bol sont plateau et tapis ; seul le tapis touche le coussin), donc 1.

```
produits = ["bol", "tapis", "lampe", "plateau", "coussin"]
aretes = [("bol", "plateau"), ("bol", "tapis"), ("tapis", "lampe"),
          ("tapis", "coussin"), ("lampe", "coussin")]
idx = {p: i for i, p in enumerate(produits)}
A = np.zeros((5, 5), dtype=int)
for a_, b_ in aretes:
    A[idx[a_], idx[b_]] = A[idx[b_], idx[a_]] = 1
A2 = A @ A
print("plateau-tapis :", A2[idx["plateau"], idx["tapis"]])
print("bol-coussin   :", A2[idx["bol"], idx["coussin"]])
```

```
plateau-tapis : 1
bol-coussin   : 1
```

Chapitre 2 : Probabilités — exercices et applications

Ce chapitre du cahier accompagne le chapitre 2 du livre (probabilités, lois, espérance et variance, loi des grands nombres, théorème central limite). Les **applications** sont de petites études guidées avec du code ; les **exercices** se travaillent d'abord à la main, puis se vérifient par le calcul. Vous aurez besoin de Python avec NumPy et SciPy. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse.

Applications

Application 2.1 — Classer des avis clients avec la formule de Bayes

Contexte. La gérante reçoit des dizaines d'avis par semaine. Elle a lu et étiqueté 40 avis : 28 positifs et 12 négatifs. Pour trois mots-clés, elle a compté dans combien d'avis de chaque type ils apparaissent :

Mot	Avis positifs (sur 28)	Avis négatifs (sur 12)
« cassé »	2	9
« rapide »	18	2
« déçu »	1	8

Objectif. Calculer la probabilité qu'un nouvel avis soit négatif, d'après les mots qu'il contient.

Étape 1 : un seul mot, à la main. Un avis contient « cassé ». Avec $P(\text{nég}) = 12/40 = 0,3$, $P(\text{cassé} | \text{nég}) = 9/12$ et $P(\text{cassé} | \text{pos}) = 2/28$, la formule de Bayes donne

$$P(\text{nég} | \text{cassé}) = \frac{0,75 \times 0,3}{0,75 \times 0,3 + \frac{2}{28} \times 0,7} = \frac{0,225}{0,225 + 0,05} \approx 0,818.$$

Étape 2 : plusieurs mots à la fois. Le filtre **naïf** de Bayes suppose les mots indépendants *sachant la classe* : on multiplie les vraisemblances de chaque mot, en tenant compte aussi des mots **absents** (probabilité $1 - p$). On compare ensuite les deux classes. Écrivons-le en quelques lignes :

```
import numpy as np
n_pos, n_neg = 28, 12
vus = {"cassé": (2, 9), "rapide": (18, 2), "déçu": (1, 8)} # (avis positifs, avis négatifs)
```

```
def prob_negatif(presents, lissage=0):
    """P(négatif | mots présents), les autres mots du dictionnaire étant absents."""
    vrais = {"pos": n_pos / 40, "neg": n_neg / 40} # a priori
    for mot, (kp, kn) in vus.items():
        pp = (kp + lissage) / (n_pos + 2 * lissage)
        pn = (kn + lissage) / (n_neg + 2 * lissage)
        vrais["pos"] *= pp if mot in presents else 1 - pp
        vrais["neg"] *= pn if mot in presents else 1 - pn
    return vrais["neg"] / (vrais["pos"] + vrais["neg"])

print("« cassé » et « déçu », sans « rapide » :", round(prob_negatif({"cassé", "déçu"}), 4))
print("« rapide » seulement : ", round(prob_negatif({"rapide"}), 4))
```

```
« cassé » et « déçu », sans « rapide » : 0.9949
« rapide » seulement : 0.0102
```

Le premier avis a 99,5 % de chances d'être négatif ; le second, seulement 1,0 % (il est donc presque certainement positif). Les mots présents *et* les mots absents pèsent dans la balance.

Étape 3 : le piège des zéros. Un nouveau mot, « remboursement », apparaît dans 0 avis positif et 3 avis négatifs. Sa vraisemblance pour la classe positive vaut 0 : un seul de ces mots suffit à écraser toute l'information des autres, et le produit est nul. La parade classique est le **lissage de Laplace** : on ajoute 1 « fictif » à chaque comptage (et 2 au total), ce qui ne laisse jamais une probabilité à zéro.

```
vus["remboursement"] = (0, 3)
avis = {"cassé", "rapide", "remboursement"}
print("sans lissage :", round(prob_negatif(avis), 4))
print("avec lissage :", round(prob_negatif(avis, lissage=1), 4))
```

```
sans lissage : 1.0
avec lissage : 0.7726
```

Sans lissage, le mot jamais vu dans un avis positif impose une probabilité de 100 % d'être négatif, quoi que dise le mot « rapide ». Avec lissage, on obtient 77 % : un verdict nuancé, où la contradiction entre « rapide » et « remboursement » est enfin visible.

Pour aller plus loin. Que se passe-t-il quand on change l'a priori (par exemple 50/50 au lieu de 70/30) ? Quelle est la sensibilité du résultat au lissage (essayez `lissage=0.1`) ?

Application 2.2 — Voir pour croire : trois résultats par simulation

Contexte. Trois résultats du chapitre peuvent sembler incroyables. La simulation est la meilleure façon de s'en convaincre : on joue le jeu des milliers de fois et on compte.

Partie A — Monty Hall. Trois portes, une voiture ; vous choisissez la porte 0 ; l'animateur ouvre une porte vide parmi les deux autres. Gagne-t-on plus en changeant ?

```
import numpy as np
rng = np.random.default_rng(3)
voiture = rng.integers(0, 3, size=100_000) # porte gagnante de chaque partie
# on a toujours choisi la porte 0 ; si on change, on gagne exactement quand 0 était mauvaise
print("gain en restant :", round((voiture == 0).mean(), 3))
print("gain en changeant :", round((voiture != 0).mean(), 3))
```

```
gain en restant : 0.333
gain en changeant : 0.667
```

Partie B — Anniversaires. Dans un groupe de n personnes, quelle est la probabilité qu'au moins

deux partagent leur anniversaire ? On compare la formule exacte (par le complémentaire) et la simulation de 20 000 groupes.

```
def exacte(n):
    return 1 - np.prod([(365 - i) / 365 for i in range(n)])

rng = np.random.default_rng(7)
for n in (10, 23, 40):
    groupes = rng.integers(0, 365, size=(20_000, n))
    simulee = np.mean([len(set(g)) < n for g in groupes]) # y a-t-il un doublon ?
    print(f"n = {n:>2} : exacte = {exacte(n):.3f}    simulée = {simulee:.3f}")

n = 10 : exacte = 0.117    simulée = 0.119
n = 23 : exacte = 0.507    simulée = 0.503
n = 40 : exacte = 0.891    simulée = 0.892
```

Partie C — Un processus de Poisson par ses temps d'attente. Les commandes arrivent au rythme de 3 par heure. On construit le processus **uniquement** à partir de temps d'attente exponentiels, puis on compte les commandes de chaque heure et on compare à la loi de Poisson de paramètre 3.

```
from scipy import stats
rng = np.random.default_rng(15)
lam, n_heures = 3, 20_000
comptes = np.empty(n_heures, dtype=int)
for h in range(n_heures):
    t, k = rng.exponential(1 / lam), 0
    while t <= 1.0: # tant qu'on est dans l'heure
        k += 1
        t += rng.exponential(1 / lam) # temps d'attente jusqu'à la commande suivante
    comptes[h] = k
print("moyenne, variance des comptes :", comptes.mean().round(3), comptes.var().round(3), "(théorie : 3 et 3)")
for k in range(5):
    print(f"P(N = {k}) : simulée = {(comptes == k).mean():.4f}    Poisson(3) = {stats.poisson(3).pmf(k):.4f}")

moyenne, variance des comptes : 2.998 2.932 (théorie : 3 et 3)
P(N = 0) : simulée = 0.0493    Poisson(3) = 0.0498
P(N = 1) : simulée = 0.1479    Poisson(3) = 0.1494
P(N = 2) : simulée = 0.2207    Poisson(3) = 0.2240
P(N = 3) : simulée = 0.2313    Poisson(3) = 0.2240
P(N = 4) : simulée = 0.1688    Poisson(3) = 0.1680
```

Repères : en B, l'écart entre exacte et simulée ne dépasse pas 0,004 ; en C, la moyenne des comptes vaut 2,998 et leur variance 2,93.

Questions. (1) Que valent les écarts entre simulation et théorie, et comment évoluent-ils si l'on multiplie le nombre de répétitions par 100 ? (2) Dans la partie C, la variance observée est-elle proche de la moyenne ? Que cela dit-il de la loi de Poisson ?

Application 2.3 — Dimensionner un échantillon : Tchebychev contre TCL

Contexte. La boutique veut estimer son taux de conversion (proche de 0,205) à ± 2 points avec 95 % de confiance. Combien de visiteurs faut-il observer ? Le livre donne deux réponses : la borne de Tchebychev (valable pour toute loi, donc pessimiste) et celle du théorème central limite. Voyons ce que valent vraiment ces deux bornes.

Étape 1 : les deux tailles.

```
import numpy as np
p, eps = 0.205, 0.02
n_tcheb = p * (1 - p) / (0.05 * eps**2)
n_tcl = (1.96 / eps) ** 2 * p * (1 - p)
print(f"Tchebychev : {n_tcheb:,.0f} visiteurs    TCL : {n_tcl:,.0f} visiteurs    rapport : {n_tcheb / n_tcl:,.2f}")
```

Tchebychev : 8,149 visiteurs TCL : 1,565 visiteurs rapport : 5.21

Étape 2 : la couverture réelle. Pour une taille n donnée, on simule 100 000 échantillons et on calcule la proportion de ceux dont le taux observé est à moins de 2 points de la vérité. Cette proportion devrait dépasser 95 %.

```
rng = np.random.default_rng(6)
for n in (500, 1000, 1565, 3000, 8150):
    taux = rng.binomial(n, p, size=100_000) / n
    print(f"n = {n:>5} : couverture = {(np.abs(taux - p) <= eps).mean():.4f}")
```

```
n = 500 : couverture = 0.7321
n = 1000 : couverture = 0.8827
n = 1565 : couverture = 0.9508
n = 3000 : couverture = 0.9932
n = 8150 : couverture = 1.0000
```

Lecture et questions. La couverture passe de 73 % pour 500 visiteurs à 95,1 % pour 1 565 : c'est la prédiction du TCL. À 8 150, elle vaut 100 % (à la précision de la simulation).

- (1) Quelle taille est vraiment nécessaire ? Tchebychev est-il « faux » ? (2) À $n = 8\,150$, quelle est la couverture ? Que signifie « borne pessimiste » ? (3) Si la gérante veut une marge de ± 1 point au lieu de ± 2 , par combien faut-il multiplier n ? Vérifiez par la formule, puis par simulation.

Application 2.4 — Chaîne de Markov : la valeur à long terme d'un client

Contexte. Chaque mois, un client est Actif (A), Occasionnel (O) ou Inactif (I). La gérante a estimé la matrice de transition $\mathbf{P}$ (lignes : état actuel ; colonnes : état du mois suivant) et la contribution mensuelle moyenne de chaque état : 30 €, 10 € et 0 €. Elle envisage une campagne de relance qui ferait passer la probabilité Inactif→Actif de 0,10 à 0,20.

```
import numpy as np
P = np.array([[0.80, 0.15, 0.05],
              [0.30, 0.50, 0.20],
              [0.10, 0.20, 0.70]])
valeur = np.array([30, 10, 0])

def loi_stationnaire(P):
    """Vecteur propre de P^T pour la valeur propre 1, normalisé pour sommer à 1."""
    w, V = np.linalg.eig(P.T)
    pi = np.real(V[:, np.argmax(np.abs(w - 1))])
    return pi / pi.sum()
```

```
pi = loi_stationnaire(P)
print("loi stationnaire :", pi.round(3), "   revenu mensuel moyen :", round(pi @ valeur, 2), "€")
```

```
loi stationnaire : [0.5  0.25 0.25]   revenu mensuel moyen : 17.5 €
```

Étape 2 : la relance. On modifie la ligne « Inactif » (0,20 vers Actif, 0,20 vers Occasionnel, donc 0,60 de rester inactif) et on recalcule.

```
P_relance = P.copy()
P_relance[2] = [0.20, 0.20, 0.60]
gain = loi_stationnaire(P_relance) @ valeur - pi @ valeur
print("nouveau revenu :", round(loi_stationnaire(P_relance) @ valeur, 2), "€ ; gain :", round(gain,
↳ 2), "€ par client et par mois")
print("avec 2 000 clients :", round(gain * 2000), "€ par mois")
```

```
nouveau revenu : 19.3 € ; gain : 1.8 € par client et par mois
avec 2 000 clients : 3596 € par mois
```

Étape 3 : jusqu'où la relance vaut-elle la peine ? On fait varier la probabilité Inactif→Actif de 0,10 à 0,40 (en retirant ce qu'on ajoute à « rester inactif »).

```
for q in (0.10, 0.20, 0.30, 0.40):
    Pq = P.copy(); Pq[2] = [q, 0.20, 0.80 - q]
    print(f"Inactif→Actif = {q:.2f} : revenu mensuel moyen = {loi_stationnaire(Pq) @ valeur:.2f} €")
```

```
Inactif→Actif = 0.10 : revenu mensuel moyen = 17.50 €
Inactif→Actif = 0.20 : revenu mensuel moyen = 19.30 €
Inactif→Actif = 0.30 : revenu mensuel moyen = 20.43 €
Inactif→Actif = 0.40 : revenu mensuel moyen = 21.20 €
```

Les gains successifs diminuent : passer de 0,10 à 0,20 rapporte 1,80 € par client et par mois, de 0,20 à 0,30 seulement 1,13 €, de 0,30 à 0,40 0,77 €.

Questions. (1) Que représente chaque valeur propre de **P** en dehors de 1 ? (2) Si la campagne coûte 1,50 € par client et par mois, est-elle rentable pour $q = 0,20$? Pour $q = 0,15$? (3) Le modèle suppose que les probabilités de transition sont constantes. Citez deux raisons pour lesquelles c'est discutable.

Application 2.5 — Diversification et matrice de covariance

Contexte. Deux produits ont des ventes quotidiennes de moyenne 50 et d'écart-type 10. Plus leurs ventes sont corrélées négativement, plus le total est stable. On vérifie la formule $\text{Var}(X + Y) = \text{Var } X + \text{Var } Y + 2 \text{Cov}(X, Y)$ sur une famille de corrélations, puis on lit une matrice de covariance sur des données simulées.

Partie A — La variabilité du total selon la corrélation.

```
import numpy as np
rng = np.random.default_rng(8)
print("   rho   écart-type théorique   écart-type simulé")
for rho in (-0.8, -0.5, 0.0, 0.5, 0.8):
    cov = [[100, rho * 100], [rho * 100, 100]]
    total = rng.multivariate_normal([50, 50], cov, size=100_000).sum(axis=1)
    print(f"{rho:+5.1f}   {np.sqrt(200 + 200 * rho):18.2f}   {total.std():17.2f}")
```

rho	écart-type théorique	écart-type simulé
-0.8	6.32	6.34
-0.5	10.00	9.97
+0.0	14.14	14.14
+0.5	17.32	17.30

+0.8	18.97	18.97
------	-------	-------

Partie B — Une matrice de covariance, trois variables. Sur 365 jours, la température influence les visites, qui influencent les ventes.

```
rng = np.random.default_rng(2)
n = 365
temperature = rng.normal(25, 6, n)
visites = 80 + 3 * temperature + rng.normal(0, 15, n)
ventes = 0.2 * visites + rng.normal(0, 3, n)
donnees = np.column_stack([temperature, visites, ventes])
print("covariances :\n", np.cov(donnees.T).round(1))
print("corrélations :\n", np.corrcoef(donnees.T).round(2))
```

```
covariances :
[[ 36.7 106.2  22.6]
 [106.2 529.4 107.1]
 [ 22.6 107.1  31.1]]
corrélations :
[[1.  0.76 0.67]
 [0.76 1.  0.84]
 [0.67 0.84 1.  ]]
```

Le total est d'autant plus stable que la corrélation est négative : l'écart-type passe de 18,97 pour $\rho = +0,8$ à 6,32 pour $\rho = -0,8$, et la simulation retombe sur la théorie à 0,03 près.

Questions. (1) Pour quelle valeur de ρ la variabilité du total est-elle minimale, et quelle est-elle ? (2) Dans la partie B, expliquez pourquoi la corrélation température–ventes (0,67) est plus faible que les deux autres. (3) Calculez à la main $\text{Var}(\text{visites} + \text{ventes})$ à partir de la matrice de covariance, puis vérifiez avec `np.var(donnees[:,1]+donnees[:,2], ddof=1)`.

Application 2.6 — Les dépenses « à zéros » : une loi mixte

Contexte. Un visiteur dépense 0 € avec une probabilité de 70 % ; sinon sa dépense suit une loi exponentielle de moyenne 80 €. On calcule les résumés de cette loi mixte à la main, puis on les vérifie par simulation et on voit pourquoi un seul chiffre ne suffit pas.

À la main. $E[X] = 0,3 \times 80 = 24$. Pour la variance : $E[X^2] = 0,3 \times E[Y^2]$ avec $Y \sim \text{Exp}$ de moyenne 80, donc $E[Y^2] = 2 \times 80^2 = 12\,800$ et $E[X^2] = 3\,840$; ainsi $\text{Var}(X) = 3\,840 - 24^2 = 3\,264$ et $\sigma \approx 57,1$.

```
import numpy as np
rng = np.random.default_rng(31)
n = 500_000
achete = rng.random(n) < 0.3
depense = np.where(achete, rng.exponential(80, size=n), 0.0)
print("part de zéros      :", round((depense == 0).mean(), 4))
print("moyenne, écart-type :", depense.mean().round(2), depense.std().round(2), "(théorie : 24 et 57,1)")
print("médiane           :", np.median(depense).round(2))
print("moyenne chez les acheteurs :", depense[achete].mean().round(2))
```

```
part de zéros      : 0.6998
moyenne, écart-type : 24.03 57.09 (théorie : 24 et 57,1)
médiane           : 0.0
moyenne chez les acheteurs : 80.04
```

La simulation donne 70,0 % de zéros, une moyenne de 24,03 et un écart-type de 57,09 (théorie : 24 et 57,1) ; la médiane est nulle, et la dépense moyenne des seuls acheteurs vaut 80,04.

Questions. (1) Quelle est la médiane de X en théorie ? Pourquoi la moyenne et la médiane racontent-elles des histoires opposées ? (2) Comment présenteriez-vous ces dépenses à la gérante en deux chiffres plutôt qu'un ? (3) Quelle est la part du chiffre d'affaires total réalisée par les 10 % de clients qui dépensent le plus ? Estimez-la avec `np.sort` et `cumsum`.

Exercices

Exercice 2.1 ★ — Union (section 2.1)

Pour une newsletter : $P(\text{ouvre}) = 0,35$, $P(\text{clique}) = 0,25$, $P(\text{ouvre et clique}) = 0,10$. Quelle est la probabilité qu'un destinataire fasse **au moins une** des deux actions ? Qu'il ne fasse **aucune** ?

Exercice 2.2 ★★ — Bayes (section 2.1)

Chez un fournisseur, 2 % des articles sont défectueux. Un contrôle automatique signale 90 % des articles défectueux, mais aussi 4 % des articles corrects. Un article est signalé : quelle est la probabilité qu'il soit réellement défectueux ? Interprétez.

Exercice 2.3 ★ — « Au moins un » (section 2.1)

Chaque colis a 2 % de chances d'être endommagé, indépendamment des autres. (a) Probabilité qu'au moins un colis soit endommagé parmi 50 ? (b) Combien de colis faut-il pour que cette probabilité dépasse 90 % ?

Exercice 2.4 ★ — Binomiale (section 2.2)

Sur 15 visiteurs d'une publicité, chacun achète avec la probabilité 0,3. Calculez $P(X = 5)$, l'espérance et l'écart-type de X , et $P(X \geq 8)$.

Exercice 2.5 ★★ — Poisson et exponentielle (section 2.2)

Le service client reçoit en moyenne 4 appels par heure (processus de Poisson). (a) Probabilité de ne recevoir **aucun** appel pendant une demi-heure ? (b) Le temps d'attente entre deux appels est exponentiel : quelle est sa moyenne, et quelle est la probabilité d'attendre plus de 20 minutes ? (c) On a déjà attendu 10 minutes sans appel : quelle est la probabilité d'attendre **encore** 20 minutes ?

Exercice 2.6 ★★ — Normale (section 2.2)

Le poids des colis expédiés suit $\mathcal{N}(500 \text{ g}, 40^2)$. (a) Probabilité qu'un colis pèse moins de 450 g ? (b) Entre 460 et 540 g ? (c) Quel poids n'est dépassé que par 1 % des colis ?

Exercice 2.7 ★★ — Espérance et variance (section 2.3)

Un jeu de fidélité distribue : 0 € avec la probabilité 0,80 ; 10 € avec 0,15 ; 50 € avec 0,05. Calculez l'espérance, la variance et l'écart-type du gain. Si participer coûte 5 €, le jeu est-il favorable au client ? Et si le client joue 100 fois, quelle est l'espérance et l'écart-type de son gain total ?

Exercice 2.8 ★★ — Covariance (section 2.3)

Cinq clients ont noté le délai de livraison ($x = 1, 2, 3, 4, 5$ jours) et leur satisfaction ($y = 2, 4, 5, 4, 5$ sur 5). Calculez à la main la covariance (divisée par n), la variance de chaque variable et la corrélation. Vérifiez avec NumPy.

Exercice 2.9 ★★★ — Théorème central limite (section 2.4)

Le panier d'un client a une moyenne de 45 € et un écart-type de 30 € (loi très asymétrique). On observe 100 clients. (a) Quelle est la loi approchée de la moyenne ? (b) Probabilité que la moyenne dépasse 50 € ? (c) Combien de clients faut-il observer pour que l'erreur-type de la moyenne soit inférieure à 1 € ? (d) Vérifiez (b) par simulation. Attention : une loi exponentielle de moyenne 45 a un écart-type de 45, pas 30 ; quelle loi asymétrique de moyenne 45 et d'écart-type 30 peut-on utiliser à la place ?

Exercice 2.10 ★★★ — Chaîne de Markov (section 2.6)

Chaque jour, une machine est **en marche** (M) ou **en panne** (P). Si elle est en marche, elle tombe en panne le lendemain avec probabilité 0,1. Si elle est en panne, elle est réparée le lendemain avec probabilité 0,4. (a) Écrivez la matrice de transition. (b) Si elle est en marche aujourd'hui, quelle est la probabilité qu'elle le soit après-demain ? (c) Quelle est la proportion de temps passée en panne à long terme ?

Corrigés**Corrigé 2.1**

$P(\text{au moins une}) = 0,35 + 0,25 - 0,10 = 0,50$. Aucune : $1 - 0,50 = 0,50$. (On retranche l'intersection pour ne pas la compter deux fois ; le complémentaire de « au moins une » est « aucune ».)

Corrigé 2.2

Sur 10 000 articles : 200 défectueux, dont 180 signalés ; 9 800 corrects, dont 392 signalés à tort. Total des signalements : 572, dont 180 justifiés : $180/572 \approx 0,315$.

```
p_def, sens, fausse = 0.02, 0.90, 0.04
p_signal = sens * p_def + fausse * (1 - p_def)
print("P(défectueux | signalé) =", round(sens * p_def / p_signal, 4))
```

$P(\text{défectueux} \mid \text{signalé}) = 0.3147$

Seulement **31,5 %** : même avec un bon contrôle, **deux articles signalés sur trois sont corrects**, car les défauts sont rares (erreur du taux de base). Ils justifient une seconde vérification plutôt qu'un rejet automatique.

Corrigé 2.3

(a) $1 - 0,98^{50} \approx 0,636$. (b) On veut $1 - 0,98^n > 0,9 \iff 0,98^n < 0,1 \iff n > \frac{\ln 0,1}{\ln 0,98} \approx 113,97$, donc **114 colis**.

```
import numpy as np
from scipy import stats
print("(a)", round(1 - 0.98**50, 4))
print("(b)", np.log(0.1) / np.log(0.98), "-> n =", int(np.ceil(np.log(0.1) / np.log(0.98))))
```

(a) 0.6358
(b) 113.97408559184939 -> n = 114

Corrigé 2.4

$P(X = 5) = \binom{15}{5} 0,3^5 0,7^{10}$. $E[X] = np = 4,5$; $\sigma = \sqrt{np(1-p)} = \sqrt{3,15} \approx 1,775$; $P(X \geq 8) = 1 - P(X \leq 7)$.

```
X = stats.binom(15, 0.3)
print("P(X=5) =", round(X.pmf(5), 4))
print("E, sigma=", X.mean(), round(X.std(), 3))
print("P(X>=8) =", round(X.sf(7), 4))
```

P(X=5) = 0.2061
E, sigma= 4.5 1.775
P(X>=8) = 0.05

Corrigé 2.5

(a) Sur une demi-heure, $N \sim \text{Poisson}(4 \times 0,5 = 2)$: $P(N = 0) = e^{-2} \approx 0,135$. (b) Le temps d'attente est $\text{Exp}(4/h)$, de moyenne $1/4 \text{ h} = 15 \text{ min}$. $20 \text{ min} = 1/3 \text{ h}$: $P(T > 1/3) = e^{-4/3} \approx 0,264$. (c) Par absence de mémoire, c'est la même probabilité : **0,264**.

```
print("(a)", round(np.exp(-2), 4), round(stats.poisson(2).pmf(0), 4))
T = stats.expon(scale=1 / 4) # unité : heure
print("(b)", round(T.sf(1 / 3), 4))
print("(c)", round(T.sf(10 / 60 + 1 / 3) / T.sf(10 / 60), 4))
```

(a) 0.1353 0.1353
(b) 0.2636
(c) 0.2636

Corrigé 2.6

(a) $z = (450 - 500)/40 = -1,25$, $P = \Phi(-1,25) \approx 0,106$. (b) z de -1 à $+1$: $\approx 0,683$. (c) $z_{0,99} \approx 2,326$, donc $500 + 2,326 \times 40 \approx 593 \text{ g}$.

```
W = stats.norm(500, 40)
print("(a)", round(W.cdf(450), 4))
print("(b)", round(W.cdf(540) - W.cdf(460), 4))
print("(c)", round(W.ppf(0.99), 1))
```

(a) 0.1056
(b) 0.6827
(c) 593.1

Corrigé 2.7

$E = 0,15 \times 10 + 0,05 \times 50 = 1,5 + 2,5 = 4$ €. $E[X^2] = 0,15 \times 100 + 0,05 \times 2500 = 15 + 125 = 140$, $\text{Var} = 140 - 16 = 124$, $\sigma \approx 11,1$ €. À 5 € la partie, l'espérance du **gain net** est $4 - 5 = -1$ € : défavorable en moyenne (c'est favorable à la boutique). Sur 100 parties (indépendantes) : espérance $100 \times 4 = 400$ €, variance $100 \times 124 = 12\,400$, écart-type $\sqrt{12\,400} \approx 111$ €. Remarquez que l'écart-type relatif diminue : $111/400 = 28$ % contre $11,1/4 = 278$ % pour une seule partie.

```
gains = np.array([0, 10, 50]); probas = np.array([0.8, 0.15, 0.05])
E = (gains * probas).sum(); V = (gains**2 * probas).sum() - E**2
print("E =", E, " Var =", round(V, 2), " sigma =", round(np.sqrt(V), 2))
print("100 parties : E =", 100 * E, " sigma =", round(np.sqrt(100 * V), 1))
```

```
E = 4.0 Var = 124.0 sigma = 11.14
100 parties : E = 400.0 sigma = 111.4
```

Corrigé 2.8

Moyennes $\bar{x} = 3$, $\bar{y} = 4$. Écarts : $x - \bar{x} = (-2, -1, 0, 1, 2)$, $y - \bar{y} = (-2, 0, 1, 0, 1)$. Produits : $4, 0, 0, 0, 2$, de somme 6, donc $\text{Cov} = 6/5 = 1,2$. $\text{Var}(x) = (4 + 1 + 0 + 1 + 4)/5 = 2$; $\text{Var}(y) = (4 + 0 + 1 + 0 + 1)/5 = 1,2$. $\rho = \frac{1,2}{\sqrt{2 \times 1,2}} = \frac{1,2}{1,549} \approx 0,775$. Lecture : la corrélation est *positive* : plus la livraison est lente, plus la satisfaction serait élevée ? Ce n'est pas plausible : avec seulement cinq points, c'est probablement un hasard de l'échantillon. Nous verrons au chapitre 3 comment tester si une corrélation observée est significative.

```
x = np.array([1, 2, 3, 4, 5.0]); y = np.array([2, 4, 5, 4, 5.0])
print("cov =", ((x - x.mean()) * (y - y.mean())).mean(), " var x, y =", x.var(), y.var())
print("rho =", round(np.corrcoef(x, y)[0, 1], 4))
```

```
cov = 1.2 var x, y = 2.0 1.2
rho = 0.7746
```

Corrigé 2.9

- (a) Par le TCL, $\bar{X}_{100} \approx \mathcal{N}(45, 30^2/100) = \mathcal{N}(45, 3^2)$: erreur-type 3 €. (b) $z = (50 - 45)/3 \approx 1,667$, $P(\bar{X} > 50) \approx 0,048$. (c) $\sigma/\sqrt{n} < 1 \iff n > 900$. (d) Pour la simulation, une exponentielle de moyenne 45 a un écart-type de 45 (pas 30) : elle ne convient pas. On utilise une loi Gamma de moyenne 45 et d'écart-type 30 (forme $k = (45/30)^2 = 2,25$, échelle $\theta = 30^2/45 = 20$). La simulation donne 5,1 % contre 4,8 % par le TCL : l'écart vient de l'asymétrie résiduelle à $n = 100$.

```
print("(b) TCL :", round(stats.norm(45, 3).sf(50), 4))
rng = np.random.default_rng(9)
echantillons = rng.gamma(shape=2.25, scale=20, size=(100_000, 100))
print("moyenne, écart-type du panier simulé :", echantillons.mean().round(2),
      "\n", echantillons.std().round(2))
print("(b) simulée :", round((echantillons.mean(axis=1) > 50).mean(), 4))
print("(c) n minimal :", (30 / 1) ** 2)
```

```
(b) TCL : 0.0478
moyenne, écart-type du panier simulé : 45.01 30.0
(b) simulée : 0.0512
(c) n minimal : 900.0
```

Corrigé 2.10

(a) $\mathbf{P} = \begin{pmatrix} 0,9 & 0,1 \\ 0,4 & 0,6 \end{pmatrix}$ (lignes M, P). (b) $\mathbf{P}_{MM}^2 = 0,9 \times 0,9 + 0,1 \times 0,4 = 0,85$. (c) On cherche $\pi = (\pi_M, \pi_P)$ avec $\pi\mathbf{P} = \pi$: de la seconde colonne, $0,1\pi_M + 0,6\pi_P = \pi_P$, soit $0,1\pi_M = 0,4\pi_P$, donc $\pi_M = 4\pi_P$; avec $\pi_M + \pi_P = 1$: $\pi_P = 0,2$. La machine est en panne **20 % du temps** à long terme.

```
P = np.array([[0.9, 0.1], [0.4, 0.6]])
print("P^2[M,M] =", np.linalg.matrix_power(P, 2)[0, 0].round(4))
w, V = np.linalg.eig(P.T)
pi = np.real(V[:, np.argmin(np.abs(w - 1))]); pi /= pi.sum()
print("loi stationnaire :", pi.round(3))
```

```
P^2[M,M] = 0.85
loi stationnaire : [0.8 0.2]
```

Chapitre 3 : Statistique — exercices et applications

Ce chapitre du cahier accompagne le chapitre 3 du livre (statistique descriptive, estimation, intervalles de confiance, tests, puissance, sondages, méthodes non paramétriques). Il contient **sept applications guidées** (avec le code complet des simulations que le livre ne montre pas) et **douze exercices corrigés**. Toutes les applications travaillent sur le jeu de 400 commandes `donnees/commandes.csv` décrit au 3.1.2 du livre. Cherchez d'abord à la main, vérifiez ensuite par le code, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse.

Préparation commune. Le chargement ci-dessous sert à toutes les applications.

```
import numpy as np
import pandas as pd
from scipy import stats

df = pd.read_csv("donnees/commandes.csv")      # canal, montant, livraison, satisfaction
m = df["montant"]
print(df.shape)
```

```
(400, 4)
```

Applications

Application 3.1 — Décrire un jeu de commandes

Section 3.1 du livre. La gérante vous remet 400 commandes et demande un portrait fidèle **avant** toute analyse. Vous allez résumer, regarder la forme, comparer les canaux, puis relier délai et satisfaction.

Étape 1 : position et dispersion du montant.

```
resume = pd.DataFrame({"moyenne": m.mean(), "médiane": m.median(), "écart-type": m.std(),
                        "IQR": m.quantile(0.75) - m.quantile(0.25)}, index=["montant"])
print(resume.round(2))
print(m.quantile([0.05, 0.25, 0.5, 0.75, 0.95]).round(1).to_dict())
```

```
      moyenne  médiane  écart-type   IQR
montant   60.25    51.0     38.02  41.65
{0.05: 19.0, 0.25: 34.2, 0.5: 51.0, 0.75: 75.8, 0.95: 128.6}
```

La moyenne dépasse la médiane : c'est le signe d'une asymétrie à droite, que l'étape suivante quantifie.

Étape 2 : la forme. On mesure l'asymétrie, on compte les valeurs atypiques (règle de $1,5 \times \text{IQR}$) et l'on regarde si le logarithme symétrise.

```
q1, q3 = m.quantile([0.25, 0.75])
seuil = q3 + 1.5 * (q3 - q1)
print("asymétrie :", round(stats.skew(m), 2), " asymétrie du logarithme :",
      ↪ round(stats.skew(np.log(m)), 2))
print(f"valeurs atypiques (> {seuil:.1f} €) :", int((m > seuil).sum()))
print(df.loc[m > seuil, "canal"].value_counts().to_dict())
```

```
asymétrie : 1.75    asymétrie du logarithme : -0.07
valeurs atypiques (> 138.3 €) : 17
{'Boutique': 9, 'Site': 6, 'Réseaux': 2}
```

Étape 3 : comparer les canaux.

```
par_canal = df.groupby("canal").agg(commandes=("montant", "size"), panier_moyen=("montant", "mean"),
                                   ↪ "median"), panier_median=("montant", "median"), delai_moyen=("livraison",
                                   ↪ "mean"),
                                   satisfaction=("satisfaction", "mean")).round(2)
print(par_canal)
```

canal	commandes	panier_moyen	panier_median	delai_moyen	satisfaction
Boutique	114	74.81	64.85	0.00	4.49
Réseaux	138	49.01	41.50	4.49	3.72
Site	148	59.50	49.50	4.70	3.79

Étape 4 : relier délai et satisfaction.

```
print("Pearson :", round(df["livraison"].corr(df["satisfaction"]), 2),
      ↪ " Spearman :", round(stats.spearmanr(df["livraison"], df["satisfaction"]).statistic, 2))
classes = pd.cut(df["livraison"], [-1, 0, 3, 6, 20], labels=["retrait", "1-3 j", "4-6 j", "7 j et +"])
print(df.groupby(classes, observed=True)["satisfaction"].agg(["size", "mean"]).round(2))
```

```
Pearson : -0.53    Spearman : -0.51
      size mean
livraison
retrait      114  4.49
1-3 j         88  4.03
4-6 j        156  3.76
7 j et +      42  3.17
```

À vous. Rédigez en cinq lignes, pour la gérante, le portrait des commandes (une phrase sur le centre, une sur la dispersion, une sur la forme, une sur les canaux, une sur le délai). Lequel des résumés n'est pas fiable si l'on garde la boutique dans le calcul du lien délai-satisfaction, et pourquoi ?

Application 3.2 — Biais, variance et maximum de vraisemblance par simulation

Section 3.2 du livre. On vérifie par simulation ce que la théorie affirme, puis on compare deux lois pour les montants.

Étape 1 : diviser par n ou par $n - 1$? Cent mille échantillons de taille 5 d'une loi de variance 1.

```
rng = np.random.default_rng(3)
x = rng.normal(0, 1, size=(100_000, 5))
v_n, v_n1 = x.var(axis=1, ddof=0), x.var(axis=1, ddof=1)
print("moyenne, division par n :", round(v_n.mean(), 3), " (théorie 0,8)")
```

```
print("moyenne, division par n-1 :", round(v_n1.mean(), 3))
```

```
moyenne, division par n : 0.801 (théorie 0,8)
moyenne, division par n-1 : 1.001
```

Étape 2 : l'erreur quadratique moyenne. Le biais est un défaut, mais ce n'est pas le seul critère (3.2.2). Calculons l'EQM des deux estimateurs de la variance (la vraie valeur est 1) :

```
print("EQM, division par n :", round(((v_n - 1) ** 2).mean(), 3), " (théorie 0,36)")
print("EQM, division par n-1 :", round(((v_n1 - 1) ** 2).mean(), 3), " (théorie 0,50)")
```

```
EQM, division par n : 0.358 (théorie 0,36)
EQM, division par n-1 : 0.497 (théorie 0,50)
```

L'estimateur **biaisé** a une EQM plus faible : il est un peu décentré mais bien moins variable. C'est l'illustration de la décomposition $EQM = Var + Biais^2$.

Étape 3 : le maximum de vraisemblance numérique d'une loi Gamma.

```
from scipy import optimize

def neg_loglik(params, x):
    k, theta = params
    return np.inf if k <= 0 or theta <= 0 else -stats.gamma.logpdf(x, a=k, scale=theta).sum()

xbar, s2 = m.mean(), m.var()
depart = [xbar**2 / s2, s2 / xbar] # estimation par les moments
res = optimize.minimize(neg_loglik, depart, args=(m.to_numpy(),), method="Nelder-Mead")
print("moments :", np.round(depart, 3), " log-vraisemblance =", round(-neg_loglik(depart,
    ↪ m.to_numpy()), 1))
print("EMV      :", np.round(res.x, 3), " log-vraisemblance =", round(-res.fun, 1))
```

```
moments : [ 2.511 23.991] log-vraisemblance = -1942.2
EMV      : [ 3.001 20.074] log-vraisemblance = -1938.9
```

Étape 4 : Gamma ou log-normale ? Les deux lois ont deux paramètres : on peut comparer directement leurs log-vraisemblances maximales.

```
k, _, theta = stats.gamma.fit(m, floc=0)
s, _, scale = stats.lognorm.fit(m, floc=0)
print("Gamma      :", round(stats.gamma.logpdf(m, k, 0, theta).sum(), 1))
print("Log-normale :", round(stats.lognorm.logpdf(m, s, 0, scale).sum(), 1))
```

```
Gamma      : -1938.9
Log-normale : -1931.0
```

À vous. Laquelle des deux lois décrit le mieux les montants ? L'écart est-il grand ? Comment trancheriez-vous si les deux modèles avaient un nombre de paramètres différent (critère d'Akaike) ?

Application 3.3 — Intervalles de confiance : couverture et bootstrap

Section 3.3 du livre.

Étape 1 : que veut dire « 95 % » ? On traite les 400 commandes comme la population, on tire 20 000 échantillons de 40 commandes (sans remise) et l'on compte les intervalles de Student qui contiennent la vraie moyenne.

```
rng = np.random.default_rng(21)
population, mu, n_ech = m.to_numpy(), m.mean(), 40
```

```

t_crit = stats.t.ppf(0.975, n_ech - 1)
couvre, largeurs = 0, []
for _ in range(20_000):
    e = rng.choice(population, size=n_ech, replace=False)
    demi = t_crit * e.std(ddof=1) / np.sqrt(n_ech)
    couvre += abs(e.mean() - mu) <= demi
    largeurs.append(2 * demi)
print("couverture :", round(couvre / 20_000, 4), " largeur moyenne :", round(np.mean(largeurs), 2),
      " €")

```

```
couverture : 0.9451  largeur moyenne : 23.88 €
```

Étape 2 : le bootstrap. Intervalle « percentile » pour la médiane et pour le 90e centile des montants.

```

rng = np.random.default_rng(42)
B = 10_000
tirages = rng.choice(population, size=(B, len(population)), replace=True)
for nom, stat in [("médiane", np.median), ("90e centile", lambda a, axis: np.percentile(a, 90,
      axis=axis))]:
    valeurs = stat(tirages, axis=1)
    print(f"{nom:12s} observée = {stat(population, axis=0):6.1f}  IC95 % = {np.percentile(valeurs,
      [2.5, 97.5]).round(1)}")

```

```

médiane      observée = 51.0  IC95 % = [47.3 55.1]
90e centile  observée = 107.4 IC95 % = [ 98.1 119.1]

```

Étape 3 : Wald contre Wilson, couverture exacte. Pour une vraie proportion p et un échantillon de taille n , on peut calculer la couverture **sans simulation** : on additionne les probabilités binomiales des résultats k dont l'intervalle contient p .

```

def couverture(n, p, methode):
    k = np.arange(n + 1); ph = k / n; z = 1.96
    if methode == "Wald":
        demi = z * np.sqrt(ph * (1 - ph) / n); lo, hi = ph - demi, ph + demi
    else:
        centre = (k + z**2 / 2) / (n + z**2)
        demi = z / (n + z**2) * np.sqrt(k * (n - k) / n + z**2 / 4); lo, hi = centre - demi, centre +
    demi
    return float((stats.binom.pmf(k, n, p) * ((lo <= p) & (p <= hi))).sum())

for n, p in [(20, 0.05), (20, 0.30), (100, 0.05), (1000, 0.20)]:
    print(f"n = {n:>4}, p = {p:.2f} : Wald {couverture(n, p, 'Wald'):.3f}  Wilson {couverture(n, p,
      'Wilson'):.3f}")

```

```

n = 20, p = 0.05 : Wald 0.639  Wilson 0.925
n = 20, p = 0.30 : Wald 0.947  Wilson 0.975
n = 100, p = 0.05 : Wald 0.877  Wilson 0.966
n = 1000, p = 0.20 : Wald 0.947  Wilson 0.947

```

À vous. Pour quels (n, p) la méthode de Wald est-elle très en dessous de 95 % ? Est-ce cohérent avec le conseil du livre (3.3.4) ?

Application 3.4 — Comparer les trois canaux

Section 3.4 du livre. Il y a trois comparaisons possibles entre canaux. Pour chacune : test de Welch, intervalle de confiance de la différence et d de Cohen ; puis une correction de Holm pour tenir compte des trois tests (3.5.5).

```
from itertools import combinations
groupes = {c: g["montant"].to_numpy() for c, g in df.groupby("canal")}
lignes = []
for a, b in combinations(["Boutique", "Site", "Réseaux"], 2):
    x, y = groupes[a], groupes[b]
    t = stats.ttest_ind(x, y, equal_var=False)
    ic = t.confidence_interval()
    s_commun = np.sqrt(((len(x) - 1) * x.var(ddof=1) + (len(y) - 1) * y.var(ddof=1)) / (len(x) +
    ↪ len(y) - 2))
    lignes.append({"comparaison": f"{a} - {b}", "écart": x.mean() - y.mean(), "IC_bas": ic.low,
                  "IC_haut": ic.high, "p": t.pvalue, "d_Cohen": (x.mean() - y.mean()) / s_commun})
tab = pd.DataFrame(lignes)

# Holm : on trie les p, on multiplie la k-ième plus petite par (m - k + 1), puis on rend la suite
↪ croissante
ordre = tab["p"].argsort().to_numpy()
brut, ajuste, courant = tab["p"].to_numpy(), np.empty(3), 0.0
for rang, i in enumerate(ordre):
    courant = max(courant, min(1.0, (3 - rang) * brut[i]))
    ajuste[i] = courant
tab["p_Holm"] = ajuste
with pd.option_context("display.float_format", "{:.4g}".format, "display.width", 120):
    print(tab.to_string(index=False))
```

	comparaison	écart	IC_bas	IC_haut	p	d_Cohen	p_Holm
	Boutique - Site	15.31	5.571	25.04	0.002188	0.3889	0.004377
	Boutique - Réseaux	25.8	16.66	34.94	8.016e-08	0.7222	2.405e-07
	Site - Réseaux	10.49	2.393	18.59	0.0113	0.2996	0.0113

Un test d'indépendance. La satisfaction (note ≥ 4) dépend-elle du canal ?

```
df["satisfait"] = df["satisfaction"] >= 4
tableau = pd.crosstab(df["canal"], df["satisfait"])
chi2, p, ddl, attendus = stats.chi2_contingency(tableau)
v_cramer = np.sqrt(chi2 / (tableau.values.sum() * (min(tableau.shape) - 1)))
print(tableau, f"\nKhi-deux = {chi2:.2f}, ddl = {ddl}, p = {p:.1e}, V de Cramér = {v_cramer:.2f}",
    ↪ sep="")
```

satisfait	False	True
canal		
Boutique	5	109
Réseaux	51	87
Site	45	103

Khi-deux = 38.40, ddl = 2, p = 4.6e-09, V de Cramér = 0.31

À vous. Les trois écarts restent-ils significatifs après correction de Holm ? Lequel est le plus important en pratique (regardez d et l'intervalle) ? Pourquoi ne peut-on pas conclure que le canal *cause* la différence de satisfaction (rappel : la boutique n'a pas de délai de livraison) ?

Application 3.5 — Puissance, arrêt prématuré et tests multiples

Section 3.5 du livre.

Étape 1 : la p-valeur sous H_0 est uniforme.

```
rng = np.random.default_rng(1)
p0 = np.array([stats.ttest_ind(rng.normal(0, 1, 30), rng.normal(0, 1, 30)).pvalue for _ in
    ↪ range(5_000)])
print("part de p < 0,05 :", round((p0 < 0.05).mean(), 3), " part de p < 0,50 :", round((p0 <
    ↪ 0.50).mean(), 3))
print("histogramme (10 classes) :", np.histogram(p0, bins=10, range=(0, 1))[0])
```

```
part de p < 0,05 : 0.05    part de p < 0,50 : 0.489
histogramme (10 classes) : [493 483 487 506 476 493 535 480 538 509]
```

Étape 2 : la puissance du test A/B (12 % contre 15 %, 1 000 visiteurs par version), par la formule puis par simulation.

```
def puissance(p1, p2, n, alpha=0.05):
    se = np.sqrt(p1 * (1 - p1) / n + p2 * (1 - p2) / n)
    return stats.norm.cdf(abs(p2 - p1) / se - stats.norm.ppf(1 - alpha / 2))

rng = np.random.default_rng(2)
a, b = rng.binomial(1000, 0.12, 10_000), rng.binomial(1000, 0.15, 10_000)
pp = (a + b) / 2000
z = (b - a) / 1000 / np.sqrt(pp * (1 - pp) * 2 / 1000)
print("formule :", round(puissance(0.12, 0.15, 1000), 3), " simulation :", round((np.abs(z) >
    ↪ 1.96).mean(), 3))
n_requis = next(n for n in range(100, 20_000, 50) if puissance(0.12, 0.15, n) >= 0.80)
print("visiteurs par version pour 80 % de puissance :", n_requis)
```

```
formule : 0.502    simulation : 0.5
visiteurs par version pour 80 % de puissance : 2050
```

Étape 3 : l'arrêt prématuré (*peeking*). Aucune différence réelle (12 % des deux côtés). On regarde les résultats après chaque tranche de 100 visiteurs par version (20 regards au total) et l'on s'arrête dès que $p < 0,05$.

```
rng = np.random.default_rng(7)
essais, regards, pas = 4_000, 20, 100
a = rng.binomial(1, 0.12, (essais, regards * pas)).cumsum(axis=1)[: , pas - 1::pas]
b = rng.binomial(1, 0.12, (essais, regards * pas)).cumsum(axis=1)[: , pas - 1::pas]
n = pas * np.arange(1, regards + 1)
pp = (a + b) / (2 * n)
z = (b - a) / n / np.sqrt(np.maximum(pp * (1 - pp) * 2 / n, 1e-12))
print("faux positifs avec un seul regard final :", round((np.abs(z[:, -1]) > 1.96).mean(), 3))
print("faux positifs en s'arrêtant au premier p < 0,05 :", round((np.abs(z) >
    ↪ 1.96).any(axis=1).mean(), 3))
```

```
faux positifs avec un seul regard final : 0.054
faux positifs en s'arrêtant au premier p < 0,05 : 0.24
```

Étape 4 : tests multiples. Cent comparaisons dont dix vrais effets ($d = 1$), 40 observations par groupe.

```
rng = np.random.default_rng(5)
vrai = np.array([True] * 10 + [False] * 90)
p = np.array([stats.ttest_ind(rng.normal(float(v), 1, 40), rng.normal(0, 1, 40)).pvalue for v in
    ↪ vrai])
ordre = np.argsort(p); rangs = np.arange(1, 101)
holm = np.zeros(100, bool); holm[ordre[np.cumprod(p[ordre] < 0.05 / (101 - rangs)).astype(bool)]] =
    ↪ True
```

```
ok = p[ordre] <= rangs / 100 * 0.05
bh = np.zeros(100, bool)
if ok.any():
    bh[ordre[: np.max(np.where(ok)[0]) + 1]] = True
for nom, r in [("sans correction", p < 0.05), ("Bonferroni", p < 0.05 / 100), ("Holm", holm),
    ↪ ("Benjamini-Hochberg", bh)]:
    print(f"{nom:20s} découvertes = {r.sum():>2}    vraies = {(r & vrai).sum():>2}    fausses = {(r &
    ↪ ~vrai).sum():>2}")
```

sans correction	découvertes = 12	vraies = 9	fausses = 3
Bonferroni	découvertes = 7	vraies = 7	fausses = 0
Holm	découvertes = 7	vraies = 7	fausses = 0
Benjamini-Hochberg	découvertes = 9	vraies = 9	fausses = 0

À vous. Combien de regards faut-il pour que le taux de faux positifs dépasse 20 % ? Que se passe-t-il pour Benjamini-Hochberg si l'on porte à 30 le nombre de vrais effets ?

Application 3.6 — Stratifier et pondérer

Section 3.6 du livre.

Étape 1 : le gain de la stratification. Une base de 10 000 clients en trois canaux (4 000, 3 500 et 2 500) ; on estime la dépense moyenne avec 200 clients, au hasard ou par strates proportionnelles.

```
rng = np.random.default_rng(50)
tailles = {"Réseaux": 4000, "Site": 3500, "Boutique": 2500}
base = {"Réseaux": 3.7, "Site": 3.9, "Boutique": 4.1}
strates = {c: np.exp(rng.normal(base[c], 0.55, size=n)) for c, n in tailles.items()}
pop = np.concatenate(list(strates.values())); N = len(pop)
n_h = {c: round(200 * n / N) for c, n in tailles.items()} # 80, 70, 50
eas, strat = [], []
for _ in range(5_000):
    eas.append(rng.choice(pop, 200, replace=False).mean())
    strat.append(sum(tailles[c] / N * rng.choice(strates[c], n_h[c], replace=False).mean() for c in
    ↪ tailles))
print("vraie moyenne :", round(pop.mean(), 2))
print("EAS : erreur-type =", round(np.std(eas), 2), "\nstratifié : erreur-type =",
    ↪ round(np.std(strat), 2))
```

```
vraie moyenne : 56.4
EAS : erreur-type = 2.46
stratifié : erreur-type = 2.36
```

Étape 2 : corriger un échantillon déséquilibré. Parmi 300 réponses à un questionnaire, 150 viennent de Réseaux, 120 du site et 30 de la boutique, alors que la clientèle est répartie en 40 %, 35 % et 25 %. Les taux de satisfaits par canal sont 63 %, 70 % et 96 %.

```
taux = pd.Series({"Réseaux": 0.63, "Site": 0.70, "Boutique": 0.96})
reponses = pd.Series({"Réseaux": 150, "Site": 120, "Boutique": 30})
population = pd.Series({"Réseaux": 0.40, "Site": 0.35, "Boutique": 0.25})
poids = population / (reponses / reponses.sum())
print("poids :", poids.round(2).to_dict())
print("moyenne naïve :", round((reponses * taux).sum() / reponses.sum(), 3),
    ↪ " pondérée :", round((reponses * poids * taux).sum() / (reponses * poids).sum(), 3))
```

```
poids : {'Réseaux': 0.8, 'Site': 0.87, 'Boutique': 2.5}
moyenne naïve : 0.691 pondérée : 0.737
```

À vous. Que devient l'erreur-type de l'estimateur stratifié si l'on interroge **autant** de clients dans chaque strate (allocation égale) ? Pourquoi la pondération ne peut-elle pas corriger un biais de non-réponse à l'intérieur d'un canal ?

Application 3.7 — Tests sans hypothèse de loi : rangs et permutations

Section 3.7 du livre.

Étape 1 : Welch ou Mann-Whitney ? Deux petits groupes dont l'un contient une valeur extrême.

```
ga = np.array([12, 15, 14, 10, 13, 40]); gb = np.array([9, 8, 11, 10, 7, 12])
print("Welch :", round(stats.ttest_ind(ga, gb, equal_var=False).pvalue, 3),
      " Mann-Whitney :", round(stats.mannwhitneyu(ga, gb).pvalue, 3))
```

```
Welch : 0.15    Mann-Whitney : 0.02
```

Étape 2 : un test de permutation sur la médiane, statistique pour laquelle il n'y a pas de formule simple (boutique contre Réseaux).

```
rng = np.random.default_rng(12)
b = df.loc[df["canal"] == "Boutique", "montant"].to_numpy()
r = df.loc[df["canal"] == "Réseaux", "montant"].to_numpy()
valeurs, n_b = np.concatenate([b, r]), len(b)
obs = np.median(b) - np.median(r)
difs = np.empty(20_000)
for k in range(20_000):
    melange = rng.permutation(valeurs)
    difs[k] = np.median(melange[:n_b]) - np.median(melange[n_b:])
print("différence de médianes observée :", round(obs, 2), "€")
print("p-valeur de permutation :", (np.sum(np.abs(difs) >= abs(obs)) + 1) / (len(difs) + 1))
```

```
différence de médianes observée : 23.35 €
p-valeur de permutation : 4.999750012499375e-05
```

Étape 3 : la normalité, canal par canal.

```
for c, g in df.groupby("canal"):
    print(f"{c:9s} Shapiro (montant) p = {stats.shapiro(g['montant']).pvalue:.1e}    Shapiro (log) p = {stats.shapiro(np.log(g['montant'])).pvalue:.3f}")
```

```
Boutique  Shapiro (montant) p = 4.0e-07    Shapiro (log) p = 0.301
Réseaux   Shapiro (montant) p = 5.9e-09    Shapiro (log) p = 0.710
Site      Shapiro (montant) p = 2.0e-13    Shapiro (log) p = 0.243
```

À vous. La conclusion du test de permutation sur la médiane diffère-t-elle de celle du test de Welch sur la moyenne ? Pourquoi la p-valeur ne peut-elle pas valoir exactement zéro ?

Exercices

Exercice 3.1 ★ — Résumer huit commandes (section 3.1 du livre)

Huit commandes en € : 12, 15, 15, 18, 20, 22, 25, 60. Calculez la moyenne, la médiane, le mode, l'écart-type (avec $n - 1$) et l'écart interquartile. Quelle mesure de position est la plus représentative, et pourquoi ?

Exercice 3.2 ★ — La variance sans biais (section 3.2 du livre)

Un échantillon de 5 délais de livraison : 4, 8, 6, 5, 7 jours. Calculez la variance en divisant par n puis par $n - 1$. Laquelle utiliser pour estimer la variance de **tous** les délais ?

Exercice 3.3 ★★ — Maximum de vraisemblance d'un taux d'arrivée (section 3.2 du livre)

Le nombre de commandes par heure a été relevé 8 fois : 2, 3, 1, 4, 0, 3, 2, 5. On modélise par une loi de Poisson(λ). (a) Écrivez la log-vraisemblance et trouvez $\hat{\lambda}$ en dérivant. (b) Avec cette estimation, quelle est la probabilité de ne recevoir aucune commande pendant une heure ?

Exercice 3.4 ★ — Intervalle de confiance d'une moyenne (section 3.3 du livre)

Sur $n = 36$ commandes : $\bar{x} = 52$ €, $s = 12$ €. Donnez un intervalle de confiance à 95 % de la moyenne, puis à 99 %. Quelle est la signification du « 95 % » ?

Exercice 3.5 ★★ — Intervalle de confiance d'une proportion (section 3.3 du livre)

Un questionnaire envoyé à 60 clients donne 18 « très satisfaits ». Calculez l'IC à 95 % de la vraie proportion par la méthode de Wald et par celle de Wilson. Pourquoi sont-ils différents ?

Exercice 3.6 ★★ — Le transporteur tient-il sa promesse ? (section 3.4 du livre)

Le transporteur promet un délai moyen de 3 jours. Sur 25 colis, on mesure en moyenne 3,6 jours avec un écart-type de 1,5 jour. (a) Testez $H_0 : \mu = 3$ contre $H_1 : \mu \neq 3$ à 5 %. (b) Que change un test unilatéral $H_1 : \mu > 3$? (c) Peut-on dire que le transporteur respecte sa promesse ?

Exercice 3.7 ★★ — Un test A/B (section 3.4 et 3.5 du livre)

Version A : 45 achats sur 500 visiteurs. Version B : 66 achats sur 500 visiteurs. (a) B est-elle meilleure, à 5 % ? (b) Donnez un IC de la différence. (c) Quelle était la puissance de ce test si le vrai écart est celui observé ?

Exercice 3.8 ★★ — Un code promo a-t-il un effet ? (section 3.4 du livre)

Parmi 100 clients ayant reçu un code promo, 30 ont acheté ; parmi 100 clients sans code, 45 ont acheté. Le code a-t-il un effet ? Construisez le tableau, calculez les effectifs attendus et le test du khi-deux. Comparez avec le test de deux proportions.

Exercice 3.9 ★★★ — Dimensionner une expérience (section 3.5 du livre)

Le taux de conversion actuel est de 20 %. La gérante veut détecter une hausse à 24 % avec une puissance de 80 % et $\alpha = 5\%$. (a) Combien de visiteurs par version ? (b) Et pour détecter 22 % ? (c) Expliquez le rapport entre les deux résultats.

Exercice 3.10 ★★★ — Tests multiples (section 3.5 du livre)

Un analyste teste 8 hypothèses et obtient les p-valeurs 0,001; 0,008; 0,012; 0,030; 0,040; 0,200; 0,500; 0,700. Lesquelles sont rejetées à 5 % (a) sans correction, (b) avec Bonferroni, (c) avec Holm, (d) avec Benjamini-Hochberg ?

Exercice 3.11 ★ — Marge d'erreur d'un sondage (section 3.6 du livre)

Une enquête de satisfaction recueille 625 réponses tirées au hasard dans la clientèle. (a) Quelle est la marge d'erreur maximale à 95 % sur une proportion ? (b) Combien de réponses faut-il pour une marge de ± 2 points ? (c) Que ne dit pas cette marge ?

Exercice 3.12 ★★ — Mann-Whitney à la main (section 3.7 du livre)

Trois commandes du site : 12, 15, 14 € ; trois commandes de la boutique : 9, 8, 11 €. (a) Rangez les six valeurs et calculez la somme des rangs du premier groupe. (b) Quelle est la probabilité qu'une commande tirée au hasard dans le premier groupe dépasse une commande tirée dans le second ? (c) Parmi les $\binom{6}{3} = 20$ répartitions possibles des six valeurs en deux groupes de trois, combien donnent un résultat au moins aussi extrême ? Quelle est la plus petite p-valeur bilatérale accessible avec des groupes de 3 ?

Corrigés

Corrigé 3.1

Moyenne : $187/8 = 23,375$. Triées : 12, 15, 15, 18, 20, 22, 25, 60 : la médiane est $(18 + 20)/2 = 19$. Mode : 15. Écart-type ($n - 1$) : voir le code ($\approx 15,4$). Les quartiles (méthode de NumPy) donnent un IQR de 7,75. La **médiane** (19) est la plus représentative : la moyenne (23,4) est tirée vers le haut par la commande de 60 € (valeur extrême), qui est supérieure de plus du double au reste.

```
import numpy as np
from scipy import stats
x = np.array([12, 15, 15, 18, 20, 22, 25, 60])
print("moyenne :", x.mean(), " médiane :", np.median(x), " mode :", stats.mode(x).mode)
print("écart-type (n-1) :", round(x.std(ddof=1), 2))
print("IQR :", np.percentile(x, 75) - np.percentile(x, 25))
```

```
moyenne : 23.375  médiane : 19.0  mode : 15
écart-type (n-1) : 15.38
IQR : 7.75
```

Corrigé 3.2

$\bar{x} = 6$; écarts : $-2, 2, 0, -1, 1$; carrés : $4, 4, 0, 1, 1$; somme = 10. Division par n : $10/5 = 2$. Division par $n - 1$: $10/4 = 2,5$. Pour estimer la variance de la **population**, on utilise $n - 1$ (estimateur sans biais, 3.2.3).

```
d = np.array([4, 8, 6, 5, 7])
print("var (n) :", d.var(ddof=0), " var (n-1) :", d.var(ddof=1))
```

```
var (n) : 2.0  var (n-1) : 2.5
```

Corrigé 3.3

(a) $\ell(\lambda) = \sum_i (-\lambda + x_i \ln \lambda - \ln x_i!) = -n\lambda + \ln \lambda \sum x_i - \sum \ln x_i!$. $\ell'(\lambda) = -n + \frac{\sum x_i}{\lambda} = 0 \Rightarrow \hat{\lambda} = \bar{x} = \frac{20}{8} = 2,5$ (c'est un maximum car $\ell'' = -\sum x_i/\lambda^2 < 0$). (b) $P(N = 0) = e^{-2,5} \approx 0,082$.

```
x = np.array([2, 3, 1, 4, 0, 3, 2, 5])
lam = x.mean()
print("lambda chapeau =", lam, " P(0 commande) =", round(np.exp(-lam), 4))
grille = np.linspace(0.5, 6, 1101)
print("maximum sur grille :", round(grille[np.argmax([stats.poisson.logpmf(x, g).sum() for g in
↵ grille])], 2))
```

```
lambda chapeau = 2.5   P(0 commande) = 0.0821
maximum sur grille : 2.5
```

Corrigé 3.4

Erreur-type : $12/\sqrt{36} = 2$. Valeur critique de Student à 35 ddl : 2,030 (95 %) et 2,724 (99 %). IC 95 % : $52 \pm 2,030 \times 2 = [47,9; 56,1]$. IC 99 % : $52 \pm 2,724 \times 2 = [46,6; 57,4]$ (plus large : plus de confiance coûte en précision). « 95 % » : si l'on répétait l'enquête de nombreuses fois, environ 95 % des intervalles ainsi construits contiendraient la vraie moyenne.

```
n, xbar, s = 36, 52, 12
se = s / np.sqrt(n)
for niveau in (0.95, 0.99):
    t = stats.t.ppf(1 - (1 - niveau) / 2, n - 1)
    print(f"{niveau:.0%} : t = {t:.3f}   IC = [{xbar - t * se:.1f} ; {xbar + t * se:.1f}]")
```

```
95% : t = 2.030   IC = [47.9 ; 56.1]
99% : t = 2.724   IC = [46.6 ; 57.4]
```

Corrigé 3.5

$\hat{p} = 18/60 = 0,30$. Wald : erreur-type $\sqrt{0,3 \times 0,7/60} = 0,0592$, IC $0,30 \pm 0,116 = [0,184; 0,416]$. Wilson (voir le code) donne environ $[0,199; 0,425]$: légèrement décalé vers le centre et plus large à droite. Wilson est plus fiable car il tient compte du fait que l'erreur-type dépend de p lui-même, d'où une meilleure couverture pour n modeste.

```
k, n = 18, 60
p = k / n
d = 1.96 * np.sqrt(p * (1 - p) / n)
print("Wald : [", round(p - d, 3), ";", round(p + d, 3), "]")
w = stats.binomtest(k, n).proportion_ci(confidence_level=0.95, method="wilson")
print("Wilson : [", round(w.low, 3), ";", round(w.high, 3), "]")
```

```
Wald : [ 0.184 ; 0.416 ]
Wilson : [ 0.199 ; 0.425 ]
```

Corrigé 3.6

(a) $t = \frac{3,6 - 3}{1,5/\sqrt{25}} = \frac{0,6}{0,3} = 2,0$ avec 24 ddl. Valeur critique bilatérale à 5 % : 2,064. Comme $2,0 < 2,064$ (et $p \approx 0,057$), on **ne rejette pas** H_0 de justesse. (b) En unilatéral, $p \approx 0,028 < 0,05$: on rejette. Mais on ne peut choisir le sens du test **qu'avant** de voir les données et si l'on n'est vraiment intéressé que par un dépassement ; décider après coup serait tricher. (c)

Honnêtement : les données sont **à la limite** : le retard moyen estimé est de 0,6 jour, l'IC à 95 % ($3,6 \pm 2,064 \times 0,3 = [2,98; 4,22]$) inclut 3 de justesse. On ne peut ni affirmer que la promesse est tenue, ni qu'elle ne l'est pas ; il faut davantage de colis.

```
n, xbar, s, mu0 = 25, 3.6, 1.5, 3
t = (xbar - mu0) / (s / np.sqrt(n))
print("t =", t, " p bilatérale =", round(2 * stats.t.sf(t, n - 1), 4), " p unilatérale =",
      ↪ round(stats.t.sf(t, n - 1), 4))
tc = stats.t.ppf(0.975, n - 1)
print("IC95 % : [", round(xbar - tc * s / np.sqrt(n), 2), ";", round(xbar + tc * s / np.sqrt(n), 2),
      ↪ "]")
```

```
t = 2.0000000000000004    p bilatérale = 0.0569    p unilatérale = 0.0285
IC95 % : [ 2.98 ; 4.22 ]
```

Corrigé 3.7

- (a) $\hat{p}_A = 0,09$, $\hat{p}_B = 0,132$, $\hat{p} = \frac{111}{1000} = 0,111$. $z = \frac{0,042}{\sqrt{0,111 \times 0,889 \times (2/500)}} = \frac{0,042}{0,01987} \approx 2,11$, $p \approx 0,035$: significatif à 5 %. (b) Erreur-type non regroupée $\approx 0,0194$, donc IC = $0,042 \pm 0,039 = [0,003; 0,081]$: l'écart est positif, mais très imprécis (de 0,3 à 8 points !). (c) La puissance, si le vrai écart est celui observé, est d'environ 56 % : l'expérience était sous-dimensionnée (voir le code).

```
kA, kB, n = 45, 66, 500
pA, pB = kA / n, kB / n
pp = (kA + kB) / (2 * n)
z = (pB - pA) / np.sqrt(pp * (1 - pp) * 2 / n)
print("z =", round(z, 3), " p =", round(2 * stats.norm.sf(z), 4))
se = np.sqrt(pA * (1 - pA) / n + pB * (1 - pB) / n)
print("IC95 % de la différence : [", round(pB - pA - 1.96 * se, 4), ";", round(pB - pA + 1.96 * se,
      ↪ 4), "]")
print("puissance :", round(stats.norm.cdf((pB - pA) / se - 1.96), 3))
```

```
z = 2.114    p = 0.0345
IC95 % de la différence : [ 0.0031 ; 0.0809 ]
puissance : 0.563
```

Corrigé 3.8

Tableau : code promo : 30 achats, 70 non ; sans code : 45 achats, 55 non. Totaux : colonnes 75 et 125 ; lignes 100 et 100. Effectifs attendus (sous indépendance) : achats $100 \times 75/200 = 37,5$ dans chaque ligne ; non-achats 62,5. Chaque case s'écarte de son attendu de 7,5, donc $\chi^2 = \sum \frac{(O-E)^2}{E} = 2 \times \frac{7,5^2}{37,5} + 2 \times \frac{7,5^2}{62,5} = 3,0 + 1,8 = 4,8$ (sans correction de continuité), 1 ddl, $p \approx 0,029$ (0,041 avec la correction de Yates, plus prudente). **Attention : le code promo est associé à moins d'achats** (30 % contre 45 %) : le test signale une différence, pas son sens. (Et le test à deux proportions donne $z^2 = \chi^2$: même p .)

```
from scipy.stats import chi2_contingency
tab = np.array([[30, 70], [45, 55]])
chi2, p, ddl, att = chi2_contingency(tab, correction=False)
print("attendus :\n", att)
print("khi-deux =", round(chi2, 3), " p =", round(p, 4), " (avec correction de Yates :",
      ↪ round(chi2_contingency(tab)[1], 4), ")")
pp = 75 / 200
z = (0.45 - 0.30) / np.sqrt(pp * (1 - pp) * 2 / 100)
print("z =", round(z, 3), " z^2 =", round(z**2, 3), " p =", round(2 * stats.norm.sf(abs(z)), 4))
```

```

attendus :
[[37.5 62.5]
 [37.5 62.5]]
khi-deux = 4.8  p = 0.0285 (avec correction de Yates : 0.0409 )
z = 2.191  z^2 = 4.8  p = 0.0285

```

Corrigé 3.9

(a) $n = \frac{(1,96 + 0,8416)^2 [0,2 \times 0,8 + 0,24 \times 0,76]}{0,04^2} \approx 1\,680$ par version. (b) Pour 22 %, l'effet est deux fois plus petit : $n \approx 6\,500$. (c) Diviser l'effet par 2 multiplie n par **environ 4** : $n \propto 1/\text{effet}^2$.

```

za, zb = stats.norm.ppf(0.975), stats.norm.ppf(0.80)
def n_groupe(p1, p2):
    return (za + zb) ** 2 * (p1 * (1 - p1) + p2 * (1 - p2)) / (p2 - p1) ** 2
print("20 -> 24 % :", int(np.ceil(n_groupe(0.20, 0.24))), " 20 -> 22 % :", int(np.ceil(n_groupe(0.20,
↪ 0.22))))
print("rapport :", round(n_groupe(0.20, 0.22) / n_groupe(0.20, 0.24), 2))

```

```

20 -> 24 % : 1680  20 -> 22 % : 6507
rapport : 3.87

```

Corrigé 3.10

(a) Sans correction : $p < 0,05$ pour les 5 premières. (b) Bonferroni : seuil $0,05/8 = 0,00625$: seule 0,001 est rejetée. (c) Holm : seuils $0,05/8 = 0,00625$, $0,05/7 = 0,00714$, $0,05/6 = 0,00833$, ... : $0,001 < 0,00625$ ✓ ; $0,008 > 0,00714$ ✗ : on s'arrête. Un seul rejet, comme Bonferroni. (d) BH : seuils $\frac{k}{8} \times 0,05$: $0,00625$; $0,0125$; $0,01875$; $0,025$; $0,03125$; ... ; $p_{(1)} = 0,001 \leq 0,00625$ ✓, $p_{(2)} = 0,008 \leq 0,0125$ ✓, $p_{(3)} = 0,012 \leq 0,01875$ ✓, $p_{(4)} = 0,030 > 0,025$ ✗, $p_{(5)} = 0,040 > 0,03125$ ✗. Le plus grand k qui convient est 3 : les **trois** premières sont rejetées.

```

p = np.array([0.001, 0.008, 0.012, 0.030, 0.040, 0.200, 0.500, 0.700])
m = len(p)
print("sans correction :", int((p < 0.05).sum()), "rejets")
print("Bonferroni      :", int((p < 0.05 / m).sum()), "rejets")
rang = np.arange(1, m + 1)
holm = np.cumprod(p < 0.05 / (m - rang + 1)) # on s'arrête au premier échec
print("Holm          :", int(holm.sum()), "rejets")
ok = p <= rang / m * 0.05
print("Benjamini-Hochberg :", int(np.max(np.where(ok)[0]) + 1) if ok.any() else 0, "rejets")

```

```

sans correction : 5 rejets
Bonferroni      : 1 rejets
Holm            : 1 rejets
Benjamini-Hochberg : 3 rejets

```

Corrigé 3.11

(a) $1,96\sqrt{0,25/625} = 1,96 \times 0,02 = 0,0392$: environ **±3,9 points**. (b) $n \geq (1,96/0,02)^2 \times 0,25 \approx 2\,401$ réponses. (c) Cette marge ne couvre que l'**erreur d'échantillonnage** ; elle ne dit rien du biais de sélection ni de la non-réponse (3.6.1), souvent plus grands.

```

print("marge pour n = 625 :", round(1.96 * np.sqrt(0.25 / 625) * 100, 2), "points")
print("n pour ±2 points  :", int(np.ceil((1.96 / 0.02) ** 2 * 0.25)))

```

marge pour $n = 625$: 3.92 points
 n pour ± 2 points : 2401

Corrigé 3.12

- (a) Valeurs triées : 8, 9, 11, 12, 14, 15 ; rangs 1 à 6. Le premier groupe (12, 15, 14) occupe les rangs 4, 6, 5 : somme = 15. (b) Chaque commande du premier groupe dépasse chaque commande du second : $U = 9$ sur $3 \times 3 = 9$ paires, donc probabilité = 1. (c) Une seule répartition donne le premier groupe tout en haut, et une autre le met tout en bas : 2 répartitions sur 20 sont aussi extrêmes (dans les deux sens), d'où une p-valeur exacte bilatérale de $2/20 = 0,10$. Avec des groupes de 3, la plus petite p-valeur possible est donc 0,10 : **aucun test ne peut rejeter à 5 %**, même avec une séparation parfaite. Les petits échantillons manquent structurellement de puissance.

```
A, B = np.array([12, 15, 14]), np.array([9, 8, 11])
print("somme des rangs du groupe A :", stats.rankdata(np.concatenate([A, B]))[:3].sum())
print("p exacte de Mann-Whitney :", stats.mannwhitneyu(A, B, method="exact").pvalue)
```

```
somme des rangs du groupe A : 15.0
p exacte de Mann-Whitney : 0.1
```

Chapitre 4 : Programmation — exercices et applications

Ce chapitre du cahier accompagne le chapitre 4 du livre (Python, R, algorithmes, NumPy et pandas, graphiques, objets et tests, complexité). Il rassemble **huit applications guidées** (de petits programmes réels, construits pas à pas) et **quatorze exercices corrigés**. Les données sont celles du livre : `donnees/commandes.csv` (400 commandes). Cherchez d'abord à la main, vérifiez ensuite par le code, et seulement après lisez le corrigé.

Préparation

Presque toutes les applications et tous les exercices utilisent le tableau des commandes, enrichi de deux colonnes **simulées** (une date et un numéro de client), exactement comme en 4.4.5 du livre. Voici la préparation, à exécuter une fois :

```
import numpy as np
import pandas as pd

df = pd.read_csv("donnees/commandes.csv")
rng = np.random.default_rng(7)
jours = rng.integers(0, 140, size=len(df)) # un jour tiré parmi 140 (= 20 semaines)
df["date"] = pd.Timestamp("2026-01-05") + pd.to_timedelta(jours, unit="D")
df["id_client"] = rng.integers(1, 121, size=len(df)) # clients numérotés de 1 à 120
df = df.sort_values("date").reset_index(drop=True)
df.insert(0, "id_commande", np.arange(1, len(df) + 1))
df["semaine"] = df["date"].dt.to_period("W").dt.start_time # le lundi de la semaine de chaque
↳ commande
print(df.shape, "| CA total :", round(df["montant"].sum(), 1), "€")
```

```
(400, 8) | CA total : 24098.3 €
```

Applications

Application 4.1 — Le ticket de caisse complet (section 4.1.10)

Objectif. Écrire un programme qui édite un ticket de caisse. Règles : prix du catalogue hors taxe ; **remise fidélité de 10 %** si le sous-total HT dépasse 100 € ; **TVA de 19 %** sur le montant après remise ; le ticket affiche chaque ligne, le sous-total, la remise, la TVA et le total TTC.

Le calcul à la main d'abord, pour le panier « 2 bols, 1 plateau, 3 bougies » (prix HT : bol 12,5 ; plateau 45 ; bougie 15,9) : sous-total 25 + 45 + 47,7 = 117,70 € ; remise 11,77 € ; net 105,93 € ; TVA 20,13 € ; total TTC 126,06 €. Ce sont les valeurs que le programme devra retrouver.

Étape 1 — les règles de calcul, sous forme de petites fonctions :

```
CATALOGUE = {"bol": 12.5, "tasse": 8.0, "plateau": 45.0, "bougie": 15.9}
TVA, SEUIL_REMISE, TAUX_REMISE = 0.19, 100, 0.10

def sous_total(panier):
    """panier : liste de couples (produit, quantité)."""
    return sum(CATALOGUE[nom] * qte for nom, qte in panier)

def remise(montant_ht):
    return montant_ht * TAUX_REMISE if montant_ht > SEUIL_REMISE else 0.0
```

Étape 2 — le ticket. La fonction `ticket` calcule les montants avec les deux précédentes, puis met le tout en forme avec des f-strings (:<8 aligne à gauche, :>8.2f aligne à droite avec 2 décimales). Elle renvoie le texte **et** le total, ce qui permet de la tester :

```
def ticket(panier):
    ht = sous_total(panier)
    r = remise(ht)
    net = ht - r
    tva = net * TVA
    lignes = ["=== BOUTIQUE ==="]
    for nom, qte in panier:
        lignes.append(f"{qte} x {nom:<8} {CATALOGUE[nom]:>6.2f} {qte * CATALOGUE[nom]:>8.2f}")
    lignes.append(f"{'Sous-total HT':<20}{ht:>8.2f}")
    lignes.append(f"{'Remise fidélité':<20}{-r:>8.2f}")
    lignes.append(f"{'TVA 19 %':<20}{tva:>8.2f}")
    lignes.append(f"{'TOTAL TTC':<20}{net + tva:>8.2f}")
    return "\n".join(lignes), round(net + tva, 2)
```

```
texte, total = ticket([("bol", 2), ("plateau", 1), ("bougie", 3)])
print(texte)
```

```
=== BOUTIQUE ===
2 x bol      12.50    25.00
1 x plateau  45.00    45.00
3 x bougie   15.90    47.70
Sous-total HT      117.70
Remise fidélité    -11.77
TVA 19 %           20.13
TOTAL TTC          126.06
```

Étape 3 — les tests aux limites. Un `assert` ne dit rien quand la condition est vraie et arrête le programme sinon :

```
assert remise(100) == 0.0 # pile au seuil : pas de remise (condition « > »)
assert ticket([("tasse", 2)])[1] == 19.04 # 2 tasses : 16,00 HT, TVA 3,04
assert ticket([])[1] == 0.0 # panier vide
assert ticket([("bol", 2), ("plateau", 1), ("bougie", 3)])[1] == 126.06
print("tous les tests passent")
```

tous les tests passent

Lecture. Le ticket retombe sur les valeurs de la main : 117,70 ; 11,77 ; 20,13 ; 126,06. Les trois questions qui prolongent l'application (produit absent du catalogue, message aimable, code promo) sont les exercices 4.2 et 4.3.

Application 4.2 — Le ticket de caisse en R (section 4.2)

Objectif. Refaire le calcul du sous-total et du ticket de l'application 4.1 en R, avec un vecteur nommé qui joue le rôle du dictionnaire, et vérifier que les deux langages donnent les mêmes nombres.

```
catalogue <- c(bol = 12.5, tasse = 8.0, plateau = 45.0, bougie = 15.9)
panier    <- c(bol = 2, plateau = 1, bougie = 3)

sous_total <- sum(catalogue[names(panier)] * panier)      # les prix des produits du panier, fois les
  ↪ quantités
remise <- if (sous_total > 100) 0.10 * sous_total else 0
net <- sous_total - remise
c(HT = sous_total, remise = remise, TVA = 0.19 * net, TTC = 1.19 * net)
```

HT	remise	TVA	TTC
117.7000	11.7700	20.1267	126.0567

Lecture. On retrouve 117,70 € de sous-total, 11,77 € de remise, 20,13 € de TVA et 126,06 € de total TTC (aux arrondis près) : le même ticket qu'en Python, en quatre lignes grâce à la vectorisation.

Application 4.3 — Une file d'attente à l'atelier d'emballage (section 4.3.3)

Objectif. Simuler l'attente des commandes devant **une seule** emballeuse. Cinq commandes arrivent aux minutes 0, 1, 2, 10 et 11 ; emballer une commande dure 4 minutes ; les commandes sont traitées dans l'ordre d'arrivée.

À la main :

Commande	Arrivée	Début d'emballage	Attente
1	0	0	0
2	1	4 (la 1 se termine à 4)	3
3	2	8	6
4	10	12 (la 3 se termine à 12)	2
5	11	16	5

Attente moyenne : $(0 + 3 + 6 + 2 + 5)/5 = 3,2$ minutes. Le programme utilise une **file** (deque) et retient l'instant où l'emballuse sera libre :

```
from collections import deque

def simuler_file(arrivees, duree):
    libre_a, attentes = 0, []
    file = deque(arrivees)      # les commandes en attente, dans l'ordre
    while file:
        arrivee = file.popleft() # premier arrivé, premier servi
        debut = max(arrivee, libre_a) # on commence quand la commande est là ET l'emballuse libre
        attentes.append(debut - arrivee)
        libre_a = debut + duree
    return attentes
```

```
attentes = simuler_file([0, 1, 2, 10, 11], duree=4)
print("attentes :", attentes, "| moyenne :", sum(attentes) / len(attentes))
```

```
attentes : [0, 3, 6, 2, 5] | moyenne : 3.2
```

Lecture. Le programme retrouve les attentes [0, 3, 6, 2, 5] et la moyenne de 3,2 minutes. Cette mini-simulation est le premier pas vers la **théorie des files d'attente** (processus de Poisson et loi exponentielle du chapitre 2). L'exercice 4.6 la prolonge avec deux emballeuses.

Application 4.4 — Le seuil de livraison gratuite (section 4.3.5)

Objectif. La gérante veut offrir la livraison à partir d'un seuil tel qu'environ 30 % des commandes y aient droit. Une liste triée et la recherche dichotomique répondent à « quelle part des commandes dépasse s € ? ». Le module `bisect` contient la recherche dichotomique toute faite : `bisect_left(liste_triee, s)` donne le nombre de montants **strictement inférieurs** à s .

```
import bisect

triee = sorted(df["montant"])

def part_au-dessus(seuil):
    return 1 - bisect.bisect_left(triee, seuil) / len(triee)

for seuil in (50, 60, 70, 80, 90, 100):
    print(f"seuil {seuil:3d} € : {part_au-dessus(seuil):6.1%} des commandes y ont droit")
print("quantile 70 % (numpy) :", round(float(np.quantile(df["montant"], 0.70)), 1), "€")
```

```
seuil 50 € : 51.2% des commandes y ont droit
seuil 60 € : 41.0% des commandes y ont droit
seuil 70 € : 29.2% des commandes y ont droit
seuil 80 € : 21.8% des commandes y ont droit
seuil 90 € : 17.2% des commandes y ont droit
seuil 100 € : 13.0% des commandes y ont droit
quantile 70 % (numpy) : 68.9 €
```

Lecture. Un seuil de 70 € concerne 29,2 % des commandes, soit à peu près les 30 % visés ; le quantile à 70 % calculé par NumPy (68,9 €) pointe au même endroit, puisque par définition 30 % des commandes lui sont supérieures. Nous avons retrouvé par un algorithme de recherche ce que les quantiles du 3.1.3 donnaient directement : un bon moyen de comprendre ce que « quantile » veut dire, *la position dans la liste triée*.

Application 4.5 — Le rapport hebdomadaire de la gérante (section 4.4.13)

Objectif. Produire, pour une semaine donnée, le rapport que la gérante lit chaque lundi. D'abord le **tableau de bord hebdomadaire** : une ligne par semaine, avec une moyenne mobile sur quatre semaines pour lisser les à-coups.

```
hebdo = df.groupby("semaine").agg(
    commandes=("montant", "count"),
    ca=("montant", "sum"),
    panier_moyen=("montant", "mean"),
    part_reseaux=("canal", lambda s: (s == "Réseaux").mean()), # moyenne d'un masque = proportion
).round(2)
hebdo["ca_lisse"] = hebdo["ca"].rolling(4).mean().round(1)
print(hebdo.head(4))
print(hebdo.tail(2))
```

	commandes	ca	panier_moyen	part_reseaux	ca_lisse
semaine					
2026-01-05	20	1082.4	54.12	0.25	NaN
2026-01-12	19	1083.0	57.00	0.53	NaN
2026-01-19	18	1403.0	77.94	0.22	NaN
2026-01-26	20	750.7	37.54	0.40	1079.8
	commandes	ca	panier_moyen	part_reseaux	ca_lisse
semaine					
2026-05-11	23	1306.3	56.80	0.39	1225.5
2026-05-18	27	1585.4	58.72	0.37	1433.7

Puis une première fonction qui **calcule** les chiffres d'une semaine, et une seconde qui les **met en forme** (séparer le calcul de la présentation rend chacune plus facile à tester) :

```
def chiffres_semaine(df, debut):
    debut = pd.Timestamp(debut)
    sem = df[df["semaine"] == debut]
    prec = df[df["semaine"] == debut - pd.Timedelta(weeks=1)]
    return {"debut": debut, "n": len(sem),
            "ca": sem["montant"].sum(), "ca_prec": prec["montant"].sum(),
            "panier": sem["montant"].mean(), "note": sem["satisfaction"].mean(),
            "par_canal": sem.groupby("canal")["montant"].sum().sort_values(ascending=False),
            "top": sem.groupby("id_client")["montant"].sum().nlargest(3)}
```

Puis la seconde, qui met en forme (et le contrôle croisé avec le tableau hebdomadaire) :

```
def rapport_hebdo(df, debut):
    c = chiffres_semaine(df, debut)
    if c["n"] == 0:
        return f"Aucune commande la semaine du {c['debut']:%d/%m/%Y}."
    ligne_ca = f" chiffre d'affaires : {c['ca']:.2f} €"
    if c["ca_prec"] > 0:
        ligne_ca += f" ({(c['ca'] / c['ca_prec'] - 1) * 100:+.1f} % vs semaine précédente)"
    return "\n".join([f"Semaine du {c['debut']:%d/%m/%Y}",
                      f" commandes : {c['n']} | panier moyen : {c['panier']:.2f} €", ligne_ca,
                      " par canal : " + ", ".join(f"{k} {v:.0f} €" for k, v in c["par_canal"].items()),
                      " meilleurs clients : " + ", ".join(f"n°{i} ({v:.0f} €)" for i, v in
                      ↪ c["top"].items()),
                      f" satisfaction moyenne : {c['note']:.2f}/5" + (" (!) à surveiller" if
                      ↪ c["note"] < 3.5 else "")])

print(rapport_hebdo(df, "2026-03-02"))
print(rapport_hebdo(df, "2025-01-06")) # une semaine sans données
print(hebdo.loc["2026-03-02", ["commandes", "ca"]].tolist()) # contrôle croisé avec le tableau
↪ hebdomadaire
```

```
Semaine du 02/03/2026
commandes : 23 | panier moyen : 72.96 €
chiffre d'affaires : 1678.00 € (+59.4 % vs semaine précédente)
par canal : Site 707 €, Boutique 502 €, Réseaux 469 €
meilleurs clients : n°42 (189 €), n°7 (170 €), n°83 (160 €)
satisfaction moyenne : 3.78/5
Aucune commande la semaine du 06/01/2025.
[23.0, 1678.0]
```

Lecture. Le contrôle croisé final confirme que le rapport et le tableau hebdomadaire disent la même chose (23 commandes et 1 678 € pour la semaine du 2 mars). Ce petit programme utilise presque tout le chapitre : filtrage, groupby, tri, dates, formatage. Il est surtout **réutilisable** : la semaine suivante, on change la date et on obtient le nouveau rapport sans rien refaire.

Application 4.6 — Le tableau de bord de la gérante (section 4.5.7)

Objectif. Une **fonction** qui produit en une fois la figure que la gérante joint à son rapport : courbe lissée du chiffre d'affaires, part de chaque canal, répartition des notes. Chaque panneau a un **titre qui énonce sa conclusion, calculée à partir des données** : si les chiffres changent, le titre reste vrai.

Préparation graphique : une palette (couleurs de la même famille partout dans le livre) et un style discret.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

BLEU, ORANGE, AQUA = "#2a78d6", "#eb6834", "#1baf7a"
COULEURS = {"Réseaux": BLEU, "Site": ORANGE, "Boutique": AQUA}
plt.rcParams.update({"axes.spines.top": False, "axes.spines.right": False, "axes.grid": True,
                    "grid.color": "#e1e0d9", "grid.linewidth": 0.6, "axes.titlesize": 11})
```

Les chiffres des trois panneaux se calculent d'abord (c'est de la pure analyse, sans dessin) :

```
def resumes(df):
    ca = df.groupby("semaine")["montant"].sum()
    return {"ca": ca, "lisse": ca.rolling(4).mean().dropna(),
            "par_canal": df.groupby("canal")["montant"].sum().sort_values(),
            "notes": df["satisfaction"].value_counts().sort_index()}
```

Chaque panneau est ensuite dessiné par une petite fonction qui reçoit l'axes (ax) où dessiner. Son **titre énonce la conclusion**, calculée à partir des données :

```
def panneau_ca(ax, r):
    lisse = r["lisse"]
    sens = "monte" if lisse.iloc[-1] > lisse.iloc[0] else "baisse"
    ax.plot(r["ca"].index, r["ca"].values, color=BLEU, alpha=0.35, marker="o", ms=3)
    ax.plot(lisse.index, lisse.values, color=BLEU, lw=2.5)
    ax.set_title(f"Le chiffre d'affaires hebdomadaire {sens} de {lisse.iloc[0]:.0f} à
↳ {lisse.iloc[-1]:.0f} € (moyenne mobile)")
    ax.set_ylabel("€ par semaine")
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%d/%m"))

def panneau_canaux(ax, par_canal):
    ax.barh(par_canal.index, par_canal.values, color=[COULEURS[c] for c in par_canal.index],
↳ alpha=0.85)
    ax.set_title(f"{par_canal.index[-1]} : {par_canal.iloc[-1] / par_canal.sum():.0%} du chiffre
↳ d'affaires".replace("%", " %"))
    ax.set_xlabel("chiffre d'affaires (€)")
    ax.grid(axis="y", visible=False)

def panneau_notes(ax, notes, part_satisfaits):
    ax.bar(notes.index, notes.values, color=[ORANGE if n <= 2 else BLEU for n in notes.index],
↳ alpha=0.85)
    ax.set_title(f"{part_satisfaits:.0%} des clients notent 4 ou 5".replace("%", " %"))
    ax.set_xlabel("note de satisfaction")
    ax.set_ylabel("commandes")
    ax.grid(axis="x", visible=False)
```

Enfin, la fonction qui assemble les trois panneaux (un grand en haut, deux en bas) et enregistre la figure :

```
def tableau_de_bord(df, chemin):
    r = resumes(df)
    fig = plt.figure(figsize=(10.5, 6.6))
```

```

grille = fig.add_gridspec(2, 2, height_ratios=[1.15, 1], hspace=0.5, wspace=0.3)
panneau_ca(fig.add_subplot(grille[0, :]), r)                                # panneau du haut : toute la
↪ largeur                                                                    # largeur
panneau_canaux(fig.add_subplot(grille[1, 0]), r["par_canal"])
panneau_notes(fig.add_subplot(grille[1, 1]), r["notes"], (df["satisfaction"] >= 4).mean())
fig.savefig(chemin, dpi=150, bbox_inches="tight")
plt.close(fig)
return r["par_canal"].index[-1], round(float(r["lisse"].iloc[-1]), 1)

print(tableau_de_bord(df, "figures/ch04-tableau-de-bord.png"))

('Site', 1433.7)

```

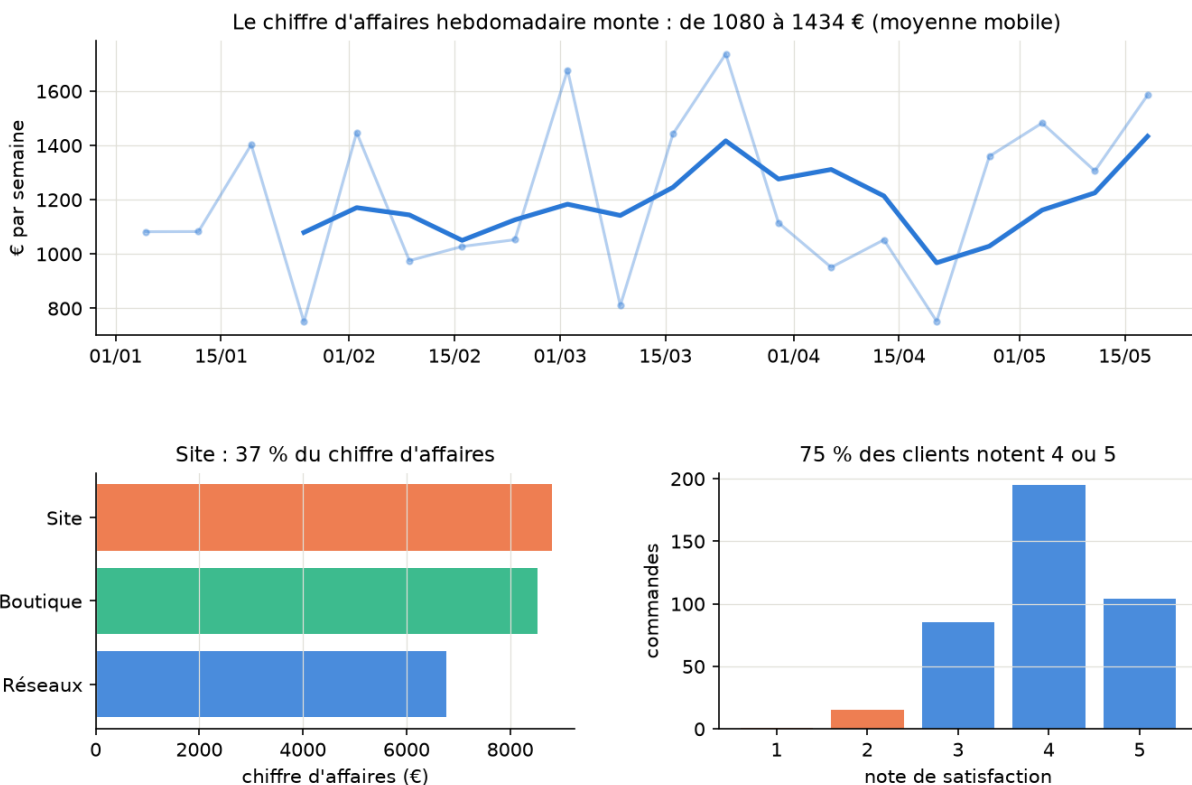


Figure 7: Le tableau de bord produit par la fonction : trois panneaux, chacun avec un titre qui énonce sa conclusion.

Lecture. La tendance du chiffre d'affaires est à la hausse (1 080 → 1 434 € par semaine en moyenne mobile), le Site et la Boutique pèsent chacun plus d'un tiers des ventes, et trois clients sur quatre sont satisfaits (4 ou 5). Voilà le chemin complet : des données brutes aux tableaux (pandas) puis à des figures prêtes à insérer dans un rapport, dans un script que l'on peut relancer chaque semaine.

Application 4.7 — Le module de la boutique et ses tests (section 4.6)

Objectif. Construire un petit module orienté objet et sa suite de tests, comme dans un vrai projet. On écrit des fichiers depuis un terminal (`cat > fichier <<'FIN' ... FIN`) et on lance `pytest`.

Règles métier, vérifiées à la main avant de coder : TVA 19 % ; 2 savons à 20 € et 1 plateau à 30 € valent 70 € HT, soit $70 \times 1,19 = 83,30$ € TTC ; avec 10 % de remise sur le hors-taxe : 63 € HT, soit 74,97 € TTC ; sur le site, la livraison coûte 7 €, offerte dès 100 € TTC ; le retrait en boutique est gratuit.

Étape 1 — la remise (le fichier `remises.py`, que le module importera) :

```
cat > remises.py <<'FIN'
def prix_apres_remise(prix, taux):
    """Prix après une remise de `taux` (0,25 pour 25 %)."""
    if not 0 <= taux <= 1:
        raise ValueError(f"le taux doit être entre 0 et 1, reçu {taux}")
    return prix * (1 - taux)
FIN
echo "remises.py écrit"
```

```
remises.py écrit
```

Étape 2 — les classes `Article` (immuable et validée) et `Panier` :

```
cat > boutique.py <<'FIN'
"""Modèle objet minimal de la boutique."""
from dataclasses import dataclass, field

from remises import prix_apres_remise

TVA = 0.19
SEUIL_LIVRAISON_OFFERTE = 100.0      # €, panier TTC
FRAIS_LIVRAISON_SITE = 7.0          # €

@dataclass(frozen=True)              # frozen : on ne peut plus modifier un article créé
class Article:
    """Un article du catalogue : un nom et un prix hors taxe (€)."""
    nom: str
    prix_ht: float

    def __post_init__(self) -> None:
        if self.prix_ht < 0:
            raise ValueError(f"prix négatif pour {self.nom!r} : {self.prix_ht}")
FIN
echo "article écrit"
```

```
article écrit
```

Puis le **panier** (on ajoute à la suite du fichier avec `>>`) :

```
cat >> boutique.py <<'FIN'

@dataclass
class Panier:
    """Un panier : des lignes (article, quantité)."""
    lignes: list = field(default_factory=list)

    def ajouter(self, article: Article, quantite: int = 1) -> None:
        if quantite <= 0:
            raise ValueError("la quantité doit être strictement positive")
        self.lignes.append((article, quantite))

    def __len__(self) -> int:          # len(panier) = nombre total d'unités
        return sum(quantite for _, quantite in self.lignes)

    def total_ht(self) -> float:
        return sum(article.prix_ht * quantite for article, quantite in self.lignes)

    def total_ttc(self, remise: float = 0.0) -> float:
        """Total TTC arrondi au centime, après une remise sur le hors-taxé."""
        return round(prix_apres_remise(self.total_ht(), remise) * (1 + TVA), 2)
```

FIN

Étape 3 — les commandes (héritage : la commande du site ne redéfinit que les frais de livraison) :

```
cat >> boutique.py <<'FIN'

class Commande:
    """Une commande en boutique : retrait gratuit."""

    def __init__(self, panier: Panier) -> None:
        self.panier = panier

    def frais_livraison(self) -> float:
        return 0.0

    def total_a_payer(self) -> float:
        return round(self.panier.total_ttc() + self.frais_livraison(), 2)

class CommandeSite(Commande):
    """Une commande sur le site : 7 € de livraison, offerts dès 100 € TTC."""

    def frais_livraison(self) -> float:
        if self.panier.total_ttc() >= SEUIL_LIVRAISON_OFFERTE:
            return 0.0
        return FRAIS_LIVRAISON_SITE

FIN
echo "module écrit : $(wc -l < boutique.py) lignes"
```

```
module écrit : 63 lignes
```

Étape 4 — les tests. Chaque règle métier vérifiée à la main devient un test (schéma *Arrange – Act – Assert*) :

```
cat > test_boutique.py <<'FIN'
import pytest

from boutique import Article, Commande, CommandeSite, Panier

@pytest.fixture
def panier():
    """Le panier de l'exemple : 2 savons à 20 € + 1 plateau à 30 €."""
    p = Panier()
    p.ajouter(Article("savon", 20.0), 2)
    p.ajouter(Article("plateau", 30.0))
    return p

def test_nombre_d_unites(panier):
    assert len(panier) == 3

def test_total_ht(panier):
    assert panier.total_ht() == 70.0
FIN
```

```
cat >> test_boutique.py <<'FIN'
```

```
def test_total_ttc(panier):
```

```

    assert panier.total_ttc() == pytest.approx(83.30)

def test_total_ttc_avec_remise(panier):
    assert panier.total_ttc(remise=0.10) == pytest.approx(74.97)
FIN

```

```

cat >> test_boutique.py <<'FIN'

@pytest.mark.parametrize("quantite", [0, -2])
def test_quantite_invalide(quantite):
    with pytest.raises(ValueError):
        Panier().ajouter(Article("savon", 20.0), quantite)

def test_prix_negatif_refuse():
    with pytest.raises(ValueError):
        Article("cadeau", -1.0)

FIN

```

Enfin les tests de la commande : retrait gratuit, livraison payante sous le seuil, offerte au-dessus.

```

cat >> test_boutique.py <<'FIN'

def test_retrait_boutique_gratuit(panier):
    assert Commande(panier).total_a_payer() == pytest.approx(83.30)

def test_livraison_payante_sous_le_seuil(panier):
    assert CommandeSite(panier).total_a_payer() == pytest.approx(90.30)      # 83,30 + 7

def test_livraison_offerte_au_dessus_du_seuil(panier):
    panier.ajouter(Article("coffret", 20.0))      # 90 € HT -> 107,10 € TTC
    assert CommandeSite(panier).total_a_payer() == pytest.approx(107.10)
FIN
python -m pytest -q --color=no --tb=short -p no:cacheprovider 2>&1 | sed -E 's/ in [0-9.]+s///'

..... [100%]
10 passed

```

Étape 5 — casser pour voir. Mettons un zéro de trop au seuil de livraison offerte (une faute de frappe plausible) et relançons les tests : un seul doit passer au rouge, celui qui protège précisément cette règle. Puis remettons la bonne valeur.

```

sed -i 's/SEUIL_LIVRAISON_OFFERTE = 100.0/SEUIL_LIVRAISON_OFFERTE = 1000.0/' boutique.py
python -m pytest -q --color=no --tb=no -p no:cacheprovider 2>&1 | sed -E 's/ in [0-9.]+s///' | tail -2
sed -i 's/SEUIL_LIVRAISON_OFFERTE = 1000.0/SEUIL_LIVRAISON_OFFERTE = 100.0/' boutique.py
python -m pytest -q --color=no -p no:cacheprovider 2>&1 | sed -E 's/ in [0-9.]+s///' | tail -1

FAILED test_boutique.py::test_livraison_offerte_au_dessus_du_seuil - assert 1...
1 failed, 9 passed
10 passed

```

Étape 6 — tester une fonction d'analyse. Le panier moyen par canal, testé sur un petit tableau dont on connaît la réponse à la main (Site : $(10 + 30)/2 = 20$; Boutique : 50) :

```

cat > analyse.py <<'FIN'
import pandas as pd

def panier_moyen_par_canal(commandes: pd.DataFrame) -> pd.Series:
    """Montant moyen des commandes pour chaque canal de vente."""
    return commandes.groupby("canal")["montant"].mean()
FIN
cat > test_analyse.py <<'FIN'
import pandas as pd
import pytest

from analyse import panier_moyen_par_canal

def test_panier_moyen_par_canal():
    petit = pd.DataFrame({"canal": ["Site", "Site", "Boutique"], "montant": [10.0, 30.0, 50.0]})
    resultat = panier_moyen_par_canal(petit)
    assert resultat["Site"] == pytest.approx(20.0)          # (10 + 30) / 2
    assert resultat["Boutique"] == pytest.approx(50.0)      # une seule commande
FIN
python -m pytest -q --color=no -p no:cacheprovider 2>&1 | sed -E 's/ in [0-9.]+s//' | tail -1

11 passed

```

Lecture. Les tests vérifient les calculs de la main (83,30 ; 74,97 ; 90,30 ; 107,10). Les onze tests de l'ensemble (dix pour le module, un pour l'analyse) passent ; en cassant le seuil, un seul devient rouge (« 1 failed, 9 passed »), celui qui protège précisément cette règle, et la bonne valeur remise, tout repasse au vert. Dans un vrai projet, on range le code dans `src/` et les tests dans `tests/`, et `pytest` trouve alors seul les fichiers `test_*.py`.

Application 4.8 — Règles d'association : quels produits s'achètent ensemble ? (section 4.8.7)

Objectif. La gérante voudrait savoir quels produits sont **souvent achetés ensemble** pour proposer des offres groupées. Le catalogue compte 40 produits. Énumérer tous les paniers *possibles* est hors de portée ($\binom{40}{10} = 847\,660\,528$ rien que pour les paniers de 10 produits) ; mais on n'a besoin que des paniers **réellement achetés** : on compte, panier par panier, les paires qu'il contient.

Les données sont simulées (20 000 paniers de 2 à 5 produits, avec en plus le couple 7 + 12 acheté ensemble par 15 % des clientes) :

```

from collections import Counter
from itertools import combinations

rng = np.random.default_rng(21)
paniers = []
for _ in range(20_000):
    taille = int(rng.integers(2, 6))          # entre 2 et 5 produits
    panier = set(rng.choice(40, size=taille, replace=False).tolist())
    if rng.random() < 0.15:                  # 15 % des clientes achètent le couple 7 + 12
        panier |= {7, 12}
    paniers.append(sorted(panier))
print(len(paniers), "paniers ; exemple :", paniers[0])

20000 paniers ; exemple : [7, 12, 15, 24, 29]

```

Étape 1 — compter les paires et les produits.

```

paires, produits = Counter(), Counter()
for panier in paniers:
    produits.update(panier)
    paires.update(combinations(panier, 2))           # toutes les paires DU panier
print("paires comptées :", sum(paires.values()))
print(paires.most_common(3))

```

```

paires comptées : 120761
[((7, 12), 3017), ((7, 8), 395), ((5, 12), 392)]

```

Étape 2 — trois mesures. Pour un couple (a, b) : le **soutien** est la part des paniers qui contiennent les deux ; la **confiance** de $a \Rightarrow b$ est la part des paniers contenant a qui contiennent aussi b ; le **lift** compare cette confiance à la part de b dans l'ensemble des paniers (un lift de 1 signifie « aucun lien », un lift grand signifie que a et b vont ensemble bien plus que ne le voudrait le hasard).

```

n = len(paniers)

def mesures(a, b):
    soutien = paires[(a, b)] / n
    confiance = paires[(a, b)] / produits[a]
    lift = confiance / (produits[b] / n)
    return round(soutien, 3), round(confiance, 3), round(lift, 2)

print("couple (7, 12) :", mesures(7, 12))           # soutien, confiance, lift
print("couple (7, 8)  :", mesures(7, 8))

```

```

couple (7, 12) : (0.151, 0.687, 3.11)
couple (7, 8)  : (0.02, 0.09, 1.0)

```

Lecture. Le couple (7, 12) ressort nettement : il est présent dans 15,1 % des paniers (soutien), 68,7 % des paniers qui contiennent le produit 7 contiennent aussi le 12 (confiance), et le lift vaut 3,11 : le 7 et le 12 vont ensemble trois fois plus souvent que ne le voudrait le hasard. À l'inverse, une paire due au hasard comme (7, 8) a un lift de 1,0 (aucun lien). La méthode est **linéaire** en nombre de paniers : chaque panier de k produits fournit $\binom{k}{2}$ paires. C'est exactement l'idée derrière les algorithmes de recommandation : on compte ce qui s'est vraiment passé, on n'explore pas ce qui aurait pu se passer.

Exercices

★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les exercices 4.2 et 4.3 prolongent l'application 4.1 ; les exercices 4.13 et 4.14 reprennent les sections optionnelles 4.6 et 4.8. Les corrigés sont en fin de chapitre.

Exercice 4.1 ★ — dictionnaires et boucles (section 4.1)

Cinq commandes (canal, montant) : ("Site", 40), ("Réseaux", 25), ("Site", 60), ("Boutique", 100), ("Réseaux", 35). Calculez à la main le chiffre d'affaires de chaque canal et sa part du total. Écrivez ensuite le programme avec un dictionnaire et une boucle, sans bibliothèque.

Exercice 4.2 ★★ — un produit absent du catalogue (section 4.1.10)

Reprenez le programme de la caisse. (a) Que se passe-t-il pour le panier [("bol", 2), ("vase", 1)] ? (b) Faites afficher un message **aimable** (nom du produit fautif et liste des produits disponibles) au lieu d'un plantage. (c) Variante : ignorez les produits inconnus, calculez le total TTC du reste du panier et listez ce qui a été ignoré. Quel total TTC attendez-vous à la main ?

Exercice 4.3 ★★ — un code promo (section 4.1.10)

La gérante crée le code BIENVENUE15 (15 % de remise sur le hors-tax) et le code RENTREE5 (5 %). Règle : la remise promo **ne se cumule pas** avec la remise fidélité (10 % au-delà de 100 €) ; on applique la **meilleure des deux**. Un code inconnu est ignoré avec un message. Calculez à la main le TTC du panier « 2 bols, 1 plateau, 3 bougies » avec BIENVENUE15, puis celui de « 2 tasses » avec le même code, et vérifiez par le code.

Exercice 4.4 ★ — R (section 4.2)

Quelle part des commandes est « satisfaite » (note ≥ 4) dans chaque canal ? (a) Sur les notes 5, 4, 2, 3, 5 de l'exemple, calculez la part à la main. (b) Répondez sur `donnees/commandes.csv` en R de base (tapply sur un test logique), puis (c) avec dplyr, et (d) contrôlez le résultat avec pandas.

Exercice 4.5 ★ — une pile (section 4.3.3)

La notation polonaise inversée (NPI) écrit l'opérateur **après** ses opérandes : `3 4 +` signifie $3 + 4$. Elle se calcule avec une pile et n'a besoin d'aucune parenthèse. (a) Évaluez à la main `3 4 + 2 *`, puis `5 1 2 + 4 * + 3 -`. (b) Écrivez la fonction d'évaluation. (c) Le prix TTC d'un article à 80 € HT avec 10 % de remise s'écrit `80 1 0.1 - * 1.19 *` : calculez-le. (d) Que doit faire la fonction devant `3 + ?`

Exercice 4.6 ★★ — une file à deux emballeuses (section 4.3.3)

Six commandes arrivent aux minutes 0, 1, 2, 3, 10 et 11 ; emballer une commande dure 4 minutes. Calculez à la main l'attente de chaque commande (a) avec **une** emballeuse, (b) avec **deux**, puis écrivez la simulation. Indice : retenez, pour chaque emballeuse, l'instant où elle sera libre ; la prochaine commande est confiée à celle qui est libre **le plus tôt** (le module `heapq` sait trouver le plus petit élément).

Exercice 4.7 ★★ — une dichotomie à variante (section 4.3.5)

`bisect_left(liste, x)` renvoie la **première** position où `x` pourrait être inséré dans une liste triée. (a) Évaluez à la main cette position pour $x = 2$ dans `[1, 2, 2, 2, 5]`. (b) Écrivez votre propre version par dichotomie, en énonçant son invariant. (c) Déduisez-en le nombre de commandes ayant la note 5 dans la colonne `satisfaction` triée, et comparez avec `Counter`. (d) Vérifiez votre fonction contre `bisect_left` sur 1 000 listes aléatoires.

Exercice 4.8 ★ — tri stable (section 4.3.6)

Rangez les commandes ("Site", 40.1), ("Boutique", 65.8), ("Site", 19.6), ("Boutique", 17.4), ("Réseaux", 30.1), ("Site", 75.0) **par canal croissant puis, dans chaque canal, par montant décroissant**, avec uniquement des appels à `sorted`. Dans quel ordre faut-il enchaîner les deux tris ? Que se passe-t-il si on les inverse ?

Exercice 4.9 ★ — NumPy et broadcasting (section 4.4)

Les quantités vendues sur trois jours pour (bol, tasse, plateau, bougie) sont les lignes de $Q = \begin{pmatrix} 2 & 0 & 1 & 3 \\ 1 & 1 & 0 & 0 \\ 0 & 4 & 2 & 1 \end{pmatrix}$, les prix HT sont $[12.5, 8, 45, 15.9]$. (a) Calculez à la main le chiffre d'affaires de chaque jour. (b) Une promotion retire $[0, 10\%, 0, 20\%]$ aux prix ; recalculez le chiffre d'affaires quotidien **en une seule ligne** grâce au broadcasting. (c) Quel produit s'est le mieux vendu en nombre d'unités ?

Exercice 4.10 ★★ — groupby et dates (section 4.4)

Six ventes : (5 janv., Site, 30), (20 janv., Réseaux, 50), (2 févr., Site, 70), (10 févr., Site, 20), (11 févr., Boutique, 100), (1er mars, Réseaux, 40). (a) Dressez à la main le tableau **mois × canal** des montants et les totaux mensuels. (b) Obtenez-le avec `pivot_table`. (c) Sur les 400 commandes (avec les colonnes `date` et `id_client` simulées en 4.4.5), quel mois rapporte le plus, et quel jour de la semaine compte le plus de commandes ?

Exercice 4.11 ★★ — jointure et clients dormants (section 4.4.10)

- (a) Quatre commandes portent les numéros de client 1, 1, 3, 5 ; la table des clients contient les numéros 1 à 5. Qui n'a jamais commandé ? (b) Sur les données du livre (table `clients` de 4.4.10), combien de clients n'ont **aucune** commande ? Répondez de deux façons : avec `isin`, puis avec une jointure `left` et le comptage des valeurs manquantes. (c) Quel est le chiffre d'affaires par ville ?

Exercice 4.12 ★★ — un graphique honnête (section 4.5)

Tracez le montant moyen par canal en barres **triées par ordre décroissant**, avec un axe qui commence à zéro, un titre qui énonce le message, des axes nommés avec leur unité. Vérifiez par le code que les hauteurs des barres sont bien les moyennes, que l'axe commence à 0 et que le fichier est enregistré. Citez deux défauts qu'aurait une version avec axe tronqué et camembert 3D.

Exercice 4.13 ★★ — un test unitaire (section 4.6)

Règle de livraison : retrait en boutique gratuit ; sur le Site ou Réseaux, 7 €, **offerts à partir de 100 € TTC** (100 compris) ; un canal inconnu est une erreur. (a) Dressez la liste des cas à tester, en pensant aux **bornes**. (b) Écrivez la fonction **avec le défaut classique** (`>` au lieu de `>=`) et les tests avec `pytest` ; constatez l'échec. (c) Corrigez et relancez.

Exercice 4.14 ★★★ — coût d'un algorithme (section 4.8)

On veut compter les **paires de commandes passées par le même client**. (a) Combien de paires de commandes y a-t-il en tout parmi $n = 400$? Et parmi $n = 800$? Quel est le facteur ? (b) Écrivez la version « double boucle » et comptez ses tours. (c) Écrivez une version en $O(n)$ avec un dictionnaire de compteurs (un client avec c commandes produit $c(c-1)/2$ paires). (d) Vérifiez qu'elles donnent le même résultat. (e) Estimez, avec la formule, le nombre de tours de la double boucle pour un million de commandes.

Corrigés

Corrigé 4.1

Site : $40 + 60 = 100$; Réseaux : $25 + 35 = 60$; Boutique : 100. Total : 260. Parts : $100/260 \approx 38,5\%$ pour le Site, $60/260 \approx 23,1\%$ pour Réseaux, $38,5\%$ pour la Boutique (la somme des parts doit faire 100 %).

```
commandes = [("Site", 40), ("Réseaux", 25), ("Site", 60), ("Boutique", 100), ("Réseaux", 35)]
ca = {}
for canal, montant in commandes:
    ca[canal] = ca.get(canal, 0) + montant          # .get évite la KeyError au premier passage
total = sum(ca.values())
for canal, valeur in sorted(ca.items()):
    print(f"{canal:<10} {valeur:>4} €   {valeur / total:6.1%}")
print("total :", total, "| somme des parts :", round(sum(v / total for v in ca.values()), 6))
```

Boutique	100 €	38.5%
Réseaux	60 €	23.1%
Site	100 €	38.5%
total : 260 somme des parts : 1.0		

Corrigé 4.2

- (a) Le programme cherche `CATALOGUE["vase"]`, qui n'existe pas : Python s'arrête avec `KeyError` : 'vase' (voir 4.1.8 du livre). (b) On rattrape l'erreur avec `try / except KeyError`, et on utilise la clé fautive que l'exception transporte. (c) On sépare les produits connus des inconnus **avant** de calculer. À la main : 2 bols = 25 € HT, sous le seuil de remise ; TVA $25 \times 0,19 = 4,75$; TTC = 29,75 €.

```
CATALOGUE = {"bol": 12.5, "tasse": 8.0, "plateau": 45.0, "bougie": 15.9}          # identique à 4.1.10
TVA, SEUIL_REMISE, TAUX_REMISE = 0.19, 100, 0.10
```

```
def sous_total(panier):
    return sum(CATALOGUE[nom] * qte for nom, qte in panier)

def remise(montant_ht):
    return montant_ht * TAUX_REMISE if montant_ht > SEUIL_REMISE else 0.0

panier = [("bol", 2), ("vase", 1)]
try:
    sous_total(panier)
except KeyError as e:
    print("(a) KeyError, clé fautive :", e)
    print(f"(b) Désolé, {e.args[0]!r} n'est pas au catalogue. Disponibles : {'',
        ↪ ' '.join(sorted(CATALOGUE))}.")
```

(a) `KeyError`, clé fautive : 'vase'

(b) Désolé, 'vase' n'est pas au catalogue. Disponibles : bol, bougie, plateau, tasse.

Pour la variante (c), on sépare les produits connus des inconnus **avant** de calculer :

```
def total_prudent(panier):
    """Renvoie (total TTC, liste des produits ignorés)."""
    connus = [(nom, qte) for nom, qte in panier if nom in CATALOGUE]
    ignores = [nom for nom, _ in panier if nom not in CATALOGUE]
```

```

    ht = sous_total(connus)
    net = ht - remise(ht)
    return round(net * (1 + TVA), 2), ignores

print("(c)", total_prudent(panier))

```

```
(c) (29.75, ['vase'])
```

On retrouve le TTC de 29,75 € calculé à la main, et 'vase' est signalé comme ignoré. Le choix entre « refuser le panier » (b) et « ignorer et signaler » (c) est une **décision métier**, pas technique : ce qui compte, c'est de ne jamais avaler une erreur en silence.

Piège. Un except : nu (sans type d'erreur) rattrape *tout*, y compris vos propres fautes de frappe : on ne rattrape que l'erreur **attendue** (KeyError, ValueError...).

Corrigé 4.3

Panier de 4.1.10 : sous-total HT = 117,70. Remise fidélité : 10 % soit 11,77 € ; remise promo : 15 % soit 17,655 €. On garde la meilleure, la promo : net = 117,70 × 0,85 = 100,045 ; TTC = 100,045 × 1,19 = 119,05355 ≈ 119,05 € (contre 126,06 sans code). Panier de 2 tasses : HT = 16, pas de remise fidélité (sous 100 €), promo 15 % : net = 13,60, TTC = 13,60 × 1,19 = 16,184 ≈ 16,18 €.

```
CODES = {"BIENVENUE15": 0.15, "RENTREE5": 0.05}
```

```

def total_ttc(panier, code=None):
    ht = sous_total(panier)
    taux = TAUX_REMISE if ht > SEUIL_REMISE else 0.0      # remise fidélité
    if code is not None:
        if code in CODES:
            taux = max(taux, CODES[code])                # la meilleure des deux, sans cumul
        else:
            print(f" code {code!r} inconnu : ignoré")
    return round(ht * (1 - taux) * (1 + TVA), 2)

gros = [("bol", 2), ("plateau", 1), ("bougie", 3)]
print("gros panier sans code      :", total_ttc(gros))
print("gros panier BIENVENUE15    :", total_ttc(gros, "BIENVENUE15"))
print("gros panier RENTREE5       :", total_ttc(gros, "RENTREE5"), "(la fidélité de 10 % est meilleure)")
print("2 tasses BIENVENUE15       :", total_ttc([("tasse", 2)], "BIENVENUE15"))
print("2 tasses code bidon        :", total_ttc([("tasse", 2)], "PROMO99"))
assert total_ttc(gros, "BIENVENUE15") == 119.05 and total_ttc([("tasse", 2)], "BIENVENUE15") == 16.18

```

```

gros panier sans code      : 126.06
gros panier BIENVENUE15    : 119.05
gros panier RENTREE5       : 126.06 (la fidélité de 10 % est meilleure)
2 tasses BIENVENUE15      : 16.18
   code 'PROMO99' inconnu : ignoré
2 tasses code bidon       : 19.04

```

Les valeurs coïncident avec la main. Remarquez le troisième cas : RENTREE5 (5 %) est **moins bon** que la fidélité (10 %), donc le total reste celui sans code.

Corrigé 4.4

- (a) Notes 5,4,2,3,5 : trois notes sur cinq sont ≥ 4 , soit 60 %. (b) En R, un test logique vaut TRUE (1) ou FALSE (0) : la **moyenne** d'un test logique est donc la **proportion** de TRUE. (c) La version dplyr. (d) Le contrôle avec pandas utilise exactement la même idée.

```
notes <- c(5, 4, 2, 3, 5)
mean(notes >= 4) # (a) : 0.6

df <- read.csv("donnees/commandes.csv")
round(tapply(df$satisfaction >= 4, df$canal, mean), 3) # (b) R de base

suppressPackageStartupMessages(library(dplyr))
df |> # (c) dplyr
  group_by(canal) |>
  summarise(n = n(), part_satisfaits = round(mean(satisfaction >= 4), 3))
```

```
[1] 0.6
Boutique Réseaux Site
0.956 0.630 0.696
# A tibble: 3 × 3
  canal      n part_satisfaits
  <chr> <int>         <dbl>
1 Boutique 114         0.956
2 Réseaux 138         0.63
3 Site 148         0.696
```

```
commandes = pd.read_csv("donnees/commandes.csv")
print((commandes["satisfaction"] >= 4).groupby(commandes["canal"]).mean().round(3)) # (d) pandas
```

```
canal
Boutique 0.956
Réseaux 0.630
Site 0.696
Name: satisfaction, dtype: float64
```

Les trois méthodes donnent les mêmes proportions : 95,6 % de commandes satisfaites en **Boutique**, 69,6 % sur le **Site** et 63,0 % sur **Réseaux**. La Boutique est loin devant : le retrait est immédiat, or la satisfaction baisse d'environ 0,18 point par jour de délai (4.2.5). C'est une description de l'échantillon : savoir si un écart est *significatif* demanderait un test, comme au 3.4.

Corrigé 4.5

- (a) $3 \ 4 \ + \ 2 \ *$: on empile 3 puis 4 ; + dépile 4 et 3 et empile 7 ; on empile 2 ; * dépile 2 et 7 et empile 14. Résultat : $(3 + 4) \times 2 = 14$. Pour $5 \ 1 \ 2 \ + \ 4 \ * \ + \ 3 \ -$: pile [5], [5,1], [5,1,2] ; + $\rightarrow$ [5,3] ; 4 $\rightarrow$ [5,3,4] ; * $\rightarrow$ [5,12] ; + $\rightarrow$ [17] ; 3 $\rightarrow$ [17,3] ; - $\rightarrow$ [14]. Soit $5 + (1 + 2) \times 4 - 3 = 14$. **Attention à l'ordre** : pour - et /, le **premier** dépilé est l'opérande de **droite**. (c) $80 \times (1 - 0,1) \times 1,19 = 85,68$ €.

```
def evaluer_npi(expression):
    operations = {"+": lambda a, b: a + b, "-": lambda a, b: a - b,
                  "*": lambda a, b: a * b, "/": lambda a, b: a / b}
    pile = []
    for jeton in expression.split():
        if jeton in operations:
            if len(pile) < 2:
                raise ValueError(f"expression invalide : il manque un opérande pour {jeton!r}")
            droite = pile.pop() # le dernier empilé est l'opérande de droite
            gauche = pile.pop()
            pile.append(operations[jeton](gauche, droite))
        else:
```

```

        pile.append(float(jeton))
    if len(pile) != 1:
        raise ValueError("expression invalide : il reste plusieurs valeurs sur la pile")
    return pile[0]

print(evaluer_npi("3 4 + 2 *"))
print(evaluer_npi("5 1 2 + 4 * + 3 -"))
print(round(evaluer_npi("80 1 0.1 - * 1.19 *"), 2))
try:
    evaluer_npi("3 +")
except ValueError as e:
    print("(d) ValueError :", e)

```

```

14.0
14.0
85.68
(d) ValueError : expression invalide : il manque un opérande pour '+'

```

- (d) Devant 3 +, la pile ne contient qu'une valeur quand arrive + : la fonction doit **signaler l'erreur** plutôt que de planter obscurément sur un pop de pile vide.

Corrigé 4.6

- (a) Une emballeuse (libre en 0, 4, 8, 12, ...) : débuts 0, 4, 8, 12, 16, 20 ; attentes 0, 3, 6, 9, 6, 9 ; moyenne $33/6 = 5,5$ min. (b) Deux emballeuses : la commande 0 démarre en 0 (emballeuse A, libre à 4) ; la 1 démarre en 1 (B, libre à 5), attente 0 ; la 2 attend A jusqu'à 4 : attente 2 (A libre à 8) ; la 3 attend B jusqu'à 5 : attente 2 (B libre à 9) ; la 4 (arrivée 10) trouve tout libre : attente 0 ; la 5 (arrivée 11) trouve B libre depuis 9 : attente 0. Attentes 0, 0, 2, 2, 0, 0 ; moyenne $4/6 \approx 0,67$ min.

```

import heapq
from collections import deque

def simuler(arrivees, duree, nb_emballeuses):
    libres = [0] * nb_emballeuses          # instant où chaque emballeuse sera libre (un « tas »)
    heapq.heapify(libres)
    file = deque(arrivees)
    attentes = []
    while file:
        arrivee = file.popleft()            # premier arrivé, premier servi
        libre_a = heapq.heappop(libres)     # l'emballeuse libre le plus tôt
        debut = max(arrivee, libre_a)
        attentes.append(debut - arrivee)
        heapq.heappush(libres, debut + duree)
    return attentes

arrivees = [0, 1, 2, 3, 10, 11]
for k in (1, 2):
    a = simuler(arrivees, 4, k)
    print(f"{k} emballeuse(s) : attentes {a} | moyenne {sum(a) / len(a):.2f} min")

```

```

1 emballeuse(s) : attentes [0, 3, 6, 9, 6, 9] | moyenne 5.50 min
2 emballeuse(s) : attentes [0, 0, 2, 2, 0, 0] | moyenne 0.67 min

```

Doubler l'effectif divise l'attente moyenne **par plus de huit** (de 5,5 à 0,67 minute) : les files d'attente sont non linéaires, et c'est ce qu'étudie la théorie évoquée au chapitre 2.

Corrigé 4.7

- (a) Dans [1, 2, 2, 2, 5], la première position où l'on peut insérer 2 sans casser l'ordre est l'indice 1 (juste devant le premier 2). (b) On cherche le **plus petit indice p tel que $\text{liste}[p] \geq x$** (ou n s'il n'existe pas). *Invariant* : tous les éléments d'indice $< g$ sont $< x$ et tous ceux d'indice $\geq d$ sont $\geq x$. *Initialisation* : $g = 0, d = n$ (zones vides). *Conservation* : si $\text{liste}[m] < x$, tous ceux d'indice $\leq m$ sont $< x$ (liste triée) donc $g = m + 1$; sinon tous ceux d'indice $\geq m$ sont $\geq x$ donc $d = m$. *Terminaison* : la zone $[g, d[$ perd au moins la moitié de sa taille à chaque tour. À la sortie $g = d$, et l'invariant dit que g est la réponse. À la main sur $x = 2$: $[g, d[= [0, 5[, m = 2, \text{liste}[2] = 2$ n'est pas < 2 donc $d = 2$; $m = 1$, même chose, $d = 1$; $m = 0, \text{liste}[0] = 1 < 2$ donc $g = 1$; $g = d = 1$: réponse 1. (c) Les notes sont entières : le nombre de 5 vaut $\text{première_position}(\text{liste}, 6) - \text{première_position}(\text{liste}, 5)$; le code trouve 104 notes de 5 sur 400, comme Counter.

```
import bisect
import random
from collections import Counter

def première_position(triee, x):
    g, d = 0, len(triee)
    while g < d:
        m = (g + d) // 2
        if trie[m] < x:
            g = m + 1
        else:
            d = m
    return g
```

Vérifions (a) et (c), puis comparons la fonction à `bisect_left` sur 1 000 listes aléatoires :

```
print("(a)", première_position([1, 2, 2, 2, 5], 2), "| occurrences de 2 :",
      première_position([1, 2, 2, 2, 5], 3) - première_position([1, 2, 2, 2, 5], 2))

notes = sorted(pd.read_csv("donnees/commandes.csv")["satisfaction"])
nb5 = première_position(notes, 6) - première_position(notes, 5)
print("(c) notes 5 :", nb5, "| Counter :", Counter(notes)[5])

random.seed(1)
for _ in range(1000):
    liste = sorted(random.choices(range(20), k=random.randint(0, 30)))
    x = random.randint(-1, 21)
    assert première_position(liste, x) == bisect.bisect_left(liste, x)
print("(d) 1000 listes aléatoires : même réponse que bisect_left")
```

```
(a) 1 | occurrences de 2 : 3
(c) notes 5 : 104 | Counter : 104
(d) 1000 listes aléatoires : même réponse que bisect_left
```

Corrigé 4.8

Les deux tris sont **stables** : le second préserve l'ordre produit par le premier pour les éléments à égalité. Il faut donc trier d'abord selon le critère **secondaire** (montant décroissant), puis selon le critère **principal** (canal).

```
cmds = [("Site", 40.1), ("Boutique", 65.8), ("Site", 19.6), ("Boutique", 17.4), ("Réseaux", 30.1),
        ("Site", 75.0)]
bon = sorted(sorted(cmds, key=lambda c: c[1], reverse=True), key=lambda c: c[0])
print("secondaire puis principal :")
```

```

for c in bon:
    print(" ", c)
mauvais = sorted(sorted(cmds, key=lambda c: c[0]), key=lambda c: c[1], reverse=True)
print("ordre inversé (faux) :")
for c in mauvais:
    print(" ", c)

```

```

secondaire puis principal :
('Boutique', 65.8)
('Boutique', 17.4)
('Réseaux', 30.1)
('Site', 75.0)
('Site', 40.1)
('Site', 19.6)
ordre inversé (faux) :
('Site', 75.0)
('Boutique', 65.8)
('Site', 40.1)
('Réseaux', 30.1)
('Site', 19.6)
('Boutique', 17.4)

```

Le bon résultat est Boutique (65,8 puis 17,4), Réseaux (30,1), Site (75,0 ; 40,1 ; 19,6). Dans la version inversée, le dernier tri est celui des montants : les canaux sont mélangés, et leur ordre n'est conservé que pour les montants égaux, ce qui n'arrive pas ici. Raccourci équivalent : `sorted(cmds, key=lambda c: (c[0], -c[1]))`.

Corrigé 4.9

- (a) Jour 1 : $2 \times 12,5 + 0 \times 8 + 1 \times 45 + 3 \times 15,9 = 25 + 45 + 47,7 = 117,7$. Jour 2 : $12,5 + 8 = 20,5$. Jour 3 : $4 \times 8 + 2 \times 45 + 15,9 = 32 + 90 + 15,9 = 137,9$. (b) Prix remisés : 12,5 ; 7,2 ; 45 ; 12,72. Jour 1 : $25 + 45 + 3 \times 12,72 = 108,16$; jour 2 : $12,5 + 7,2 = 19,7$; jour 3 : $4 \times 7,2 + 90 + 12,72 = 131,52$. (c) Unités par produit (somme des colonnes) : 3, 5, 3, 4 : la tasse.

```

import numpy as np

Q = np.array([[2, 0, 1, 3], [1, 1, 0, 0], [0, 4, 2, 1]])
prix = np.array([12.5, 8, 45, 15.9])
promo = np.array([0, 0.10, 0, 0.20])

print("(a) CA par jour      :", Q @ prix)
print("(b) CA avec promotion :", (Q * (prix * (1 - promo))).sum(axis=1))
print("  formes :", Q.shape, "*", prix.shape, "->", (Q * prix).shape, "(le vecteur est répété sur  
  ↪ chaque ligne)")
produits = ["bol", "tasse", "plateau", "bougie"]
print("(c) unités par produit :", {p: int(q) for p, q in zip(produits, Q.sum(axis=0))}, "-> meilleur  
  ↪ :", produits[Q.sum(axis=0).argmax()])

```

```

(a) CA par jour      : [117.7  20.5 137.9]
(b) CA avec promotion : [108.16  19.7 131.52]
    formes : (3, 4) * (4,) -> (3, 4) (le vecteur est répété sur chaque ligne)
(c) unités par produit : {'bol': 3, 'tasse': 5, 'plateau': 3, 'bougie': 4} -> meilleur : tasse

```

$Q * \text{prix}$ marche parce que les formes (3,4) et (4,) sont **compatibles** : NumPy « étire » le vecteur sur les trois lignes (broadcasting, 4.4.3). `axis=1` somme **le long des colonnes**, c'est-à-dire produit un total par ligne (par jour).

Corrigé 4.10

- (a) Janvier : Réseaux 50, Site 30, Boutique 0 (total 80). Février : Site 70 + 20 = 90, Boutique 100 (total 190). Mars : Réseaux 40 (total 40). (b) et (c) :

```
mini = pd.DataFrame({
    "date": pd.to_datetime(["2026-01-05", "2026-01-20", "2026-02-02", "2026-02-10", "2026-02-11",
        ↪ "2026-03-01"]),
    "canal": ["Site", "Réseaux", "Site", "Site", "Boutique", "Réseaux"],
    "montant": [30, 50, 70, 20, 100, 40],
})
mini["mois"] = mini["date"].dt.month
tab = mini.pivot_table(index="mois", columns="canal", values="montant", aggfunc="sum", fill_value=0)
tab["total"] = tab.sum(axis=1)
print(tab)
```

canal	Boutique	Réseaux	Site	total
mois				
1	0	50	30	80
2	100	0	90	190
3	0	40	0	40

Sur les 400 commandes (tableau df de la préparation, avec ses colonnes date et id_client) :

```
# --- les 400 commandes (tableau `df` de la préparation)
df["mois"] = df["date"].dt.month

par_mois = df.pivot_table(index="mois", columns="canal", values="montant", aggfunc="sum",
    ↪ fill_value=0).round(0)
par_mois["total"] = par_mois.sum(axis=1)
print()
print(par_mois)
print("mois le plus rentable :", par_mois["total"].idxmax(), "(", par_mois["total"].max(), "€)")

noms = ["lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi", "dimanche"]
par_jour = df["date"].dt.dayofweek.value_counts().sort_index()
print()
print({noms[j]: int(n) for j, n in par_jour.items()})
print("jour le plus chargé :", noms[par_jour.idxmax()])
```

canal	Boutique	Réseaux	Site	total
mois				
1	1861.0	853.0	1483.0	4197.0
2	1316.0	1366.0	1863.0	4545.0
3	1948.0	2073.0	2037.0	6058.0
4	1648.0	998.0	1752.0	4398.0
5	1755.0	1473.0	1672.0	4900.0

mois le plus rentable : 3 (6058.0 €)

```
{'lundi': 53, 'mardi': 63, 'mercredi': 65, 'jeudi': 53, 'vendredi': 51, 'samedi': 52, 'dimanche': 63}
jour le plus chargé : mercredi
```

Le tableau de la main est reproduit exactement par pivot_table (le fill_value=0 remplace les cases vides par 0 au lieu de NaN). Sur les 400 commandes, **mars** est le mois le plus rentable (6 058 €). Attention toutefois à comparer des mois **inégalement couverts** : nos données vont du 5 janvier au 24 mai, donc janvier (27 jours) et mai (24 jours) sont incomplets, alors que mars l'est : un mois incomplet est un piège classique de lecture. Quant au jour de la semaine, les 140 jours de la période contiennent exactement 20 lundis, 20 mardis, etc. ; les dates ayant été tirées **au hasard et uniformément**, les écarts (de 51 commandes le vendredi à 65 le mercredi) ne sont que du hasard d'échantillonnage. Sur de vraies ventes, de telles différences guideraient les horaires d'ouverture ; ici, il ne faut surtout pas les « interpréter ».

Corrigé 4.11

- (a) Les numéros 2 et 4 n'apparaissent dans aucune commande : ce sont les clients **dormants**.
(b) et (c) :

```
petit_clients = pd.DataFrame({"id_client": [1, 2, 3, 4, 5]})
petites_cmds = pd.DataFrame({"id_client": [1, 1, 3, 5]})
print("(a) sans commande :",
      ↪ petit_clients[~petit_clients["id_client"].isin(petites_cmds["id_client"])]["id_client"].tolist())

rng_c = np.random.default_rng(11) # même recette qu'en 4.4.10
clients = pd.DataFrame({
    "id_client": np.arange(1, 126),
    "ville": rng_c.choice(["Ville H", "Ville F", "Ville G", "Ville E", "Ville B"], size=125, p=[0.4,
        ↪ 0.2, 0.2, 0.1, 0.1]),
})

# méthode 1 : isin
dormants1 = clients[~clients["id_client"].isin(df["id_client"])]
# méthode 2 : jointure « left » depuis les clients, vers le nombre de commandes par client
nb_cmd = df.groupby("id_client").size().rename("nb_commandes").reset_index()
jointure = clients.merge(nb_cmd, on="id_client", how="left")
dormants2 = jointure[jointure["nb_commandes"].isna()]
print("(b) clients dormants :", len(dormants1), "=", len(dormants2), "| identiques :",
      ↪ dormants1["id_client"].tolist() == dormants2["id_client"].tolist())
print("    dont les clients 121 à 125 :", sorted(set(range(121, 126)) & set(dormants1["id_client"])))

ca_ville = df.merge(clients, on="id_client",
    ↪ how="left").groupby("ville")["montant"].sum().round(0).sort_values(ascending=False)
print("(c) CA par ville :")
print(ca_ville)
print("somme :", ca_ville.sum(), "| total des commandes :", round(df["montant"].sum()))
```

```
(a) sans commande : [2, 4]
(b) clients dormants : 11 = 11 | identiques : True
    dont les clients 121 à 125 : [121, 122, 123, 124, 125]
(c) CA par ville :
ville
Ville H    10491.0
Ville G     5492.0
Ville F     4518.0
Ville E     2352.0
Ville B     1245.0
Name: montant, dtype: float64
somme : 24098.0 | total des commandes : 24098
```

Le contrôle de cohérence final (la **somme par ville égale le total**) est un réflexe : une jointure qui dupliquerait ou perdrait des lignes (clés en double, clés absentes avec inner) fausserait silencieusement les totaux. C'est pourquoi on précise `how="left"` quand on veut conserver toutes les commandes. Onze clients sur 125 n'ont jamais commandé : les cinq clients 121 à 125 (dormants **par construction**, ce que le résultat confirme) et six autres que le hasard a laissés de côté. Ville H réalise à elle seule près de 44 % du chiffre d'affaires.

Corrigé 4.12

Le message est visible d'un coup d'œil si les barres sont triées et si l'axe part de zéro (la longueur de la barre porte l'information, voir 4.5.6).

```
import os
import tempfile
import matplotlib.pyplot as plt
```

```

moy = df.groupby("canal")["montant"].mean().sort_values(ascending=False)
fig, ax = plt.subplots(figsize=(6, 3.4))
barres = ax.bar(moy.index, moy.values, color="#2a78d6")
ax.bar_label(barres, fmt="%.1f") # valeur écrite au-dessus de chaque
    ↪ barre
ax.set_ylim(bottom=0)
ax.set_title("La boutique a le panier moyen le plus élevé")
ax.set_xlabel("canal de vente")
ax.set_ylabel("montant moyen par commande (€)")

chemin = os.path.join(tempfile.mkdtemp(), "panier-moyen.png")
fig.savefig(chemin, dpi=150, bbox_inches="tight")

print("ordre des barres :", [t.get_text() for t in ax.get_xticklabels()])
print("hauteurs = moyennes :", np.allclose([b.get_height() for b in barres], moy.values))
print("axe vertical démarre à :", ax.get_ylim()[0])
print("fichier enregistré :", os.path.getsize(chemin) > 0)
plt.close(fig)

```

```

ordre des barres : ['Boutique', 'Site', 'Réseaux']
hauteurs = moyennes : True
axe vertical démarre à : 0.0
fichier enregistré : True

```

Les moyennes par canal sont celles obtenues au 4.1.9 (Boutique devant le Site, puis Réseaux). **Défauts d'une version truquée :** (1) un axe tronqué (par exemple de 45 à 75 €) ferait paraître la barre de la Boutique plusieurs fois plus haute que celle de Réseaux alors que l'écart réel est de l'ordre de 50 % ; (2) un camembert en 3D déforme les aires par la perspective et force l'œil à comparer des angles, ce qu'il fait très mal ; pour trois catégories, des barres triées sont toujours plus lisibles.

Corrigé 4.13

- (a) Cas à tester : boutique à n'importe quel montant (0) ; Site à 99,99 (7) ; Site à **100,00 exactement** (0, la borne) ; Site à 150 (0) ; Réseaux à 50 (7) ; canal inconnu (erreur). Les **bornes** sont l'endroit où se cachent les bogues. (b) La version fautive utilise > : un panier à 100,00 € pile paierait 7 €.

```

cat > livraison_frais.py <<'FIN'
def frais_livraison(canal, total_ttc):
    """Frais de livraison en € : 0 en boutique ; 7 € sinon, offerts dès 100 € TTC."""
    if canal == "Boutique":
        return 0.0
    if canal in ("Site", "Réseaux"):
        return 0.0 if total_ttc > 100 else 7.0 # <-- défaut volontaire : devrait être >=
    raise ValueError(f"canal inconnu : {canal!r}")
FIN

```

On écrit ensuite les tests, avec `pytest.mark.parametrize` pour rejouer le même test sur plusieurs cas, puis on les lance :

```

cat > test_livraison_frais.py <<'FIN'
import pytest
from livraison_frais import frais_livraison

@pytest.mark.parametrize("canal, total, attendu", [
    ("Boutique", 20.0, 0.0),
    ("Site", 99.99, 7.0),
    ("Site", 100.0, 0.0), # la borne : 100 compris
    ("Site", 150.0, 0.0),
    ("Réseaux", 50.0, 7.0),
])

```

```

))
def test_frais(canal, total, attendu):
    assert frais_livraison(canal, total) == attendu

def test_canal_inconnu():
    with pytest.raises(ValueError):
        frais_livraison("Telephone", 10.0)
FIN

python -m pytest -q --color=no --tb=short -p no:cacheprovider test_livraison_frais.py 2>&1 | sed -E
↪ 's/ in [0-9.]+s//'
```

```

..F... [100%]
===== FAILURES =====
_____ test_frais[Site-100.0-0.0] _____
test_livraison_frais.py:12: in test_frais
    assert frais_livraison(canal, total) == attendu
E   AssertionError: assert 7.0 == 0.0
E   + where 7.0 = frais_livraison('Site', 100.0)
===== short test summary info =====
FAILED test_livraison_frais.py::test_frais[Site-100.0-0.0] - AssertionError: ...
1 failed, 5 passed
```

Le test de la borne a trouvé le défaut : pour 100,00 €, la fonction renvoie 7 au lieu de 0 ; les cinq autres tests passent, preuve qu'un test « du milieu » n'aurait rien vu. (c) On corrige ($\geq$) et on relance :

```

sed -i 's/total_ttc > 100 else/total_ttc >= 100 else/; s/ *# <-- défaut volontaire.*//'
↪ livraison_frais.py
python -m pytest -q --color=no --tb=short -p no:cacheprovider test_livraison_frais.py 2>&1 | sed -E
↪ 's/ in [0-9.]+s//'
```

```

..... [100%]
6 passed
```

À retenir. On teste les **bornes** (juste en dessous, pile, juste au-dessus) et les **cas d'erreur**. Ces tests, écrits une fois, protégeront la règle des 100 € de toute régression future.

Corrigé 4.14

- (a) Le nombre de paires parmi n éléments est $\binom{n}{2} = n(n-1)/2$: pour $n = 400$, $400 \times 399/2 = 79\,800$; pour $n = 800$, $800 \times 799/2 = 319\,600$. Le facteur est $319\,600/79\,800 \approx 4,005$: **doubler n quadruple** le travail, signature d'un coût en $O(n^2)$. (b)–(d) :

```

from math import comb
import time

ids = df["id_client"].tolist() # les 400 numéros de client (4.4.5)

def paires_double_boucle(ids):
    tours, paires = 0, 0
    for i in range(len(ids)):
        for j in range(i + 1, len(ids)):
            tours += 1
            if ids[i] == ids[j]:
                paires += 1
    return paires, tours
```

```
def paires_dictionnaire(ids):
    compteurs = {}
    for c in ids:
        compteurs[c] = compteurs.get(c, 0) + 1
    return sum(k * (k - 1) // 2 for k in compteurs.values()), len(ids)
```

On compare les deux versions sur les 200 premières puis sur les 400 commandes, et on mesure :

```
for n in (200, 400):
    p1, tours = paires_double_boucle(ids[:n])
    p2, passages = paires_dictionnaire(ids[:n])
    print(f"n = {n} : double boucle {tours:>6} tours | dictionnaire {passages:>3} passages | "
          f"paires de même client : {p1} = {p2}")

t0 = time.perf_counter(); paires_double_boucle(ids); t1 = time.perf_counter()
paires_dictionnaire(ids); t2 = time.perf_counter()
print("la version dictionnaire est plus rapide :", (t2 - t1) < (t1 - t0))
print("rapport des tours quand n double (200 -> 400) :", round(79800 / 19900, 2), "pour la double
↪ boucle, 2.0 pour le dictionnaire")
print("(e) pour n = 1 000 000 :", f"{comb(1_000_000, 2):,}".replace(",", " "), "tours de double
↪ boucle")
```

```
n = 200 : double boucle 19900 tours | dictionnaire 200 passages | paires de même client : 212 = 212
n = 400 : double boucle 79800 tours | dictionnaire 400 passages | paires de même client : 702 = 702
la version dictionnaire est plus rapide : True
rapport des tours quand n double (200 -> 400) : 4.01 pour la double boucle, 2.0 pour le dictionnaire
(e) pour n = 1 000 000 : 499 999 500 000 tours de double boucle
```

Les deux versions comptent **le même nombre de paires** (702 parmi les 400 commandes, 212 parmi les 200 premières), mais l'une fait 79 800 tours et l'autre 400 passages. Quand n passe de 200 à 400, la double boucle est ≈ 4 fois plus longue ($19\,900 \rightarrow 79\,800$), le dictionnaire seulement 2 fois. À un million de commandes la double boucle ferait environ 5×10^{11} tours, soit des heures voire des jours de calcul contre une fraction de seconde pour le dictionnaire : **même résultat, deux mondes de coûts**. Seule la mesure du temps dépend de la machine ; le booléen « le dictionnaire est plus rapide » vaut True partout, et les comptes de tours sont exacts.

Chapitre 5 : Bases de données et SQL

— exercices et applications

Ce chapitre du cahier accompagne le chapitre 5 du livre. Les **applications** sont de petites études guidées (reconstruire la base, repérer les meilleurs clients, segmenter la clientèle...) ; les **exercices** sont à chercher seul(e) avant de lire les **corrigés**. Tout s'appuie sur la base de la boutique du livre (donnees/boutique.db : 80 clients, 16 produits, 400 commandes, 693 lignes de commande). **La date « du jour » est le 31 décembre 2025**. Prérequis : les sections 5.1 à 5.4 du livre.

Préparation

Le cahier est autonome : nous ouvrons une **copie en mémoire** de la base fournie, pour pouvoir y créer et supprimer des tables d'essai sans jamais modifier le fichier d'origine. Les blocs sql s'exécutent sur cette connexion con.

```
import sqlite3
import pandas as pd

con = sqlite3.connect(":memory:")
sqlite3.connect("donnees/boutique.db").backup(con)      # copie de la base fournie
con.execute("PRAGMA foreign_keys = ON")                # SQLite n'applique les clés étrangères que sur
↳ demande
print([r[0] for r in con.execute("SELECT name FROM sqlite_master WHERE type = 'table' ORDER BY
↳ name")])
```

```
['categories', 'clients', 'commandes', 'lignes_commande', 'produits']
```

Applications

Application 5.1 — Reconstruire la base de la boutique

Objectif. Comprendre comment la base du livre a été fabriquée : écrire son squelette, puis la régénérer avec le script fourni et vérifier qu'elle est identique au fichier.

Étape 1 : le squelette. Les trois premières tables (la **catégorie**, le **produit** et le **client**) ne dépendent que d'elles-mêmes ou de tables déjà créées. Créons-les dans une base vide.

```
squelette = sqlite3.connect(":memory:")
squelette.execute("PRAGMA foreign_keys = ON")
squelette.executescript("""
CREATE TABLE categories (id_categorie INTEGER PRIMARY KEY, nom TEXT NOT NULL UNIQUE);
CREATE TABLE produits (
    id_produit INTEGER PRIMARY KEY, nom TEXT NOT NULL,
    id_categorie INTEGER NOT NULL REFERENCES categories(id_categorie),
    prix_catalogue REAL NOT NULL CHECK (prix_catalogue > 0));
CREATE TABLE clients (
    id_client INTEGER PRIMARY KEY, prenom TEXT NOT NULL, nom TEXT NOT NULL, ville TEXT NOT NULL,
    date_inscription TEXT NOT NULL, telephone TEXT,
    id_parrain INTEGER REFERENCES clients(id_client));
""")
```

Étape 2 : les tables qui référencent les précédentes. Une commande référence un client ; une ligne de commande référence une commande et un produit, et sa clé primaire est le couple (commande, produit).

```
squelette.executescript("""
CREATE TABLE commandes (
    id_commande INTEGER PRIMARY KEY,
    id_client INTEGER NOT NULL REFERENCES clients(id_client),
    date_commande TEXT NOT NULL,
    canal TEXT NOT NULL CHECK (canal IN ('Réseaux', 'Site', 'Boutique')),
    montant REAL NOT NULL, delai_livraison INTEGER NOT NULL,
    satisfaction INTEGER CHECK (satisfaction BETWEEN 1 AND 5));
CREATE TABLE lignes_commande (
    id_commande INTEGER NOT NULL REFERENCES commandes(id_commande),
    id_produit INTEGER NOT NULL REFERENCES produits(id_produit),
    quantite INTEGER NOT NULL CHECK (quantite > 0), prix_unitaire REAL NOT NULL,
    PRIMARY KEY (id_commande, id_produit));
""")
print([r[0] for r in squelette.execute("SELECT name FROM sqlite_master WHERE type = 'table' ORDER BY
↵ name")])
```

```
['categories', 'clients', 'commandes', 'lignes_commande', 'produits']
```

Étape 3 : le remplissage. Les lignes de cette base ne sont pas tapées à la main : elles sont **simulées** par la fonction `construire()` du script `build/base_sql.py`, avec une graine fixée. Elle lit les 400 commandes du fichier `donnees/commandes.csv`, leur attribue une date et un client, invente la liste des clients (dont quelques-uns qui ne commandent jamais) et compose chaque commande de un à trois produits, de sorte que la somme (quantité × prix) retombe sur le montant. Régénérons la base et comparons-la au fichier fourni, table par table.

```
import sys
sys.path.insert(0, "build")
from base_sql import construire

neuve = construire()
for table in ["categories", "produits", "clients", "commandes", "lignes_commande"]:
    a = neuve.execute(f"SELECT * FROM {table} ORDER BY 1, 2").fetchall()
    b = con.execute(f"SELECT * FROM {table} ORDER BY 1, 2").fetchall()
    print(f"{table:16s} {len(a):4d} lignes ; identique au fichier fourni : {a == b}")
```

```
categories      4 lignes ; identique au fichier fourni : True
produits        16 lignes ; identique au fichier fourni : True
clients         80 lignes ; identique au fichier fourni : True
commandes       400 lignes ; identique au fichier fourni : True
lignes_commande 693 lignes ; identique au fichier fourni : True
```

Étape 4 : contrôler la cohérence. Une base fabriquée doit être **vérifiée**. Trois contrôles : chaque commande retombe sur son montant ; chaque client référencé existe ; toutes les dates sont dans l'année 2025.

```
SELECT
  (SELECT COUNT(*) FROM commandes AS c
   WHERE ABS(c.montant - (SELECT SUM(quantite * prix_unitaire) FROM lignes_commande AS l
                           WHERE l.id_commande = c.id_commande)) > 0.005) AS
    ↳ commandes_incoherentes,
  (SELECT COUNT(*) FROM commandes WHERE id_client NOT IN (SELECT id_client FROM clients)) AS
    ↳ clients_inconnus,
  (SELECT COUNT(*) FROM commandes WHERE date_commande NOT BETWEEN '2025-01-01' AND '2025-12-31') AS
    ↳ dates_hors_2025;
```

```
commandes_incoherentes  clients_inconnus  dates_hors_2025
                        0                  0                  0
```

Pour aller plus loin. Changez la graine dans build/base_sql.py (copie de travail !) et relancez : quelles statistiques de la boutique restent stables (le nombre de commandes ? le panier moyen ?) et lesquelles bougent (les noms ? les clients dormants ?) ?

Application 5.2 — Importer un CSV et lui donner un vrai schéma

Objectif. Partir d'un fichier CSV brut, comme on en reçoit en pratique, et en faire une table **conçue**, avec clé primaire et contraintes.

Étape 1 : l'import rapide. pandas charge le CSV dans SQLite d'un trait ; SQLite devine les types, mais ne pose aucune règle.

```
brut = sqlite3.connect(":memory:")
pd.read_csv("donnees/commandes.csv").to_sql("commandes_csv", brut, index=False)
print(brut.execute("SELECT sql FROM sqlite_master").fetchone()[0])
```

```
CREATE TABLE "commandes_csv" (
  "canal" TEXT,
  "montant" REAL,
  "livraison" INTEGER,
  "satisfaction" INTEGER
)
```

Étape 2 : la table conçue. On crée une vraie table avec un numéro, des types, et des règles de gestion, puis on y copie les données du CSV avec INSERT ... SELECT.

```
brut.executescript("""
CREATE TABLE commandes_propres (
  id_commande INTEGER PRIMARY KEY,
  canal TEXT NOT NULL CHECK (canal IN ('Réseaux', 'Site', 'Boutique')),
  montant REAL NOT NULL CHECK (montant > 0),
  livraison INTEGER NOT NULL CHECK (livraison >= 0),
  satisfaction INTEGER CHECK (satisfaction BETWEEN 1 AND 5));
INSERT INTO commandes_propres (canal, montant, livraison, satisfaction)
SELECT canal, montant, livraison, satisfaction FROM commandes_csv;
""")
print(brut.execute("SELECT COUNT(*), ROUND(AVG(montant), 2) FROM commandes_propres").fetchone())
```

```
(400, 60.25)
```

Étape 3 : la table se défend. Essayons d'y glisser des lignes absurdes : un montant négatif, une note de 9, un canal inconnu.

```
for ligne in [ "('Site', -5, 2, 4)", "('Site', 50, 2, 9)", "('Marché', 50, 2, 4)"]:
    try:
        brut.execute(f"INSERT INTO commandes_propres (canal, montant, livraison, satisfaction) VALUES
↳ {ligne}")
        print(ligne, "-> accepté (!)")
    except sqlite3.IntegrityError as erreur:
        print(ligne, "-> refusé :", erreur)
```

```
('Site', -5, 2, 4) -> refusé : CHECK constraint failed: montant > 0
('Site', 50, 2, 9) -> refusé : CHECK constraint failed: satisfaction BETWEEN 1 AND 5
('Marché', 50, 2, 4) -> refusé : CHECK constraint failed: canal IN ('Réseaux', 'Site', 'Boutique')
```

Questions. (1) Quelle contrainte a refusé chacune des trois lignes ? (2) Quels autres contrôles ajouteriez-vous (une borne haute pour le montant ? une date ?) ? (3) Pourquoi la table du CSV ne pouvait-elle pas refuser ces lignes ?

Application 5.3 — Les dix meilleurs clients

Contexte. La gérante veut connaître ses dix meilleurs clients. On joint clients et commandes, on regroupe par client, on trie par chiffre d'affaires décroissant.

```
SELECT cl.id_client,
       cl.prenom || ' ' || cl.nom AS client,
       cl.ville,
       COUNT(*) AS commandes,
       ROUND(SUM(c.montant), 2) AS chiffre_affaires,
       ROUND(AVG(c.montant), 2) AS panier_moyen,
       MAX(c.date_commande) AS derniere_commande
FROM clients AS cl
JOIN commandes AS c ON c.id_client = cl.id_client
GROUP BY cl.id_client
ORDER BY chiffre_affaires DESC
LIMIT 10;
```

id_client	client	ville	commandes	chiffre_affaires	panier_moyen	derniere_commande
2	Sam Fontaine	Ville A	23	1874.3	81.49	2025-12-24
1	Yann Lambert	Ville C	28	1701.7	60.77	2025-12-20
47	Inès Michel	Ville C	22	1317.3	59.88	2025-12-17
11	Luc Garcia	Ville H	18	1218.9	67.72	2025-12-29
45	Anna Bernard	Ville A	15	1009.3	67.29	2025-12-25
17	Léa Fontaine	Ville A	15	937.6	62.51	2025-12-22
39	Lou Simon	Ville F	10	833.1	83.31	2025-12-10
27	Zoé Garcia	Ville G	13	808.9	62.22	2025-12-28
14	Jules Garcia	Ville G	5	566.0	113.20	2025-11-09
51	Elsa Girard	Ville D	12	552.0	46.00	2025-12-10

Lecture. Le meilleur client par le chiffre d'affaires est Sam Fontaine (1 874 €, panier moyen de 81 €) ; le plus fidèle est Yann Lambert (28 commandes). Le dixième (Elsa Girard) a un chiffre d'affaires de 552 €, **plus de trois fois moins** que le premier : la clientèle est très inégale. Remarquez le GROUP BY cl.id_client et non GROUP BY cl.nom : regrouper par nom aurait fusionné les homonymes.

À faire. (1) Ajoutez la part de chaque client dans le chiffre d'affaires total (utilisez une sous-requête, 5.2.6). (2) Ajoutez le canal le plus utilisé par le client (indice : une fonction fenêtre ou une sous-requête corrélée).

Application 5.4 — Chiffre d'affaires par mois et par canal

Contexte. On veut un tableau avec un canal par colonne : un **tableau croisé** (*pivot*). L'astuce `SUM(CASE WHEN ... THEN ... ELSE 0 END)` place chaque canal dans sa propre colonne.

```
SELECT strftime('%Y-%m', date_commande) AS mois,
       ROUND(SUM(CASE WHEN canal = 'Réseaux' THEN montant ELSE 0 END)) AS reseaux,
       ROUND(SUM(CASE WHEN canal = 'Site'     THEN montant ELSE 0 END)) AS site,
       ROUND(SUM(CASE WHEN canal = 'Boutique' THEN montant ELSE 0 END)) AS boutique,
       ROUND(SUM(montant)) AS total
FROM commandes
GROUP BY mois
ORDER BY mois;
```

mois	reseaux	site	boutique	total
2025-01	262.0	379.0	356.0	996.0
2025-02	382.0	203.0	582.0	1168.0
2025-03	737.0	824.0	126.0	1687.0
2025-04	367.0	919.0	557.0	1843.0
2025-05	693.0	679.0	942.0	2314.0
2025-06	816.0	723.0	951.0	2491.0
2025-07	647.0	992.0	932.0	2571.0
2025-08	601.0	1161.0	672.0	2435.0
2025-09	454.0	675.0	584.0	1713.0
2025-10	324.0	531.0	416.0	1272.0
2025-11	551.0	759.0	974.0	2284.0
2025-12	928.0	960.0	1437.0	3325.0

Lecture. Le site est en tête sept mois sur douze, la boutique cinq mois (février, mai, juin, novembre et décembre, où elle réalise 1 437 € sur 3 325 €), le canal Réseaux jamais ; son meilleur mois est décembre (928 €). Les trois colonnes se somment à la colonne total : c'est un contrôle de bon sens.

À faire. Ajoutez une colonne « part du canal Réseaux » en pourcentage du total du mois, puis repérez le mois où elle est la plus forte.

Application 5.5 — Repérer les clients « dormants »

Contexte. Une cliente qui achetait régulièrement et ne revient plus est un signe d'alerte (on parle de *churn* quand elle part pour de bon). Un client **dormant** a passé **au moins 3 commandes** mais **aucune depuis plus de 90 jours** avant la date d'arrêt (le 31 décembre 2025). Le seuil se calcule : `date('2025-12-31', '-90 days')` donne le 2 octobre 2025.

```
SELECT cl.id_client,
       cl.prenom || ' ' || cl.nom AS client,
       COUNT(*) AS commandes,
       MAX(c.date_commande) AS derniere_commande,
       CAST(julianday('2025-12-31') - julianday(MAX(c.date_commande)) AS INTEGER) AS jours_sans_achat
FROM clients AS cl
JOIN commandes AS c ON c.id_client = cl.id_client
GROUP BY cl.id_client
HAVING COUNT(*) >= 3 AND MAX(c.date_commande) < '2025-10-02'
ORDER BY jours_sans_achat DESC;
```

id_client	client	commandes	derniere_commande	jours_sans_achat
20	Adam Garcia	3	2025-05-07	238
62	Anna Faure	3	2025-06-16	198
37	Elsa Lambert	3	2025-07-21	163
16	Alex Lefebvre	5	2025-07-26	158
40	Inès Faure	3	2025-08-14	139
50	Lina Faure	4	2025-08-17	136
8	Théo Michel	6	2025-08-30	123

6	Hugo Lefebvre	6	2025-09-24	98
29	Noé Lefebvre	7	2025-09-26	96

Lecture. Neuf clients répondent à ces critères : voilà la liste de relance de la gérante. Le plus ancien (Adam Garcia, dernière commande le 7 mai) n'est pas revenu depuis 238 jours. Notez l'emploi de **HAVING** avec **deux conditions sur le groupe** : le nombre de commandes **et** la date de la dernière.

À faire. Ajoutez le chiffre d'affaires passé de chaque client dormant et classez la liste de relance par valeur décroissante : à qui écrire en premier ?

Application 5.6 — Segmenter la clientèle par quartiles de dépenses

Contexte. Les marketeurs aiment les segmentations de type **RFM** : **R**écence (depuis combien de temps le client n'a-t-il pas acheté ?), **F**réquence (combien de fois a-t-il acheté ?), **M**ontant (combien a-t-il dépensé ?). Deux étapes : une première CTE calcule les trois indicateurs par client ; la seconde utilise **NTILE(4)**, qui découpe les clients, triés par montant décroissant, en **4 groupes d'effectifs égaux** (quartiles).

```
WITH rfm AS (
    SELECT id_client,
           CAST(julianday('2025-12-31') - julianday(MAX(date_commande)) AS INTEGER) AS recence_jours,
           COUNT(*) AS frequence,
           ROUND(SUM(montant), 2) AS montant
    FROM commandes
    GROUP BY id_client
),
segments AS (
    SELECT *, NTILE(4) OVER (ORDER BY montant DESC) AS quartile
    FROM rfm
)
SELECT quartile,
       COUNT(*) AS clients,
       ROUND(MIN(montant)) AS depense_min,
       ROUND(MAX(montant)) AS depense_max,
       ROUND(SUM(montant)) AS depense_totale,
       ROUND(AVG(frequence), 1) AS achats_moyens,
       ROUND(AVG(recence_jours)) AS jours_depuis_dernier_achat
FROM segments
GROUP BY quartile
ORDER BY quartile;
```

quartile	clients	depense_min	depense_max	depense_totale	achats_moyens	jours_depuis_dernier_achat
1	17	427.0	1874.0	14063.0	12.6	20.0
2	17	270.0	427.0	5678.0	5.8	49.0
3	16	122.0	270.0	3086.0	3.6	66.0
4	16	33.0	122.0	1270.0	1.8	153.0

Lecture. Le quartile 1 (les 17 plus gros clients) dépense **14 063 € sur 24 098**, soit **58 %** du chiffre d'affaires, et ces clients sont revenus il y a 20 jours en moyenne. Le quartile 4 (16 clients) ne pèse que 5 % et n'est pas revenu depuis 153 jours en moyenne. On retrouve la loi de **Pareto** (« 80-20 », ici plutôt « 25-58 ») : une minorité de clients fait une majorité du chiffre d'affaires. Cette information change la stratégie : chouchouter le quartile 1, relancer le quartile 3, ne pas s'acharner sur le quartile 4.

À faire. Remplacez **NTILE(4)** par **NTILE(10)** : quelle part du chiffre d'affaires fait le meilleur décile ?

Application 5.7 — Le temps dans les données : un calendrier complet, et le délai entre commandes

Contexte. Une table de ventes ne contient que les jours où il y a eu des ventes : les jours **sans** vente n'apparaissent pas, ce qui fausse les moyennes quotidiennes. La parade classique : générer **tous les jours** de l'année avec une CTE récursive, puis les joindre aux commandes par un LEFT JOIN.

```
WITH RECURSIVE jours(d) AS (
    SELECT '2025-01-01'
    UNION ALL
    SELECT date(d, '+1 day') FROM jours WHERE d < '2025-12-31'
),
ventes_par_jour AS (
    SELECT j.d, COUNT(c.id_commande) AS commandes
    FROM jours AS j
    LEFT JOIN commandes AS c ON c.date_commande = j.d
    GROUP BY j.d
)
SELECT strftime('%Y-%m', d) AS mois,
       COUNT(*) AS jours,
       SUM(commandes = 0) AS jours_sans_commande,
       ROUND(AVG(commandes), 2) AS commandes_par_jour
FROM ventes_par_jour
GROUP BY mois
ORDER BY mois;
```

mois	jours	jours_sans_commande	commandes_par_jour
2025-01	31	19	0.52
2025-02	28	15	0.71
2025-03	31	15	0.81
2025-04	30	7	1.20
2025-05	31	9	1.16
2025-06	30	10	1.10
2025-07	31	7	1.35
2025-08	31	7	1.42
2025-09	30	8	1.03
2025-10	31	18	0.65
2025-11	30	7	1.33
2025-12	31	10	1.84

Lecture. En janvier, **19 jours sur 31** se sont passés sans la moindre commande, et ils n'apparaissent dans aucune table de ventes. Sans le calendrier, la « moyenne par jour » de décembre serait calculée seulement sur les jours où l'on a vendu (donc **surestimée**) ; avec lui, elle l'est sur les 31 jours. C'est un exemple concret de **biais de sélection** dans une requête : les jours sans vente sont absents du tableau, donc invisibles.

Deuxième partie : le délai entre deux commandes d'un même client. Pour chaque commande, LAG(date_commande) calculé *dans la partition du client* donne la date de sa commande précédente. Voici ce que cela donne pour le client n° 1.

```
SELECT id_client, date_commande,
       LAG(date_commande) OVER (PARTITION BY id_client ORDER BY date_commande, id_commande) AS
       ↳ commande_precedente,
       CAST(julianday(date_commande)
            - julianday(LAG(date_commande) OVER (PARTITION BY id_client ORDER BY date_commande,
            ↳ id_commande))
            AS INTEGER) AS jours_ecoules
FROM commandes
WHERE id_client = 1
ORDER BY date_commande, id_commande
LIMIT 6;
```

id_client	date_commande	commande_precedente	jours_ecoules
1	2025-01-05	NaN	NaN
1	2025-02-01	2025-01-05	27.0
1	2025-02-11	2025-02-01	10.0
1	2025-02-14	2025-02-11	3.0
1	2025-03-08	2025-02-14	22.0
1	2025-03-09	2025-03-08	1.0

À faire. Combien de jours sans commande y a-t-il au total sur l'année ? Quel mois a la plus longue série de jours consécutifs sans commande (indice : numérotez les jours, 5.3.2) ?

Application 5.8 — Le réseau de parrainage : chaînes et chiffre d'affaires généré

Contexte. Le livre a compté les filleuls de chaque ambassadeur. On veut maintenant voir **la chaîne complète** de parrainage de chaque membre d'un réseau, et chiffrer ce que le réseau rapporte. Le texte chemin recolle les prénoms le long de la branche (il permet aussi de trier dans l'ordre de parcours d'un arbre).

```
WITH RECURSIVE arbre(id_client, profondeur, chemin) AS (
    SELECT id_client, 0, prenom || ' ' || nom
    FROM clients
    WHERE id_client = 10
    UNION ALL
    SELECT c.id_client, a.profondeur + 1, a.chemin || ' > ' || c.prenom || ' ' || c.nom
    FROM clients AS c
    JOIN arbre AS a ON c.id_parrain = a.id_client
)
SELECT id_client, profondeur, chemin
FROM arbre
ORDER BY chemin;
```

id_client	profondeur	chemin
10	0	Lou Michel
14	1	Lou Michel > Jules Garcia
23	2	Lou Michel > Jules Garcia > Hugo Girard
28	3	Lou Michel > Jules Garcia > Hugo Girard > Anna Martin
55	4	Lou Michel > Jules Garcia > Hugo Girard > Anna Martin > Théo Girard
18	1	Lou Michel > Sam Michel

Lecture. Le chemin de chaque client donne **toute sa chaîne de parrainage** depuis l'ambassadeur : Théo Girard (profondeur 4) est arrivé par Anna Martin, venue par Hugo Girard, venu par Jules Garcia, venu par Lou Michel. Remarquez que Lou Michel (client n° 10) **n'a jamais passé de commande** : elle figure parmi les quatorze clients inscrits sans achat. Pourtant elle a amené cinq clients : une requête qui ne regarderait que les achats la jugerait sans valeur.

À faire. Calculez le chiffre d'affaires **généré** par ce réseau (la somme des commandes de ses membres, sans la racine) : c'est la base d'une prime au parrainage. (Le corrigé de l'exercice 5.10 fait ce calcul pour les trois plus gros réseaux.)

Application 5.9 — Mesurer le gain d'un index

Contexte. Sur 400 lignes, un index ne change rien. Pour **le voir**, on construit une table de 500 000 lignes avec une CTE récursive, puis on chronomètre la même recherche avant et après la création d'un index.

```
import time

con.executescript("""
CREATE TABLE ex_gros (id INTEGER PRIMARY KEY, id_client INTEGER, montant REAL);
```

```

WITH RECURSIVE s(i) AS (SELECT 1 UNION ALL SELECT i + 1 FROM s WHERE i < 500000)
INSERT INTO ex_gros SELECT i, (i * 7919) % 50000, ROUND(10 + (i * 31) % 190, 2) FROM s;
""")

def duree(repetitions=20):
    debut = time.perf_counter()
    for k in range(repetitions):
        con.execute("SELECT COUNT(*), SUM(montant) FROM ex_gros WHERE id_client = ?", (123 +
↪ k,)).fetchone()
    return (time.perf_counter() - debut) / repetitions

sans_index = duree()
con.execute("CREATE INDEX idx_gros_client ON ex_gros(id_client)")
avec_index = duree()
print("lignes dans la table :", con.execute("SELECT COUNT(*) FROM ex_gros").fetchone()[0])
print("l'index accélère la recherche :", sans_index > 10 * avec_index)
con.execute("DROP TABLE ex_gros")

```

```

lignes dans la table : 500000
l'index accélère la recherche : True

```

Lecture. Les durées exactes dépendent de votre machine ; le programme n'affiche donc qu'un **verdict** robuste (le gain dépasse un facteur dix). Affichez `sans_index / avec_index` chez vous pour connaître le facteur exact sur votre machine. Vérifiez aussi, avec `EXPLAIN QUERY PLAN`, que la requête passe de `SCAN` à `SEARCH`.

À faire. Mesurez l'effet inverse : combien de temps prend l'insertion de 100 000 lignes avec et sans index ? (L'index n'est pas gratuit : voir 5.4.5.)

Application 5.10 — De la base relationnelle aux documents JSON

Contexte. Une base de documents range une commande dans **un seul document**. On reconstruit ces documents à partir de nos cinq tables, on les interroge en Python, et on compare avec la réponse SQL.

```

import json

lignes = {}
for i, produit, categorie, quantite, prix in con.execute("""
    SELECT l.id_commande, p.nom, ca.nom, l.quantite, l.prix_unitaire
    FROM lignes_commande AS l
    JOIN produits AS p ON p.id_produit = l.id_produit
    JOIN categories AS ca ON ca.id_categorie = p.id_categorie
    ORDER BY l.id_commande, l.id_produit"""):
    lignes.setdefault(i, []).append({"produit": produit, "categorie": categorie, "quantite":
↪ quantite, "prix": prix})

documents = [
    {"_id": i, "date": date, "canal": canal, "montant": montant,
     "client": {"id": id_client, "nom": f"{prenom} {nom}", "ville": ville}, "lignes": lignes[i]}
    for i, date, canal, montant, id_client, prenom, nom, ville in con.execute("""
        SELECT c.id_commande, c.date_commande, c.canal, c.montant, cl.id_client, cl.prenom, cl.nom,
↪ cl.ville
        FROM commandes AS c JOIN clients AS cl ON cl.id_client = c.id_client ORDER BY c.id_commande""")
]
print(len(documents), "documents")
print(json.dumps(documents[2], ensure_ascii=False))

```

```

400 documents
{"_id": 3, "date": "2025-01-04", "canal": "Réseaux", "montant": 88.2, "client": {"id": 3, "nom":
↪ "Adam Michel", "ville": "Ville F"}, "lignes": [{"produit": "Bol en céramique", "categorie":
↪ "Poterie", "quantite": 1, "prix": 17.84}, {"produit": "Vase peint à la main", "categorie":
↪ "Poterie", "quantite": 1, "prix": 64.42}, {"produit": "Savon à l'huile d'olive", "categorie":
↪ "Cosmétiques", "quantite": 1, "prix": 5.94}]}

```

Interroger les documents. Les commandes d'au moins 100 € qui contiennent un bijou, en Python sur les documents, puis en SQL sur les tables : les deux réponses doivent coïncider.

```
avec_bijou = [d["_id"] for d in documents
               if d["montant"] >= 100 and any(l["categorie"] == "Bijoux" for l in d["lignes"])]
sql = """SELECT COUNT(DISTINCT c.id_commande) FROM commandes AS c
        JOIN lignes_commande AS l ON l.id_commande = c.id_commande
        JOIN produits AS p ON p.id_produit = l.id_produit
        WHERE c.montant >= 100 AND p.id_categorie = 3"""
print("version documents (Python) :", len(avec_bijou))
print("version relationnelle (SQL):", con.execute(sql).fetchone()[0])
print("documents à modifier si le client n° 2 déménage :", sum(1 for d in documents if
↪ d["client"]["id"] == 2))
```

```
version documents (Python) : 27
version relationnelle (SQL): 27
documents à modifier si le client n° 2 déménage : 23
```

Lecture. Même résultat, deux philosophies : l'une **reconstruit** les liens à la lecture (jointure), l'autre les a **pré-assemblés** à l'écriture (imbrication). Mais l'imbrication a un prix : la ville du client est recopiée dans chacun de ses 23 documents, et il faudrait les modifier tous : c'est l'anomalie de mise à jour du 5.4.1.

À faire. Stockez ces documents dans une table SQLite (CREATE TABLE ex_docs (doc TEXT)) et calculez le panier moyen par canal avec json_extract.

Application 5.11 — Un cache en Python

Contexte. Le cas d'usage numéro un des bases clé-valeur comme Redis est le **cache** : stocker le résultat d'une requête lente pour ne pas la recalculer à chaque visite. Voici le mécanisme avec un simple dictionnaire.

```
cache = {}
nb_requetes_sql = 0

def chiffre_affaires_du_canal(canal):
    global nb_requetes_sql
    cle = f"ca:{canal}"
    if cle in cache:
        return cache[cle]
    nb_requetes_sql += 1
    cache[cle] = con.execute("SELECT ROUND(SUM(montant)) FROM commandes WHERE canal = ?",
↪ (canal,)).fetchone()[0]
    return cache[cle]

for canal in ["Site", "Site", "Boutique", "Site", "Boutique", "Réseaux", "Site"]:
    print(f"{canal:10s}", chiffre_affaires_du_canal(canal))
print("7 demandes, requêtes SQL réellement exécutées :", nb_requetes_sql)
```

```
Site      8807.0
Site      8807.0
Boutique  8528.0
Site      8807.0
Boutique  8528.0
Réseaux   6764.0
Site      8807.0
7 demandes, requêtes SQL réellement exécutées : 3
```

Lecture. Sept demandes, trois requêtes seulement. Reste le problème classique : **quand périmé le cache** ? Si une nouvelle commande arrive, la valeur en cache devient fausse.

À faire. Ajoutez une durée de vie à chaque entrée (stockez l'instant d'écriture avec la valeur et refusez les valeurs plus vieilles que n secondes). Qu'avez-vous réimplémenté ? (Réponse : le paramètre EX de Redis.)

Exercices

Cherchez d'abord seul(e) (sur papier, ou en écrivant la requête dans votre éditeur), vérifiez ensuite en exécutant, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse.

Exercice 5.1 ★ — Clés et contraintes (section 5.1 du livre)

La gérante veut enregistrer les **avis** des clients sur les produits. Un avis a un numéro, est écrit par **un** client sur **un** produit, à une date, avec une note de 1 à 5 et un commentaire facultatif. Un client ne peut laisser **qu'un seul avis par produit**. (a) Quelle est la clé primaire, quelles sont les clés étrangères ? (b) Écrivez le CREATE TABLE avec toutes les contraintes. (c) Vérifiez qu'un avis valide est accepté, et que trois avis invalides (note 6, doublon client/produit, produit inexistant) sont refusés.

Exercice 5.2 ★ — Filtrer (section 5.2 du livre)

Combien de commandes de **plus de 100 €** ont été passées **en boutique** en **décembre 2025** ? Affichez les trois plus grosses.

Exercice 5.3 ★ — Agréger et joindre (section 5.2 du livre)

Pour chaque **ville de client**, donnez le nombre de clients ayant commandé, le nombre de commandes, le chiffre d'affaires et le panier moyen, classées par chiffre d'affaires décroissant. Quelle ville a le meilleur panier moyen ? Est-ce aussi celle qui a le plus gros chiffre d'affaires ?

Exercice 5.4 ★★ — HAVING, jointure externe (section 5.2 du livre)

Quels produits se sont vendus à **moins de 40 unités** sur l'année ? Pour chacun, donnez les unités vendues et le chiffre d'affaires. Faut-il arrêter de vendre tous ces produits ?

Exercice 5.5 ★★ — Anti-jointure (section 5.2 du livre)

Les clients qui habitent près de la boutique (**Ville H, Ville C, Ville A**) mais n'y ont **jamais acheté** sont une cible de choix pour une invitation. Listez-les, de deux façons différentes (NOT EXISTS et LEFT JOIN ... IS NULL), et vérifiez que les deux donnent le même nombre.

Exercice 5.6 ★★ — NULL (section 5.2.7 du livre)

- (a) Pour chaque ville, quel est le **pourcentage de clients dont le téléphone est renseigné** ?
- (b) Un stagiaire écrit WHERE id_parrain != 3 pour compter les clients **qui n'ont pas été parrainés par le client n° 3**. Combien de lignes obtient-il ? Combien devrait-il en obtenir ? Corrigez.

Exercice 5.7 ★★ — Dates, et regard statistique (sections 5.2.7 et 3.4 du livre)

Quel est le **jour de la semaine** le plus chargé (en nombre de commandes et en chiffre d'affaires) ? Cette différence entre jours est-elle significative, ou du bruit ? (Utilisez un test du khi-deux d'adéquation, chapitre 3.)

Exercice 5.8 ★★★ — Fenêtre (section 5.3 du livre)

Pour **chaque catégorie**, quel est le produit au **plus gros chiffre d'affaires** ?

Exercice 5.9 ★★★ — Cte et fenêtres (section 5.3 du livre)

- (a) Quelle proportion des clients actifs a commandé **au moins deux fois** (taux de réachat) ? (b) Pour ces clients, comparez le montant de leur **première** commande à celui de leur **dernière** : combien dépensent plus à la fin qu'au début ?

Exercice 5.10 ★★★ — Récursivité (section 5.3.6 du livre)

Pour les trois clients ayant le plus de filleuls (directs ou non), calculez le **nombre de membres** de leur réseau (sans eux-mêmes) et le **chiffre d'affaires cumulé de ces membres**. Qui est l'ambassadeur le plus rentable ?

Exercice 5.11 ★★★ — Dépendances fonctionnelles (section 5.4 du livre)

La boutique enregistre ses livraisons dans une seule table : livraisons(id_livraison, id_commande, transporteur, tel_transporteur, ville_livraison, frais). Règles de gestion : une livraison concerne une commande, est assurée par un transporteur et part vers une ville ; un transporteur n'a qu'un seul numéro de téléphone ; les **frais ne dépendent que de la ville** de livraison (barème par ville). (a) Écrivez les dépendances fonctionnelles. (b) Déterminez la clé. (c) La table est-elle en 2FN ? en 3FN ? (d) Proposez une décomposition en 3FN, et vérifiez-la avec la fonction fermeture du 5.4.2.

Corrigés

Corrigé 5.1

- (a) La clé primaire est id_avis. Les clés étrangères sont id_client (vers clients) et id_produit (vers produits). La règle « un avis par client et par produit » se traduit par une contrainte UNIQUE (id_client, id_produit) : le couple est une **clé candidate** en plus de id_avis. (b) et (c) :

```
CREATE TABLE ex_avis (
  id_avis      INTEGER PRIMARY KEY,
  id_client    INTEGER NOT NULL REFERENCES clients(id_client),
  id_produit   INTEGER NOT NULL REFERENCES produits(id_produit),
  date_avis    TEXT     NOT NULL,
  note        INTEGER NOT NULL CHECK (note BETWEEN 1 AND 5),
  commentaire  TEXT,
  UNIQUE (id_client, id_produit)
);
INSERT INTO ex_avis VALUES (1, 2, 8, '2025-12-02', 5, 'Magnifique broderie');
```

```

essais = {
    "note de 6": "INSERT INTO ex_avis VALUES (2, 3, 8, '2025-12-03', 6, NULL)",
    "doublet client/produit": "INSERT INTO ex_avis VALUES (3, 2, 8, '2025-12-04', 4, NULL)",
    "produit inexistant": "INSERT INTO ex_avis VALUES (4, 2, 99, '2025-12-05', 4, NULL)",
}
for nom, requete in essais.items():
    try:
        con.execute(requete)
        print(f"{nom:24s} -> accepté (!)")
    except sqlite3.IntegrityError as erreur:
        print(f"{nom:24s} -> refusé : {erreur}")
print("avis enregistrés :", con.execute("SELECT COUNT(*) FROM ex_avis").fetchone()[0])
con.execute("DROP TABLE ex_avis")

```

```

note de 6          -> refusé : CHECK constraint failed: note BETWEEN 1 AND 5
doublet client/produit -> refusé : UNIQUE constraint failed: ex_avis.id_client, ex_avis.id_produit
produit inexistant  -> refusé : FOREIGN KEY constraint failed
avis enregistrés : 1

```

L'avis valide est enregistré, les trois autres sont refusés chacun par une contrainte différente (CHECK, UNIQUE, clé étrangère). Le commentaire est facultatif : c'est la seule colonne sans NOT NULL.

Corrigé 5.2

On filtre sur trois conditions (AND) ; les dates ISO se comparent comme du texte (5.1.4) : « à partir du 1er décembre » suffit, puisque la base s'arrête au 31.

```

SELECT COUNT(*) AS nb_commandes
FROM commandes
WHERE canal = 'Boutique' AND date_commande >= '2025-12-01' AND montant > 100;

```

```

nb_commandes
6

```

```

SELECT id_commande, date_commande, montant
FROM commandes
WHERE canal = 'Boutique' AND date_commande >= '2025-12-01' AND montant > 100
ORDER BY montant DESC
LIMIT 3;

```

```

id_commande date_commande  montant
362      2025-12-16      208.8
376      2025-12-22      147.6
391      2025-12-29      128.5

```

Corrigé 5.3

Il faut joindre clients (la ville) et commandes (les montants). Pour compter les *clients distincts*, COUNT(DISTINCT ...).

```

SELECT cl.ville,
       COUNT(DISTINCT cl.id_client) AS clients_actifs,
       COUNT(*) AS commandes,
       ROUND(SUM(c.montant)) AS chiffre_affaires,
       ROUND(AVG(c.montant), 2) AS panier_moyen
FROM clients AS cl
JOIN commandes AS c ON c.id_client = cl.id_client
GROUP BY cl.ville
ORDER BY chiffre_affaires DESC;

```

ville	clients_actifs	commandes	chiffre_affaires	panier_moyen
Ville A	13	100	6456.0	64.56
Ville C	9	80	4619.0	57.73
Ville H	9	49	3407.0	69.53
Ville G	7	38	2200.0	57.89
Ville D	6	41	2101.0	51.25
Ville F	6	34	2043.0	60.09
Ville B	8	31	1735.0	55.95
Ville E	8	27	1538.0	56.96

Ville A est en tête pour le chiffre d'affaires (6 456 €) mais pas pour le panier : c'est **Ville H** qui a le meilleur panier moyen (69,53 €), avec un nombre de commandes beaucoup plus faible (49 contre 100). Le chiffre d'affaires est le produit *nombre de commandes* × *panier moyen* : un fort volume de petits paniers peut battre un faible volume de gros paniers.

Corrigé 5.4

LEFT JOIN pour ne pas perdre un éventuel produit **jamais vendu** (il aurait 0 unité, ou NULL : voir COALESCE). Le filtre sur une valeur agrégée s'écrit avec HAVING.

```
SELECT p.nom                                AS produit,
       COALESCE(SUM(l.quantite), 0)         AS unites,
       ROUND(COALESCE(SUM(l.quantite * l.prix_unitaire), 0)) AS chiffre_affaires
FROM produits AS p
LEFT JOIN lignes_commande AS l ON l.id_produit = p.id_produit
GROUP BY p.id_produit
HAVING COALESCE(SUM(l.quantite), 0) < 40
ORDER BY unites;
```

produit	unites	chiffre_affaires
Petit tapis	13	1593.0
Vase peint à la main	37	2411.0
Écharpe en soie	37	2031.0

Trois produits. Mais **faible volume ne veut pas dire faible intérêt** : le petit tapis, à 120 € l'unité, ne s'est vendu qu'à 13 exemplaires et rapporte pourtant 1 593 €, plus que bien des produits très vendus. Le vase peint et l'écharpe en soie (37 unités chacun) sont même parmi les produits au plus gros chiffre d'affaires du magasin. Arrêter de les vendre serait une erreur : le bon indicateur dépend de la question (rotation, chiffre d'affaires, marge). Aucun de nos produits n'est resté invendu.

Corrigé 5.5

Version NOT EXISTS : on garde les clients de ces villes pour lesquels il n'existe **aucune** commande en boutique. Version LEFT JOIN : on joint **seulement** les commandes de boutique (la condition sur le canal va dans le ON, pas dans le WHERE !), puis on garde ceux qui n'ont aucun partenaire.

```
SELECT cl.id_client, cl.prenom, cl.nom, cl.ville
FROM clients AS cl
WHERE cl.ville IN ('Ville H', 'Ville C', 'Ville A')
AND NOT EXISTS (SELECT 1 FROM commandes AS c
                WHERE c.id_client = cl.id_client AND c.canal = 'Boutique')
ORDER BY cl.id_client;
```

id_client	prenom	nom	ville
10	Lou	Michel	Ville H
20	Adam	Garcia	Ville H
28	Anna	Martin	Ville A
32	Théo	Roux	Ville C

55	Théo	Girard	Ville C
65	Elsa	Girard	Ville H
75	Lina	Girard	Ville C
78	Léa	Lefebvre	Ville H

```
SELECT COUNT(*) AS avec_left_join
FROM clients AS cl
LEFT JOIN commandes AS c ON c.id_client = cl.id_client AND c.canal = 'Boutique'
WHERE cl.ville IN ('Ville H', 'Ville C', 'Ville A')
AND c.id_commande IS NULL;
```

```
avec_left_join
      8
```

Huit clients, quelle que soit la méthode. Si l'on avait placé `c.canal = 'Boutique'` dans le `WHERE`, la requête aurait éliminé justement les lignes « sans partenaire » (dont `c.canal` est `NULL`), et le résultat aurait été vide : un cas d'école du 5.2.7. Parmi ces huit clients, certains n'ont **jamais** acheté du tout (comme Lou Michel, la grande ambassadrice du 5.3.6) : l'invitation à la boutique serait pour eux un premier achat.

Corrigé 5.6

(a) `telephone IS NOT NULL` vaut 1 ou 0 : sa moyenne est la proportion de numéros renseignés.

```
SELECT ville,
       COUNT(*)
       SUM(telephone IS NOT NULL)
       ROUND(100.0 * AVG(telephone IS NOT NULL), 1)
FROM clients
GROUP BY ville
ORDER BY pourcentage DESC;
```

ville	clients	avec_telephone	pourcentage
Ville D	7	7	100.0
Ville B	11	11	100.0
Ville G	11	10	90.9
Ville F	7	6	85.7
Ville E	10	8	80.0
Ville H	11	8	72.7
Ville A	13	9	69.2
Ville C	10	6	60.0

Les numéros sont toujours renseignés à Ville D et à Ville B, mais seulement à 60 % à Ville C. (b) Le comparatif `!=` renvoie *inconnu* quand `id_parrain` est `NULL` : les 49 clients **sans parrain** sont éliminés, alors qu'ils ne sont évidemment pas parrainés par le client n° 3.

```
SELECT (SELECT COUNT(*) FROM clients WHERE id_parrain != 3) AS naif,
       (SELECT COUNT(*) FROM clients WHERE id_parrain != 3 OR id_parrain IS NULL) AS corrige,
       (SELECT COUNT(*) FROM clients WHERE id_parrain = 3) AS parraines_par_3,
       (SELECT COUNT(*) FROM clients WHERE id_parrain IS NULL) AS sans_parrain;
```

```
naif  corrige  parraines_par_3  sans_parrain
  30      79           1           49
```

Le stagiaire obtient 30 lignes, alors qu'il en faut 79 (80 clients moins l'unique filleul du client n° 3). Les 49 sans parrain manquent à l'appel ; $30 + 49 = 79$. On peut aussi écrire `WHERE id_parrain IS NOT 3` (opérateur de SQLite qui traite proprement `NULL`) ou `COALESCE(id_parrain, 0) != 3`.

Corrigé 5.7

`strftime('%w', ...)` donne 0 pour dimanche... 6 pour samedi ; un CASE donne des noms lisibles.

```
SELECT CASE strftime('%w', date_commande)
  WHEN '0' THEN 'dimanche' WHEN '1' THEN 'lundi'   WHEN '2' THEN 'mardi'
  WHEN '3' THEN 'mercredi' WHEN '4' THEN 'jeudi'   WHEN '5' THEN 'vendredi'
  ELSE 'samedi' END
  AS jour,
  COUNT(*)
  AS commandes,
  ROUND(SUM(montant))
  AS chiffre_affaires
FROM commandes
GROUP BY strftime('%w', date_commande)
ORDER BY commandes DESC, chiffre_affaires DESC;
```

jour	commandes	chiffre_affaires
mercredi	66	3722.0
lundi	63	4071.0
vendredi	61	3794.0
dimanche	56	3350.0
mardi	54	3203.0
jeudi	50	3319.0
samedi	50	2641.0

Le mercredi est en tête pour le nombre de commandes (66), le lundi pour le chiffre d'affaires. Mais l'écart est-il autre chose que du bruit ? Test du khi-deux d'adéquation (3.4) de l'hypothèse « les commandes se répartissent **uniformément** sur les sept jours » :

```
from scipy import stats
effectifs = [r[0] for r in con.execute(
    "SELECT COUNT(*) FROM commandes GROUP BY strftime('%w', date_commande) ORDER BY strftime('%w',
    ↳ date_commande)")]
khi2, p = stats.chisquare(effectifs)
print("effectifs (dimanche ... samedi) :", effectifs)
print(f"khi-deux = {khi2:.2f}, p-valeur = {p:.3f}")
```

```
effectifs (dimanche ... samedi) : [56, 63, 54, 66, 50, 61, 50]
khi-deux = 4.21, p-valeur = 0.648
```

Avec une p-valeur largement supérieure à 5 %, on **ne peut pas rejeter** l'uniformité : les différences entre jours sont compatibles avec le simple hasard. (C'est d'ailleurs normal : ces dates ont été simulées sans aucun effet de jour de semaine.) Leçon : un classement (« le mercredi est le meilleur jour ») n'est pas une découverte tant qu'on ne l'a pas confronté au hasard.

Corrigé 5.8

On calcule d'abord le chiffre d'affaires par produit (CTE `par_produit`), puis un RANK par catégorie, et l'on ne garde que le rang 1 (5.3.2).

```
WITH par_produit AS (
  SELECT cat.nom AS categorie, p.nom AS produit,
    ROUND(SUM(l.quantite * l.prix_unitaire)) AS chiffre_affaires
  FROM lignes_commande AS l
  JOIN produits AS p ON p.id_produit = l.id_produit
  JOIN categories AS cat ON cat.id_categorie = p.id_categorie
  GROUP BY p.id_produit
)
SELECT categorie, produit, chiffre_affaires
FROM (
  SELECT *, RANK() OVER (PARTITION BY categorie ORDER BY chiffre_affaires DESC) AS rang
  FROM par_produit
)
WHERE rang = 1
```

ORDER BY categorie;

categorie	produit	chiffre_affaires
Bijoux	Bague en argent	2638.0
Cosmétiques	Huile de soin	1128.0
Poterie	Vase peint à la main	2411.0
Textile	Écharpe en soie	2031.0

La bague en argent domine les bijoux, le vase peint la poterie, l'écharpe en soie le textile et l'huile de soin les cosmétiques. (RANK laisserait apparaître deux lignes en cas d'égalité parfaite ; ici il n'y en a pas.)

Corrigé 5.9

- (a) Un client « actif » est un client qui a au moins une commande. (b) On numérote les commandes de chaque client dans les deux sens (ROW_NUMBER croissant et décroissant) : la première a le rang 1 dans l'ordre croissant, la dernière a le rang 1 dans l'ordre décroissant.

```
SELECT COUNT(*) AS clients_actifs,
       SUM(n >= 2) AS rechetent,
       ROUND(100.0 * SUM(n >= 2) / COUNT(*), 1) AS taux_de_reachat_pct
FROM (SELECT id_client, COUNT(*) AS n FROM commandes GROUP BY id_client);
```

clients_actifs	rechetent	taux_de_reachat_pct
66	58	87.9

```
WITH numerotees AS (
  SELECT id_client, montant,
         ROW_NUMBER() OVER (PARTITION BY id_client ORDER BY date_commande, id_commande) AS
         ↳ depuis_le_debut,
         ROW_NUMBER() OVER (PARTITION BY id_client ORDER BY date_commande DESC, id_commande DESC) AS
         ↳ depuis_la_fin,
         COUNT(*) OVER (PARTITION BY id_client) AS
         ↳ nb_commandes
  FROM commandes
),
premiere_et_derniere AS (
  SELECT id_client,
         MAX(CASE WHEN depuis_le_debut = 1 THEN montant END) AS premiere,
         MAX(CASE WHEN depuis_la_fin = 1 THEN montant END) AS derniere
  FROM numerotees
  WHERE nb_commandes >= 2
  GROUP BY id_client
)
SELECT COUNT(*) AS clients,
       SUM(derniere > premiere) AS dependent_plus_a_la_fin,
       ROUND(AVG(premiere), 2) AS premiere_moyenne,
       ROUND(AVG(derniere), 2) AS derniere_moyenne
FROM premiere_et_derniere;
```

clients	dependent_plus_a_la_fin	premiere_moyenne	derniere_moyenne
58	29	61.91	57.4

Sur 66 clients actifs, 58 ont recommandé au moins une fois : un **taux de réachat de 87,9 %**, excellent. Parmi eux, la moitié exactement (29 sur 58) dépense davantage lors de la dernière commande que lors de la première, et les montants moyens sont proches (61,91 € contre 57,40 €) : **pas de tendance** à dépenser plus avec le temps dans ces données (là encore, un test de comparaison de moyennes au sens du chapitre 3 serait à faire avant de conclure à autre chose que du hasard).

Corrigé 5.10

Une CTE récursive parcourt les réseaux, en gardant la **racine** de chacun comme au 5.3.6. On retire la racine elle-même (profondeur 0) du décompte et du chiffre d'affaires.

```
WITH RECURSIVE reseau(id_client, racine, profondeur) AS (
    SELECT id_client, id_client, 0 FROM clients WHERE id_parrain IS NULL
    UNION ALL
    SELECT c.id_client, r.racine, r.profondeur + 1
    FROM clients AS c JOIN reseau AS r ON c.id_parrain = r.id_client
),
ca_client AS (
    SELECT id_client, SUM(montant) AS ca FROM commandes GROUP BY id_client
)
SELECT r.racine,
       cl.prenom || ' ' || cl.nom      AS ambassadeur,
       COUNT(*)                      AS membres,
       ROUND(COALESCE(SUM(ca.ca), 0), 2) AS ca_des_membres
FROM reseau AS r
JOIN clients AS cl ON cl.id_client = r.racine
LEFT JOIN ca_client AS ca ON ca.id_client = r.id_client
WHERE r.profondeur > 0
GROUP BY r.racine
ORDER BY membres DESC, ca_des_membres DESC
LIMIT 3;
```

racine	ambassadeur	membres	ca_des_membres
10	Lou Michel	5	808.2
7	Paul Martin	4	2048.7
1	Yann Lambert	4	1117.0

Le `LEFT JOIN` est indispensable : un membre qui n'a jamais commandé n'a pas de ligne dans `ca_client` ; avec un `JOIN` ordinaire, il disparaîtrait du décompte des membres. Les trois plus gros réseaux sont ceux de Lou Michel (n° 10, 5 membres), de Paul Martin (n° 7, 4 membres) et de Yann Lambert (n° 1, 4 membres ; il est classé après Paul car on départage les ex æquo par le chiffre d'affaires). L'ambassadeur le plus **rentable** est **Paul Martin** : ses quatre filleuls ont dépensé 2 048,70 €, contre 1 117,00 € pour ceux de Yann Lambert et 808,20 € seulement pour les cinq membres du plus grand réseau, celui de Lou Michel. Le réseau le plus **grand** n'est donc pas le plus **rentable** : il faut mesurer ce qu'on veut optimiser.

Corrigé 5.11

(a) Dépendances :

- `id_livraison` → `id_commande`, `transporteur`, `ville_livraison` (une livraison fixe sa commande, son transporteur et sa destination) ;
- `transporteur` → `tel_transporteur` ;
- `ville_livraison` → `frais`.

(b) La fermeture de `id_livraison` contient tous les attributs (elle atteint `tel_transporteur` par `transporteur` et `frais` par `ville_livraison`), et c'est un attribut seul : c'est donc **la clé**. (c) **2FN** : oui, trivialement, car la clé est formée d'un seul attribut (il ne peut pas y avoir de dépendance partielle). **3FN** : **non**, car deux dépendances sont **transitives** : `id_livraison` → `transporteur` → `tel_transporteur`, et `id_livraison` → `ville_livraison` → `frais`. Conséquence concrète : le téléphone d'un transporteur est répété sur toutes ses livraisons, et le barème d'une ville sur chaque livraison vers elle (anomalies du 5.4.1). (d) Décomposition : `livraisons(id_livraison, id_commande, transporteur, ville_livraison)`, `transporteurs(transporteur, tel_transporteur)`, `tarifs(ville_livraison, frais)`. Vérification par le calcul :

```
def fermeture(X, deps):
    """Attributs déterminés par X (algorithme de point fixe)."""
    res, change = set(X), True
    while change:
        change = False
        for gauche, droite in deps:
            if set(gauche) <= res and not set(droite) <= res:
                res |= set(droite)
                change = True
    return res

deps = [
    (["id_livraison"], ["id_commande", "transporteur", "ville_livraison"]),
    (["transporteur"], ["tel_transporteur"]),
    (["ville_livraison"], ["frais"]),
]
tous = {"id_livraison", "id_commande", "transporteur", "tel_transporteur", "ville_livraison", "frais"}
print("fermeture de {id_livraison} :", sorted(fermeture(["id_livraison"], deps)))
print("c'est une clé :", fermeture(["id_livraison"], deps) == tous)
```

```
fermeture de {id_livraison} : ['frais', 'id_commande', 'id_livraison', 'tel_transporteur',
↳ 'transporteur', 'ville_livraison']
c'est une clé : True
```

```
# Dans chaque table de la décomposition, la clé détermine bien toutes les colonnes de la table.
decomposition = {
    "livraisons": ({"id_livraison", "id_commande", "transporteur", "ville_livraison"},
↳ {"id_livraison"}),
    "transporteurs": ({"transporteur", "tel_transporteur"}, {"transporteur"}),
    "tarifs": ({"ville_livraison", "frais"}, {"ville_livraison"}),
}
for nom, (attributs, cle) in decomposition.items():
    print(f"{nom:14s} clé {sorted(cle)} détermine toute la table : {fermeture(cle, deps) >=
↳ attributs}")
```

```
livraisons      clé ['id_livraison'] détermine toute la table : True
transporteurs    clé ['transporteur'] détermine toute la table : True
tarifs          clé ['ville_livraison'] détermine toute la table : True
```

Dans chaque table, la clé détermine toutes les autres colonnes, et les deux dépendances transitives ont été isolées chacune dans sa propre table (aucune dépendance entre colonnes non-clés ne subsiste) : le schéma est en 3FN, et même en BCNF puisque le seul déterminant de chaque table est sa clé. La décomposition est **sans perte** (théorème de Heath, 5.4.3) : chaque découpage sépare un attribut déterminant (transporteur, ville_livraison) de ce qu'il détermine, et le garde dans la table de gauche comme clé étrangère.

Chapitre 6 : Outils de travail — exercices et applications

Ce chapitre du cahier accompagne le chapitre 6 du livre (Git, notebooks, ligne de commande, scripts, reproductibilité). Les **applications** reprennent, pas à pas et en entier, les séances que le livre ne fait qu'esquisser ; les **exercices** (avec corrigés) servent à s'entraîner seul. Tout se tape dans un **terminal** (Linux, macOS, Git Bash ou WSL) ; le fichier de données est `commandes.csv`, dans le dossier que la variable `$DONNEES` désigne (chez vous : l'endroit où vous avez rangé le dossier `donnees/` du livre). Les exemples Python s'exécutent depuis la racine du volume. Les sorties affichées sont les vraies.

```
mkdir -p ~/atelier
git config --global user.name "La gérante"
git config --global user.email "gerante@boutique.example"
git config --global init.defaultBranch main
```

Une astuce de reproductibilité (à ne pas reproduire chez vous). Pour que les empreintes Git affichées ci-dessous soient toujours les mêmes, la date de toutes les versions est fixée artificiellement (variables `GIT_AUTHOR_DATE` et `GIT_COMMITTER_DATE`). Chez vous, Git utilise l'horloge : vos empreintes seront différentes, ce qui n'a aucune importance.

Applications

Application 6.1 — Une séance Git de A à Z

Objectif. Rejouer toute la vie d'un petit dépôt : le créer, photographier le travail, comparer, regarder à l'intérieur d'un commit, défaire des erreurs, ignorer des fichiers, étiqueter une version. *Section du livre : 6.1.*

Étape 1 : un dossier, un script, un dépôt.

```
mkdir -p ~/atelier/boutique
cd ~/atelier/boutique
cp "$DONNEES/commandes.csv" .
git init -q
cat > analyse.py <<'FIN'
import pandas as pd

df = pd.read_csv("commandes.csv")
print("Nombre de commandes :", len(df))
print("Montant moyen :", round(df["montant"].mean(), 2), "€")
```

FIN

```
python analyse.py
git status
```

```
Nombre de commandes : 400
Montant moyen : 60.25 €
On branch main
```

```
No commits yet
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  analyse.py
  commandes.csv
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

Git voit deux fichiers **non suivis** (*untracked*) : il les a remarqués mais ne les surveille pas encore. Rappel des trois zones : dossier de travail → (git add) → index → (git commit) → dépôt.

Étape 2 : deux commits propres. On enregistre d'abord le script, puis les données, en deux photos distinctes.

```
git add analyse.py
git status --short
git commit -q -m "Premier script : montant moyen des commandes"
git add commandes.csv
git commit -q -m "Ajout du fichier de données (400 commandes)"
git log --oneline
```

```
A analyse.py
?? commandes.csv
1a81aa5 Ajout du fichier de données (400 commandes)
07eefcb Premier script : montant moyen des commandes
```

git status --short affiche une lettre par fichier : A (ajouté à l'index) pour analyse.py, ?? (non suivi) pour commandes.csv. Après les deux commits, git log --oneline montre une ligne par version.

Étape 3 : regarder à l'intérieur d'un commit. L'empreinte d'un blob est le SHA-1 de la chaîne blob, de la taille du contenu, d'un caractère nul et du contenu : on la calcule avec Git, puis à la main avec shasum.

```
echo "bonjour" > bonjour.txt
git hash-object bonjour.txt
printf 'blob 8\0bonjour\n' | shasum
rm bonjour.txt
```

```
1cd909e05d33f0f6bc4ea1caf19b5749b434ceb3
1cd909e05d33f0f6bc4ea1caf19b5749b434ceb3 -
```

Les deux empreintes sont identiques : Git hache simplement le contenu. Voyons le dernier commit et son arbre :

```
git cat-file -p HEAD
echo "---"
git cat-file -p 'HEAD^{tree}'
```

```
tree 25e41e984c9bfc34cd88069b6821dcf2a9176257
parent 07eefcb2bdc2698f8f0e9eb534afa347c96a5088
author La gérante <gerante@boutique.example> 1772442000 +0100
```

```

committer La gérante <gerante@boutique.example> 1772442000 +0100

Ajout du fichier de données (400 commandes)
---
100644 blob e6c6d1eb7801facd05a2164e7d875a2e82b1f3e0 analyse.py
100644 blob 3872f3a74145a4a12a3c77704adfe45a309b6f7b commandes.csv

```

On lit d'abord le commit (empreinte de l'arbre, du parent, auteur, message), puis l'arbre : une ligne par fichier, avec le mode, le type blob, l'empreinte et le nom.

Étape 4 : le cycle modifier, comparer, enregistrer. La gérante veut le montant moyen par canal.

```

cat >> analyse.py <<'FIN'
print()
print("Montant moyen par canal :")
print(df.groupby("canal")["montant"].mean().round(2))
FIN
git status --short
git diff

```

```

M analyse.py
diff --git a/analyse.py b/analyse.py
index e6c6d1e..7d5d285 100644
--- a/analyse.py
+++ b/analyse.py
@@ -3,3 +3,6 @@ import pandas as pd
 df = pd.read_csv("commandes.csv")
 print("Nombre de commandes :", len(df))
 print("Montant moyen :", round(df["montant"].mean(), 2), "€")
+print()
+print("Montant moyen par canal :")
+print(df.groupby("canal")["montant"].mean().round(2))

```

git diff montre les différences entre le dossier de travail et la dernière photo (+ : ligne ajoutée ; - : ligne supprimée). Lançons le script, enregistrons, puis relisons l'historique avec le détail des fichiers touchés :

```

python analyse.py
git add analyse.py
git commit -q -m "Ajout du montant moyen par canal"
git log --stat | head -n 12
git show --stat HEAD~1 | head -n 8

```

```

Nombre de commandes : 400
Montant moyen : 60.25 €

Montant moyen par canal :
canal
Boutique      74.81
Réseaux       49.01
Site          59.50
Name: montant, dtype: float64
commit 511c5a6e737e6dde7fee3937a80290aa5e92e0d7
Author: La gérante <gerante@boutique.example>
Date: Mon Mar 2 10:00:00 2026 +0100

    Ajout du montant moyen par canal

analyse.py | 3 +++
1 file changed, 3 insertions(+)

commit 1a81aa5eef7d5801ef4e929907514907fe0db2c8
Author: La gérante <gerante@boutique.example>

```

```
Date: Mon Mar 2 10:00:00 2026 +0100
commit 1a81aa5eef7d5801ef4e929907514907fe0db2c8
Author: La gérante <gerante@boutique.example>
Date: Mon Mar 2 10:00:00 2026 +0100
```

Ajout du fichier de données (400 commandes)

```
commandes.csv | 401 ++++++++++++++++++++++++++++++++++++++++++++++++++++++
1 file changed, 401 insertions(+)
```

Étape 5 : défaire une erreur, trois situations. *Situation 1 : « J'ai cassé un fichier, je veux la version du dernier commit. »* La gérante, fatiguée, efface tout le contenu de son script.

```
wc -c analyse.py
> analyse.py
wc -c analyse.py
python analyse.py
git restore analyse.py
wc -c analyse.py
python analyse.py | head -n 2
```

```
256 analyse.py
0 analyse.py
256 analyse.py
Nombre de commandes : 400
Montant moyen : 60.25 €
```

`wc -c` compte les octets. La ligne `> analyse.py` **vide** le fichier : 0 octet, et le script n'affiche plus rien. `git restore` ramène le fichier à l'état du dernier commit. Ce que Git n'a jamais photographié, il ne peut pas le retrouver : **commits petits et fréquents**.

Situation 2 : « J'ai enregistré une erreur ; je l'ai déjà partagée ou je veux garder une trace. » On annule par un **nouveau commit** qui fait l'inverse (`git revert`) : l'historique reste intact.

```
echo 'print("TODO supprimer cette ligne")' >> analyse.py
git commit -q -am "Ligne de debug oubliée"
git revert --no-edit HEAD
git log --format='%h %s'
python analyse.py | tail -n 1
```

```
[main fe75e16] Revert "Ligne de debug oubliée"
Date: Mon Mar 2 10:00:00 2026 +0100
1 file changed, 1 deletion(-)
fe75e16 Revert "Ligne de debug oubliée"
c057bea Ligne de debug oubliée
511c5a6 Ajout du montant moyen par canal
1a81aa5 Ajout du fichier de données (400 commandes)
07eefcb Premier script : montant moyen des commandes
Name: montant, dtype: float64
```

(L'option `-a` de `commit` ajoute automatiquement tous les fichiers **déjà suivis** et modifiés ; on vérifie quand même avec `git status` avant.) L'historique contient l'erreur *et* son annulation, et le script est redevenu propre.

Situation 3 : « Je veux revoir l'état du dossier à une date passée. » On se déplace temporairement dans l'historique avec `git switch --detach`, puis on revient.

```
git switch --detach HEAD~3 2>&1 | head -n 1
cat analyse.py
git switch main
```

```
HEAD is now at 1a81aa5 Ajout du fichier de données (400 commandes)
import pandas as pd

df = pd.read_csv("commandes.csv")
print("Nombre de commandes :", len(df))
print("Montant moyen :", round(df["montant"].mean(), 2), "€")
Previous HEAD position was 1a81aa5 Ajout du fichier de données (400 commandes)
Switched to branch 'main'
```

On y voit le script tel qu'il était, **sans** le calcul par canal. On peut regarder et lancer le code, mais on ne doit pas y travailler (Git parle d'état *detached HEAD*).

Étape 6 : ignorer des fichiers. Les fichiers temporaires, l'environnement virtuel et surtout les **secrets** n'ont pas leur place dans l'historique.

```
mkdir -p .env __pycache__
touch .env/pyvenv.cfg __pycache__/analyse.cpython-313.pyc secrets.env
git status --short
cat > .gitignore <<'FIN'
# environnement et fichiers temporaires
.env/
__pycache__/
*.pyc
.ipynb_checkpoints/
# secrets : JAMAIS dans Git
*.env
FIN
git status --short
```

```
?? .env/
?? __pycache__/
?? secrets.env
?? .gitignore
```

Après l'ajout du `.gitignore`, il ne reste que ce fichier lui-même, que l'on **veut** suivre (pour que toute l'équipe ignore les mêmes choses). `git check-ignore -v` explique pourquoi un fichier est ignoré, et la dernière commande pose une étiquette sur la version :

```
git add .gitignore
git commit -q -m "Ajout du .gitignore"
git check-ignore -v secrets.env
git tag -a v1.0-rapport-banque -m "Version remise à la banque (mars 2026)"
git tag
git show --stat v1.0-rapport-banque | head -n 6
```

```
.gitignore:7:*.env secrets.env
v1.0-rapport-banque
tag v1.0-rapport-banque
Tagger: La gérante <gerante@boutique.example>
Date: Mon Mar 2 10:00:00 2026 +0100

Version remise à la banque (mars 2026)
```

Un secret commité est un secret perdu. Le retirer dans un commit suivant ne suffit pas : il reste lisible dans l'historique. Mettez `.gitignore` en place **avant** de créer le fichier secret.

Pour aller plus loin. Écrivez un message de commit que vous comprendriez dans six mois pour chacun des commits de l'étape 4 ; puis essayez `git diff HEAD~1 HEAD` et `git diff --staged` après avoir modifié le script sans le commiter.

Application 6.2 — Branches, fusions et conflits

Objectif. Tester une idée sur une branche, fusionner (cas facile, cas automatique, cas conflictuel) et résoudre un conflit à la main. *À la suite de l'application 6.1 (même dépôt ~/atelier/boutique). Section du livre : 6.1.8.*

Une expérience sur une branche. La gérante se demande à partir de quel montant offrir la livraison. Elle ouvre une branche dédiée.

```
cd ~/atelier/boutique
git branch
git switch -c seuil-livraison
git branch
```

```
* main
Switched to a new branch 'seuil-livraison'
  main
* seuil-livraison
```

L'étoile * marque la branche courante. Travaillons dessus : on ajoute un seuil de livraison gratuite et on calcule la part de commandes concernées.

```
cat >> analyse.py <<'FIN'

SEUIL = 80 # seuil de livraison gratuite (€)
part = (df["montant"] >= SEUIL).mean()
print(f"Part des commandes dès {SEUIL} € : {part:.1%}")
FIN
python analyse.py | tail -n 2
git commit -q -am "Seuil de livraison gratuite à 80 €"
git switch main
tail -n 3 analyse.py
```

```
Name: montant, dtype: float64
Part des commandes dès 80 € : 21.8%
Switched to branch 'main'
print()
print("Montant moyen par canal :")
print(df.groupby("canal")["montant"].mean().round(2))
```

Retour sur main : les lignes du seuil ont **disparu** du fichier, car elles n'existent que sur la branche. En repassant sur la branche, tout revient.

Fusion facile : l'avance rapide. main n'a pas bougé depuis la création de la branche : Git se contente d'avancer le pointeur de main (*fast-forward*).

```
git merge seuil-livraison
git log --oneline --graph | head -n 4
git branch -d seuil-livraison
```

```
Updating 9687a07..247efd8
Fast-forward
 analyse.py | 4 +++
 1 file changed, 4 insertions(+)
* 247efd8 Seuil de livraison gratuite à 80 €
* 9687a07 Ajout du .gitignore
* fe75e16 Revert "Ligne de debug oubliée"
* c057bea Ligne de debug oubliée
Deleted branch seuil-livraison (was 247efd8).
```

Les deux ont bougé : fusion automatique. Sam teste un seuil plus bas (60 €) sur sa branche ; la gérante, en parallèle sur main, ajoute une moyenne de satisfaction.

```

git switch -c seuil-sam
sed -i 's/^SEUIL = 80./SEUIL = 60 # seuil proposé par Sam (€)/' analyse.py
git commit -q -am "Seuil à 60 € (proposition de Sam)"
git switch main
cat >> analyse.py <<'FIN'

print("Satisfaction moyenne :", round(df["satisfaction"].mean(), 2))
FIN
git commit -q -am "Ajout de la satisfaction moyenne"
git log --oneline --graph --all | head -n 4

```

```

Switched to a new branch 'seuil-sam'
Switched to branch 'main'
* 112a992 Ajout de la satisfaction moyenne
| * b82cc85 Seuil à 60 € (proposition de Sam)
|/
* 247efd8 Seuil de livraison gratuite à 80 €

```

Le graphe a la forme d'un « Y » : les deux branches sont parties du même commit et ont chacune avancé. Fusionnons celle de Sam :

```

git merge --no-edit seuil-sam
git log --oneline --graph | head -n 5
python analyse.py | tail -n 3

```

```

Auto-merging analyse.py
Merge made by the 'ort' strategy.
analyse.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
* 752a0ca Merge branch 'seuil-sam'
|\
| * b82cc85 Seuil à 60 € (proposition de Sam)
* | 112a992 Ajout de la satisfaction moyenne
|/
Name: montant, dtype: float64
Part des commandes dès 60 € : 41.0%
Satisfaction moyenne : 3.96

```

Git a réussi **automatiquement** : les deux changements portaient sur des zones différentes du fichier. Il a créé un **commit de fusion** à deux parents. Python confirme que le script fonctionne et applique bien les deux modifications.

Un conflit. Un conflit survient quand deux branches modifient **la même ligne** de façons différentes. La gérante veut un seuil de 70 €, Sam de 65 €.

```

git switch -c seuil-gerante
sed -i 's/^SEUIL = .*/SEUIL = 70 # seuil retenu par la gérante (€)/' analyse.py
git commit -q -am "Seuil à 70 €"
git switch main
sed -i 's/^SEUIL = .*/SEUIL = 65 # compromis de Sam (€)/' analyse.py
git commit -q -am "Seuil à 65 €"
git merge --no-edit seuil-gerante

```

```

Switched to a new branch 'seuil-gerante'
Switched to branch 'main'
Auto-merging analyse.py
CONFLICT (content): Merge conflict in analyse.py
Automatic merge failed; fix conflicts and then commit the result.

```

Git s'arrête : **CONFLICT (content)**. Le dépôt est en cours de fusion ; consultons l'état puis le fichier.

```
git status --short
grep -n -A6 '<<<<<<' analyse.py
```

```
UU analyse.py
10:<<<<<< HEAD
11-SEUIL = 65 # compromis de Sam (€)
12-=====
13-SEUIL = 70 # seuil retenu par la gérante (€)
14->>>>>> seuil-gerante
15-part = (df["montant"] >= SEUIL).mean()
16-print(f"Part des commandes dès {SEUIL} € : {part:.1%}")
```

Git a écrit, autour de la ligne litigieuse, trois **marqueurs** : <<<<<< HEAD (version de la branche où l'on se trouve : 65 €), ===== (séparation) et >>>>>> seuil-gerante (version qu'on fusionne : 70 €). Résoudre le conflit, c'est **éditer le fichier** pour ne garder que ce qu'on veut (ici, après discussion, 70 €), supprimer les marqueurs, déclarer le conflit réglé avec `git add`, et conclure par un commit.

```
sed -i '/^<<<<<< /d; /^=====$/d; /^>>>>>> /d; /^SEUIL = 65/d' analyse.py
grep -n '^SEUIL' analyse.py
python analyse.py | tail -n 3
git add analyse.py
git commit -q --no-edit
git log --oneline --graph | head -n 8
git branch -d seuil-sam seuil-gerante
```

```
10:SEUIL = 70 # seuil retenu par la gérante (€)
Name: montant, dtype: float64
Part des commandes dès 70 € : 29.2%
Satisfaction moyenne : 3.96
* 0f9316c Merge branch 'seuil-gerante'
|\
| * ec2c633 Seuil à 70 €
* | 56af79a Seuil à 65 €
|/
* 752a0ca Merge branch 'seuil-sam'
|\
| * b82cc85 Seuil à 60 € (proposition de Sam)
Deleted branch seuil-sam (was b82cc85).
Deleted branch seuil-gerante (was ec2c633).
```

(Le long `sed` enlève les trois lignes de marqueurs et la ligne de la version de Sam. Avec un éditeur comme VS Code, vous cliquez sur « Accepter la modification actuelle / entrante » ; le principe est identique.) Avant de valider, **toujours relancer le code**.

Pour aller plus loin. Provoquez un second conflit, mais cette fois résolvez-le en gardant **les deux** modifications (par exemple deux seuils affichés côte à côte).

Application 6.3 — Un dépôt partagé, simulé sans Internet

Objectif. Comprendre `push`, `clone` et `pull` avec un dépôt « distant » qui n'est qu'un autre dossier. À la suite de l'application 6.2. Section du livre : 6.1.10.

Un dépôt distant est souvent « nu » (*bare* : sans dossier de travail, car personne n'y édite directement). La gérante crée un dépôt partagé et y envoie son travail ; Sam le clone.

```
cd ~/atelier
git init -q --bare depot-partage.git
cd boutique
git remote add origin ~/atelier/depot-partage.git
git remote -v | sed "s|$HOME|~|"
```

```
git push -q -u origin main --tags 2>&1 | tail -n 3
cd ..
git clone -q depot-partage.git copie-sam
cd copie-sam
git config user.name "Sam"
git config user.email "sam@boutique.example"
git log --oneline | wc -l
```

```
origin ~/atelier/depot-partage.git (fetch)
origin ~/atelier/depot-partage.git (push)
13
```

(Le `sed` ne sert qu'à abrégé le chemin en `~` pour l'affichage ; les deux `git config` donnent à ce clone sa propre identité, Sam travaillant sur une autre machine.) La dernière commande compte les commits reçus : tout l'historique de la gérante, fusions comprises. Sam travaille et envoie sa modification ; la gérante la récupère.

```
echo 'print("Commandes Réseaux :", (df["canal"] == "Réseaux").sum())' >> analyse.py
git commit -q -am "Ajout du nombre de commandes Réseaux"
git push -q origin main 2>&1 | tail -n 3
cd ../boutique
git pull -q origin main 2>&1 | tail -n 3
git log --format='%an : %s' | head -n 3
python analyse.py | tail -n 1
```

```
Sam : Ajout du nombre de commandes Réseaux
La gérante : Merge branch 'seuil-gerante'
La gérante : Seuil à 65 €
Commandes Réseaux : 138
```

Chaque commit porte le nom de son auteur, et le script affiche bien la nouvelle ligne. Avec *GitHub*, le principe est le même (adresse de la forme `git@github.com:nom/depot.git`), avec en plus la **pull request** pour relire avant de fusionner (non exécuté ici : cela demande un compte et une connexion).

Pour aller plus loin. Faites modifier la **même ligne** par la gérante et par Sam avant le `pull` : que se passe-t-il ? Résolvez le conflit comme dans l'application 6.2.

Application 6.4 — Un notebook : le fabriquer, l'exécuter, l'exporter

Objectif. Constater qu'un notebook n'est qu'un fichier JSON, l'exécuter depuis un programme, puis l'exporter en script, en Markdown et en HTML. *Section du livre : 6.2.2 et 6.2.7.*

Fabriquer le carnet. Trois cellules : un titre, le chargement des données, un calcul par canal.

```
import json
import nbformat
from nbformat import v4 as nbformat

nb = nbformat.new_notebook()
nb.metadata["kernelspec"] = {"name": "python3", "display_name": "Python 3", "language": "python"}
nb.cells = [
    nbformat.new_markdown_cell("# Ventes de la boutique\nMontant moyen des commandes, par canal.",
    ↪ id="titre"),
    nbformat.new_code_cell("import pandas as pd\nndf = pd.read_csv('donnees/commandes.csv')\nndf.shape",
    ↪ id="chargement"),
    nbformat.new_code_cell("df.groupby('canal')['montant'].mean().round(2)", id="par-canal"),
]
texte = nbformat.writes(nb)
print(texte[:700])
```

```
{
  "cells": [
    {
      "cell_type": "markdown",
      "id": "titre",
      "metadata": {},
      "source": [
        "# Ventes de la boutique\n",
        "Montant moyen des commandes, par canal."
      ]
    },
    {
      "cell_type": "code",
      "execution_count": null,
      "id": "chargement",
      "metadata": {},
      "outputs": [],
      "source": [
        "import pandas as pd\n",
        "df = pd.read_csv('donnees/commandes.csv')\n",
        "df.shape"
      ]
    },
    {
      "cell_type": "code",
      "execution_count": null,
      "id": "par-canal",
      "metadata": {},
      "outputs": [],
      "source": [
        "df.groupby('canal')['montant'].mean().round(2)"
      ]
    }
  ],
  "metadata": {
    "kernel_spec": {
      "display_name": "Python 3",
      "language": "p"
    }
  }
}
```

On y reconnaît : une liste de cells ; pour chaque cellule, son cell_type (markdown ou code), sa source, et, pour les cellules de code, execution_count (vide tant que la cellule n'a pas tourné) et outputs (vide pour l'instant). Exécutons-le avec nbclient, la bibliothèque qui pilote un noyau depuis un programme (c'est ce que fait le bouton « Exécuter tout ») :

```
from nbclient import NotebookClient

NotebookClient(nb, timeout=120, kernel_name="python3", record_timing=False,
               resources={"metadata": {"path": "."}}).execute()

for cellule in nb.cells:
    if cellule.cell_type == "code":
        print(f"[{cellule.execution_count}] {cellule.source.splitlines()[-1]}")
        for sortie in cellule.outputs:
            print("    ->", sortie.output_type, ":", sortie.data["text/plain"].replace("\n", "\n↵"))
```

```
[1] df.shape
    -> execute_result : (400, 4)
[2] df.groupby('canal')['montant'].mean().round(2)
    -> execute_result : canal
        Boutique    74.81
        Réseaux     49.01
        Site        59.50
```

```
Name: montant, dtype: float64
```

Chaque cellule de code porte désormais son **numéro d'exécution** et ses **sorties**, enregistrées dans le fichier. Un graphique serait stocké sous `image/png`, encodé en texte : un notebook avec beaucoup de graphiques devient volumineux.

Exporter. nbconvert transforme un notebook en d'autres formats. D'abord en script Python (le code seul), puis en rapport Markdown :

```
from nbconvert import PythonExporter, MarkdownExporter

script, _ = PythonExporter().from_notebook_node(nb)
print(script)
rapport, _ = MarkdownExporter().from_notebook_node(nb)
print(rapport.replace("`", "~~~")) # ~~~ à la place des accents graves, pour l'affichage dans le
↪ cahier
```

```
#!/usr/bin/env python
# coding: utf-8

# # Ventes de la boutique
# Montant moyen des commandes, par canal.

# In[1]:

import pandas as pd
df = pd.read_csv('donnees/commandes.csv')
df.shape

# In[2]:

df.groupby('canal')['montant'].mean().round(2)

# Ventes de la boutique
Montant moyen des commandes, par canal.

~~~python
import pandas as pd
df = pd.read_csv('donnees/commandes.csv')
df.shape
~~~

(400, 4)

~~~python
df.groupby('canal')['montant'].mean().round(2)
~~~

canal
Boutique    74.81
Réseaux     49.01
Site        59.50
Name: montant, dtype: float64
```

Le script contient le code de chaque cellule, précédé d'un commentaire `# In[1]:` ; le texte Mark-

down est transformé en commentaires. Le rapport contient le titre, le code de chaque cellule et ses sorties. **En ligne de commande**, avec l'option `--execute` qui rejoue d'abord tout le notebook dans un noyau neuf (la règle « Restart & Run All », automatisée) :

```
mkdir -p ~/atelier/notebook
cd ~/atelier/notebook
cp "$DONNEES/commandes.csv" .
python - <<'FIN'
import nbformat
from nbformat import v4 as nbformat
nb = nbformat.new_notebook(cells=[
    nbformat.new_markdown_cell("# Satisfaction par canal", id="a"),
    nbformat.new_code_cell("import pandas as pd\n", id="b"),
    nbformat.new_code_cell("df = pd.read_csv('commandes.csv')\ndf.groupby('canal')['satisfaction'].mean().round(2)", id="c"),
])
nb.metadata["kernelspec"] = {"name": "python3", "display_name": "Python 3", "language": "python"}
nbformat.write(nb, "satisfaction.ipynb")
FIN
jupyter nbconvert --to markdown --execute satisfaction.ipynb 2>&1 | grep -v -i warning
sed 's/`~/`~`/' satisfaction.md
```

```
[NbConvertApp] Converting notebook satisfaction.ipynb to markdown
[NbConvertApp] Writing 265 bytes to satisfaction.md
# Satisfaction par canal
```

```
~~~python
import pandas as pd
df = pd.read_csv('commandes.csv')
df.groupby('canal')['satisfaction'].mean().round(2)
~~~
```

```
canal
Boutique    4.49
Réseaux     3.72
Site        3.79
Name: satisfaction, dtype: float64
```

Pour obtenir un fichier HTML, remplacez `markdown` par `html` ; pour un PDF, `--to pdf` demande en plus une installation de LaTeX.

Pour aller plus loin. Ajoutez une cellule qui trace un histogramme des montants : que devient la taille du fichier `.ipynb` après exécution ?

Application 6.5 — L'état caché, le détecteur d'ordre et le nettoyeur de sorties

Objectif. Reproduire le piège principal des notebooks (l'état caché), puis écrire deux petits outils : un détecteur de notebooks douteux et un nettoyeur de sorties avant commit. *À la suite de l'application 6.4 (le notebook nb est réutilisé).* Sections du livre : 6.2.3, 6.2.4, 6.2.6.

Simuler un noyau. Un dictionnaire `memoire` joue la mémoire du noyau, `executer` joue le rôle de `Maj + Entrée`. Trois cellules calculent le prix d'une commande : trois articles à 50 €, TVA à 19 %.

```
memoire = {}

def executer(code):
    exec(code, memoire)

cellules = {
    "A": "prix_unitaire = 50",
    "B": "total = prix_unitaire * 3 * 1.19",
```

```

    "C": "print('Total TTC :', round(total, 2), '€')",
}
for nom in "ABC":
    executer(cellules[nom])

```

Total TTC : 178.5 €

Le total attendu est $50 \times 3 \times 1,19 = 178,5$ €. Maintenant la gérante corrige le prix à 80 €, relance A, puis C... en oubliant B :

```

cellules["A"] = "prix_unitaire = 80"
executer(cellules["A"])
executer(cellules["C"])

```

Total TTC : 178.5 €

Le total est **toujours 178,5 €** : la variable total en mémoire date de l'ancienne exécution de B. Voici ce que donne une exécution propre, sur un noyau neuf :

```

memoire = {}
for nom in "ABC":
    executer(cellules[nom])
print("Vérification à la main :", 80 * 3 * 1.19)

```

Total TTC : 285.6 €

Vérification à la main : 285.59999999999997

Le vrai total est **285,6 €**. (La ligne de vérification affiche 285.59999999999997 : l'artefact de virgule flottante vu en 1.5.) **Second exemple : la cellule supprimée.** La gérante définit une remise dans une cellule, l'utilise plus bas, puis supprime la cellule de la remise « pour faire propre » : tout marche encore, jusqu'à ce qu'un collègue ouvre le notebook.

```

memoire = {}
executer("remise = 0.10")
executer("prix_remise = 80 * (1 - remise)")
print("prix remisé (noyau de la gérante) :", memoire["prix_remise"])

memoire = {}
try:
    executer("prix_remise = 80 * (1 - remise)")
except NameError as erreur:
    print("collègue : NameError :", erreur)

```

prix remisé (noyau de la gérante) : 72.0

collègue : NameError : name 'remise' is not defined

Le détecteur d'ordre suspect. Un notebook exécuté d'un trait, sur un noyau neuf, affiche [1], [2], [3]... sans trou ni saut. Écrivons une fonction qui repère les numéros désordonnés, les cellules jamais exécutées et les erreurs enregistrées.

```

import copy

def audit(carnet):
    """Retourne la liste des problèmes détectés dans un notebook déjà exécuté."""
    problemes = []
    code = [c for c in carnet.cells if c.cell_type == "code"]
    comptes = [c.execution_count for c in code]
    if any(n is None for n in comptes):
        problemes.append("au moins une cellule n'a jamais été exécutée")
    vus = [n for n in comptes if n is not None]
    if vus != list(range(1, len(vus) + 1)):

```

```

    problemes.append(f"numéros d'exécution {comptes} : pas 1, 2, 3... (ordre ou noyau douteux)")
    if any(s.output_type == "error" for c in code for s in c.outputs):
        problemes.append("une erreur est enregistrée dans les sorties")
    return problemes or ["OK : notebook exécuté dans l'ordre, sans erreur"]

```

```

print("notebook de la gérante :", audit(nb))
douteux = copy.deepcopy(nb) # un notebook « bidouillé » : ordre d'exécution désordonné
douteux.cells[1].execution_count = 3
douteux.cells[2].execution_count = 1
print("notebook bidouillé :", audit(douteux))

```

```

notebook de la gérante : ["OK : notebook exécuté dans l'ordre, sans erreur"]
notebook bidouillé : ["numéros d'exécution [3, 1] : pas 1, 2, 3... (ordre ou noyau douteux)"]

```

Le détecteur ne *prouve* pas qu'un notebook est reproductible (la seule preuve est de le réexécuter), mais il repère les cas flagrants en une milliseconde. **Le nettoyeur de sorties.** Comme les sorties et les numéros sont stockés dans le fichier, la moindre ré-exécution modifie des dizaines de lignes : on ne garde dans Git que les sources.

```

def nettoyer(carnet):
    """Copie du notebook sans sorties ni numéros d'exécution."""
    propre = copy.deepcopy(carnet)
    for c in propre.cells:
        if c.cell_type == "code":
            c.outputs = []
            c.execution_count = None
    return propre

complet, propre = nbformat.writes(nb), nbformat.writes(nettoyer(nb))
print("lignes du JSON avec sorties :", complet.count("\n"))
print("lignes du JSON sans sorties :", propre.count("\n"))
cellule = json.loads(propre)["cells"][2]
print("cellule 3 nettoyée :", cellule["execution_count"], cellule["outputs"])

```

```

lignes du JSON avec sorties : 81
lignes du JSON sans sorties : 55
cellule 3 nettoyée : None []

```

Dans la pratique, l'outil `nbstripout` fait ce nettoyage automatiquement à chaque commit. **Pour aller plus loin.** Étendez `audit` pour signaler un notebook dont la dernière cellule de code n'a pas de sortie, ou dont une cellule contient un chemin absolu (par exemple une chaîne qui commence par `/home/`).

Application 6.6 — Explorer un fichier avec les outils du terminal

Objectif. Se repérer dans une arborescence, lire un CSV sans l'ouvrir, interroger les données avec `grep`, `cut`, `sort`, `uniq` et `awk`, et sauvegarder des résultats par redirection. *Sections du livre : 6.3.1 à 6.3.4.*

Un projet bien rangé. Les données brutes ne se modifient jamais, les notebooks explorent, `src` contient le code réutilisable, `rapports` reçoit les résultats.

```

cd ~/atelier
mkdir -p etude-ventes/{donnees,notebooks,src,rapports}
cd etude-ventes
cp "$DONNEES/commandes.csv" donnees/
touch README.md src/utils.py
pwd | sed "s|^$HOME|~|"
ls
find . -not -name '.' | sort

```

```
~/atelier/etude-ventes
README.md
donnees
notebooks
rapports
src
./README.md
./donnees
./donnees/commandes.csv
./notebooks
./rapports
./src
./src/outils.py
```

La syntaxe {a,b,c} est une **expansion d'accolades**. Pour **déplacer, renommer, supprimer** :

```
mv README.md LISEZMOI.md      # renommer
mv LISEZMOI.md README.md      # et on revient en arrière
cp -r rapports rapports-copie  # copier un dossier entier (-r : récursif)
rm -r rapports-copie          # supprimer un dossier et son contenu (sans corbeille !)
ls
```

```
README.md
donnees
notebooks
rapports
src
```

Lire un fichier.

```
cd donnees
head -n 5 commandes.csv
echo "... "
tail -n 3 commandes.csv
wc -l commandes.csv
```

```
canal,montant,livraison,satisfaction
Boutique,44.8,0,4
Site,34.5,2,4
Réseaux,88.2,5,4
Réseaux,30.1,4,4
...
Site,62.6,4,3
Réseaux,37.5,3,4
Site,31.4,4,4
401 commandes.csv
```

Interroger les données. Combien de commandes viennent des réseaux sociaux ? Combien par canal ? Quelles sont les trois plus grosses ?

```
grep -c Réseaux commandes.csv
tail -n +2 commandes.csv | cut -d, -f1 | sort | uniq -c | sort -rn
tail -n +2 commandes.csv | sort -t, -k2 -n -r | head -n 3
```

```
138
  148 Site
  138 Réseaux
  114 Boutique
Site,255.7,4,3
Site,243.8,5,3
Site,217.1,7,4
```

Le pipeline de la deuxième ligne : `tail -n +2` supprime l'en-tête, `cut -d, -f1` garde la colonne

des canaux, sort regroupe, `uniq -c` compte, `sort -rn` classe. La troisième trie sur la 2^e colonne (`-k2`), numériquement (`-n`), de la plus grande à la plus petite (`-r`) : la plus grosse commande atteint le maximum vu avec `describe()` au chapitre 3. Pour le montant moyen, il faut calculer, ce que fait `awk` : `$2` désigne la 2^e colonne, `NR` le numéro de ligne.

```
awk -F, 'NR > 1 { somme += $2; n++ } END { printf "montant moyen : %.2f € sur %d commandes\n",
↪ somme/n, n }' commandes.csv
awk -F, 'NR > 1 { somme[$1] += $2; n[$1]++ }
↪ END { for (canal in n) printf "%-10s %.2f\n", canal, somme[canal]/n[canal] }' commandes.csv |
↪ sort
```

```
montant moyen : 60.25 € sur 400 commandes
Boutique      74.81
Réseaux       49.01
Site          59.50
```

La seconde commande calcule une moyenne **par canal** grâce à un tableau associatif (l'équivalent d'un dictionnaire Python) : ce sont les trois moyennes que `df.groupby("canal")["montant"].mean()` donne dans `pandas`.

Rediriger vers des fichiers. `>` écrase, `>>` ajoute, `2>` capte les messages d'erreur ; `$?` donne le code de sortie de la dernière commande (0 : succès).

```
cd ..
tail -n +2 donnees/commandes.csv | cut -d, -f1 | sort | uniq -c > rapports/commandes-par-canal.txt
echo "# généré par la ligne de commande" >> rapports/commandes-par-canal.txt
cat rapports/commandes-par-canal.txt

ls donnees/inexistant.csv 2> rapports/erreurs.txt
echo "code de sortie : $?"
cat rapports/erreurs.txt
```

```
114 Boutique
138 Réseaux
148 Site
# généré par la ligne de commande
code de sortie : 2
ls: cannot access 'donnees/inexistant.csv': No such file or directory
```

Pour aller plus loin. Écrivez, avec seulement des outils du terminal, la liste des commandes de plus de 150 € livrées en plus de 5 jours (colonnes 2 et 3), puis comptez-les.

Application 6.7 — Un outil en ligne de commande, avec ses codes de sortie

Objectif. Écrire un script Python qui prend un fichier en argument et respecte la convention des codes de sortie (0 : succès ; non nul : erreur). À la suite de l'application 6.6. Section du livre : 6.3.5.

Les arguments tapés après le nom du script sont dans la liste `sys.argv` (`sys.argv[0]` est le nom du script).

```
cat > src/resume.py <<'FIN'
"""Résumé rapide d'un fichier CSV : python src/resume.py fichier.csv"""
import sys
import pandas as pd

if len(sys.argv) != 2:
    print("usage : python src/resume.py fichier.csv")
    sys.exit(2)

try:
    df = pd.read_csv(sys.argv[1])
```

```
except FileNotFoundError:
    print(f"fichier introuvable : {sys.argv[1]}")
    sys.exit(1)

print(f"{len(df)} lignes, {df.shape[1]} colonnes")
print(df.describe().loc[["mean", "min", "max"]].round(2))
FIN
python src/resume.py donnees/commandes.csv
echo "code de sortie : $?"
python src/resume.py donnees/absent.csv
echo "code de sortie : $?"
python src/resume.py
echo "code de sortie : $?"
```

```
400 lignes, 4 colonnes
      montant  livraison  satisfaction
mean      60.25        3.29          3.96
min       8.60         0.00          1.00
max      255.70       13.00          5.00
code de sortie : 0
fichier introuvable : donnees/absent.csv
code de sortie : 1
usage : python src/resume.py fichier.csv
code de sortie : 2
```

Le script renvoie **0** quand tout va bien, **1** pour un fichier absent, **2** pour un mauvais usage. Ces codes permettent à d'autres programmes d'enchaîner les étapes automatiquement et de s'arrêter proprement en cas de problème. **Pour aller plus loin.** Ajoutez une option facultative donnant le nom de la colonne à résumer, et un code de sortie 3 si cette colonne n'existe pas.

Application 6.8 — Des scripts shell et un petit pipeline

Objectif. Écrire un script shell avec arguments, test, boucle et substitution de commande, l'enchaîner avec le script Python, et mesurer ce que `set -e` apporte. À la suite de l'application 6.7. Section du livre : 6.4.1 et 6.4.2.

Un premier script. Il affiche, pour un fichier de commandes, le nombre de commandes par canal et leur part. On l'écrit en deux morceaux (> crée le fichier, >> ajoute la suite).

```
mkdir -p scripts
cat > scripts/rapport.sh <<'FIN'
#!/usr/bin/env bash
# rapport.sh : nombre de commandes par canal, avec leur part
# usage : bash scripts/rapport.sh donnees/commandes.csv
set -euo pipefail

if [ "$#" -ne 1 ]; then
    echo "usage : $0 fichier.csv" >&2
    exit 2
fi
fichier="$1"
if [ ! -f "$fichier" ]; then
    echo "fichier introuvable : $fichier" >&2
    exit 1
fi
FIN
```

Seconde moitié du script : le calcul et la boucle sur les canaux.

```
cat >> scripts/rapport.sh <<'FIN'
```

```
total=$(( $(wc -l < "$fichier") - 1 ))
echo "== Rapport sur $fichier ($total commandes) =="
for canal in Boutique Réseaux Site; do
    n=$(grep -c "^$canal," "$fichier")
    part=$(awk -v n="$n" -v t="$total" 'BEGIN { printf "%.1f", 100 * n / t }')
    echo "  $canal : $n commandes ($part %)"
done
FIN
bash scripts/rapport.sh donnees/commandes.csv
```

```
== Rapport sur donnees/commandes.csv (400 commandes) ==
Boutique : 114 commandes (28.5 %)
Réseaux : 138 commandes (34.5 %)
Site : 148 commandes (37.0 %)
```

Relisez le script : le **shebang** (`#!/usr/bin/env bash`) dit avec quel programme l'exécuter ; `set -euo pipefail` est la ceinture de sécurité (`-e` : s'arrêter à la première erreur ; `-u` : refuser une variable jamais définie ; `-o pipefail` : faire échouer un pipeline si l'une de ses commandes échoue) ; `$#` est le nombre d'arguments, `>&2` écrit sur la sortie d'erreur ; `$(( ... ))` fait de l'arithmétique entière ; `awk` calcule le pourcentage (le shell ne sait pas faire de décimales). On peut rendre le script exécutable et tester ses garde-fous :

```
chmod +x scripts/rapport.sh
./scripts/rapport.sh donnees/commandes.csv | head -n 2
bash scripts/rapport.sh donnees/absent.csv
echo "code de sortie : $?"
bash scripts/rapport.sh
echo "code de sortie : $?"
```

```
== Rapport sur donnees/commandes.csv (400 commandes) ==
Boutique : 114 commandes (28.5 %)
fichier introuvable : donnees/absent.csv
code de sortie : 1
usage : scripts/rapport.sh fichier.csv
code de sortie : 2
```

Une boucle sur des fichiers. Séparons le fichier de données en un fichier par canal, en recopiant l'en-tête dans chacun. Le `*` est un **joker** qui désigne tous les fichiers d'un motif.

```
mkdir -p donnees/par-canal
for canal in Boutique Réseaux Site; do
    (head -n 1 donnees/commandes.csv; grep "^$canal," donnees/commandes.csv) >
    ↪ "donnees/par-canal/$canal.csv"
done
wc -l donnees/par-canal/*.csv
```

```
115 donnees/par-canal/Boutique.csv
139 donnees/par-canal/Réseaux.csv
149 donnees/par-canal/Site.csv
403 total
```

Chaque fichier compte une ligne de plus que son nombre de commandes (l'en-tête) ; le total est donc de 400 commandes + 3 en-têtes.

Un petit pipeline. Un vrai projet est une chaîne d'étapes. Ce script appelle le script shell puis le programme Python de l'application 6.7.

```
cat > scripts/pipeline.sh <<'FIN'
#!/usr/bin/env bash
set -euo pipefail
echo "[1/3] comptage par canal"
```

```
bash scripts/rapport.sh "$1" > rapports/rapport.txt
echo "[2/3] résumé des colonnes numériques"
python src/resume.py "$1" >> rapports/rapport.txt
echo "[3/3] terminé : rapport dans rapports/rapport.txt"
FIN
bash scripts/pipeline.sh donnees/commandes.csv
cat rapports/rapport.txt
```

```
[1/3] comptage par canal
[2/3] résumé des colonnes numériques
[3/3] terminé : rapport dans rapports/rapport.txt
== Rapport sur donnees/commandes.csv (400 commandes) ==
  Boutique : 114 commandes (28.5 %)
  Réseaux : 138 commandes (34.5 %)
  Site : 148 commandes (37.0 %)
400 lignes, 4 colonnes
      montant livraison satisfaction
mean    60.25         3.29         3.96
min      8.60         0.00         1.00
max    255.70        13.00         5.00
```

Échouer bruyamment. Que se passe-t-il quand les données sont introuvables, avec puis sans la ceinture de sécurité (une copie du script d'où la ligne set est retirée) ?

```
echo "--- avec set -euo pipefail ---"
bash scripts/pipeline.sh donnees/absent.csv
echo "code de sortie : $?"
echo
echo "--- sans set -e (script imprudent) ---"
sed '/^set -euo/d' scripts/pipeline.sh > scripts/pipeline-imprudent.sh
bash scripts/pipeline-imprudent.sh donnees/absent.csv
echo "code de sortie : $?"
```

```
--- avec set -euo pipefail ---
[1/3] comptage par canal
fichier introuvable : donnees/absent.csv
code de sortie : 1

--- sans set -e (script imprudent) ---
[1/3] comptage par canal
fichier introuvable : donnees/absent.csv
[2/3] résumé des colonnes numériques
[3/3] terminé : rapport dans rapports/rapport.txt
code de sortie : 0
```

Avec `set -e`, le script s'arrête net à la première étape défaillante et renvoie un code d'erreur. Sans elle, il continue, annonce « terminé » et renvoie le code 0 (succès !) alors que **rien n'a été calculé** : le pire des scénarios. **Un script qui échoue doit échouer bruyamment. Pour aller plus loin.** Ajoutez une quatrième étape au pipeline qui vérifie que `rapports/rapport.txt` n'est pas vide, et testez-la en simulant une panne de l'étape 2.

Application 6.9 — Environnements virtuels : deux mondes sur une machine

Objectif. Créer un environnement virtuel, y installer une bibliothèque, figer les versions, reconstruire l'environnement ailleurs, et faire cohabiter deux versions d'une même bibliothèque. *Cette application télécharge un petit paquet (`tabulate`) : elle demande une connexion. À la suite de l'application 6.8. Sections du livre : 6.3.6 et 6.4.4.*

```
cd ~/atelier/etude-ventes
python -m venv .env
source .env/bin/activate
```

```
python -c "import sys; print('environnement virtuel actif :', sys.prefix != sys.base_prefix)"
pip list 2>&1 | grep -v -i -E "notice|warning"
```

```
environnement virtuel actif : True
Package Version
-----
pip      25.0
```

L'environnement est **vierge** : il ne contient que pip lui-même. Installons une petite bibliothèque, `tabulate` (qui formate des tableaux en texte), et vérifions qu'elle fonctionne :

```
pip install --quiet tabulate 2>&1 | grep -v -i -E "notice|warning"
python -c "from tabulate import tabulate; print(tabulate(['Boutique', 74.81], ['Réseaux', 49.01],
↳ ['Site', 59.5]), headers=['canal', 'montant moyen'], floatfmt='.2f'))"
pip freeze
pip freeze > requirements.txt
cat requirements.txt
deactivate
```

```
canal      montant moyen
-----
Boutique           74.81
Réseaux            49.01
Site               59.50
tabulate==0.10.0
tabulate==0.10.0
```

`pip freeze` liste ce qui est installé avec les **versions exactes** : c'est la clé de la reproductibilité. Quelqu'un qui reçoit le projet reconstruit **le même environnement** en trois commandes :

```
python -m venv .venv-college
source .venv-college/bin/activate
pip install --quiet -r requirements.txt 2>&1 | grep -v -i -E "notice|warning"
pip freeze
deactivate
```

```
tabulate==0.10.0
```

La liste obtenue est **identique. Deux versions d'une même bibliothèque.** Créons un second environnement, `.venv-ancien`, avec une version plus ancienne exigée par `==` ; les deux coexistent sans se gêner (on appelle ici directement le pip de chaque environnement, sans l'activer : c'est équivalent et très pratique dans un script).

```
python -m venv .venv-ancien
.venv-ancien/bin/pip install --quiet "tabulate==0.8.10" 2>&1 | grep -v -i -E "notice|warning"
echo "projet ancien :"
.venv-ancien/bin/pip freeze
echo "projet récent :"
.venv/bin/pip freeze
```

```
projet ancien :
tabulate==0.8.10
projet récent :
tabulate==0.10.0
```

Pour aller plus loin. Écrivez un `requirements.txt` à la main qui accepte n'importe quelle version 0.9.x de `tabulate` (écriture `>=0.9, <0.10`) et vérifiez ce que `pip install -r` installe.

Application 6.10 — Un rapport reproductible, de Markdown à PDF, avec make

Objectif. Assembler les outils : graines aléatoires, relevé d'environnement, rapport dont les chiffres sont calculés, conversion avec Pandoc, Makefile, empreinte des données. *À la suite de l'application 6.9. Sections du livre : 6.5.2 à 6.5.5 et 6.5.8.*

Les graines. Sans graine, deux exécutions donnent des tirages différents ; avec la même graine, des tirages identiques.

```
import numpy as np
import pandas as pd

montants = pd.read_csv("donnees/commandes.csv")["montant"].to_numpy()

def moyenne_echantillon(rng, taille=10):
    return montants[rng.choice(len(montants), size=taille, replace=False)].mean()

a = moyenne_echantillon(np.random.default_rng())
b = moyenne_echantillon(np.random.default_rng())
print("sans graine, deux exécutions identiques ?", a == b)
c = moyenne_echantillon(np.random.default_rng(42))
d = moyenne_echantillon(np.random.default_rng(42))
print("graine 42 deux fois :", round(c, 2), "et", round(d, 2), "-> identiques ?", c == d)
```

```
sans graine, deux exécutions identiques ? False
graine 42 deux fois : 46.55 et 46.55 -> identiques ? True
```

La graine sert à **figer** une exécution, pas à l'améliorer. Pour vérifier qu'un résultat n'est pas un accident de graine, on l'observe pour **plusieurs graines** :

```
moyennes = [moyenne_echantillon(np.random.default_rng(graine)) for graine in range(1, 9)]
print("moyennes pour les graines 1 à 8 :", [round(float(m), 1) for m in moyennes])
print("vraie moyenne de la population :", round(montants.mean(), 2))
```

```
moyennes pour les graines 1 à 8 : [54.1, 60.5, 45.2, 63.4, 71.8, 73.3, 59.5, 43.3]
vraie moyenne de la population : 60.25
```

Relever l'environnement. On peut imprimer, en fin de rapport, ce qui a réellement servi :

```
import sys
import importlib.metadata as meta

def empreinte_environnement(paquets=("numpy", "pandas", "scipy", "matplotlib")):
    lignes = [f"Python {sys.version.split()[0]}"]
    lignes += [f"{p} {meta.version(p)}" for p in paquets]
    return lignes

print("\n".join(empreinte_environnement()))
```

```
Python 3.13.3
numpy 2.5.3
pandas 3.0.6
scipy 1.18.1
matplotlib 3.11.2
```

Un rapport qui se fabrique tout seul. Un programme Python écrit un document Markdown en y insérant les chiffres calculés (aucun chiffre en dur). On l'écrit en deux morceaux.

```
cd ~/atelier/etude-ventes
cat > src/faire_rapport.py <<'FIN'
"""Écrit sur la sortie standard un rapport Markdown : python src/faire_rapport.py commandes.csv"""
import sys
import numpy as np
```

```
import pandas as pd

def fr(x, decimales=2):
    """Nombre au format français : virgule décimale."""
    return f"{x:.{decimales}f}".replace(".", ",")

df = pd.read_csv(sys.argv[1])
n = len(df)
m = df["montant"]
marge = 1.96 * m.std() / np.sqrt(n)          # demi-largeur de l'intervalle de confiance à 95 %
par_canal = df.groupby("canal")["montant"].agg(["count", "mean"]).sort_index()
FIN
```

Seconde moitié du programme : l'écriture du document, en Markdown.

```
cat >> src/faire_rapport.py <<'FIN'

print("---")
print('title: "Les ventes de la boutique"')
print("lang: fr")
print("---")
print()
print(f"Le fichier contient **{n} commandes**. Le montant moyen est de **{fr(m.mean())} €** "
      f"(intervalle de confiance à 95 % : de {fr(m.mean() - marge)} à {fr(m.mean() + marge)} €), "
      f"et la médiane de **{fr(m.median())} €**.")
print()
print("L'intervalle est calculé par  $\bar{x} \pm 1,96 \frac{s}{\sqrt{n}}$  ")
print()
print("| Canal | Commandes | Montant moyen (€) |")
print(":-:|-----|-----:|-----:")
for canal, ligne in par_canal.iterrows():
    print(f"| {canal} | {int(ligne['count'])} | {fr(ligne['mean'])} |")
print()
print(f"Le canal au panier moyen le plus élevé est **{par_canal['mean'].idxmax()}**.")
FIN

python src/faire_rapport.py donnees/commandes.csv > rapports/rapport-ventes.md
cat rapports/rapport-ventes.md
```

```
---
title: "Les ventes de la boutique"
lang: fr
---
```

Le fichier contient **400 commandes**. Le montant moyen est de **60,25 €** (intervalle de confiance à 95 % : de 56,52 à 63,97 €), et la médiane de **51,00 €**.

L'intervalle est calculé par $\bar{x} \pm 1,96 \frac{s}{\sqrt{n}}$.

Canal	Commandes	Montant moyen (€)
:-:	-----	-----:
Boutique	114	74,81
Réseaux	138	49,01
Site	148	59,50

Le canal au panier moyen le plus élevé est **Boutique**.

(Les doubles barres `\` dans le programme sont des échappements Python pour obtenir une seule barre `\` dans la formule LaTeX.) **Pandoc** convertit le Markdown en HTML, en texte brut, en PDF :

```
pandoc --standalone --mathml rapports/rapport-ventes.md -o rapports/rapport-ventes.html
pandoc rapports/rapport-ventes.md -t plain | head -n 14
pandoc rapports/rapport-ventes.md -o rapports/rapport-ventes.pdf --pdf-engine=xelatex
head -c 5 rapports/rapport-ventes.pdf; echo
```

[WARNING] Could not convert TeX math $\bar{x} \pm 1,96 \frac{s}{\sqrt{n}}$, rendering as TeX
 Le fichier contient 400 commandes. Le montant moyen est de 60,25 €
 (intervalle de confiance à 95 % : de 56,52 à 63,97 €), et la médiane de
 51,00 €.

L'intervalle est calculé par $\bar{x} \pm 1,96 \frac{s}{\sqrt{n}}$.

Canal	Commandes	Montant moyen (€)
Boutique	114	74,81
Réseaux	138	49,01
Site	148	59,50

Le canal au panier moyen le plus élevé est Boutique.

%PDF-

Pandoc **avertit** ([WARNING]) qu'il ne sait pas écrire une fraction en texte brut et laisse la formule en LaTeX : normal. Les cinq premiers octets du PDF sont %PDF-, la signature du format. **make : ne refaire que ce qui a changé.** On décrit dans un Makefile des règles *cible : dépendances* suivies de la commande ; make compare les dates de modification. (Les commandes doivent être indentées par une **vraie tabulation** ; nous écrivons <TAB> puis un sed le remplace.)

```
cat > Makefile <<'FIN'
.PHONY: tout propre

tout: rapports/rapport-ventes.html

rapports/rapport-ventes.md: donnees/commandes.csv src/faire_rapport.py
<TAB>python src/faire_rapport.py donnees/commandes.csv > $@

rapports/rapport-ventes.html: rapports/rapport-ventes.md
<TAB>pandoc --standalone --mathml $< -o $@

propre:
<TAB>rm -f rapports/rapport-ventes.md rapports/rapport-ventes.html
FIN
sed -i 's/^<TAB>/\t/' Makefile
make propre
make
echo "--- deuxième appel, rien n'a changé ---"
make
echo "--- on touche le fichier de données ---"
touch donnees/commandes.csv
make

rm -f rapports/rapport-ventes.md rapports/rapport-ventes.html
python src/faire_rapport.py donnees/commandes.csv > rapports/rapport-ventes.md
pandoc --standalone --mathml rapports/rapport-ventes.md -o rapports/rapport-ventes.html
--- deuxième appel, rien n'a changé ---
make: Nothing to be done for 'tout'.
--- on touche le fichier de données ---
python src/faire_rapport.py donnees/commandes.csv > rapports/rapport-ventes.md
pandoc --standalone --mathml rapports/rapport-ventes.md -o rapports/rapport-ventes.html
```

Dans les commandes, \$@ désigne la cible, \$< la première dépendance. Le premier make refait **les deux** étapes, le deuxième répond qu'il n'y a **rien à faire**, et quand on « touche » les données, les deux étapes repartent. **Une empreinte des données.** On calcule celle de commandes.csv, on la note, et n'importe qui peut vérifier ; une faute de frappe fait sonner l'alerte avant même l'analyse.

```
sha256sum donnees/commandes.csv > donnees/EMPREINTES.sha256
cat donnees/EMPREINTES.sha256
```

```
sha256sum -c donnees/EMPREINTES.sha256
echo "--- une faute de frappe glisse dans le fichier ---"
sed -i '2s/44.8/44.0/' donnees/commandes.csv
sha256sum -c donnees/EMPREINTES.sha256 || echo "ALERTE : les données ont été modifiées"
sed -i '2s/44.0/44.8/' donnees/commandes.csv
sha256sum -c donnees/EMPREINTES.sha256
```

```
27f43bf0bf7fed83c6b0e313e8a6e212547175fb6d596de3141dfde256d88acc  donnees/commandes.csv
donnees/commandes.csv: OK
--- une faute de frappe glisse dans le fichier ---
donnees/commandes.csv: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match
ALERTE : les données ont été modifiées
donnees/commandes.csv: OK
```

Pour aller plus loin. Ajoutez au Makefile une cible `verifier` qui contrôle l'empreinte des données, et faites dépendre le rapport de cette cible : `make` doit alors refuser de construire un rapport sur des données altérées.

Application 6.11 — R Markdown et LaTeX

Objectif. Compiler un vrai document dynamique R Markdown (le texte et les chiffres naissent du même code), puis un minuscule document LaTeX. *Sections du livre* : 6.5.6 et 6.5.7.

R Markdown mélange texte Markdown et blocs de code. Comme le format utilise les trois accents graves que ce cahier emploie lui-même, nous écrivons les blocs avec des tildes `~~~` et un `sed` les remplace avant la compilation. Le document calcule la satisfaction moyenne des commandes livrées, en distinguant les livraisons rapides (3 jours ou moins) des autres.

```
mkdir -p ~/atelier/etude-rmd
cd ~/atelier/etude-rmd
cp "$DONNEES/commandes.csv" .
cat > rapport.Rmd <<'FIN'
---
title: "Satisfaction des clients de la boutique"
output: html_document
---

Voici le lien entre le délai de livraison et la note de satisfaction.

~~~{r}
d <- read.csv("commandes.csv")
d <- d[d$canal != "Boutique", ]      # on ne garde que les commandes livrées
tapply(d$satisfaction, d$livraison <= 3, mean)
~~~

La satisfaction moyenne des commandes livrées est de **`r round(mean(d$satisfaction), 2)`**
sur `r nrow(d)` commandes.
FIN
sed -i 's/^~~~{r}$`{r}`/; s/^~~~$`{r}`/' rapport.Rmd
Rscript -e 'rmarkdown::render("rapport.Rmd", quiet = TRUE)'
ls
pandoc rapport.html -t plain
```

```
commandes.csv
rapport.Rmd
rapport.html
Satisfaction des clients de la boutique
```

Voici le lien entre le délai de livraison et la note de satisfaction.

```
d <- read.csv("commandes.csv")
d <- d[d$canal != "Boutique", ]      # on ne garde que les commandes livrées
tapply(d$satisfaction, d$livraison <= 3, mean)

##      FALSE      TRUE
## 3.631313 4.034091
```

La satisfaction moyenne des commandes livrées est de 3.76 sur 286 commandes.

Le chiffre du texte (l'expression `r = round(...)` écrite entre accents graves, au milieu de la phrase) est **calculé** à la compilation, et le bloc de code est exécuté avec sa sortie insérée sous lui (lignes `##`). `FALSE` correspond aux livraisons de plus de 3 jours, `TRUE` aux livraisons de 3 jours ou moins : les clients livrés vite sont plus satisfaits. Notez le détail : R affiche un point décimal, alors que notre rapport Python de l'application 6.10 écrivait une virgule grâce à sa fonction `fr()`. **LaTeX** est le langage qui met en forme les formules du livre. Compilons un minuscule document :

```
mkdir -p ~/atelier/mini-latex
cd ~/atelier/mini-latex
cat > mini.tex <<'FIN'
\documentclass{article}
\usepackage{fontspec}
\usepackage{amsmath}
\usepackage[french]{babel}
\begin{document}
\section*{Intervalle de confiance}
Pour un échantillon de taille  $n$ , de moyenne  $\bar{x}$  et d'écart-type  $s$ ,
l'intervalle de confiance à 95%, de la moyenne est
\[\bar{x} \pm 1.96 \frac{s}{\sqrt{n}}\]
\end{document}
FIN
xelatex -interaction=nonstopmode -halt-on-error mini.tex > compilation.log 2>&1
echo "code de sortie : $?"
head -c 5 mini.pdf; echo
```

```
code de sortie : 0
%PDF-
```

Le code de sortie 0 indique que la compilation a réussi, et `mini.pdf` commence bien par `%PDF-`. Si une erreur survient (une accolade oubliée), `xelatex` s'arrête et la raison se trouve dans `compilation.log`. **Pour aller plus loin.** Ajoutez au document LaTeX la formule de l'écart-type d'échantillon, puis provoquez volontairement une erreur (une accolade en moins) et retrouvez-la dans le journal.

Exercices

Cherchez d'abord par vous-même (en tapant les commandes dans un vrai terminal !), vérifiez ensuite, et seulement après lisez le corrigé. ★ = application directe, ★★ = raisonnement, ★★★ = synthèse. Les exercices utilisent un dossier de travail neuf, `~/atelier/exercices`, et le fichier `commandes.csv`.

Exercice 6.1 ★ — Se repérer (section 6.3)

Créez le dossier projet-boutique contenant deux sous-dossiers, donnees et src, copiez-y commandes.csv (dans donnees), puis (a) affichez l'arborescence créée, (b) affichez les **trois dernières** lignes du fichier, (c) comptez ses lignes.

Exercice 6.2 ★ — Tuyaux (section 6.3)

Combien de commandes ont reçu chacune des notes de satisfaction (1 à 5) ? Répondez avec **une seule ligne** de commandes enchaînées par des tuyaux, puis vérifiez avec pandas.

Exercice 6.3 ★ — Premiers pas avec Git (section 6.1)

Dans un nouveau dépôt journal, créez un fichier notes.txt contenant la ligne « Hypothèse : les commandes Réseaux sont plus petites. » et enregistrez-le (commit 1). Ajoutez une seconde ligne « Test à faire : comparer les moyennes par canal. » et enregistrez (commit 2). Par maladresse, vous supprimez ensuite notes.txt avec rm. Retrouvez-le **sans refaire à la main**, puis affichez l'historique en une ligne par commit.

Exercice 6.4 ★★ — awk (section 6.3)

Calculez, pour chaque canal, le **pourcentage de commandes notées 4 ou 5** (colonne satisfaction), avec awk. Vérifiez avec pandas.

Exercice 6.5 ★★ — Conflit (section 6.1)

Un fichier params.txt contient la ligne seuil=50. Sur une branche prudent, vous passez la valeur à seuil=40 ; sur main, votre collègue la passe à seuil=60. Fusionnez prudent dans main. Que se passe-t-il ? Résolvez le conflit en gardant la valeur la **plus élevée**, et terminez la fusion.

Exercice 6.6 ★★ — État caché (section 6.2)

Un notebook contient, de haut en bas, les cinq cellules suivantes : (A) `n = 100` ; (B) `taux = 0.19` ; (C) `tva = n * taux` ; (D) `n = 200` ; (E) `print(tva)`. La gérante les exécute dans l'ordre **A, B, D, C, E** (elle a lancé D avant C). (a) Qu'affiche E ? (b) Qu'afficherait E après « Restart & Run All » ? (c) Quelle conclusion en tirez-vous ?

Exercice 6.7 ★★ — Environnement (section 6.3)

Créez un environnement virtuel env-a, installez-y tabulate==0.8.9 et enregistrez les versions dans requirements.txt. Reconstituez ensuite, dans un second environnement env-b, **exactement les mêmes** bibliothèques à partir de ce seul fichier, et prouvez qu'elles sont identiques.

Exercice 6.8 ★★ — Script shell (section 6.4)

Écrivez un script compte.sh qui prend **deux arguments**, un fichier CSV et un numéro de colonne, et affiche le nombre d'occurrences de chaque valeur de cette colonne (sans l'en-tête). Il doit s'arrêter avec un message et un code d'erreur non nul si on lui donne un mauvais nombre d'arguments ou un fichier inexistant. Testez-le sur la colonne 1 puis sur la colonne 4.

Exercice 6.9 ★★★ — Makefile (section 6.5)

Écrivez un Makefile dont la cible `resume.txt` dépend de `commandes.csv` et du script `resume.py`, et qui la fabrique par `python resume.py commandes.csv > resume.txt`. Montrez que : (a) un premier `make` construit la cible, (b) un deuxième ne fait rien, (c) modifier le script (date de modification) relance la construction, (d) modifier un fichier **sans rapport** (`LISEZMOI.txt`) ne la relance pas.

Exercice 6.10 ★★★ — Synthèse : un projet reproductible (section 6.5)

Montez un mini-projet « rapport sur les ventes » qui réunit tout le chapitre : dépôt Git avec `.gitignore`, données, script de rapport, Makefile, empreinte sha256 des données, un commit étiqueté `v1.0`. Démontrez sa reproductibilité : **clonez** le dépôt dans un autre dossier, vérifiez l'empreinte des données, lancez `make`, et prouvez que le rapport obtenu est **identique octet pour octet** à l'original.

Corrigés

Corrigé 6.1

`mkdir -p` crée d'un coup les dossiers imbriqués (et l'accolade en crée deux) ; `find` affiche l'arborescence ; `tail -n 3` et `wc -l` répondent à (b) et (c).

```
mkdir -p ~/atelier/exercices
cd ~/atelier/exercices
mkdir -p projet-boutique/{donnees,src}
cp "$DONNEES/commandes.csv" projet-boutique/donnees/
find projet-boutique | sort
tail -n 3 projet-boutique/donnees/commandes.csv
wc -l projet-boutique/donnees/commandes.csv
```

```
projet-boutique
projet-boutique/donnees
projet-boutique/donnees/commandes.csv
projet-boutique/src
Site,62.6,4,3
Réseaux,37.5,3,4
Site,31.4,4,4
401 projet-boutique/donnees/commandes.csv
```

Le fichier compte 401 lignes : 400 commandes plus l'en-tête.

Corrigé 6.2

La note est dans la 4^e colonne. On enlève l'en-tête (`tail -n +2`), on extrait la colonne (`cut`), on **trie** (indispensable avant `uniq`), puis on compte les groupes (`uniq -c`) :

```
cd ~/atelier/exercices/projet-boutique/donnees
tail -n +2 commandes.csv | cut -d, -f4 | sort -n | uniq -c
```

```
1 1
15 2
85 3
195 4
104 5
```

Vérification avec pandas (même résultat attendu) :

```
import pandas as pd
df = pd.read_csv("donnees/commandes.csv")
print(df["satisfaction"].value_counts().sort_index())
```

```
satisfaction
1      1
2     15
3     85
4    195
5    104
Name: count, dtype: int64
```

Les deux méthodes donnent les mêmes effectifs : la note 4 est la plus fréquente, et les notes très basses sont rares (une seule note 1), ce qui est cohérent avec une satisfaction moyenne proche de 4 (3,96). Le total fait bien 400.

Corrigé 6.3

`git restore` ramène un fichier supprimé ou modifié à son état du dernier commit. Comme notre `rm` a supprimé le fichier **sans** l'enregistrer, c'est la bonne commande :

```
cd ~/atelier/exercices
mkdir journal
cd journal
git init -q
echo "Hypothèse : les commandes Réseaux sont plus petites." > notes.txt
git add notes.txt
git commit -q -m "Première hypothèse"
echo "Test à faire : comparer les moyennes par canal." >> notes.txt
git commit -q -am "Ajout du test à faire"
rm notes.txt
ls
git restore notes.txt
cat notes.txt
git log --oneline
```

```
Hypothèse : les commandes Réseaux sont plus petites.
Test à faire : comparer les moyennes par canal.
3af636d Ajout du test à faire
fe687dd Première hypothèse
```

Après le `rm`, `ls` n'affiche rien (le fichier a disparu) ; après `git restore`, le fichier est revenu avec **ses deux lignes**, car elles figuraient dans le dernier commit. Le tout a été possible parce qu'on avait enregistré : un fichier jamais commité aurait été perdu.

Corrigé 6.4

On compte, pour chaque canal, le nombre total de commandes (`n`) et le nombre de commandes notées 4 ou 5 (bons) ; à la fin, on affiche le pourcentage. (Dans `awk`, une case de tableau jamais utilisée vaut 0, donc `bons[c]` fonctionne même si un canal n'a aucune bonne note.)

```
cd ~/atelier/exercices/projet-boutique/donnees
awk -F, 'NR > 1 { n[$1]++; if ($4 >= 4) bons[$1]++ }
        END { for (c in n) printf "%-10s %.1f %%\n", c, 100 * bons[c] / n[c] }' commandes.csv | sort
```

```
Boutique    95.6 %
Réseaux     63.0 %
Site        69.6 %
```

Et la vérification avec pandas :

```
satisfaits = (df["satisfaction"] >= 4).groupby(df["canal"]).mean() * 100
print(satisfaits.round(1))
```

```
canal
Boutique    95.6
Réseaux     63.0
Site        69.6
Name: satisfaction, dtype: float64
```

Les deux approches coïncident. Les clients de la **boutique** sont les plus satisfaits : pas de délai de livraison, donc pas de pénalité (voir la construction du jeu de données au 3.1.2, où la note diminue avec le délai).

Corrigé 6.5

Les deux branches ont modifié **la même ligne** : Git ne peut pas choisir, il signale un conflit. On édite le fichier pour garder `seuil=60`, on déclare le conflit résolu avec `git add`, puis on termine par un `commit`.

```
cd ~/atelier/exercices
mkdir conflit
cd conflit
git init -q
echo "seuil=50" > params.txt
git add params.txt
git commit -q -m "Paramètre initial"
git switch -q -c prudent
echo "seuil=40" > params.txt
git commit -q -am "Seuil prudent"
git switch -q main
echo "seuil=60" > params.txt
git commit -q -am "Seuil ambitieux"
git merge --no-edit prudent
echo "----- fichier en conflit -----"
cat params.txt
```

```
Auto-merging params.txt
CONFLICT (content): Merge conflict in params.txt
Automatic merge failed; fix conflicts and then commit the result.
----- fichier en conflit -----
<<<<<< HEAD
seuil=60
=====
seuil=40
>>>>>> prudent
```

Le fichier contient maintenant les marqueurs `<<<<<<`, `=====` et `>>>>>>` autour des deux versions. On résout en écrivant la valeur voulue (ici nous la réécrivons directement, ce qui revient à supprimer les marqueurs et la ligne `seuil=40`) :

```
echo "seuil=60" > params.txt
git add params.txt
git commit -q --no-edit
git log --oneline --graph
cat params.txt
```

```
* 772e5c8 Merge branch 'prudent'
|\
| * 3723e50 Seuil prudent
* | 8d91b2b Seuil ambitieux
```

```
|/
* f3f63d7 Paramètre initial
seuil=60
```

Le graphe montre bien la **fusion** de deux lignes de travail en un commit à deux parents. N'oubliez pas, dans un vrai projet, de **relancer le code** avant de valider.

Corrigé 6.6

On simule le noyau avec un dictionnaire, comme dans l'application 6.5. Le point clé est l'**ordre d'exécution**, pas l'ordre d'écriture :

```
def jouer(ordre):
    memoire = {}
    cellules = {"A": "n = 100", "B": "taux = 0.19", "C": "tva = n * taux",
               "D": "n = 200", "E": "print(round(tva, 2))"}
    for nom in ordre:
        exec(cellules[nom], memoire)

print("(a) ordre A, B, D, C, E :", end=" ")
jouer("ABDCE")
print("(b) Restart & Run All (A, B, C, D, E) :", end=" ")
jouer("ABCDE")
```

```
(a) ordre A, B, D, C, E : 38.0
(b) Restart & Run All (A, B, C, D, E) : 19.0
```

- (a) Dans l'ordre réellement joué, n vaut déjà 200 quand C calcule la TVA : E affiche $200 \times 0,19 = 38$. (b) Dans l'ordre du fichier, C est exécutée alors que n vaut encore 100 : E affiche $100 \times 0,19 = 19$. (c) Le **même notebook** donne deux résultats différents selon l'ordre d'exécution : le chiffre que la gérante voyait à l'écran (38) n'est pas celui qu'obtiendra quiconque ouvrira le fichier et exécutera tout (19). C'est exactement l'état caché : **toujours** redémarrer et tout exécuter avant de se fier à un résultat. (Une analyse correcte de la logique du carnet dirait aussi que la cellule D, placée *après* C mais qui change n , est probablement mal placée.)

Corrigé 6.7

On crée le premier environnement, on y installe la version exacte demandée, puis on fige ; le second environnement est reconstruit uniquement depuis `requirements.txt`.

```
cd ~/atelier/exercices
mkdir env-demo
cd env-demo
python -m venv env-a
env-a/bin/pip install --quiet "tabulate==0.8.9" 2>&1 | grep -v -i -E "notice|warning"
env-a/bin/pip freeze > requirements.txt
cat requirements.txt
python -m venv env-b
env-b/bin/pip install --quiet -r requirements.txt 2>&1 | grep -v -i -E "notice|warning"
env-b/bin/pip freeze > freeze-b.txt
cmp requirements.txt freeze-b.txt && echo "les deux environnements sont identiques"
```

```
tabulate==0.8.9
les deux environnements sont identiques
```

La commande `cmp` compare deux fichiers octet par octet et ne dit rien s'ils sont identiques ; notre `&& echo` confirme alors le succès. (On appelle directement `env-a/bin/pip` au lieu d'activer l'environnement : c'est strictement équivalent.)

Corrigé 6.8

Le script vérifie ses arguments (\$#), l'existence du fichier (-f), puis enchaîne les outils de l'application 6.6 : tail (supprimer l'en-tête), cut (extraire la colonne), sort, uniq -c.

```
cd ~/atelier/exercices/projet-boutique
cat > src/compte.sh <<'FIN'
#!/usr/bin/env bash
# compte.sh : effectifs de chaque valeur d'une colonne d'un CSV
# usage : bash src/compte.sh fichier.csv numero_de_colonne
set -euo pipefail
if [ "$#" -ne 2 ]; then
    echo "usage : $0 fichier.csv numero_de_colonne" >&2
    exit 2
fi
if [ ! -f "$1" ]; then
    echo "fichier introuvable : $1" >&2
    exit 1
fi
tail -n +2 "$1" | cut -d, -f"$2" | sort | uniq -c
FIN
echo "--- colonne 1 (canal) ---"
bash src/compte.sh donnees/commandes.csv 1
echo "--- colonne 4 (satisfaction) ---"
bash src/compte.sh donnees/commandes.csv 4
echo "--- erreurs ---"
bash src/compte.sh donnees/commandes.csv; echo "code : $?"
bash src/compte.sh absent.csv 1; echo "code : $?"
```

```
--- colonne 1 (canal) ---
 114 Boutique
 138 Réseaux
 148 Site
--- colonne 4 (satisfaction) ---
   1 1
  15 2
  85 3
 195 4
 104 5
--- erreurs ---
usage : src/compte.sh fichier.csv numero_de_colonne
code : 2
fichier introuvable : absent.csv
code : 1
```

Les effectifs de la colonne 4 sont ceux de l'exercice 6.2, et ceux de la colonne 1 retrouvent les effectifs par canal. Les deux appels fautifs affichent leur message et renvoient des codes 2 (mauvais usage) et 1 (fichier absent), comme convenu.

Corrigé 6.9

Le Makefile décrit la dépendance de resume.txt à ses deux sources. Le script resume.py est écrit ici en quelques lignes pour que l'exercice soit autonome ; pour la tabulation, on emploie encore la substitution <TAB>.

```
cd ~/atelier/exercices
mkdir make-demo
cd make-demo
cp "$DONNEES/commandes.csv" .
cat > resume.py <<'FIN'
import sys
import pandas as pd
```

```
df = pd.read_csv(sys.argv[1])
print(f"{len(df)} lignes, {df.shape[1]} colonnes")
print(df.describe().loc[["mean", "min", "max"]].round(2))
FIN
echo "Notes de lecture" > LISEZMOI.txt
cat > Makefile <<'FIN'
resume.txt: commandes.csv resume.py
<TAB>python resume.py commandes.csv > resume.txt
FIN
sed -i 's/^(<TAB>)/\t/' Makefile
```

```
echo "--- (a) premier make ---"
make
echo "--- (b) deuxième make ---"
make
echo "--- (c) on touche resume.py ---"
touch resume.py
make
echo "--- (d) on touche LISEZMOI.txt ---"
touch LISEZMOI.txt
make
cat resume.txt
```

```
--- (a) premier make ---
python resume.py commandes.csv > resume.txt
--- (b) deuxième make ---
make: 'resume.txt' is up to date.
--- (c) on touche resume.py ---
python resume.py commandes.csv > resume.txt
--- (d) on touche LISEZMOI.txt ---
make: 'resume.txt' is up to date.
400 lignes, 4 colonnes
      montant  livraison  satisfaction
mean      60.25       3.29         3.96
min       8.60       0.00         1.00
max     255.70     13.00         5.00
```

- (a) Le premier make exécute la commande ; (b) le deuxième répond que la cible est déjà à jour ; (c) en modifiant la date du script, make détecte que la dépendance est plus récente que la cible et **refabrique** ; (d) LISEZMOI.txt n'est pas une dépendance : make n'y réagit pas. Seul ce qui est **déclaré** comme dépendance déclenche une reconstruction, d'où l'importance de **lister toutes** les sources dans la règle.

Corrigé 6.10

C'est le projet de synthèse du chapitre. On le construit pas à pas, puis on le **clone** pour le rejouer ailleurs. D'abord le projet, avec son script de rapport (très simple, il n'écrit ni date ni valeur aléatoire), son Makefile et l'empreinte des données :

```
cd ~/atelier/exercices
mkdir ventes-reproductibles
cd ventes-reproductibles
git init -q
mkdir donnees src rapports
cp "$DONNEES/commandes.csv" donnees/
sha256sum donnees/commandes.csv > donnees/EMPREINTES.sha256
cat > src/rapport.py <<'FIN'
import sys
import pandas as pd

df = pd.read_csv(sys.argv[1])
```

```
print("# Rapport sur les ventes")
print()
print(f"- Commandes : {len(df)}")
print(f"- Montant moyen : {df['montant'].mean():.2f} €")
print(f"- Satisfaction moyenne : {df['satisfaction'].mean():.2f} / 5")
FIN
```

On ajoute le Makefile, un .gitignore et un README, puis on construit le rapport.

```
cat > Makefile <<'FIN'
rapports/rapport.md: donnees/commandes.csv src/rapport.py
<TAB>python src/rapport.py donnees/commandes.csv > $@
FIN
sed -i 's/^<TAB>/\t/' Makefile
printf '.venv/\n__pycache__/\n*.env\n' > .gitignore
printf 'Pour tout reproduire : sha256sum -c donnees/EMPREINTES.sha256 && make\n' > README.md
make
cat rapports/rapport.md
```

```
python src/rapport.py donnees/commandes.csv > rapports/rapport.md
# Rapport sur les ventes

- Commandes : 400
- Montant moyen : 60.25 €
- Satisfaction moyenne : 3.96 / 5
```

Puis on enregistre dans Git : le **rapport produit** est lui aussi versionné ici pour pouvoir le comparer, mais en pratique on ne versionne que les sources (6.1.11). On pose l'étiquette v1.0 :

```
git add .
git commit -q -m "Projet de rapport reproductible"
git tag -a v1.0 -m "Version livrée"
git log --oneline
git status --short
```

```
ee53fd3 Projet de rapport reproductible
```

Il ne reste rien d'« en attente » (`git status` est vide). Passons à la **preuve** : un collègue clone le dépôt dans un dossier tout neuf, vérifie l'intégrité des données, reconstruit le rapport (après avoir supprimé le rapport cloné, pour être sûr que `make` le refabrique), et compare :

```
cd ~/atelier/exercices
git clone -q ventes-reproductibles colleague
cd colleague
git checkout -q v1.0
sha256sum -c donnees/EMPREINTES.sha256
rm rapports/rapport.md
make
cmp rapports/rapport.md ../ventes-reproductibles/rapports/rapport.md && echo "rapports identiques,
↪ octet pour octet"
```

```
donnees/commandes.csv: OK
python src/rapport.py donnees/commandes.csv > rapports/rapport.md
rapports identiques, octet pour octet
```

Les données sont intègres (OK), le rapport a été refabriqué par `make`, et la comparaison `cmp` confirme qu'il est **identique** à l'original. C'est la définition opérationnelle de la reproductibilité : *un tiers, sur un dossier neuf, obtient exactement le même résultat avec une commande*. Chaque élément du chapitre y a joué son rôle : Git (le code et ses versions), le .gitignore, le Makefile, l'empreinte des données, et, pour l'environnement, le `requirements.txt` que l'on ajouterait dans un vrai projet.

Projet du volume et auto-évaluation

— exercices et applications

Ce chapitre du cahier clôt le volume I : un **projet de bout en bout** qui réunit tout ce que le livre a enseigné, puis **trente questions** pour vérifier vos acquis. Il utilise la base données/boutique.db (fournie avec le livre).

Projet du volume

« Un projet de data science, ce n'est pas un modèle. C'est une **question**, des données, une réponse honnête... et un message que quelqu'un pourra lire. »

Six chapitres : des maths, des probabilités, de la statistique, du Python, du SQL, des outils. Chaque brique a été vue séparément, sur de petits exemples. Ce projet les **assemble** dans une seule étude de bout en bout, comme on le ferait dans un vrai travail : on reçoit une demande floue, on va chercher les données, on les contrôle, on les explore, on répond avec rigueur, et on rédige un rapport qu'une personne non spécialiste peut lire.

Comment lire ce projet. Il n'introduit **aucune notion nouvelle** : chaque étape renvoie à la section du livre qui l'explique. Si vous bloquez sur une étape, c'est le signal d'aller relire cette section, pas de tout recommencer. Le meilleur usage : lire d'abord le cahier des charges (P.1), **fermer le livre**, essayer de répondre à la gérante avec vos propres moyens, puis comparer.

P.1 Le cahier des charges

Fin décembre 2025. La gérante d'une boutique vous envoie ce message :

« Bonjour ! L'année est finie et j'ai un peu le vertige : des commandes partout, trois canaux de vente, et aucune idée de ce qui marche vraiment. Quatre questions me trottent dans la tête pendant les fêtes :

- 1. Comment s'est passée mon année 2025 ? (chiffre d'affaires, rythme, canaux)*
- 2. Les réseaux sociaux me prennent beaucoup de temps. Est-ce que les clients qui viennent de là dépensent vraiment moins que ceux de la boutique, ou est-ce une impression ?*
- 3. Je soupçonne que les retards de livraison font baisser la satisfaction. Est-ce vrai, et de combien ?*
- 4. Qui dois-je relancer en janvier ?*

*Je n'ai besoin ni de formules ni de jargon : juste des chiffres fiables et ce que vous me conseillez.
Merci ! »*

Transformer ce message en travail demande une **méthode**. La nôtre tient en six étapes, que nous suivrons dans l'ordre :

Étape	Question que l'on se pose	Outils du livre
1. Charger et contrôler	« Les données sont-elles fiables ? »	SQL (ch. 5), assertions (4.1.10, 4.6)
2. Photographier	« Que s'est-il passé, en gros ? »	SQL agrégé et fenêtres (5.2, 5.3), figures (4.5)
3. Comparer	« Cette différence est-elle réelle ou due au hasard ? »	tests, intervalles, tests multiples (3.3 à 3.5)
4. Relier	« Deux variables évoluent-elles ensemble ? »	corrélation, moindres carrés (1.3, 2.3, 3.1)
5. Segmenter	« Qui sont les clients à surveiller ? »	CTE, NTILE (5.3)
6. Rapporter	« Que dire, et avec quelles limites ? »	pandas, reproductibilité (4.4, ch. 6)

Intuition. Un bon analyste passe **plus de temps sur les étapes 1 et 6** que sur l'étape « sophistiquée ». Des données mal contrôlées ruinent n'importe quelle analyse ; une analyse juste mal expliquée ne change aucune décision.

P.2 Étape 1 : charger et contrôler les données

La base de la boutique a été présentée au chapitre 5 (section 5.1) ; elle est fournie avec le livre dans `donnees/boutique.db`. Commençons par ouvrir la connexion et regarder ce qu'elle contient.

```
import sqlite3
import numpy as np
import pandas as pd

con = sqlite3.connect("donnees/boutique.db")
for table in ["categories", "produits", "clients", "commandes", "lignes_commande"]:
    n = con.execute(f"SELECT COUNT(*) FROM {table}").fetchone()[0]
    print(f"{table:16s} {n:4d} lignes")
```

```
categories      4 lignes
produits        16 lignes
clients         80 lignes
commandes       400 lignes
lignes_commande 693 lignes
```

Avant la moindre analyse, on **teste** les données. Ces contrôles sont des questions dont on connaît la réponse attendue : si elle diffère, il y a un problème à comprendre avant d'aller plus loin.

```
def un_seul_chiffre(sql):
    return con.execute(sql).fetchone()[0]

controles = {
    "aucune commande sans client connu":
        un_seul_chiffre("SELECT COUNT(*) FROM commandes c LEFT JOIN clients k USING(id_client) WHERE
↪ k.id_client IS NULL") == 0,
    "identifiants de commande uniques":
        un_seul_chiffre("SELECT COUNT(*) - COUNT(DISTINCT id_commande) FROM commandes") == 0,
```

```

    "dates comprises dans 2025":
        un_seul_chiffre("SELECT COUNT(*) FROM commandes WHERE date_commande NOT BETWEEN '2025-01-01'
↪ AND '2025-12-31'") == 0,
    "satisfaction toujours entre 1 et 5":
        un_seul_chiffre("SELECT COUNT(*) FROM commandes WHERE satisfaction NOT BETWEEN 1 AND 5") == 0,
    "montants strictement positifs":
        un_seul_chiffre("SELECT COUNT(*) FROM commandes WHERE montant <= 0") == 0,
    "retrait en boutique = délai nul":
        un_seul_chiffre("SELECT COUNT(*) FROM commandes WHERE (canal = 'Boutique') <> (delai_livraison
↪ = 0)") == 0,
    "les lignes d'une commande somment à son montant":
        un_seul_chiffre("""SELECT COUNT(*) FROM commandes c
                                JOIN (SELECT id_commande, ROUND(SUM(quantite * prix_unitaire), 2) AS total
                                FROM lignes_commande GROUP BY id_commande) l USING(id_commande)
                                WHERE ABS(c.montant - l.total) > 0.005""") == 0,
}
for nom, ok in controles.items():
    print("OK " if ok else "ÉCHEC", nom)
assert all(controles.values()), "au moins un contrôle a échoué : on s'arrête là"

```

```

OK aucune commande sans client connu
OK identifiants de commande uniques
OK dates comprises dans 2025
OK satisfaction toujours entre 1 et 5
OK montants strictement positifs
OK retrait en boutique = délai nul
OK les lignes d'une commande somment à son montant

```

Pourquoi assert ? Si un contrôle échoue, le programme **s'arrête** avec un message : mieux vaut un plantage bruyant qu'un rapport faux et élégant. C'est l'application directe de l'idée du 4.6 : un test automatique vaut mieux qu'un « j'ai regardé, ça avait l'air bon ».

Le contrôle du retrait en boutique mérite un mot : il vérifie un fait que **nous** savons sur l'activité (rien n'est livré quand on vient chercher sa commande). Les données se contrôlent avec la connaissance du métier, pas seulement avec des règles techniques.

Dernière préparation : charger en une seule requête un tableau pandas « une ligne par commande », avec la ville du client, sur lequel nous travaillerons.

```

df = pd.read_sql_query("""
    SELECT c.id_commande, c.id_client, c.date_commande, c.canal, c.montant,
           c.delai_livraison, c.satisfaction, k.ville
    FROM commandes c JOIN clients k USING(id_client)
    ORDER BY c.date_commande, c.id_commande
""", con, parse_dates=["date_commande"])
df["mois"] = df["date_commande"].dt.month
print(df.shape)
print(df.dtypes.to_string())

```

```

(400, 9)
id_commande          int64
id_client            int64
date_commande        datetime64[us]
canal                str
montant              float64
delai_livraison       int64
satisfaction         int64
ville                str
mois                 int32

```

P.3 Étape 2 : photographeur l'année (question 1)

On commence par les chiffres de base, **calculés par SQL** (c'est la base qui sait agréger efficacement) :

```
total = pd.read_sql_query("""
    SELECT COUNT(*) AS commandes, COUNT(DISTINCT id_client) AS clients_actifs,
           ROUND(SUM(montant), 2) AS chiffre_affaires, ROUND(AVG(montant), 2) AS panier_moyen
    FROM commandes""", con)
print(total.to_string(index=False))
```

commandes	clients_actifs	chiffre_affaires	panier_moyen
400	66	24098.3	60.25

Puis le rythme de l'année, avec la variation d'un mois à l'autre calculée par une **fonction fenêtre** (LAG, section 5.3) :

```
mensuel = pd.read_sql_query("""
    WITH m AS (
        SELECT CAST(strftime('%m', date_commande) AS INTEGER) AS mois,
               COUNT(*) AS commandes, ROUND(SUM(montant), 2) AS ca
        FROM commandes GROUP BY mois
    )
    SELECT mois, commandes, ca,
           ROUND(100.0 * (ca - LAG(ca) OVER (ORDER BY mois)) / LAG(ca) OVER (ORDER BY mois), 1) AS
↪  variation_pct
    FROM m ORDER BY mois
""", con)
print(mensuel.to_string(index=False))
```

mois	commandes	ca	variation_pct
1	16	996.4	NaN
2	20	1167.6	17.2
3	25	1687.1	44.5
4	36	1843.0	9.2
5	36	2314.1	25.6
6	33	2490.7	7.6
7	42	2570.9	3.2
8	44	2434.9	-5.3
9	31	1713.0	-29.6
10	20	1271.7	-25.8
11	40	2284.0	79.6
12	57	3324.9	45.6

Et la répartition par canal et par catégorie de produit (une jointure à quatre tables : commandes → lignes → produits → catégories) :

```
canaux = pd.read_sql_query("""
    SELECT canal, COUNT(*) AS commandes, ROUND(SUM(montant), 2) AS ca,
           ROUND(100.0 * SUM(montant) / (SELECT SUM(montant) FROM commandes), 1) AS part_ca_pct,
           ROUND(AVG(montant), 2) AS panier_moyen
    FROM commandes GROUP BY canal ORDER BY ca DESC""", con)
print(canaux.to_string(index=False))
print()
categories = pd.read_sql_query("""
    SELECT g.nom AS categorie, SUM(l.quantite) AS articles,
           ROUND(SUM(l.quantite * l.prix_unitaire), 2) AS ca
    FROM lignes_commande l
    JOIN produits p USING(id_produit) JOIN categories g USING(id_categorie)
    GROUP BY g.nom ORDER BY ca DESC""", con)
print(categories.to_string(index=False))
```

canal	commandes	ca	part_ca_pct	panier_moyen
Site	148	8806.5	36.5	59.50
Boutique	114	8528.3	35.4	74.81
Réseaux	138	6763.5	28.1	49.01

categorie	articles	ca
Bijoux	200	7006.20
Textile	184	6994.13
Poterie	180	6950.23
Cosmétiques	196	3147.74

Application : un seul regard. Un tableau de douze lignes se lit mal ; un graphique se lit en deux secondes. Voici le chiffre d'affaires mensuel **empilé par canal** (à gauche) et, pour préparer la question 3, la satisfaction moyenne selon le délai de livraison (à droite, avec son intervalle de confiance à 95 %).

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from scipy import stats

BLEU, ORANGE, AQUA = "#2a78d6", "#eb6834", "#1baf7a"
couleur = {"Boutique": AQUA, "Site": BLEU, "Réseaux": ORANGE}
noms_mois = ["jan", "fév", "mar", "avr", "mai", "juin", "juil", "août", "sept", "oct", "nov", "déc"]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.2), gridspec_kw={"width_ratios": [1.25, 1]})

# Panneau de gauche : CA mensuel empilé par canal
ca = df.pivot_table(index="mois", columns="canal", values="montant", aggfunc="sum").reindex(range(1, 13)).fillna(0)
bas = np.zeros(12)
for canal in ["Boutique", "Site", "Réseaux"]:
    ax1.bar(range(1, 13), ca[canal], bottom=bas, color=couleur[canal], width=0.75, label=canal)
    bas += ca[canal].to_numpy()
ax1.set_xticks(range(1, 13))
ax1.set_xticklabels(noms_mois, fontsize=8)
ax1.set_ylabel("chiffre d'affaires (€)")
ax1.set_title("Chiffre d'affaires mensuel, par canal")
ax1.legend(frameon=False, ncol=3, loc="upper left", fontsize=8)
ax1.grid(axis="x", visible=False)
```

Puis le panneau de droite : la satisfaction moyenne par palier de délai, avec son intervalle de confiance à 95 % (loi de Student, volume I, section 3.3), et l'enregistrement de la figure.

```
# Panneau de droite : satisfaction moyenne selon le délai (commandes livrées)
livre = df[df["canal"] != "Boutique"]
paliers = [(1, 2), (3, 4), (5, 6), (7, 8), (9, 20)]
etiquettes = ["1-2", "3-4", "5-6", "7-8", "9+"]
moy, demi_ic = [], []
for a, b in paliers:
    s = livre.loc[livre["delai_livraison"].between(a, b), "satisfaction"]
    h = stats.t.ppf(0.975, len(s) - 1) * s.std(ddof=1) / np.sqrt(len(s))
    moy.append(s.mean()); demi_ic.append(h)
ax2.errorbar(range(len(paliers)), moy, yerr=demi_ic, fmt="o-", color=ORANGE, capsize=4, lw=1.8)
ax2.set_xticks(range(len(paliers)))
ax2.set_xticklabels(etiquettes)
ax2.set_xlabel("délai de livraison (jours)")
ax2.set_ylabel("satisfaction moyenne (1 à 5)")
ax2.set_title("Plus c'est long, moins c'est apprécié")
haut_y = max(m + h for m, h in zip(moy, demi_ic)); bas_y = min(m - h for m, h in zip(moy, demi_ic))
ax2.set_ylim(bas_y - 0.15, haut_y + 0.15) # marge pour ne couper aucune barre d'erreur
plt.tight_layout()
```

```
plt.savefig("figures/ch07-projet-vue-d-ensemble.png", dpi=200, bbox_inches="tight")
print("figure enregistrée")
```

figure enregistrée

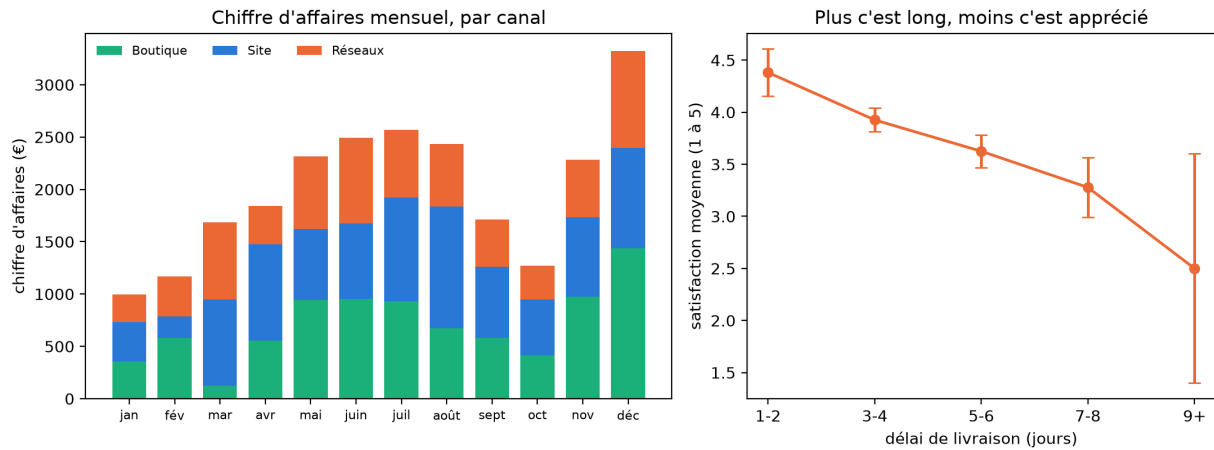


Figure 8: À gauche : chiffre d'affaires mensuel de la boutique en 2025, empilé par canal. À droite : satisfaction moyenne des commandes livrées selon le délai, avec intervalle de confiance à 95 %.

Ce que montrent ces chiffres pour la question 1. (Les nombres ci-dessous sont lus dans les sorties précédentes.)

- Le chiffre d'affaires de l'année est de **24 098,30 €** pour 400 commandes (panier moyen **60,25 €**), que reconnaîtront les lecteurs du chapitre 3.
- L'activité est **très saisonnière** : le creux est en janvier (996 €) et le pic en décembre (3 325 €), soit **un facteur 3,3** entre les deux. L'été est un plateau élevé (mai à août), suivi d'une chute en septembre-octobre, puis d'un rebond à l'approche des fêtes.
- **Aucun canal ne domine.** La boutique et le site pèsent chacun environ un tiers du chiffre d'affaires ; le canal Réseaux (réseaux sociaux), avec un nombre de commandes comparable (138, contre 148 pour le site et 114 en boutique), pèse moins (28 %) : ses paniers sont plus petits. C'est précisément le point de la question 2.

Une remarque sur le graphique de droite : le dernier palier (9 jours et plus) a un intervalle de confiance **très large**, parce qu'il ne contient que quelques commandes. Un point isolé ne prouve rien ; c'est la **tendance d'ensemble** des cinq paliers qui est convaincante. Nous la chiffrons à l'étape 4.

P.4 Étape 3 : les clients des réseaux sociaux dépensent-ils moins ? (question 2)

Reformulons la question de façon testable. La gérante voit que les paniers des clients venus des réseaux sociaux *semblent* plus petits. Ce qu'elle veut savoir : **cette différence observée dans nos 400 commandes reflète-t-elle une différence réelle entre les clientèles, ou pourrait-elle être due au hasard de l'échantillonnage ?** C'est exactement le cadre du chapitre 3 : un test de comparaison de deux moyennes (Welch, section 3.4), accompagné d'un **intervalle de confiance** (3.3) et d'une mesure de **taille d'effet**, car « significatif » ne veut pas dire « important » (3.5).

Nous avons **trois comparaisons** à faire (Boutique–Réseaux, Boutique–Site, Site–Réseaux). Selon la section 3.5, tester trois fois augmente le risque d'une fausse alerte : nous corrigerons les p-valeurs avec la méthode de **Holm**.

```

from scipy import stats

paniers = {c: g["montant"].to_numpy() for c, g in df.groupby("canal")}
paires = [("Boutique", "Réseaux"), ("Boutique", "Site"), ("Site", "Réseaux")]

lignes = []
for a, b in paires:
    x, y = paniers[a], paniers[b]
    res = stats.ttest_ind(x, y, equal_var=False)           # test de Welch
    ic = res.confidence_interval(0.95)                   # IC à 95 % de l'écart de moyennes
    s_commun = np.sqrt(((len(x) - 1) * x.var(ddof=1) + (len(y) - 1) * y.var(ddof=1)) / (len(x) +
    ↪ len(y) - 2))
    lignes.append({"comparaison": f"{a} - {b}", "ecart_eur": x.mean() - y.mean(),
                  "ic95_bas": ic.low, "ic95_haut": ic.high, "p_brute": res.pvalue,
                  "d_de_Cohen": (x.mean() - y.mean()) / s_commun})
tab = pd.DataFrame(lignes)

```

Puis la correction de Holm, écrite à la main : on trie les p-valeurs, on multiplie la plus petite par 3, la suivante par 2, la dernière par 1, on impose que la suite soit croissante et on plafonne à 1.

```

# Correction de Holm : on trie les p-valeurs, on multiplie la k-ième plus petite par (m - k + 1),
# on impose que la suite soit croissante, et on plafonne à 1.
m = len(tab)
ordre = tab["p_brute"].argsort().to_numpy()
corrigees = np.empty(m)
courant = 0.0
for rang, i in enumerate(ordre):
    courant = max(courant, min(1.0, (m - rang) * tab.loc[i, "p_brute"]))
    corrigees[i] = courant
tab["p_holm"] = corrigees

with pd.option_context("display.float_format", "{:.4g}".format, "display.width", 140):
    print(tab.to_string(index=False))

```

	comparaison	ecart_eur	ic95_bas	ic95_haut	p_brute	d_de_Cohen	p_holm
	Boutique - Réseaux	25.8	16.66	34.94	8.016e-08	0.7222	2.405e-07
	Boutique - Site	15.31	5.571	25.04	0.002188	0.3889	0.004377
	Site - Réseaux	10.49	2.393	18.59	0.0113	0.2996	0.0113

Lire le tableau. `ecart_eur` est la différence de paniers moyens ; `ic95_bas` et `ic95_haut` encadrent la différence **réelle** avec 95 % de confiance (au sens du 3.3 : la *méthode* encadre la vérité dans 95 % des cas). Le **d de Cohen** exprime l'écart en nombre d'écarts-types : environ 0,2 est « petit », 0,5 « moyen », 0,8 « grand ». Enfin `p_holm` est la p-valeur **après** correction des comparaisons multiples : c'est elle que l'on compare à 0,05.

Les trois écarts sont tous significatifs, même après correction. Mais les p-valeurs ne disent pas à quel point ces écarts comptent : le tableau montre qu'ils sont **de tailles très différentes**. L'écart entre la boutique et Réseaux est d'environ 26 € par commande (un d de Cohen de plus de 0,7 : une différence franchement visible), alors que l'écart Site-Réseaux est de l'ordre de 10 € (un d autour de 0,3, plutôt petit).

Les montants sont **asymétriques** (3.1.3 : la moyenne est tirée par quelques gros paniers). Vérifions que la conclusion ne dépend pas de ce choix en comparant aussi les **médianes**, avec un **bootstrap** (3.3.5) :

```

rng = np.random.default_rng(42)
B = 5000
x, y = paniers["Boutique"], paniers["Réseaux"]
diffs = np.empty(B)
for k in range(B):

```

```

diffs[k] = np.median(rng.choice(x, len(x))) - np.median(rng.choice(y, len(y)))
bas, haut = np.percentile(diffs, [2.5, 97.5])
print(f"médiane Boutique : {np.median(x):.2f} €   médiane Réseaux : {np.median(y):.2f} €")
print(f"écart de médianes : {np.median(x) - np.median(y):.2f} €   IC95 bootstrap : [{bas:.1f} ;
↪ {haut:.1f}]")

```

```

médiane Boutique : 64.85 €   médiane Réseaux : 41.50 €
écart de médianes : 23.35 €   IC95 bootstrap : [14.6 ; 31.6]

```

L'écart de médianes est du même ordre que l'écart de moyennes, et son intervalle exclut nettement zéro : la conclusion est **robuste**.

Ce que ces tests ne disent pas. Ils établissent que les paniers du canal Réseaux sont plus petits ; ils n'expliquent **pas pourquoi** (produits moins chers ? clientèle plus jeune ? achats d'impulsion ?). Et surtout, un panier plus petit ne signifie pas un canal moins rentable : nous n'avons ni les coûts, ni le temps passé, ni la marge. Dire « Réseaux dépense moins par commande » est un fait ; dire « Réseaux ne vaut pas le coup » serait une conclusion que ces données **ne permettent pas** de tirer.

P.5 Étape 4 : les retards font-ils baisser la satisfaction ? (question 3)

Ici, il ne s'agit plus de comparer des groupes mais de **relier deux variables** : le délai de livraison (en jours) et la satisfaction (de 1 à 5). Un détail essentiel : la boutique a un délai nul par construction (le client repart avec sa commande). Si nous l'incluons, nous mélangerions deux effets : « le client a-t-il attendu ? » et « le client est-il venu en boutique ? ». Pour isoler l'effet du **délai**, nous nous limitons aux commandes **livrées** (Site et Réseaux).

```

livre = df[df["canal"] != "Boutique"].copy()
print(len(livre), "commandes livrées")
print(livre.groupby("canal")["delai_livraison"].agg(["count", "mean", "median", "max"]).round(2))
print()
r_pearson = livre["delai_livraison"].corr(livre["satisfaction"])
r_spearman = stats.spearmanr(livre["delai_livraison"], livre["satisfaction"])
print(f"corrélation de Pearson : {r_pearson:.3f}")
print(f"corrélation de Spearman : {r_spearman.statistic:.3f} (p = {r_spearman.pvalue:.1e})")

```

286 commandes livrées

	count	mean	median	max
canal				
Réseaux	138	4.49	4.0	9
Site	148	4.70	4.0	13

```

corrélation de Pearson : -0.407
corrélation de Spearman : -0.365 (p = 1.9e-10)

```

Nous avons calculé **deux** corrélations. Pearson mesure la liaison *linéaire* ; Spearman travaille sur les rangs et ne suppose pas de linéarité (3.1.7), ce qui convient mieux à une note de 1 à 5 (variable **ordinaire**, 3.1.1). Les deux disent la même chose : une corrélation négative modérée. (Au 3.1.7, nous avons trouvé environ $-0,53$ sur **toutes** les commandes ; la valeur est ici plus faible parce que nous avons retiré la boutique, dont les délais nuls et les notes élevées renforçaient artificiellement le lien. C'est un bon exemple de la façon dont une décision de **périmètre** change un chiffre.)

Quantifions maintenant l'**effet moyen d'un jour de retard**. C'est la pente de la droite des moindres carrés (1.3 : on choisit la droite qui minimise la somme des carrés des erreurs ; la formule ci-dessous en est la solution) :

$$\hat{b} = \frac{\sum_i (d_i - \bar{d})(s_i - \bar{s})}{\sum_i (d_i - \bar{d})^2}$$

où d_i est le délai de la commande i et s_i sa note de satisfaction.

```
d = livre["delai_livraison"].to_numpy(dtype=float)
s = livre["satisfaction"].to_numpy(dtype=float)
pente = np.sum((d - d.mean()) * (s - s.mean())) / np.sum((d - d.mean()) ** 2)
ordonnee = s.mean() - pente * d.mean()
print(f"droite des moindres carrés : satisfaction = {ordonnee:.2f} + ({pente:.3f}) x délai")

# Intervalle de confiance par bootstrap (on rééchantillonne des COMMANDES entières)
rng = np.random.default_rng(7)
pentes = np.empty(5000)
for k in range(5000):
    idx = rng.integers(0, len(d), len(d))
    dk, sk = d[idx], s[idx]
    pentes[k] = np.sum((dk - dk.mean()) * (sk - sk.mean())) / np.sum((dk - dk.mean()) ** 2)
pente_bas, pente_haut = np.percentile(pentes, [2.5, 97.5])
print(f"IC95 bootstrap de la pente : [{pente_bas:.3f} ; {pente_haut:.3f}]"
```

```
droite des moindres carrés : satisfaction = 4.58 + (-0.180) x délai
IC95 bootstrap de la pente : [-0.228 ; -0.129]
```

Chaque jour de livraison supplémentaire est associé à une baisse de satisfaction d'environ **0,18 point** (sur une échelle de 5), et l'intervalle de confiance, qui ne contient pas zéro, exclut un effet nul. Pour parler en termes concrets, comparons des **groupes de délais** (puis la même pente, canal par canal, pour vérifier que l'effet n'est pas un simple artefact du mélange Site/Réseaux) :

```
livre["groupe"] = pd.cut(livre["delai_livraison"], bins=[0, 3, 6, 100], labels=["1-3 jours", "4-6
↳ jours", "7 jours et +"])
print(livre.groupby("groupe", observed=True)["satisfaction"].agg(commandes="count",
↳ moyenne="mean").round(2))
print()
for canal, g in livre.groupby("canal"):
    res = stats.linregress(g["delai_livraison"], g["satisfaction"])
    print(f"{canal:10s} pente = {res.slope:.3f} point/jour (r = {res.rvalue:.2f}, {len(g)}
↳ commandes)"
```

	commandes	moyenne
groupe		
1-3 jours	88	4.03
4-6 jours	156	3.76
7 jours et +	42	3.17

Réseaux	pente = -0.182 point/jour	(r = -0.38, 138 commandes)
Site	pente = -0.181 point/jour	(r = -0.44, 148 commandes)

Les trois groupes sont nettement ordonnés : plus le délai est long, plus la satisfaction moyenne est basse, avec un écart d'environ **0,9 point** entre les livraisons rapides (1 à 3 jours) et les livraisons lentes (7 jours et plus). Et la pente est du même ordre dans les deux canaux : le phénomène n'est pas un effet de mélange.

Une dernière question de la gérante, naturelle : « si je ramenait tous mes délais de plus de 5 jours à 5 jours, que gagnerais-je ? » Le calcul est facile avec la droite, et il faut l'énoncer avec **prudence** :

```
longs = livre[livre["delai_livraison"] > 5]
gain_par_commande = (pente * (5 - longs["delai_livraison"])).mean() # pente < 0 et (5 - délai) < 0 :
↳ gain > 0
part = len(longs) / len(livre)
print(f"{len(longs)} commandes livrées sur {len(livre)} ({100 * part:.0f} %) dépassent 5 jours")
```

```
print(f"gain de satisfaction attendu sur ces commandes : +{gain_par_commande:.2f} point")
print(f"gain sur l'ensemble des livraisons : +{part * gain_par_commande:.3f} point")
```

```
74 commandes livrées sur 286 (26 %) dépassent 5 jours
gain de satisfaction attendu sur ces commandes : +0.36 point
gain sur l'ensemble des livraisons : +0.094 point
```

Association n'est pas causalité. La pente décrit comment les deux variables **varient ensemble** dans nos données. Elle ne prouve pas que réduire les délais *fera* monter les notes : un facteur caché pourrait jouer sur les deux à la fois (les commandes volumineuses sont peut-être à la fois plus lentes à préparer et plus exigeantes). Ici les données sont simulées et la relation a été construite pour être réelle, mais dans une vraie étude, la phrase honnête serait : « *les commandes livrées plus lentement sont associées à des notes plus basses, d'environ 0,18 point par jour* ». Distinguer corrélation et causalité est l'un des grands thèmes du volume II (chapitre 7, facultatif).

P.6 Étape 5 : qui relancer en janvier ? (question 4)

Cette question n'est pas un test : c'est de la **segmentation**. On veut une liste claire de clients à contacter, et une raison de les contacter. Nous la construisons en SQL avec des CTE et NTILE (5.3). Deux critères :

- **La valeur** : le chiffre d'affaires cumulé du client en 2025 ;
- **La récence** : le nombre de jours depuis sa dernière commande, à la date de référence du 31 décembre 2025.

D'abord, le principe de **concentration** : quelle part du chiffre d'affaires vient des 25 % de clients qui achètent le plus ? (C'est la règle de Pareto, souvent « 80/20 ».)

```
concentration = pd.read_sql_query("""
    WITH par_client AS (
        SELECT id_client, SUM(montant) AS ca FROM commandes GROUP BY id_client
    ), classes AS (
        SELECT id_client, ca, NTILE(4) OVER (ORDER BY ca DESC) AS quartile FROM par_client
    )
    SELECT quartile, COUNT(*) AS clients, ROUND(SUM(ca), 2) AS ca,
        ROUND(100.0 * SUM(ca) / (SELECT SUM(ca) FROM par_client), 1) AS part_ca_pct
    FROM classes GROUP BY quartile ORDER BY quartile
""", con)
print(concentration.to_string(index=False))
```

quartile	clients	ca	part_ca_pct
1	17	14063.4	58.4
2	17	5678.4	23.6
3	16	3086.4	12.8
4	16	1270.1	5.3

Le premier quartile (les meilleurs clients) pèse un peu plus de la moitié du chiffre d'affaires : une clientèle **concentrée**. Perdre l'un de ces clients coûte beaucoup plus que d'en perdre un petit.

Cherchons maintenant les **clients précieux devenus silencieux** : ceux dont la valeur est dans les deux meilleurs quartiles mais qui n'ont plus rien commandé depuis plus de 90 jours.

```
relance = pd.read_sql_query("""
    WITH par_client AS (
        SELECT k.id_client, k.prenom, k.nom, k.ville,
            COUNT(c.id_commande) AS commandes,
            COALESCE(SUM(c.montant), 0) AS ca,
```

```

        MAX(c.date_commande) AS derniere
    FROM clients k LEFT JOIN commandes c USING(id_client)
    GROUP BY k.id_client
), classes AS (
    SELECT *, julianday('2025-12-31') - julianday(derniere) AS jours_depuis,
           NTILE(4) OVER (ORDER BY ca DESC) AS quartile_valeur
    FROM par_client WHERE commandes > 0
)
SELECT prenom || ' ' || nom AS client, ville, commandes, ROUND(ca, 2) AS ca_2025,
       derniere AS derniere_commande, CAST(jours_depuis AS INTEGER) AS jours_depuis
FROM classes
WHERE jours_depuis > 90 AND quartile_valeur <= 2
ORDER BY ca DESC
""" , con)
print(len(relance), "clients précieux à relancer :")
print(relance.to_string(index=False))
jamais = un_seul_chiffre("SELECT COUNT(*) FROM clients k WHERE NOT EXISTS (SELECT 1 FROM commandes c
↳ WHERE c.id_client = k.id_client)")
print(f"\n(par ailleurs, {jamais} clients inscrits n'ont jamais commandé : une campagne différente,
↳ de première commande)")

```

5 clients précieux à relancer :

	client	ville	commandes	ca_2025	derniere_commande	jours_depuis
	Alex Lefebvre	Ville E	5	393.0	2025-07-26	158
	Lina Faure	Ville B	4	297.8	2025-08-17	136
	Hugo Lefebvre	Ville C	6	294.9	2025-09-24	98
	Théo Michel	Ville C	6	284.0	2025-08-30	123
	Noé Lefebvre	Ville G	7	270.3	2025-09-26	96

(par ailleurs, 14 clients inscrits n'ont jamais commandé : une campagne différente, de première commande)

Deux listes, deux messages. Les clients qui ont **beaucoup acheté puis disparu** se contactent avec un message personnel (« cela fait un moment, voici les nouveautés »). Les clients **inscrits qui n'ont jamais commandé** appellent plutôt une offre de première commande. Les mélanger, c'est brouiller le message.

P.7 Étape 6 : rédiger le rapport

Tout le travail précédent n'a de valeur que s'il se transforme en **message clair**. Un rapport pour la gérante tient sur une page, ne contient aucun jargon, donne les chiffres clés, formule des recommandations et **annonce ses limites**.

Une bonne pratique, héritée du chapitre 6 (recherche reproductible) : **ne jamais recopier un chiffre à la main** dans un rapport. On génère le texte à partir des résultats déjà calculés. Si les données changent demain, le rapport se met à jour tout seul.

Une petite fonction de mise en forme à la française (espace pour les milliers, virgule décimale), puis les valeurs à citer :

```

mois_fr = ["janvier", "février", "mars", "avril", "mai", "juin", "juillet", "août",
           "septembre", "octobre", "novembre", "décembre"]

def fr(x, decimales=2):
    """Format français : 24098.3 -> '24 098,30'."""
    return f"{x:,.{decimales}f}".replace(",", " ").replace(".", ",")

ca_total = total.loc[0, "chiffre_affaires"]
mois_pic = mensuel.loc[mensuel["ca"].idxmax()]

```

```
mois_creux = mensuel.loc[mensuel["ca"].idxmin()]
t_bi = tab.iloc[0]          # Boutique - Réseaux
groupes = livre.groupby("groupe", observed=True)["satisfaction"].mean()
top_quartile_part = concentration.loc[0, "part_ca_pct"]
```

Puis le texte du rapport lui-même : un gabarit en deux morceaux, dans lequel chaque nombre est remplacé par la valeur calculée.

```
rapport1 = f"""RAPPORT 2025 : LA BOUTIQUE
{"=" * 60}

1. L'année en bref
- Chiffre d'affaires : {fr(ca_total)} € pour {int(total.loc[0, 'commandes'])} commandes
  (panier moyen : {fr(total.loc[0, 'panier_moyen'])} €).
- Un rythme très saisonnier : creux en {mois_fr[int(mois_creux['mois']) - 1]}
↳ ({fr(mois_creux['ca'], 0)} €),
  pic en {mois_fr[int(mois_pic['mois']) - 1]} ({fr(mois_pic['ca'], 0)} €).

2. Les clients des réseaux sociaux dépensent-ils moins ?
- Oui : un panier venu des réseaux sociaux est inférieur de {fr(t_bi['ecart_eur'], 1)} € à un
↳ panier en boutique
  (intervalle de confiance à 95 % : de {fr(t_bi['ic95_bas'], 1)} à {fr(t_bi['ic95_haut'], 1)} €).
- Cet écart n'est pas dû au hasard (p corrigée < 0,001) et il est franc (d de Cohen =
↳ {fr(t_bi['d_de_Cohen'])}).
- Attention : nous ne connaissons ni les marges, ni le temps passé. Un panier plus petit
  ne veut pas dire un canal moins rentable.

"""
```

Puis la suite du gabarit (retards, relances, limites) et l'impression du rapport complet :

```
rapport2 = f"""3. Les retards pèsent sur la satisfaction
- Chaque jour de livraison en plus est associé à {fr(abs(pente), 2)} point de satisfaction en moins
  (intervalle à 95 % : de {fr(abs(pente_haut), 2)} à {fr(abs(pente_bas), 2)}).
- Livraison en 1 à 3 jours : {fr(groupes.iloc[0])} / 5 ; en 7 jours et plus : {fr(groupes.iloc[2])}
↳ / 5.
- {len(longs)} livraisons sur {len(livre)} dépassent 5 jours : c'est là que l'on peut gagner.

4. Clients à relancer en janvier
- {len(relance)} clients précieux sont silencieux depuis plus de 90 jours ; les
↳ {int(concentration.loc[0, 'clients'])} meilleurs clients
  font {fr(top_quartile_part, 1)} % du chiffre d'affaires.
- {jamais} clients inscrits n'ont jamais commandé : prévoir une offre de première commande.

Limites : une seule année de données ; pas de coûts ; association n'est pas causalité.
"""

print(rapport1 + rapport2)
```

```
RAPPORT 2025 : LA BOUTIQUE
```

```
=====
```

```
1. L'année en bref
- Chiffre d'affaires : 24 098,30 € pour 400 commandes
  (panier moyen : 60,25 €).
- Un rythme très saisonnier : creux en janvier (996 €),
  pic en décembre (3 325 €).

2. Les clients des réseaux sociaux dépensent-ils moins ?
- Oui : un panier venu des réseaux sociaux est inférieur de 25,8 € à un panier en boutique
  (intervalle de confiance à 95 % : de 16,7 à 34,9 €).
- Cet écart n'est pas dû au hasard (p corrigée < 0,001) et il est franc (d de Cohen = 0,72).
- Attention : nous ne connaissons ni les marges, ni le temps passé. Un panier plus petit
  ne veut pas dire un canal moins rentable.
```

3. Les retards pèsent sur la satisfaction

- Chaque jour de livraison en plus est associé à 0,18 point de satisfaction en moins (intervalle à 95 % : de 0,13 à 0,23).
- Livraison en 1 à 3 jours : 4,03 / 5 ; en 7 jours et plus : 3,17 / 5.
- 74 livraisons sur 286 dépassent 5 jours : c'est là que l'on peut gagner.

4. Clients à relancer en janvier

- 5 clients précieux sont silencieux depuis plus de 90 jours ; les 17 meilleurs clients font 58,4 % du chiffre d'affaires.
- 14 clients inscrits n'ont jamais commandé : prévoir une offre de première commande.

Limites : une seule année de données ; pas de coûts ; association n'est pas causalité.

Relisez ce rapport comme la gérante. Y a-t-il un mot qu'elle ne comprendrait pas (« d de Cohen », « bootstrap ») ? Dans le rapport final, on garde le **chiffre** et on cache la **méthode** : l'annexe technique, c'est le reste de ce projet, rangé dans le dépôt. Dans la version ci-dessus, le « d de Cohen » est volontairement conservé pour que vous voyiez d'où vient chaque chiffre ; dans le document remis à la gérante, on écrirait simplement « un écart net ».

P.8 Rendre le projet reproductible

Un projet n'est terminé que lorsqu'**une autre personne peut le refaire**. Voici la check-list du chapitre 6, appliquée à ce projet :

Geste	Pourquoi	Où l'apprendre
Un dossier propre : <code>donnees/</code> , <code>sql/</code> , <code>analyse/</code> , <code>figures/</code> , <code>rapport/</code>	on retrouve tout	6.3
Versionner avec Git , un commit par étape logique	on peut revenir en arrière et expliquer les changements	6.1
Fixer les graines (<code>default_rng(42)</code> , <code>default_rng(7)</code>)	mêmes résultats à chaque exécution	3.7, 6.5
Lister les versions des bibliothèques (<code>requirements.txt</code>)	l'environnement se recrée	6.3
Garder les contrôles de l'étape 1 dans le script	les données douteuses sont détectées	4.6
Un notebook ou un script qui va des données brutes au rapport	un seul geste pour tout refaire	6.2, 6.5

Voici une structure de dossier possible. Elle est donnée à titre d'exemple (non exécutée) : à vous de l'adapter.

```

etude-boutique-2025/
├── README.md           <- la question, comment relancer, les versions
├── requirements.txt
├── donnees/
│   └── boutique.db
├── sql/                <- chaque requête dans son fichier : 01_ca_mensuel.sql, ...
├── analyse/
│   ├── 01_controles.py
│   ├── 02_comparaisons.py
│   └── 03_rapport.py
└── figures/

```

```
└─ rapport/
  └─ rapport-2025.md
```

P.9 Ce que cette étude ne dit pas, et la suite

Une étude honnête se termine par ses **limites** :

- **Une seule année** : impossible de distinguer une vraie saisonnalité d'un phénomène propre à 2025.
- **Pas de coûts** : nous avons parlé de chiffre d'affaires, jamais de **bénéfice**. Le canal des réseaux sociaux pourrait être très rentable si l'on dépense peu pour y vendre.
- **Pas de causalité** : les relations constatées sont des **associations** ; pour savoir si réduire les délais *améliorerait* les notes, il faudrait une expérience (volume II, chapitre 7 facultatif sur l'inférence causale, et chapitre 8 sur les plans d'expériences).
- **Variables simples** : nous n'avons pas pris en compte, par exemple, la ville de livraison, le type de produit ou le client lui-même (un client très exigeant note toujours bas). Les modèles du volume II (régression multiple, modèles mixtes) permettent de **tenir compte de plusieurs facteurs à la fois**.

À retenir. Ce que vous venez de faire, de la question de la gérante au rapport, est le **cycle de base de la data science** : *poser la question → contrôler les données → décrire → comparer ou relier avec rigueur → conclure avec prudence → rendre reproductible*. Les volumes suivants ajouteront des outils (régression, apprentissage automatique, séries temporelles...), mais ce cycle ne changera pas.

Auto-évaluation

Mode d'emploi. Répondez à **voix haute** ou **par écrit** avant de regarder le corrigé, en une ou deux phrases. Si vous ne savez pas, notez la section indiquée et allez la relire : ce n'est pas un échec, c'est le but de l'exercice. Vingt-cinq bonnes réponses sur trente signalent un volume bien assimilé.

Mathématiques (chapitre 1)

1. Que signifie l'égalité $A\mathbf{v} = \lambda\mathbf{v}$?
2. Dans quelle direction faut-il se déplacer pour **faire diminuer** le plus vite possible une fonction ?
3. Pourquoi $0.1 + 0.2 == 0.3$ vaut-il `False` en Python, et comment comparer proprement deux nombres décimaux ?
4. La gérante veut présenter 3 produits choisis parmi 8 dans une vitrine, sans tenir compte de l'ordre. Combien de vitrines possibles ?

Probabilités (chapitre 2)

5. Une maladie touche 1 % de la population. Un test la détecte dans 90 % des cas, mais donne un faux positif chez 5 % des personnes saines. Vous êtes positif : quelle est la probabilité d'être malade ?
6. Quelle différence entre la loi des grands nombres et le théorème central limite ?
7. Les montants de commandes ont un écart-type de 38 €. Quel est l'écart-type de la **moyenne** de 400 commandes ?

8. Quelle loi pour (a) le nombre de commandes reçues en une heure ; (b) le fait qu'une commande soit retournée ou non ?

Statistique (chapitre 3)

9. Pourquoi divise-t-on par $n - 1$ et non par n pour estimer une variance ?
 10. Que signifie « intervalle de confiance à 95 % » ? Que ne signifie-t-il **pas** ?
 11. Qu'est-ce qu'une p-valeur ? Citez une mauvaise interprétation fréquente.
 12. On réalise 20 tests indépendants au seuil de 5 %, alors qu'**aucun** effet n'existe. Combien de faux positifs attend-on, et quelle est la probabilité d'en obtenir **au moins un** ?
 13. Quand préférer la médiane à la moyenne ?
 14. Un test donne $p = 10^{-9}$ pour une différence de 0,3 € entre deux paniers moyens. Doit-on s'en réjouir ?

Programmation (chapitre 4)

15. Pourquoi tester l'appartenance d'un élément est-il bien plus rapide dans un set ou un dict que dans une list de grande taille ?
 16. Pourquoi `df["montant"].sum()` est-il préférable à une boucle for sur les lignes ?
 17. Que renvoie `df.groupby("canal")["montant"].mean()` : quel type, et quel index ?
 18. Citez deux manières de rendre un graphique en barres trompeur.
 19. Combien de comparaisons, au maximum, la recherche dichotomique effectue-t-elle sur une liste triée d'un million d'éléments ?
 20. À quoi sert un test unitaire, et pourquoi vaut-il mieux que « j'ai regardé, ça avait l'air bon » ?

SQL (chapitre 5)

21. Quelle est la différence entre une clé primaire et une clé étrangère ?
 22. Quelle différence entre WHERE et HAVING ?
 23. Vous voulez la liste de **tous** les clients avec leur nombre de commandes, y compris ceux qui n'ont jamais commandé. Quelle jointure ?
 24. Pourquoi `WHERE telephone = NULL` ne renvoie-t-il jamais rien, et que faut-il écrire ?
 25. En quoi une fonction fenêtre (OVER) diffère-t-elle d'un GROUP BY ?
 26. Pourquoi normaliser une base (jusqu'à la 3FN) ?

Outils (chapitre 6)

27. Quelle différence entre `git add` et `git commit` ?
 28. Pourquoi un notebook peut-il donner des résultats différents selon qui l'exécute, et comment s'en protéger ?
 29. À quoi sert un fichier `requirements.txt` ?
 30. Quelle commande compte rapidement le nombre de lignes d'un fichier CSV, et pourquoi faut-il en retrancher une ?

Vérifier les réponses chiffrées

Pour les questions numériques, plutôt que de se fier à sa mémoire, **calculons**. Le code ci-dessous vérifie les réponses des questions 3, 4, 5, 7, 12 et 19.

```
import math
```

```
# Q3 : arithmétique des flottants
print("Q3 0.1 + 0.2 == 0.3 :", 0.1 + 0.2 == 0.3, "| avec tolérance :", math.isclose(0.1 + 0.2, 0.3))

# Q4 : combinaisons
print("Q4 C(8,3) =", math.comb(8, 3))

# Q5 : formule de Bayes
prevalence, sensibilite, faux_positifs = 0.01, 0.90, 0.05
p_positif = sensibilite * prevalence + faux_positifs * (1 - prevalence)
print(f"Q5 P(malade | test positif) = {sensibilite * prevalence / p_positif:.4f}")

# Q7 : écart-type d'une moyenne
print(f"Q7 38 / sqrt(400) = {38 / math.sqrt(400):.2f} €")

# Q12 : tests multiples
print(f"Q12 faux positifs attendus : {20 * 0.05:.0f} ; P(au moins un) = {1 - 0.95 ** 20:.4f}")

# Q19 : recherche dichotomique
print("Q19 comparaisons max pour 1 000 000 éléments :", math.ceil(math.log2(1_000_000 + 1)))
```

```
Q3 0.1 + 0.2 == 0.3 : False | avec tolérance : True
Q4 C(8,3) = 56
Q5 P(malade | test positif) = 0.1538
Q7 38 / sqrt(400) = 1.90 €
Q12 faux positifs attendus : 1 ; P(au moins un) = 0.6415
Q19 comparaisons max pour 1 000 000 éléments : 20
```

Corrigés des questions

1. $\mathbf{v}$ est un **vecteur propre** de A : la matrice ne change pas sa direction, elle l'étire (ou le comprime) d'un facteur λ , la **valeur propre**. (1.1.3)
2. Dans la direction **opposée au gradient**, qui pointe vers la plus forte montée. C'est le principe de la descente de gradient. (1.2 et 1.3)
3. Les nombres décimaux sont stockés en binaire, et 0,1 n'a pas d'écriture binaire finie : on ne stocke qu'une **approximation**. On compare avec une tolérance (`math.isclose`, `np.isclose`), jamais avec `==`. (1.5)
4. $\binom{8}{3} = \frac{8!}{3!5!} = 56$ vitrines. L'ordre ne comptant pas, on divise les $8 \times 7 \times 6 = 336$ arrangements par les $3! = 6$ façons de les ranger. (1.6)
5. Environ **15,4 %**, loin des 90 % que l'on devine. Sur 1 000 personnes, 10 sont malades (9 détectées) et 990 sont saines (environ 49,5 faux positifs) : seulement 9 positifs sur 58,5 environ sont vraiment malades. C'est ce qui arrive quand la maladie est rare. (2.1)
6. La loi des grands nombres dit que la **moyenne d'échantillon converge** vers l'espérance quand n grandit ; le théorème central limite décrit **comment elle fluctue** autour de celle-ci : approximativement selon une loi normale d'écart-type $\sigma / \sqrt{n}$, quelle que soit la loi d'origine. (2.4)
7. $38 / \sqrt{400} = 38 / 20 = 1,9$ € : la moyenne est bien plus stable que chaque commande. C'est la raison pour laquelle on moyenne. (2.4)
8. (a) Une loi de **Poisson** (événements rares et indépendants dans un intervalle de temps) ; (b) une loi de **Bernoulli** (deux issues), ou binomiale si l'on compte le nombre de retours sur n commandes. (2.2)

9. Parce que l'on mesure les écarts à la moyenne **de l'échantillon**, qui est elle-même ajustée aux données : les écarts sont un peu trop petits. Diviser par $n - 1$ corrige ce biais et rend l'estimateur **sans biais**. (3.2)
10. La **méthode** produit un intervalle qui contient la vraie valeur dans 95 % des échantillons possibles. Ce n'est **pas** « 95 % de chances que la vraie valeur soit dans cet intervalle-ci » : une fois calculé, l'intervalle contient la vraie valeur ou ne la contient pas. (3.3.2)
11. La probabilité d'observer un résultat **au moins aussi extrême** que le nôtre, *si l'hypothèse nulle était vraie*. Mauvaise interprétation fréquente : « c'est la probabilité que l'hypothèse nulle soit vraie ». (3.5.1 et 3.5.2)
12. On attend $20 \times 0,05 = 1$ faux positif, et la probabilité d'au moins un est $1 - 0,95^{20} \approx 64\%$: d'où la nécessité de corriger les tests multiples. (3.5.5)
13. Quand la distribution est **asymétrique** ou contient des **valeurs extrêmes** (montants, revenus, durées) : la médiane est robuste, la moyenne est tirée par la queue. (3.1.3)
14. Pas vraiment : avec assez de données, même une différence minuscule devient « significative ». 0,3 € sur un panier de 60 € n'a **aucune importance pratique**. Il faut toujours regarder la **taille de l'effet** et l'intervalle de confiance, pas seulement la p-valeur. (3.5.3)
15. Un set ou un dict utilise une **table de hachage** : il calcule directement où se trouve l'élément (coût quasi constant). Une liste doit être **parcourue** élément par élément (coût proportionnel à sa taille). (4.3.2 et 4.8)
16. La somme vectorisée s'exécute en **code compilé** sur un tableau contigu, sans le surcoût de l'interpréteur Python à chaque ligne : elle est en général beaucoup plus rapide (au moins plusieurs fois, souvent bien davantage selon la taille du tableau), et plus courte à écrire. (4.4 et 4.8)
17. Une **Series** pandas dont l'index est le canal (Boutique, Réseaux, Site) et dont les valeurs sont les montants moyens. (4.4)
18. Par exemple : **tronquer l'axe vertical** (un écart réel de 2 % peut sembler un rapport de 5 à 1), utiliser un **camembert en 3D** (la perspective déforme les aires) ou un **double axe vertical** (on rend « visible » n'importe quelle corrélation en choisissant les échelles). Une barre doit toujours partir de zéro. (4.5.6)
19. 20 comparaisons au plus ($2^{20} = 1\,048\,576 > 10^6$) : chaque comparaison divise l'intervalle de recherche par deux. Chercher dans une liste non triée en demanderait jusqu'à un million. (4.3.5)
20. Un test unitaire **vérifie automatiquement** qu'une fonction renvoie le résultat attendu sur des cas connus. Rejoué à chaque modification, il détecte immédiatement une régression ; un coup d'œil, lui, oublie les cas limites et ne se rejoue pas. (4.6)
21. La **clé primaire** identifie de façon unique chaque ligne d'une table. Une **clé étrangère** est une colonne qui référence la clé primaire d'une autre table : c'est elle qui crée le lien entre les tables. (5.1)
22. WHERE filtre les **lignes** avant le regroupement ; HAVING filtre les **groupes** après l'agrégation (par exemple « les clients avec plus de 5 commandes »). (5.2.4)
23. Un **LEFT JOIN** de clients vers commandes : il garde tous les clients, avec NULL (ou 0 après COUNT sur la colonne de droite) pour ceux qui n'ont pas de commande. Un **INNER JOIN** les ferait disparaître. (5.2.5)
24. Parce que NULL signifie « inconnu » : comparer quoi que ce soit à NULL donne « inconnu », jamais « vrai ». Il faut écrire WHERE telephone IS NULL. (5.2.7)

25. `GROUP BY` **réduit** plusieurs lignes à une seule par groupe ; une fonction fenêtre **conserve toutes les lignes** et ajoute une colonne calculée sur une « fenêtre » de lignes voisines (classement, cumul, ligne précédente). (5.3.1)

26. Pour **éviter la redondance** (la même information écrite à plusieurs endroits) et donc les **anomalies** de mise à jour, d'insertion et de suppression : chaque fait est stocké **une seule fois**. (5.4)

27. `git add` **prépare** les modifications (zone d'index) ; `git commit` **enregistre** ce qui a été préparé dans l'historique, avec un message. Cela permet de composer des commits cohérents. (6.1.3)

28. Parce que l'on peut exécuter les cellules **dans le désordre** et que le noyau garde en mémoire des variables qui n'existent plus dans le fichier : c'est l'**état caché**. Protection : *Restart & Run All* avant de partager ou d'en tirer un résultat. (6.2.3)

29. Il **liste les bibliothèques et leurs versions** nécessaires, pour que n'importe qui puisse recréer le même environnement (`pip install -r requirements.txt`). (6.3.6)

30. `wc -l fichier.csv`. Il faut retrancher **1** : la première ligne est l'en-tête (les noms de colonnes), pas une observation. (6.3)

Votre grille d'auto-évaluation

Pour chaque ligne, cochez mentalement : **je sais l'expliquer** / **je sais le faire** / **à revoir**. Les sections à relire sont indiquées.

Compétence	Où la retravailler
Manipuler des vecteurs et des matrices, interpréter valeurs propres et SVD	1.1
Dériver, calculer un gradient, faire une descente de gradient	1.2, 1.3
Calculer avec des probabilités conditionnelles et appliquer Bayes	2.1
Choisir une loi et en calculer espérance et variance	2.2, 2.3
Expliquer la loi des grands nombres et le théorème central limite	2.4
Décrire un jeu de données (position, dispersion, forme) et le tracer	3.1
Estimer un paramètre, construire et interpréter un intervalle de confiance	3.2, 3.3
Mener un test, lire une p-valeur, corriger les tests multiples	3.4, 3.5
Écrire un programme Python avec fonctions, boucles, dictionnaires	4.1, 4.3
Manipuler un tableau avec pandas (filtrer, regrouper, joindre)	4.4
Produire un graphique honnête et lisible	4.5
Écrire des requêtes SQL avec jointures et agrégations	5.2
Utiliser fonctions fenêtres et CTE	5.3
Concevoir un schéma normalisé	5.1, 5.4
Versionner un projet avec Git	6.1
Utiliser notebooks, ligne de commande et environnements virtuels	6.2, 6.3
Mener un petit projet de bout en bout	Projet du volume